

计算机组成原理

讲师：老汤

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器



第四章：数据的表示与运算

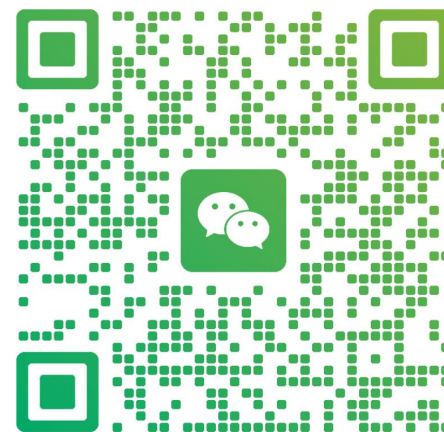
第五章：指令系统

第六章：中央处理器（CPU）

第七章：输入输出（I/O）系统

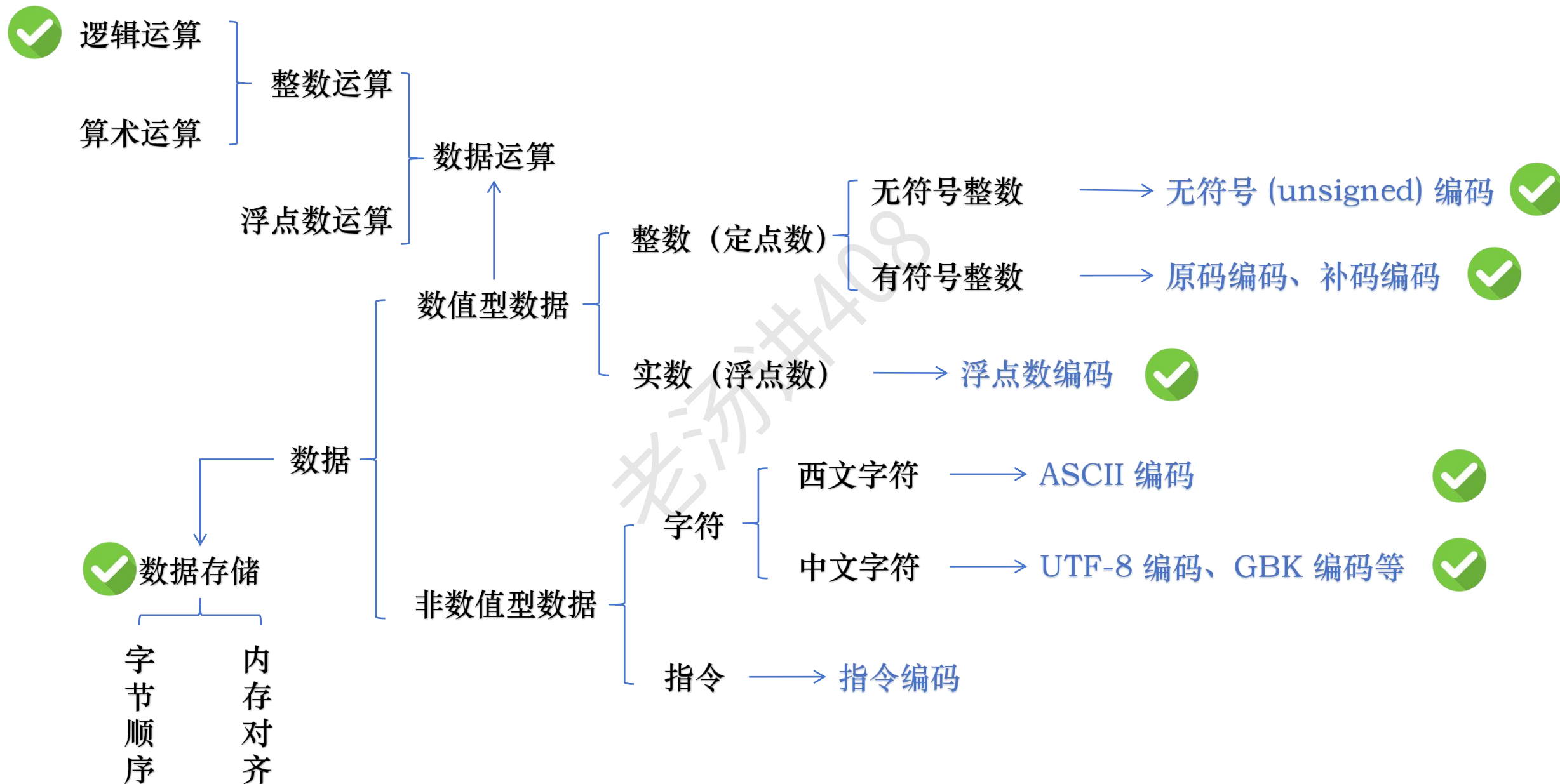


老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

数据运算



整数运算：逻辑运算

① 与、或、非、异或

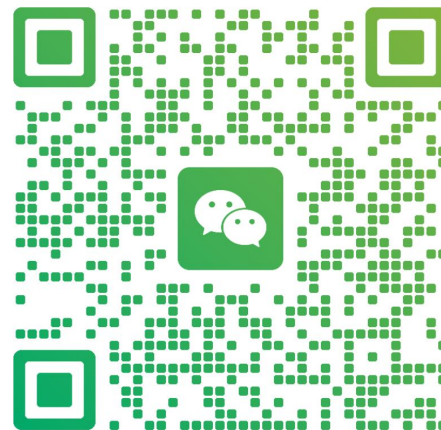
② 移位

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

整数运算：逻辑运算（与、或、非、异或）

与

$\&$	0	1
0	0	0
1	0	1

或

$ $	0	1
0	0	1
1	1	1

非

$\sim$	
0	1
1	0

异或

$\wedge$	0	1
0	0	1
1	1	0

整数运算：逻辑运算（与、或、非、异或）

机器字长 = 4

与

$$\begin{array}{r} 0110 \\ \& 1100 \\ \hline 0100 \end{array}$$



4 个逻辑与门电路

或

$$\begin{array}{r} 0110 \\ | 1100 \\ \hline 1110 \end{array}$$



4 个逻辑或门电路

非

$$\begin{array}{r} \sim 1100 \\ \hline 0011 \end{array}$$



4 个逻辑非门电路

异或

$$\begin{array}{r} 0110 \\ \wedge 1100 \\ \hline 1010 \end{array}$$



4 个逻辑异或门电路

整数运算：逻辑运算（与、或、非、异或） - 掩码

0xABCD85 $\longrightarrow$ 1010 1011 1100 1101 1000 0101
 $\&$ 1111 1111 $\longrightarrow$ 0xFF 掩码

0000 0000 0000 0000 1000 0101 $\longrightarrow$ 0x000085

145.13.3.10 $\longrightarrow$ 1001 0001 0000 1101 0000 0011 0000 1010
 $\&$ 1111 1111 1111 1111 0000 0000 0000 0000 $\longrightarrow$ 子网掩码
0xFFFF0000

1001 0001 0000 1101 0000 0000 0000 0000 $\longrightarrow$ 145.13.0.0

整数运算：逻辑运算（移位）

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

① 逻辑移位

对无符号数进行移位

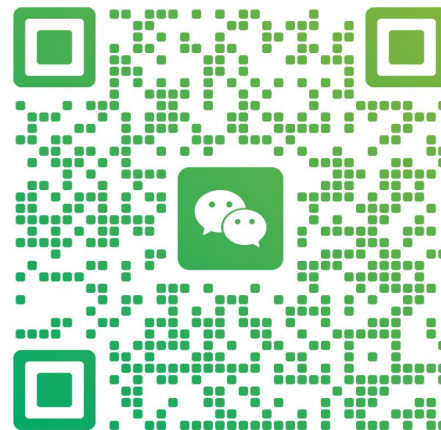
② 算术移位

对有符号数进行移位



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

整数运算：逻辑运算（逻辑移位）

机器字长 = 8

$x =$

0	1	1	0	0	0	1	0
---	---	---	---	---	---	---	---

x 左移一位: $x \ll 1$ 高位移出，低位补 0

整数运算：逻辑运算（逻辑移位）

机器字长 = 8

$x =$

1	1	0	0	0	1	1	0
---	---	---	---	---	---	---	---

x 左移一位： $x \ll 1$ 高位移出，低位补 0

最高位移出的是 1，则发生了溢出

整数运算：逻辑运算（逻辑移位）

机器字长 = 8

x =	0001 1010	26	
x << 1	001 1010 0	52	= 26 × 2
x << 1	01 1010 00	104	= 52 × 2
x << 1	1 1010 000	208	= 104 × 2
x << 1	1010 0000	160	= 208 × 2 

$0 \sim 2^8-1$ ($0 \sim 255$)

最高位移出的是 1，则发生了**溢出**

整数运算：逻辑运算（逻辑移位）

机器字长 = 8

$x =$

0	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---

x 右移一位: $x \gg 1$ 低位移出，高位补 0

整数运算：逻辑运算（逻辑移位）

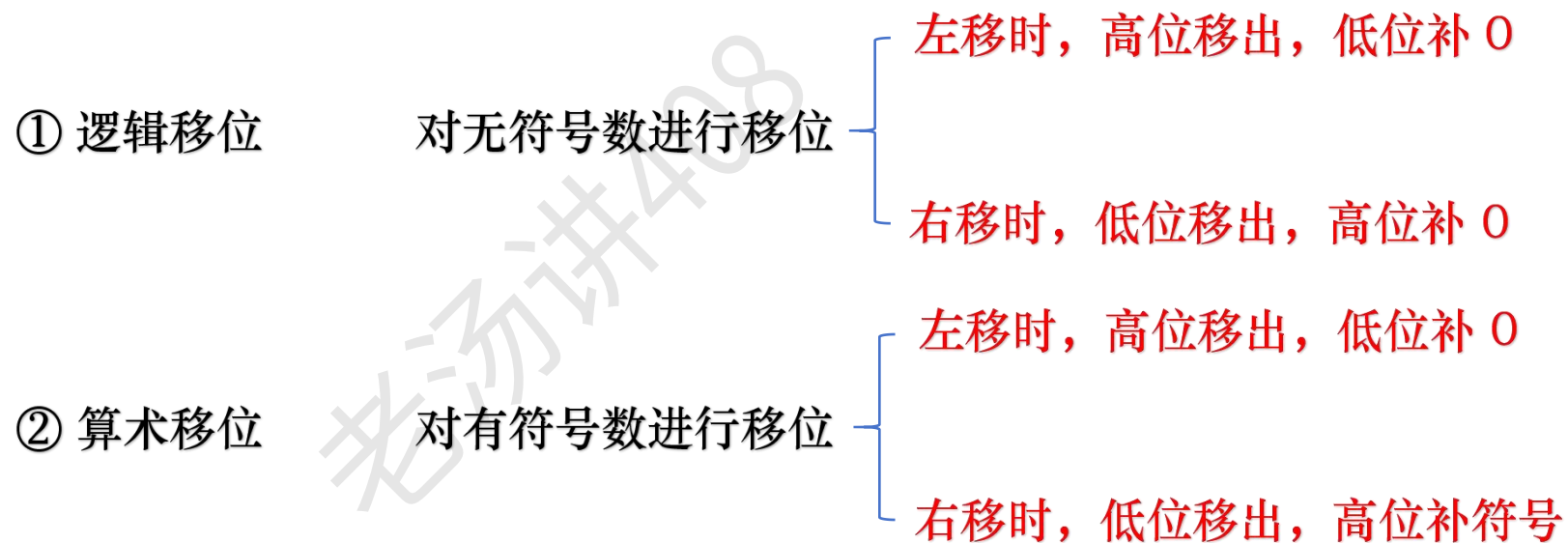
机器字长 = 8

x =	1001 1000	152	
x >> 1	01001 100	76	= 152 / 2
x >> 1	001001 10	38	= 76 / 2
x >> 1	0001001 1	19	= 38 / 2
x << 1	0000 1001	9	= 19 / 2
		9.5	



最低位移出 1，则精度降低

整数运算：逻辑运算（移位）



整数运算：逻辑运算（算术移位）

机器字长 = 8 左移时，高位移出，低位补 0

$$\begin{array}{llll} x = & 0001\ 1010 & 26 \\ x \ll 1 & 001\ 1010\color{red}0 & 52 & = 26 \times 2 \\ x \ll 1 & 01\ 1010\color{red}00 & 104 & = 52 \times 2 \\ x \ll 1 & 1\ 1010\color{red}000 & -48 & = 104 \times 2 \end{array}$$


$$-2^7 \sim 2^7-1 \quad (-128 \sim 127)$$

左移前、后的符号位不同，则发生溢出

整数运算：逻辑运算（算术移位）

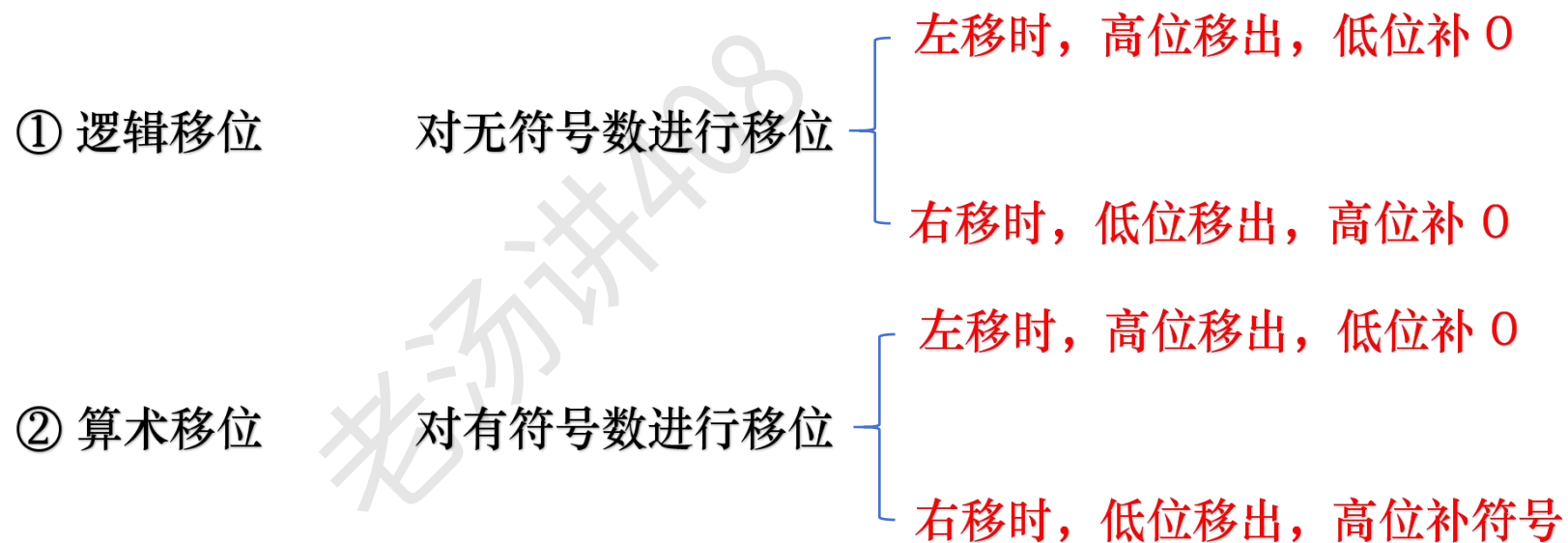
机器字长 = 8 右移时，低位移出，高位补符号

x =	1001 1000	-104	
x >> 1	11001 100	-52	= -104 / 2
x >> 1	111001 10	-26	= -52 / 2
x >> 1	1111001 1	-13	= -26 / 2
x << 1	1111 1001	-6	= -13 / 2
		-6.5	



最低位移出 1，则精度降低

整数运算：逻辑运算（移位）



整数运算：逻辑运算（移位）- 例子

已知 32 位寄存器 R1 中存放的变量 x 的机器数为 80000004H，请回答下列问题。

(1) 当 x 是 unsigned int 类型时, x 的值是多少? $x/2$ 的值是多少? $x/2$ 存放在 R1 中的机器数是什么? $2x$ 的值是多少? $2x$ 存放在 R1 中的机器数是什么?

(2) 当 x 是 signed int 类型时, x 的值是多少? $x/2$ 的值是多少? $x/2$ 存放在 R1 中的机器数是什么?
 $2x$ 的值是多少? $2x$ 存放在 R1 中的机器数是什么?

机器数：80000004H

[illegible]

无符号数: x 的真值等于 $2^{31} + 2^2$

$$x / 2 = 2^{30} + 2$$

$x / 2$ 相当于 $x \gg 1$

其机器数：40000002H

[illegible]

整数运算：逻辑运算（移位） - 例子

已知 32 位寄存器 R1 中存放的变量 x 的机器数为 80000004H，请回答下列问题。

(1) 当 x 是 unsigned int 类型时，x 的值是多少？x/2 的值是多少？x/2 存放在 R1 中的机器数是什么？2x 的值是多少？2x 存放在 R1 中的机器数是什么？

(2) 当 x 是 signed int 类型时，x 的值是多少？x/2 的值是多少？x/2 存放在 R1 中的机器数是什么？2x 的值是多少？2x 存放在 R1 中的机器数是什么？

机器数：80000004H

1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

无符号数：x 的真值等于 $2^{31} + 2^2$

$$2x = 2^{32} + 2^3$$

2x 相当于 $x \ll 1$

其机器数：00000008H

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

整数运算：逻辑运算（移位） - 例子

已知 32 位寄存器 R1 中存放的变量 x 的机器数为 80000004H，请回答下列问题。

(1) 当 x 是 unsigned int 类型时，x 的值是多少？x/2 的值是多少？x/2 存放在 R1 中的机器数是什么？2x 的值是多少？2x 存放在 R1 中的机器数是什么？

(2) 当 x 是 signed int 类型时，x 的值是多少？x/2 的值是多少？x/2 存放在 R1 中的机器数是什么？2x 的值是多少？2x 存放在 R1 中的机器数是什么？

机器数：80000004H

有符号数：x 的真值等于 $-2^{31} + 2^2$



1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

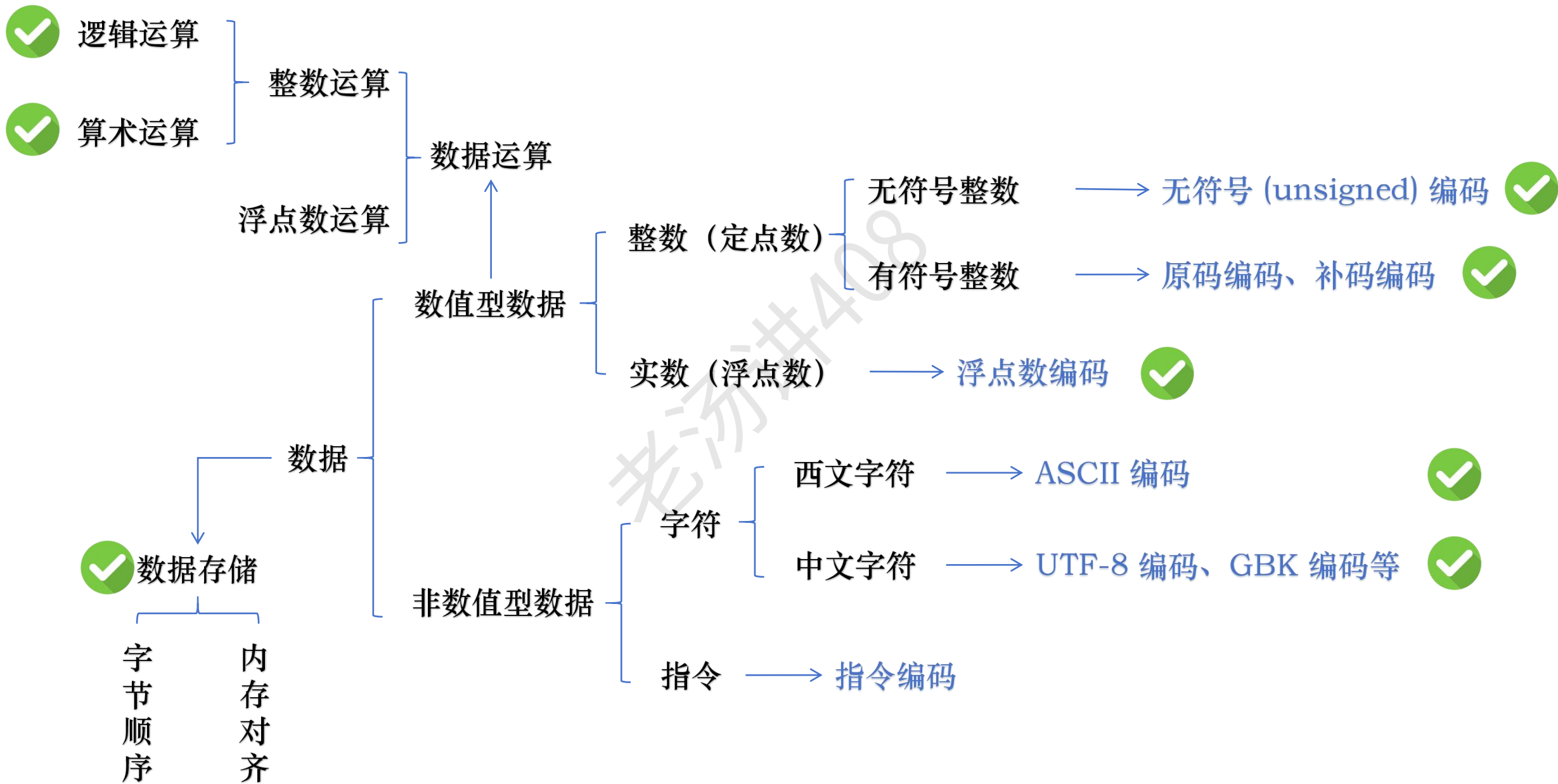
$2x = -2^{32} + 2^3$

2x 相当于 $x \ll 1$

其机器数：00000008H

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

数据运算



整数算术运算：加法和减法

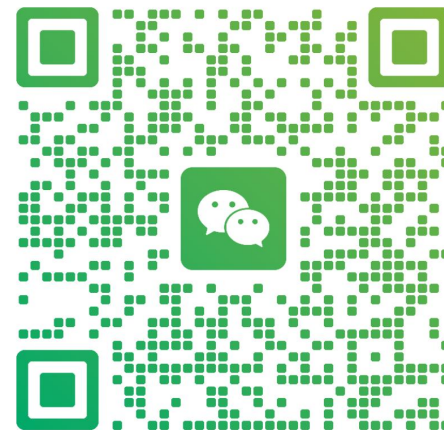
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

加法器如何实现？



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

整数算术运算：加法器

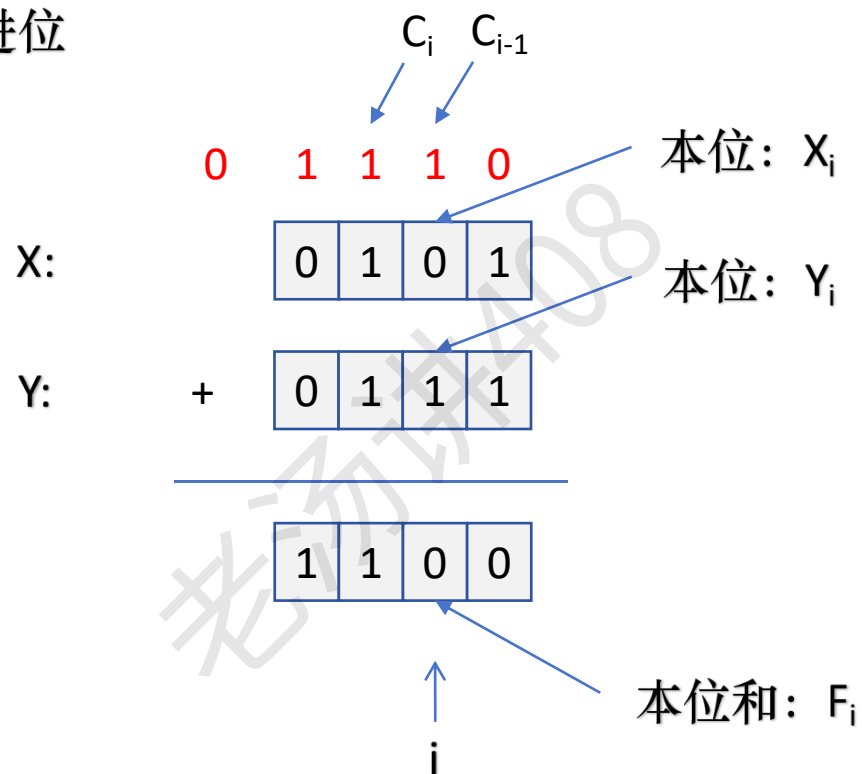
对于每一位相加都需要考虑其**低位的进位**，以及这一位相加后得到的**和**，以及**向高位的进位**

$$\begin{array}{r} \text{X:} \quad \begin{array}{|c|c|c|c|} \hline 0 & 1 & 0 & 1 \\ \hline \end{array} \\ \text{Y:} \quad + \begin{array}{|c|c|c|c|} \hline 0 & 1 & 1 & 1 \\ \hline \end{array} \\ \hline \begin{array}{|c|c|c|c|} \hline 1 & 1 & 0 & 0 \\ \hline \end{array} \end{array}$$

整数算术运算：加法器

对于每一位相加都需要考虑其**低位的进位**，以及这一位相加后得到的**和**，以及**向高位的进位**

C 是 Carry 的首字母，意思是进位



整数算术运算：加法器

输入			输出	
本位 X_i	本位 Y_i	低位进位 C_{i-1}	本位和 F_i	高位进位 C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$F_i = X_i \wedge Y_i \wedge C_{i-1}$$

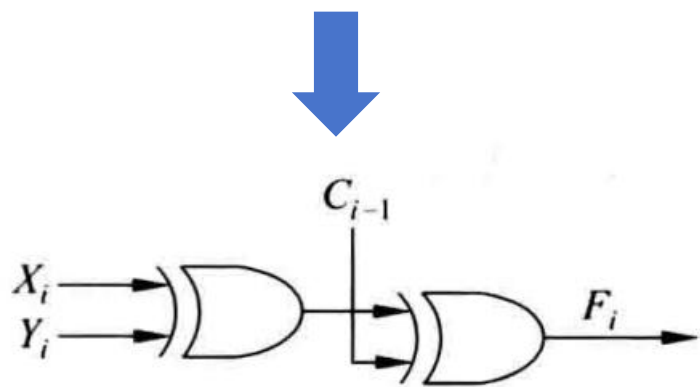
$$F_2 = 0 \wedge 1 \wedge 1 = 0$$

$$C_i = X_i \cdot C_{i-1} + Y_i \cdot C_{i-1} + X_i \cdot Y_i$$

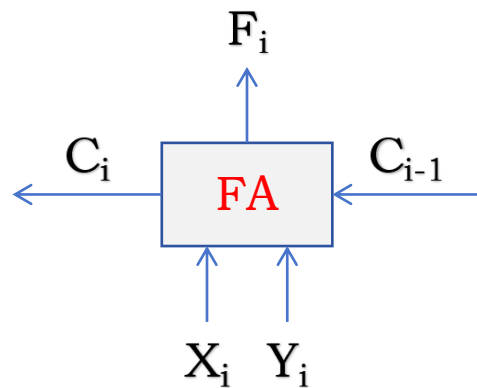
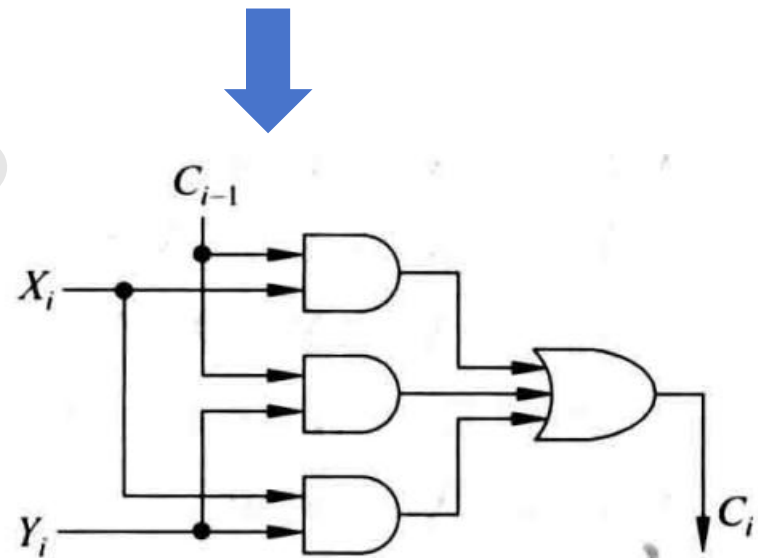
$$C_2 = 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 1$$

整数算术运算：加法器 - 全加器

$$F_i = X_i \oplus Y_i \oplus C_{i-1}$$



$$C_i = X_i \cdot C_{i-1} + Y_i \cdot C_{i-1} + X_i \cdot Y_i$$



全加器 (Full Adder, FA)

整数算术运算：串行进位加法器

X:

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Y:

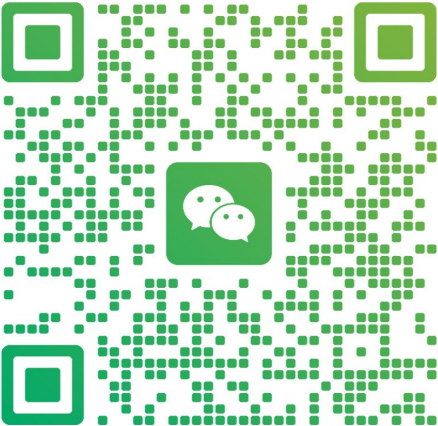
+

0	1	1	1	0	1	1	1
---	---	---	---	---	---	---	---

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

整数算术运算：串行进位加法器

0 1 1 1 0 1 1 1 0

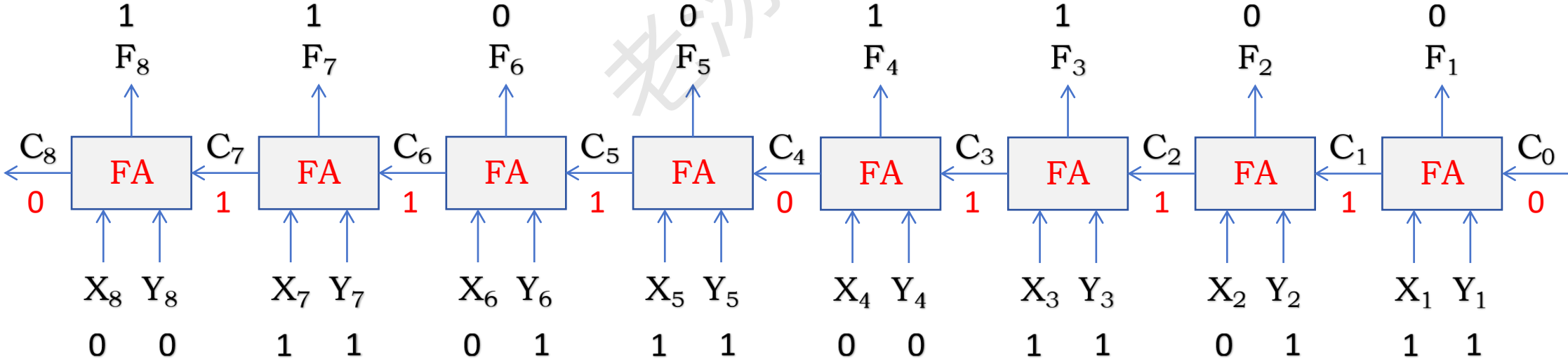
X:

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Y: +

0	1	1	1	0	1	1	1
---	---	---	---	---	---	---	---

1	1	0	0	1	1	0	0
---	---	---	---	---	---	---	---



如何改进串行进位加法器？

$$C_i = X_i \cdot C_{i-1} + Y_i \cdot C_{i-1} + X_i \cdot Y_i$$

逻辑表达式分配律： $A \cdot (B + C) = A \cdot B + A \cdot C$

$$C_1 = X_1 \cdot C_0 + Y_1 \cdot C_0 + X_1 \cdot Y_1 = X_1 \cdot Y_1 + (X_1 + Y_1) \cdot C_0$$

$$\begin{aligned} C_2 &= X_2 \cdot Y_2 + (X_2 + Y_2) \cdot C_1 = X_2 \cdot Y_2 + (X_2 + Y_2) \cdot (X_1 \cdot Y_1 + (X_1 + Y_1) \cdot C_0) \\ &= X_2 \cdot Y_2 + (X_2 + Y_2) \cdot X_1 \cdot Y_1 + (X_2 + Y_2) \cdot (X_1 + Y_1) \cdot C_0 \end{aligned}$$

$$C_3 = X_3 \cdot Y_3 + (X_3 + Y_3) \cdot C_2 = X_3 \cdot Y_3 + (X_3 + Y_3) \cdot [X_2 \cdot Y_2 + (X_2 + Y_2) \cdot X_1 \cdot Y_1 + (X_2 + Y_2) \cdot (X_1 + Y_1) \cdot C_0]$$

$$C_4 = X_4 \cdot Y_4 + (X_4 + Y_4) \cdot C_3$$

$$= X_4 \cdot Y_4 + (X_4 + Y_4) \cdot \{X_3 \cdot Y_3 + (X_3 + Y_3) \cdot [X_2 \cdot Y_2 + (X_2 + Y_2) \cdot X_1 \cdot Y_1 + (X_2 + Y_2) \cdot (X_1 + Y_1) \cdot C_0]\}$$

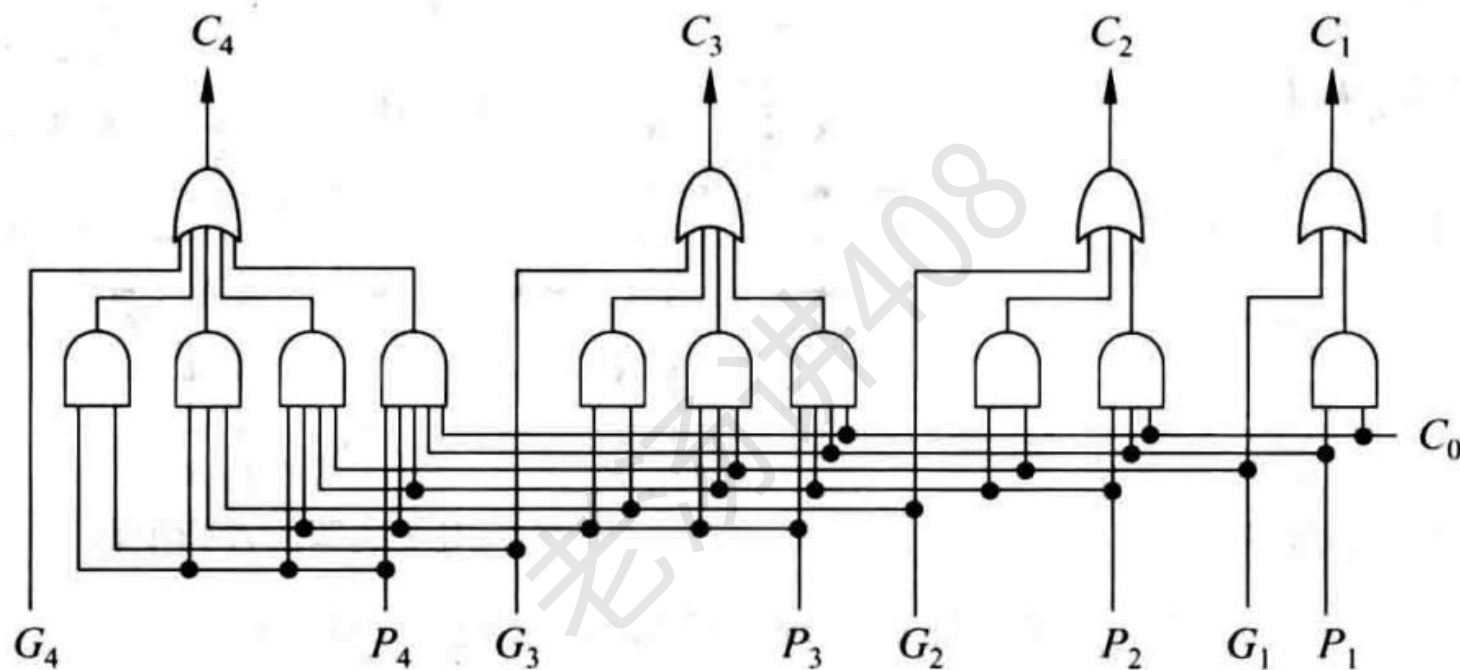
$$P_i = X_i + Y_i$$

$$G_i = X_i \cdot Y_i$$

$$C_0$$

如何改进串行进位加法器？

先行进位部件 (Carry Lookahead Unit, 简称 CLA)



$$P_4 = X_4 + Y_4$$

$$P_3 = X_3 + Y_3$$

$$P_2 = X_2 + Y_2$$

$$P_1 = X_1 + Y_1$$

$$G_4 = X_4 \cdot Y_4$$

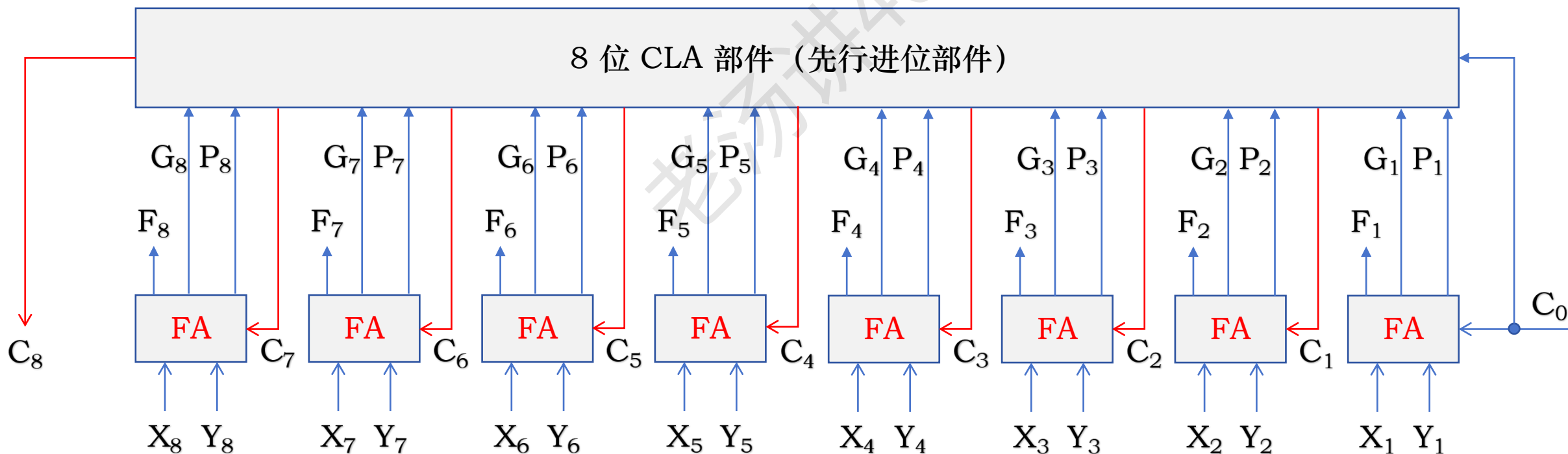
$$G_3 = X_3 \cdot Y_3$$

$$G_2 = X_2 \cdot Y_2$$

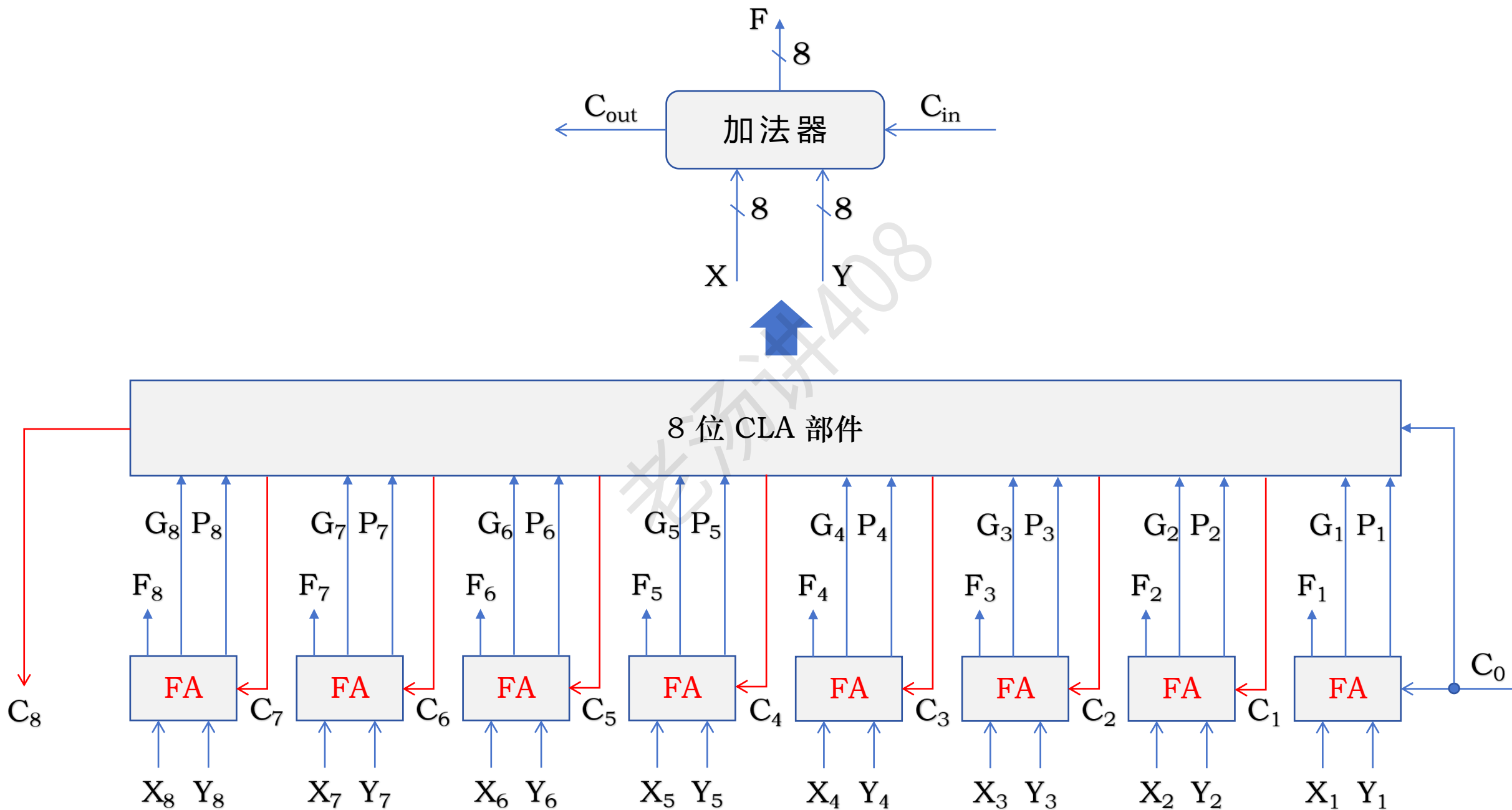
$$G_1 = X_1 \cdot Y_1$$

如何改进串行进位加法器？

并行进位加法器



并行进位加法器

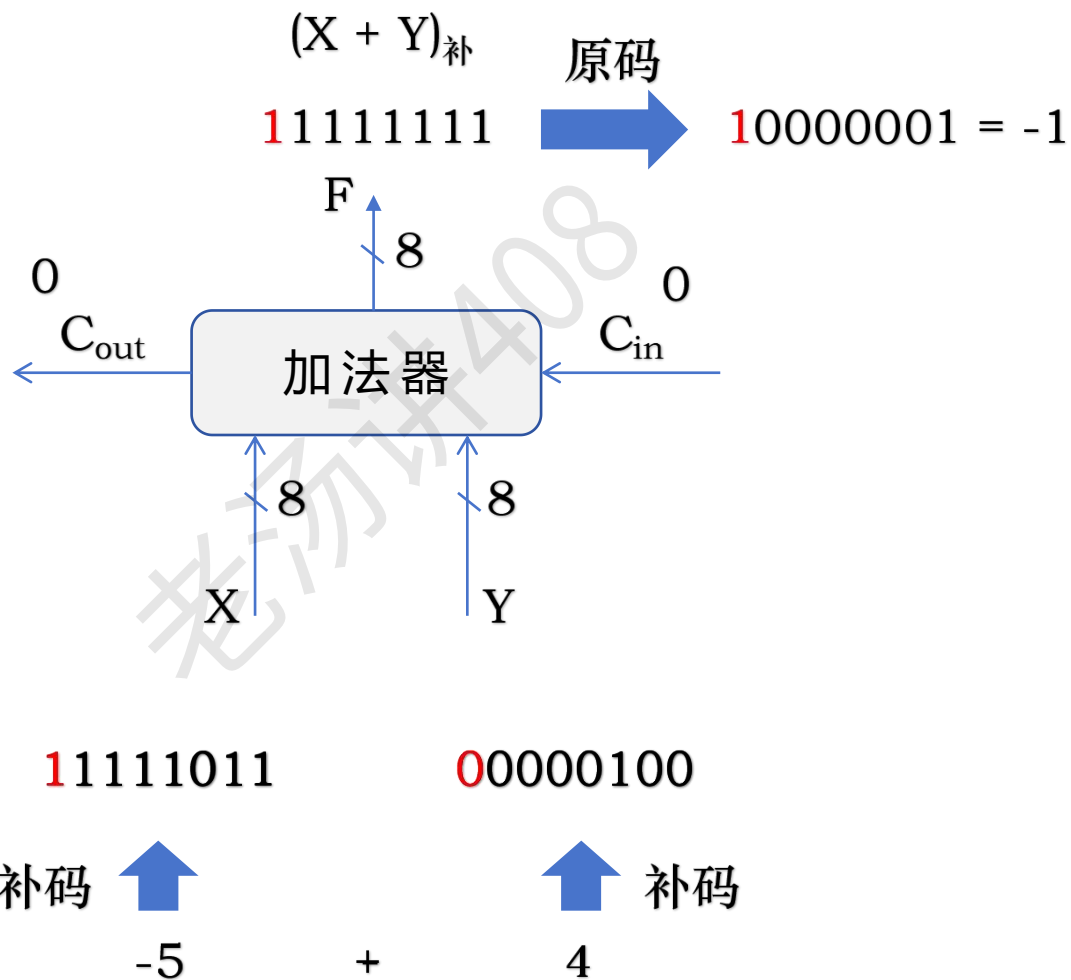


有符号整数加减运算

机器字长 = 8 位

$$(X + Y)_{\text{补}} = X_{\text{补}} + Y_{\text{补}}$$

$$(X - Y)_{\text{补}} = ?$$



有符号整数加减运算

机器字长 = 8 位

$$(X - Y)_{\text{补}} = ?$$

$$X - Y = X + (-Y)$$



$$\begin{aligned}(X - Y)_{\text{补}} &= (X + (-Y))_{\text{补}} \\ &= X_{\text{补}} + (-Y)_{\text{补}}\end{aligned}$$

$$(X + Y)_{\text{补}} = X_{\text{补}} + Y_{\text{补}}$$

已知 $Y_{\text{补}}$ ，如何求： $(-Y)_{\text{补}}$ ？

正数情况： $Y_{\text{补}} = 0y_1y_2y_3y_4y_5y_6y_7$

$$Y_{\text{原}} = 0y_1y_2y_3y_4y_5y_6y_7$$

$$(-Y)_{\text{原}} = 1y_1y_2y_3y_4y_5y_6y_7$$

$$(-Y)_{\text{补}} = 1\overline{y_1y_2y_3y_4y_5y_6y_7} + 1$$

负数情况： $Y_{\text{补}} = 1y_1y_2y_3y_4y_5y_6y_7$

$$Y_{\text{原}} = 1\overline{y_1y_2y_3y_4y_5y_6y_7} + 1$$

$$(-Y)_{\text{原}} = 0\overline{y_1y_2y_3y_4y_5y_6y_7} + 1$$

$$(-Y)_{\text{补}} = 0\overline{y_1y_2y_3y_4y_5y_6y_7} + 1$$

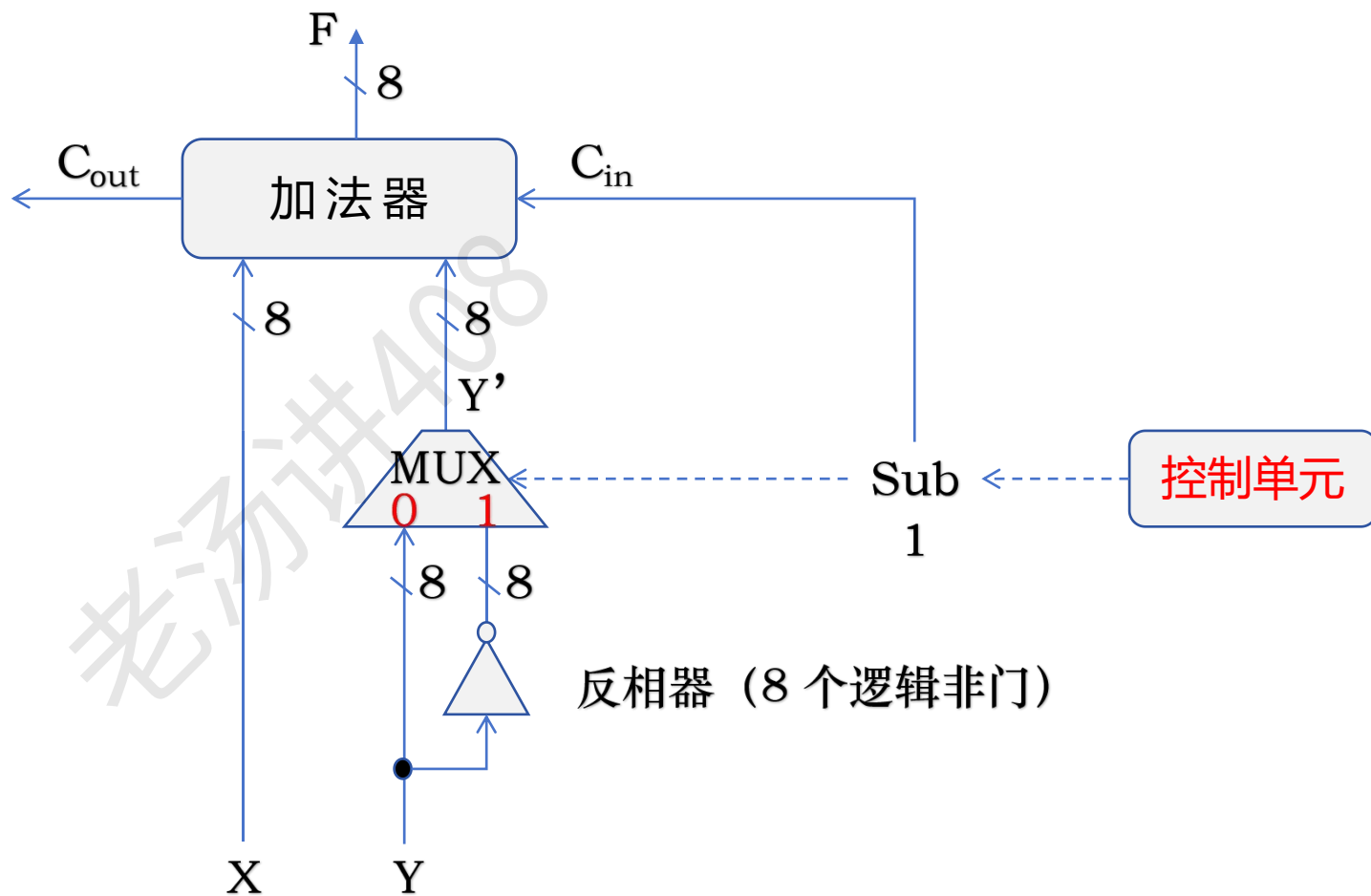
$$(-Y)_{\text{补}} = \overline{Y_{\text{补}}} + 1$$

有符号整数加减运算

机器字长 = 8 位

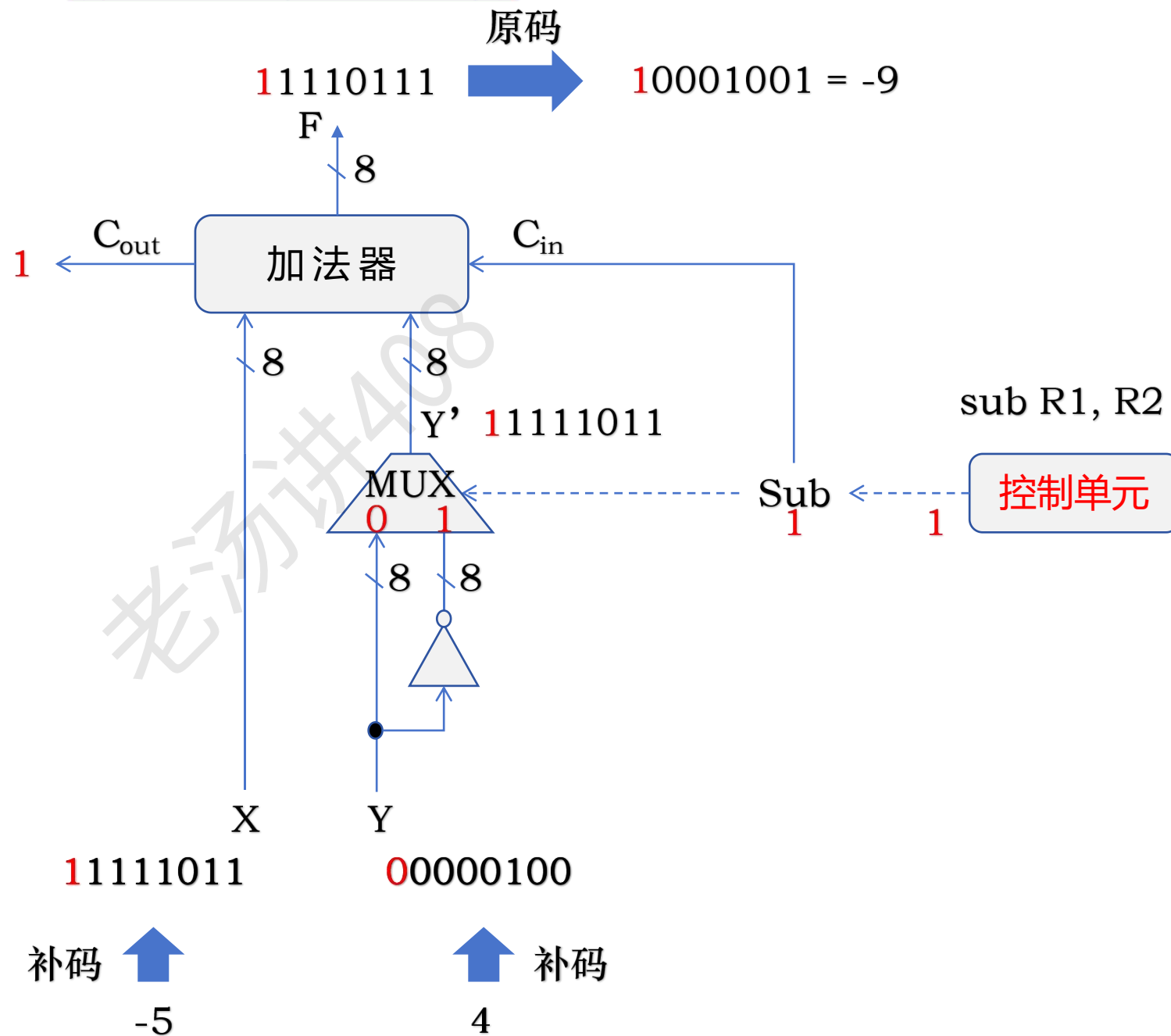
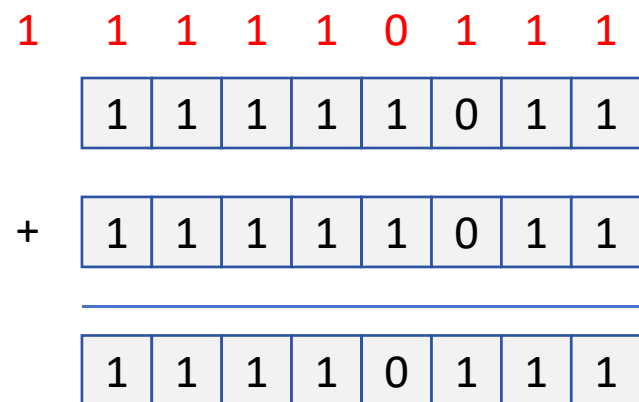
$$(X + Y)_{\text{补}} = X_{\text{补}} + Y_{\text{补}}$$

$$\begin{aligned}(X - Y)_{\text{补}} &= X_{\text{补}} + (-Y)_{\text{补}} \\ &= X_{\text{补}} + \overline{Y_{\text{补}}} + 1\end{aligned}$$



有符号整数加减运算

机器字长 = 8 位



无符号整数加减运算

机器字长 = 8 位

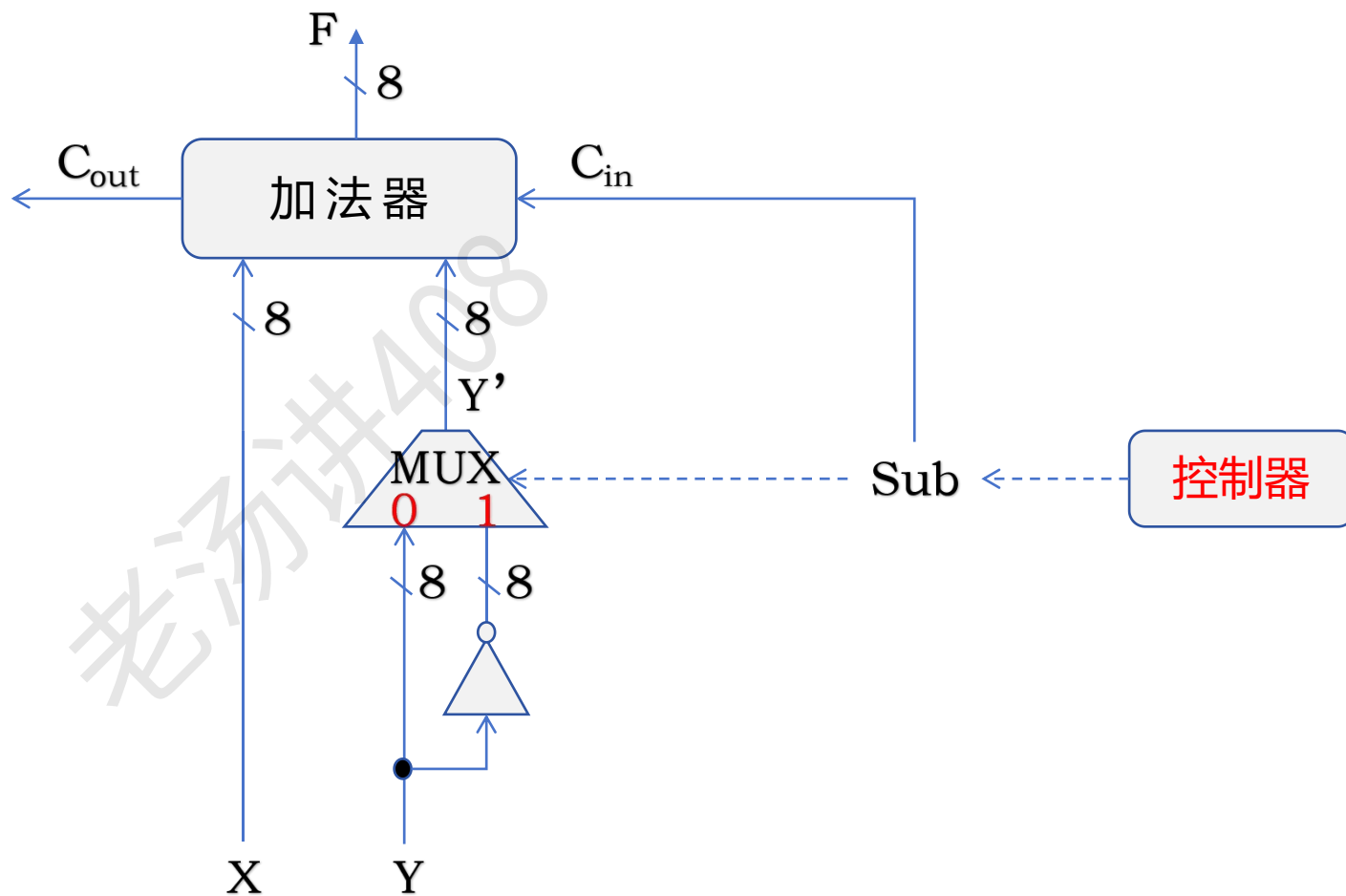
$$(X + Y)_{\text{补}} = X_{\text{补}} + Y_{\text{补}}$$

$$\begin{aligned}(X - Y)_{\text{补}} &= X_{\text{补}} + (-Y)_{\text{补}} \\ &= X_{\text{补}} + \overline{Y_{\text{补}}} + 1\end{aligned}$$

无符号整数相当于正整数

正整数的补码表示就等于其二进制本身

无符号整数的二进制表示相当于正整数的补码表示

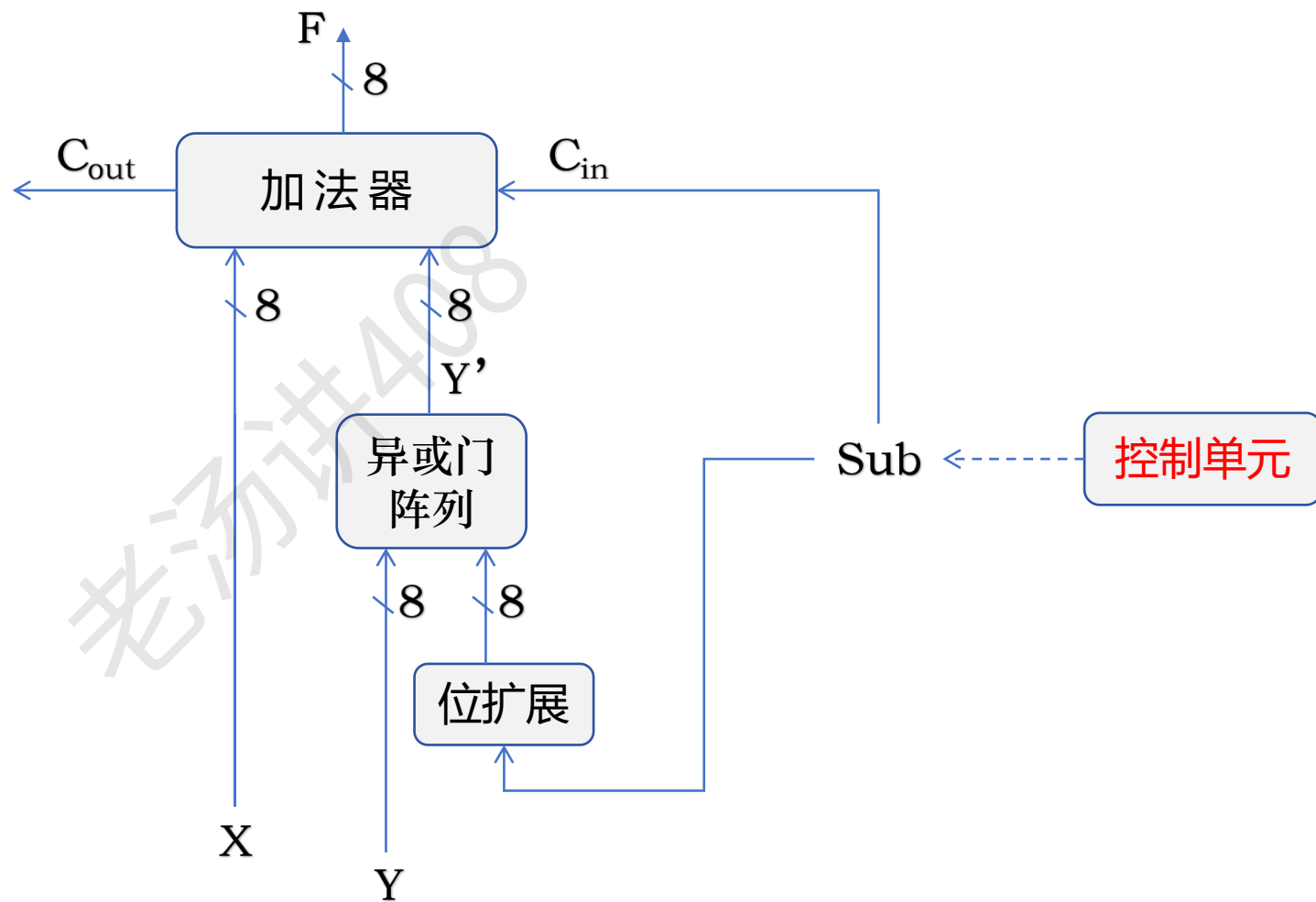


无符号整数加减运算

机器字长 = 8 位

	1	1	0	1	0	1	1	0
^	1	1	1	1	1	1	1	1
	0	0	1	0	1	0	0	1

	1	1	0	1	0	1	1	0
^	0	0	0	0	0	0	0	0
	1	1	0	1	0	1	1	0

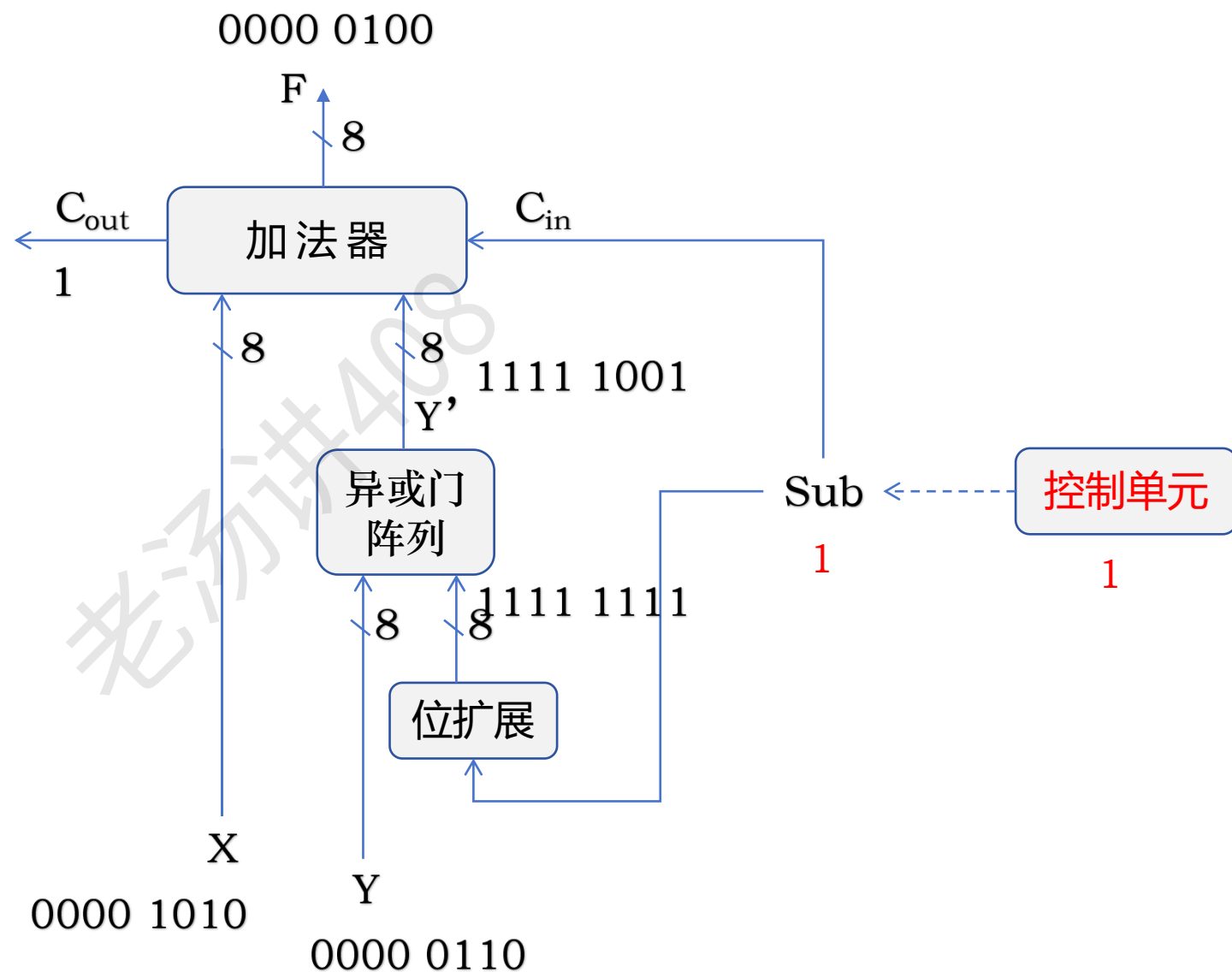


无符号整数加减运算

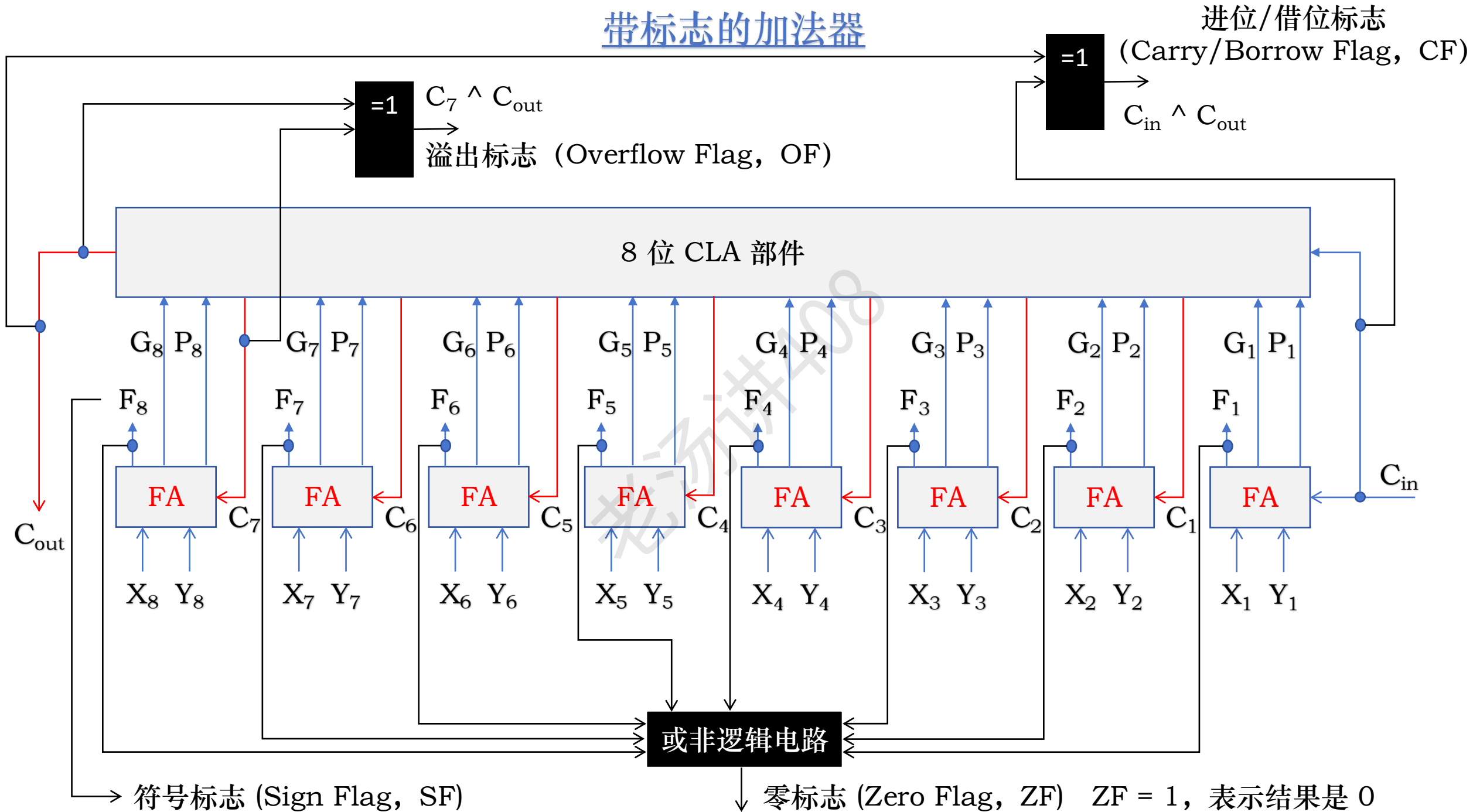
机器字长 = 8 位

无符号数相减：10 - 6

1	1	1	1	1	0	1	1	1
0	0	0	0	1	0	1	0	
+	1	1	1	1	1	0	0	1
0	0	0	0	0	1	0	0	



带标志的加法器



带标志的加法器：进位/借位标志 (CF)

CF 表示无符号数加/减运算时的进/借位

加法时 CF = 1 表示有进位，即发生溢出

		1	1	1	1	0		$C_{out} = 1$
X:		1	1	0	1		13	$C_{in} = 0$
Y:	+	1	0	1	1		11	CF = 1
		1	0	0	0		8	

		0	1	1	1	0		$C_{out} = 0$
		0	1	0	1		5	$C_{in} = 0$
	+	0	0	1	1		3	CF = 0
		1	0	0	0		8	

减法时 CF = 1 表示有借位，即不够减

带标志的加法器：进位/借位标志 (CF)

CF 表示无符号数加/减运算时的进/借位

加法时

CF = 1

表示有进位，即发生溢出

X: $\overset{1}{1} \overset{1}{1} \overset{1}{1} \overset{1}{0}$ $C_{out} = 1$
 $\begin{array}{|c|c|c|c|} \hline 1 & 1 & 0 & 1 \\ \hline \end{array}$ 13 $C_{in} = 0$
Y: $+ \begin{array}{|c|c|c|c|} \hline 1 & 0 & 1 & 1 \\ \hline \end{array}$ 11 $CF = 1$

 $\begin{array}{|c|c|c|c|} \hline 1 & 0 & 0 & 0 \\ \hline \end{array}$ 8

$\overset{0}{0} \overset{1}{1} \overset{1}{1} \overset{1}{0}$ $C_{out} = 0$
 $\begin{array}{|c|c|c|c|} \hline 0 & 1 & 0 & 1 \\ \hline \end{array}$ 5 $C_{in} = 0$
 $+ \begin{array}{|c|c|c|c|} \hline 0 & 0 & 1 & 1 \\ \hline \end{array}$ 3 $CF = 0$

 $\begin{array}{|c|c|c|c|} \hline 1 & 0 & 0 & 0 \\ \hline \end{array}$ 8

减法时

CF = 1

表示有借位，即不够减

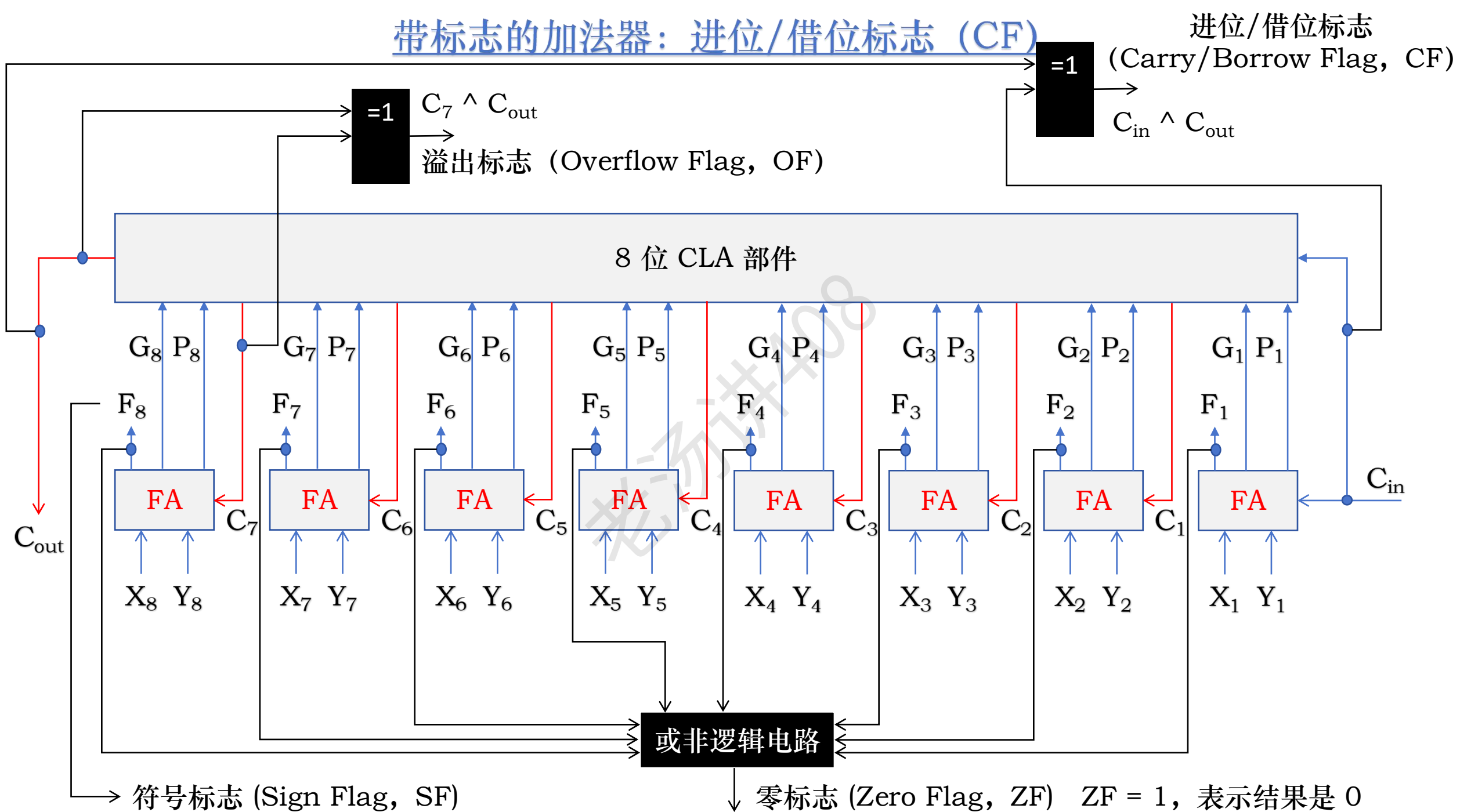
X: $\overset{0}{0} \overset{1}{1} \overset{1}{1} \overset{1}{1}$ $C_{out} = 0$
 $\begin{array}{|c|c|c|c|} \hline 0 & 0 & 1 & 1 \\ \hline \end{array}$ 3 $C_{in} = 1$
Y: $+ \begin{array}{|c|c|c|c|} \hline 0 & 1 & 1 & 1 \\ \hline \end{array}$ -8 $CF = 1$

 $\begin{array}{|c|c|c|c|} \hline 1 & 0 & 1 & 1 \\ \hline \end{array}$ -5

$\overset{1}{1} \overset{0}{0} \overset{0}{0} \overset{0}{1}$ $C_{out} = 1$
 $\begin{array}{|c|c|c|c|} \hline 1 & 0 & 0 & 0 \\ \hline \end{array}$ 8 $C_{in} = 1$
 $+ \begin{array}{|c|c|c|c|} \hline 1 & 1 & 0 & 0 \\ \hline \end{array}$ -3 $CF = 0$

 $\begin{array}{|c|c|c|c|} \hline 0 & 1 & 0 & 1 \\ \hline \end{array}$ 5

带标志的加法器：进位/借位标志 (CF)



带标志的加法器：溢出标志 (OF)

溢出标志 $OF = 1$ 表示有符号整数运算时结果发生了溢出

4 位补码可以表示的整数范围是 $-8 \sim 7$ ，
如果结果超过了这个范围，则表示溢出

用 4 位补码计算 $-7-6$ 和 $-3-5$ 的值

$$(-7)_{\text{补}} = 1001$$

$$(6)_{\text{补}} = 0110$$

$$(-3)_{\text{补}} = 1101$$

$$(5)_{\text{补}} = 0101$$

$$\begin{array}{r} 1 \quad 0 \quad 0 \quad 1 \quad 1 \\ \boxed{1} \boxed{0} \boxed{0} \boxed{1} \quad (-7)_{\text{补}} \\ + \quad \boxed{1} \boxed{0} \boxed{0} \boxed{1} \quad (-6)_{\text{补}} \\ \hline \boxed{0} \boxed{0} \boxed{1} \boxed{1} \quad +3 \end{array}$$

$$\begin{aligned} C_{\text{in}} &= 1 \\ C_{\text{out}} &= 1 \\ C_3 &= 0 \\ OF &= 1 \end{aligned}$$

$$\begin{array}{r} 1 \quad 1 \quad 1 \quad 1 \quad 1 \\ \boxed{1} \boxed{1} \boxed{0} \boxed{1} \quad (-3)_{\text{补}} \\ + \quad \boxed{1} \boxed{0} \boxed{1} \boxed{0} \quad (-5)_{\text{补}} \\ \hline \boxed{1} \boxed{0} \boxed{0} \boxed{0} \quad -8 \end{array}$$

$$\begin{aligned} C_{\text{in}} &= 1 \\ C_{\text{out}} &= 1 \\ C_3 &= 1 \\ OF &= 0 \end{aligned}$$

带标志的加法器：溢出标志 (OF)

溢出标志 $OF = 1$ 表示有符号整数运算时结果发生了溢出

4 位补码可以表示的整数范围是 $-8 \sim 7$ ，
如果结果超过了这个范围，则表示溢出

用 4 位补码计算 $-7-6$ 和 $-3-5$ 的值

$$(-7)_{\text{补}} = 1001$$

$$(6)_{\text{补}} = 0110$$

$$(-3)_{\text{补}} = 1101$$

$$(5)_{\text{补}} = 0101$$

$$\begin{array}{r} 1 \quad 0 \quad 0 \quad 1 \quad 1 \\ \boxed{1} \quad \boxed{0} \quad \boxed{0} \quad \boxed{1} \quad (-7)_{\text{补}} \\ + \quad \boxed{1} \quad \boxed{0} \quad \boxed{0} \quad \boxed{1} \quad (-6)_{\text{补}} \\ \hline \boxed{0} \quad \boxed{0} \quad \boxed{1} \quad \boxed{1} \quad +3 \end{array}$$

$$\begin{array}{l} C_{\text{in}} = 1 \\ C_{\text{out}} = 1 \\ C_3 = 0 \\ OF = 1 \end{array}$$

$$\begin{array}{r} 1 \quad 1 \quad 1 \quad 1 \quad 1 \\ \boxed{1} \quad \boxed{1} \quad \boxed{0} \quad \boxed{1} \quad (-3)_{\text{补}} \\ + \quad \boxed{1} \quad \boxed{0} \quad \boxed{1} \quad \boxed{0} \quad (-5)_{\text{补}} \\ \hline \boxed{1} \quad \boxed{0} \quad \boxed{0} \quad \boxed{0} \quad -8 \end{array}$$

$$\begin{array}{l} C_{\text{in}} = 1 \\ C_{\text{out}} = 1 \\ C_3 = 1 \\ OF = 0 \end{array}$$

用 4 位补码计算 $3+6$ 和 $3+4$ 的值

$$(3)_{\text{补}} = 0011$$

$$(6)_{\text{补}} = 0110$$

$$(3)_{\text{补}} = 0011$$

$$(4)_{\text{补}} = 0100$$

$$\begin{array}{r} 0 \quad 1 \quad 1 \quad 0 \quad 0 \\ \boxed{0} \quad \boxed{0} \quad \boxed{1} \quad \boxed{1} \quad (3)_{\text{补}} \\ + \quad \boxed{0} \quad \boxed{1} \quad \boxed{1} \quad \boxed{0} \quad (6)_{\text{补}} \\ \hline \boxed{1} \quad \boxed{0} \quad \boxed{0} \quad \boxed{1} \quad -7 \end{array}$$

$$\begin{array}{l} C_{\text{in}} = 0 \\ C_{\text{out}} = 0 \\ C_3 = 1 \\ OF = 1 \end{array}$$

$$\begin{array}{r} 0 \quad 0 \quad 0 \quad 0 \quad 0 \\ \boxed{0} \quad \boxed{0} \quad \boxed{1} \quad \boxed{1} \quad (3)_{\text{补}} \\ + \quad \boxed{0} \quad \boxed{1} \quad \boxed{0} \quad \boxed{0} \quad (4)_{\text{补}} \\ \hline \boxed{0} \quad \boxed{1} \quad \boxed{1} \quad \boxed{1} \quad +7 \end{array}$$

$$\begin{array}{l} C_{\text{in}} = 0 \\ C_{\text{out}} = 0 \\ C_3 = 0 \\ OF = 0 \end{array}$$

带标志的加法器：溢出标志 (OF)

溢出标志 $OF = 1$ 表示有符号整数运算时结果发生了溢出

4 位补码可以表示的整数范围是 $-8 \sim 7$ ，
如果结果超过了这个范围，则表示溢出

用 4 位补码计算 $-7-6$ 和 $-3-5$ 的值

$$(-7)_{\text{补}} = 1001$$

$$(6)_{\text{补}} = 0110$$

$$(-3)_{\text{补}} = 1101$$

$$(5)_{\text{补}} = 0101$$

$$\begin{array}{r} 1 \quad 0 \quad 0 \quad 1 \quad 1 \\ \boxed{1} \boxed{0} \boxed{0} \boxed{1} \quad (-7)_{\text{补}} \\ + \quad \boxed{1} \boxed{0} \boxed{0} \boxed{1} \quad (-6)_{\text{补}} \\ \hline \boxed{0} \boxed{0} \boxed{1} \boxed{1} \quad +3 \end{array}$$

$$\begin{array}{l} C_{\text{in}} = 1 \\ C_{\text{out}} = 1 \\ C_3 = 0 \\ OF = 1 \end{array}$$

$$\begin{array}{r} 1 \quad 1 \quad 1 \quad 1 \quad 1 \\ \boxed{1} \boxed{1} \boxed{0} \boxed{1} \quad (-3)_{\text{补}} \\ + \quad \boxed{1} \boxed{0} \boxed{1} \boxed{0} \quad (-5)_{\text{补}} \\ \hline \boxed{1} \boxed{0} \boxed{0} \boxed{0} \quad -8 \end{array}$$

$$\begin{array}{l} C_{\text{in}} = 1 \\ C_{\text{out}} = 1 \\ C_3 = 1 \\ OF = 0 \end{array}$$

用 4 位补码计算 $3+6$ 和 $3+4$ 的值

$$(3)_{\text{补}} = 0011$$

$$(6)_{\text{补}} = 0110$$

$$(3)_{\text{补}} = 0011$$

$$(4)_{\text{补}} = 0100$$

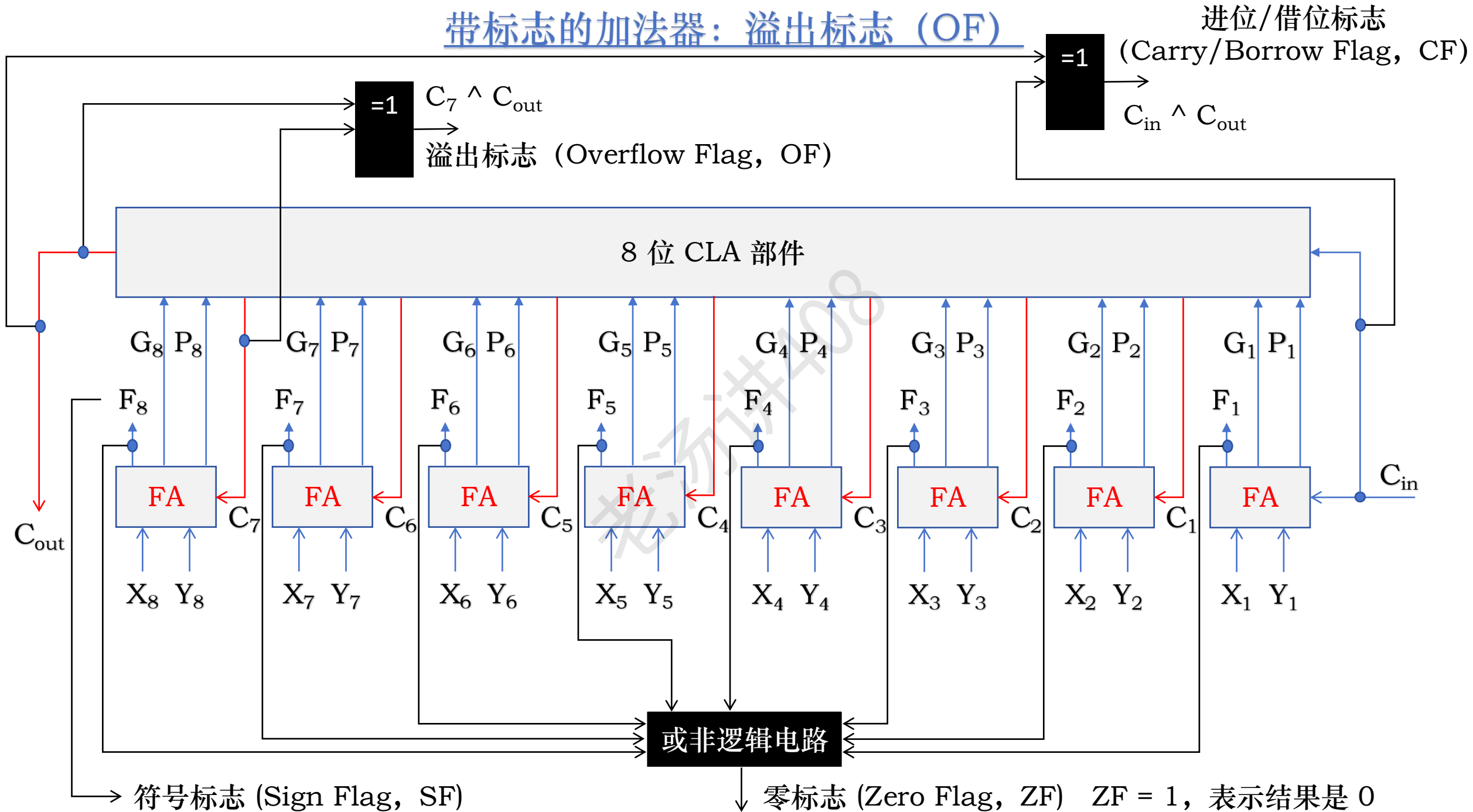
$$\begin{array}{r} 0 \quad 1 \quad 1 \quad 0 \quad 0 \\ \boxed{0} \boxed{0} \boxed{1} \boxed{1} \quad (3)_{\text{补}} \\ + \quad \boxed{0} \boxed{1} \boxed{1} \boxed{0} \quad (6)_{\text{补}} \\ \hline \boxed{1} \boxed{0} \boxed{0} \boxed{1} \quad -7 \end{array}$$

$$\begin{array}{l} C_{\text{in}} = 0 \\ C_{\text{out}} = 0 \\ C_3 = 1 \\ OF = 1 \end{array}$$

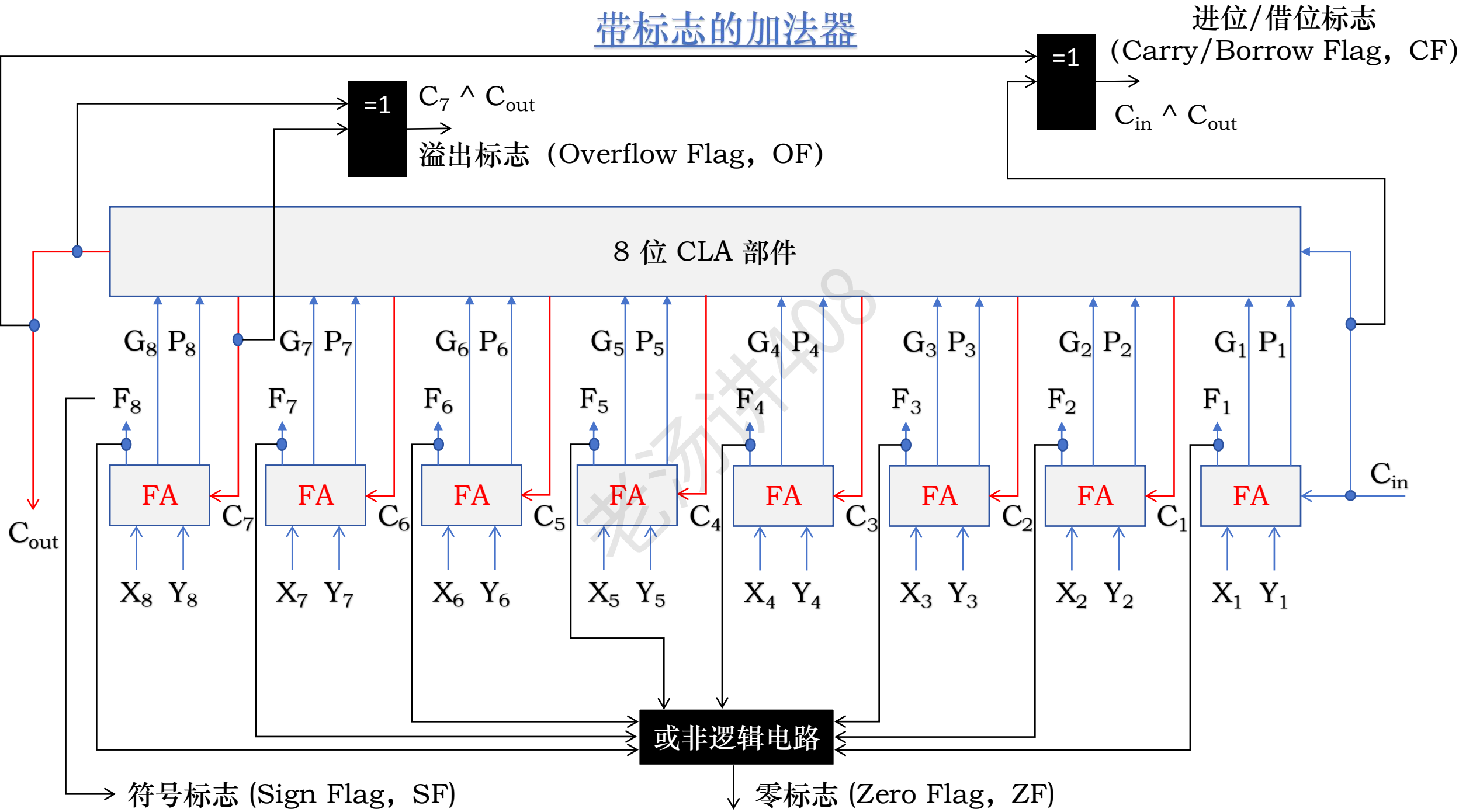
$$\begin{array}{r} 0 \quad 0 \quad 0 \quad 0 \quad 0 \\ \boxed{0} \boxed{0} \boxed{1} \boxed{1} \quad (3)_{\text{补}} \\ + \quad \boxed{0} \boxed{1} \boxed{0} \boxed{0} \quad (4)_{\text{补}} \\ \hline \boxed{0} \boxed{1} \boxed{1} \boxed{1} \quad +7 \end{array}$$

$$\begin{array}{l} C_{\text{in}} = 0 \\ C_{\text{out}} = 0 \\ C_3 = 0 \\ OF = 0 \end{array}$$

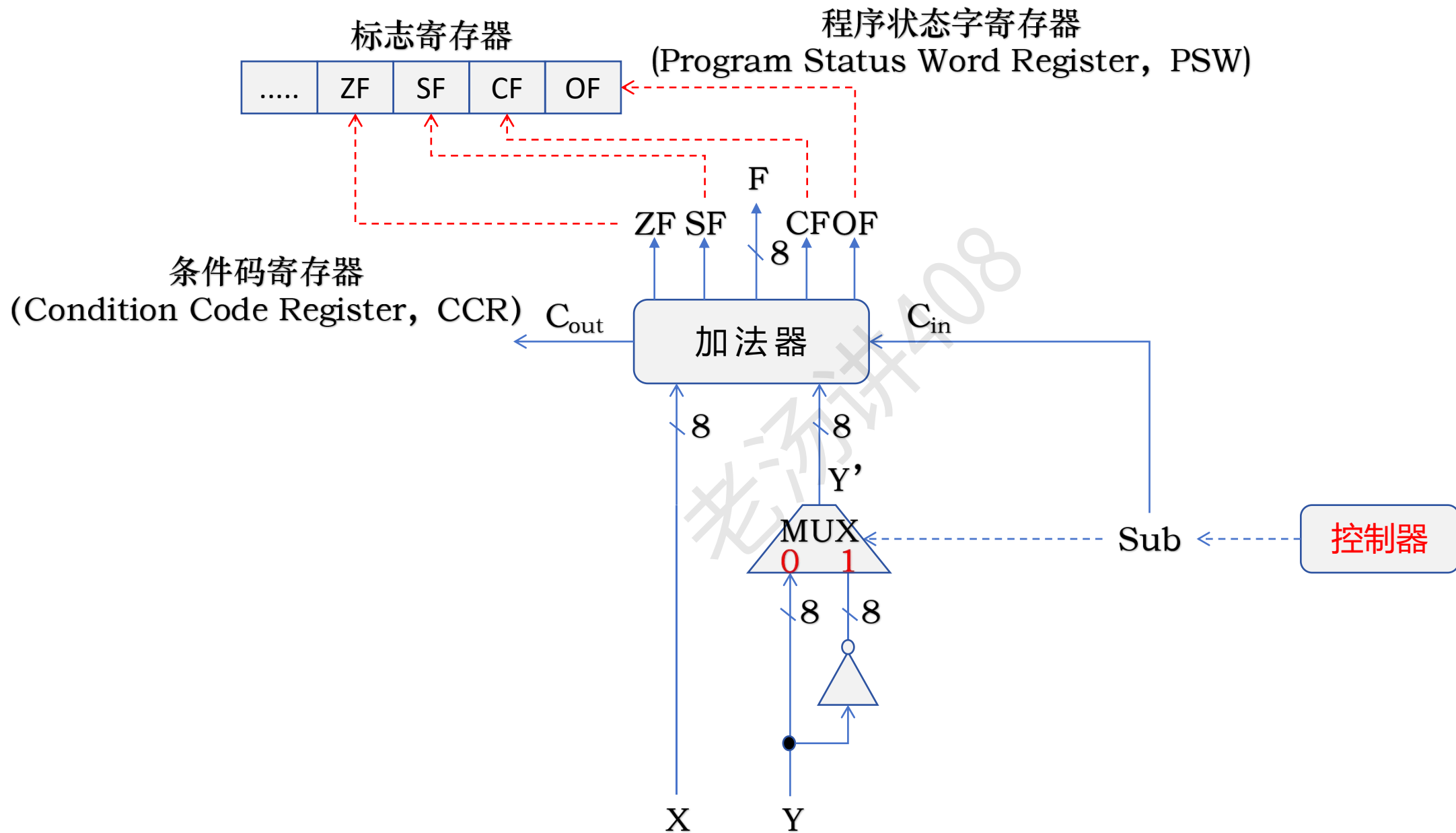
带标志的加法器：溢出标志 (OF)



带标志的加法器



带标志的加法器



有符号整数加减运算判断溢出的方法

溢出标志 (Overflow Flag, OF) = $C_{n-1} \wedge C_{out}$ = $C_{n-1} \wedge C_n$ 最高数值位的进位和符号位的进位的异或

结果和的符号和两个加数的符号是**不同**的，所以溢出

结果和的符号和两个加数的符号是**相同**的，所以没有溢出

用 4 位补码计算 -7-6 和 -3-5 的值

$(-7)_{\text{补}} = 1001$

$(6)_{\text{补}} = 0110$

$(-3)_{\text{补}} = 1101$

$(5)_{\text{补}} = 0101$

$$\begin{array}{r} 1 \quad 0 \quad 0 \quad 1 \quad 1 \\ \boxed{1} \boxed{0} \boxed{0} \boxed{1} \quad (-7)_{\text{补}} \\ + \quad \boxed{1} \boxed{0} \boxed{0} \boxed{1} \quad (-6)_{\text{补}} \\ \hline \boxed{0} \boxed{0} \boxed{1} \boxed{1} \quad +3 \end{array}$$

OF = 1

$$\begin{array}{r} 1 \quad 1 \quad 1 \quad 1 \quad 1 \\ \boxed{1} \boxed{1} \boxed{0} \boxed{1} \quad (-3)_{\text{补}} \\ + \quad \boxed{1} \boxed{0} \boxed{1} \boxed{0} \quad (-5)_{\text{补}} \\ \hline \boxed{1} \boxed{0} \boxed{0} \boxed{0} \quad -8 \end{array}$$

OF = 0

用 4 位补码计算 3+6 和 3+4 的值

$(3)_{\text{补}} = 0011$

$(6)_{\text{补}} = 0110$

$(3)_{\text{补}} = 0011$

$(4)_{\text{补}} = 0100$

$$\begin{array}{r} 0 \quad 1 \quad 1 \quad 0 \quad 0 \\ \boxed{0} \boxed{0} \boxed{1} \boxed{1} \quad (3)_{\text{补}} \\ + \quad \boxed{0} \boxed{1} \boxed{1} \boxed{0} \quad (6)_{\text{补}} \\ \hline \boxed{1} \boxed{0} \boxed{0} \boxed{1} \quad -7 \end{array}$$

OF = 1

$$\begin{array}{r} 0 \quad 0 \quad 0 \quad 0 \quad 0 \\ \boxed{0} \boxed{0} \boxed{1} \boxed{1} \quad (3)_{\text{补}} \\ + \quad \boxed{0} \boxed{1} \boxed{0} \boxed{0} \quad (4)_{\text{补}} \\ \hline \boxed{0} \boxed{1} \boxed{1} \boxed{1} \quad +7 \end{array}$$

OF = 0

有符号整数加减运算判断溢出的方法

溢出标志 (Overflow Flag, OF) = $C_{n-1} \wedge C_{out}$ = $C_{n-1} \wedge C_n$

和的符号和两个加数的符号是**不同**的，所以溢出

和的符号和两个加数的符号是**相同**的，所以没有溢出

负加数 + 负加数
正加数 + 正加数

两个加数的符号不同的话，那肯定是不溢出的

- 负加数 + 正加数
- 正加数 + 负加数

溢出标志 (Overflow Flag, OF) = $X_n Y_n \overline{F_n} + \overline{X_n} \overline{Y_n} F_n$

乘积和

输入			输出
第一加数符号	第二加数符号	结果和符号	OF
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

有符号整数加减运算判断溢出的方法

$$\text{溢出标志 (Overflow Flag, OF)} = C_{n-1} \wedge C_{\text{out}} = C_{n-1} \wedge C_n$$

$$\text{溢出标志 (Overflow Flag, OF)} = X_n Y_n \overline{F_n} + \overline{X_n} \overline{Y_n} F_n$$

一位符号位判断溢出



双符号位判断溢出

溢出标志 (OF) - 双符号位判断溢出

4 位补码

-7 的补码:

1	0	0	1
---	---	---	---



双符号补码

1	1	0	0	1
---	---	---	---	---

变形补码

模 4- 补码

0	0	0	0	1
---	---	---	---	---

真值: 1

溢出标志 (OF) - 双符号位判断溢出

5 位变形补码表示范围: -8 ~ 7

用 5 位变形补码计算 -7-6 和 -3-5 的值 $(-7)_{\text{补}} = 11001$ $(6)_{\text{补}} = 00110$ $(-3)_{\text{补}} = 11101$ $(5)_{\text{补}} = 00101$

$$\begin{array}{r} 1 \ 1 \ 0 \ 0 \ 1 \ 1 \\ \boxed{1} \ \boxed{1} \ \boxed{0} \ \boxed{0} \ \boxed{1} \quad (-7)_{\text{补}} \\ + \boxed{1} \ \boxed{1} \ \boxed{0} \ \boxed{0} \ \boxed{1} \quad (-6)_{\text{补}} \\ \hline \boxed{1} \ \boxed{0} \ \boxed{0} \ \boxed{1} \ \boxed{1} \quad \text{负溢出} \quad \text{OF} = 1 \end{array}$$

$$\begin{array}{r} 1 \ 1 \ 1 \ 1 \ 1 \ 1 \\ \boxed{1} \ \boxed{1} \ \boxed{1} \ \boxed{0} \ \boxed{1} \quad (-3)_{\text{补}} \\ + \boxed{1} \ \boxed{1} \ \boxed{0} \ \boxed{1} \ \boxed{0} \quad (-5)_{\text{补}} \\ \hline \boxed{1} \ \boxed{1} \ \boxed{0} \ \boxed{0} \ \boxed{0} \quad -8 \quad \text{OF} = 0 \end{array}$$

用 5 位变形补码计算 3+6 和 3+4 的值 $(3)_{\text{补}} = 00011$ $(6)_{\text{补}} = 00110$ $(3)_{\text{补}} = 00011$ $(4)_{\text{补}} = 00100$

$$\begin{array}{r} 0 \ 0 \ 1 \ 1 \ 0 \ 0 \\ \boxed{0} \ \boxed{0} \ \boxed{0} \ \boxed{1} \ \boxed{1} \quad (3)_{\text{补}} \\ + \boxed{0} \ \boxed{0} \ \boxed{1} \ \boxed{1} \ \boxed{0} \quad (6)_{\text{补}} \\ \hline \boxed{0} \ \boxed{1} \ \boxed{0} \ \boxed{0} \ \boxed{1} \quad \text{正溢出} \quad \text{OF} = 1 \end{array}$$

$$\begin{array}{r} 0 \ 0 \ 0 \ 0 \ 0 \ 0 \\ \boxed{0} \ \boxed{0} \ \boxed{0} \ \boxed{1} \ \boxed{1} \quad (3)_{\text{补}} \\ + \boxed{0} \ \boxed{0} \ \boxed{1} \ \boxed{0} \ \boxed{0} \quad (4)_{\text{补}} \\ \hline \boxed{0} \ \boxed{0} \ \boxed{1} \ \boxed{1} \ \boxed{1} \quad +7 \quad \text{OF} = 0 \end{array}$$

带标志的加法器：溢出标志 (OF) - 双符号位判断溢出

对于两个符号 S_1S_2 的变形补码：

$S_1S_2 = 00$ ：表示结果为正数，无溢出

$S_1S_2 = 01$ ：表示溢出，又因第 1 位符号位为 0，表示结果的真正符号为正，故 01 表示正溢出

$S_1S_2 = 10$ ：表示溢出，又因第 1 位符号位为 1，表示结果的真正符号为负，故 10 表示负溢出

$S_1S_2 = 11$ ：表示结果为负数，无溢出

溢出标志 (Overflow Flag, OF) $= S_1 \wedge S_2$

判断两个整数的大小

零标志 (Zero Flag, ZF)

符号标志 (Sign Flag, SF)

进位/借位标志 (Carry/Borrow Flag, CF)

溢出标志 (Overflow Flag, OF)

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

判断两个整数的大小：无符号整数大小的比较

零标志 (Zero Flag, ZF)

进位/借位标志 (Carry/Borrow Flag, CF)

如果 $A - B = 0$ ，即 $ZF = 1$ ，那么 $A == B$

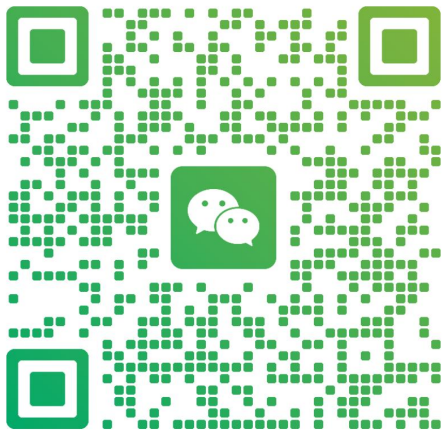
如果 $A - B \neq 0$ ，即 $ZF = 0$

$CF = 0$ ，无借位，说明够减，那么 $A - B > 0$ ，即 $A > B$

$CF = 1$ ，有借位，说明不够减，那么 $A - B < 0$ ，即 $A < B$

判断两个整数的大小

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



扫一扫上面的二维码图案，加我为朋友。

零标志 (Zero Flag, ZF)

符号标志 (Sign Flag, SF)

进位/借位标志 (Carry/Borrow Flag, CF)

溢出标志 (Overflow Flag, OF)

判断两个整数的大小：有符号数

零标志 (Zero Flag, ZF) 符号标志 (Sign Flag, SF)

如果 $A - B = 0$ ，即 $ZF = 1$ ，那么 $A == B$

如果 $A - B \neq 0$ ，即 $ZF = 0$

OF = SF 的话，那么 $A > B$ $OF \wedge SF = 0$ 的话，

当 $ZF = 0$ ，未发生溢出时，即 $OF = 0$ ，若 $SF = 0$

当 $ZF = 0$ ，发生溢出时，即 $OF = 1$ ，若 $SF = 1$ ，则必然是正数减去负数发生溢出导致结果为负， $A > B$

$OF \neq SF$ 的话，那么 $A < B$ $OF \wedge SF = 1$ 的话，那么 $A < B$

当 $ZF = 0$ ，未发生溢出时，即 $OF = 0$ ，若 $SF = 1$ ，则表示结果为负，说明 $A < B$

当 $ZF = 0$ ，发生溢出时，即 $OF = 1$ ，若 $SF = 0$ ，则必然是负数减去正数发生溢出导致结果为正， $A < B$

输入			输出
第一加数符号	第二加数符号	结果和符号	OF
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

判断两个整数的大小

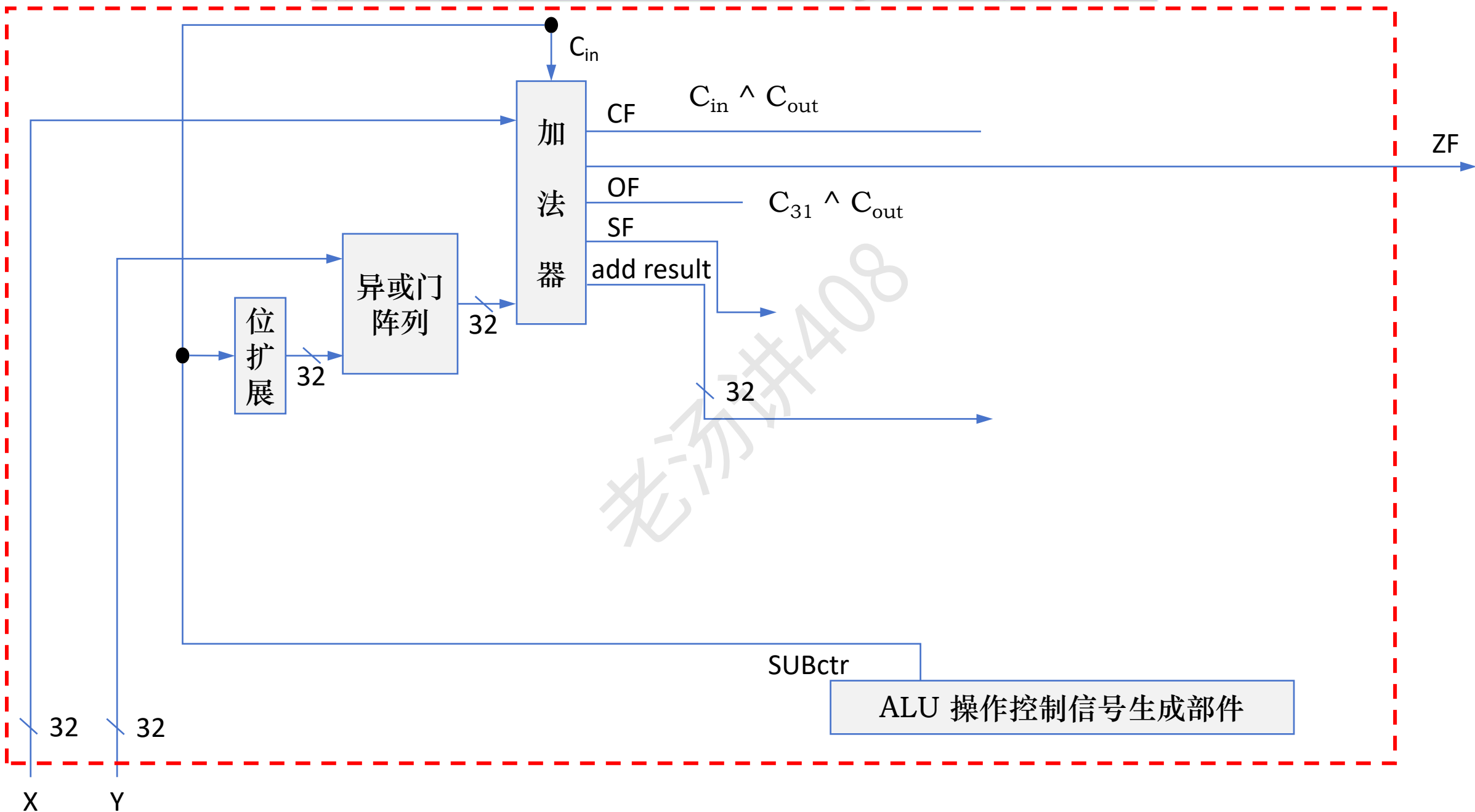
零标志 (Zero Flag, ZF)

符号标志 (Sign Flag, SF)

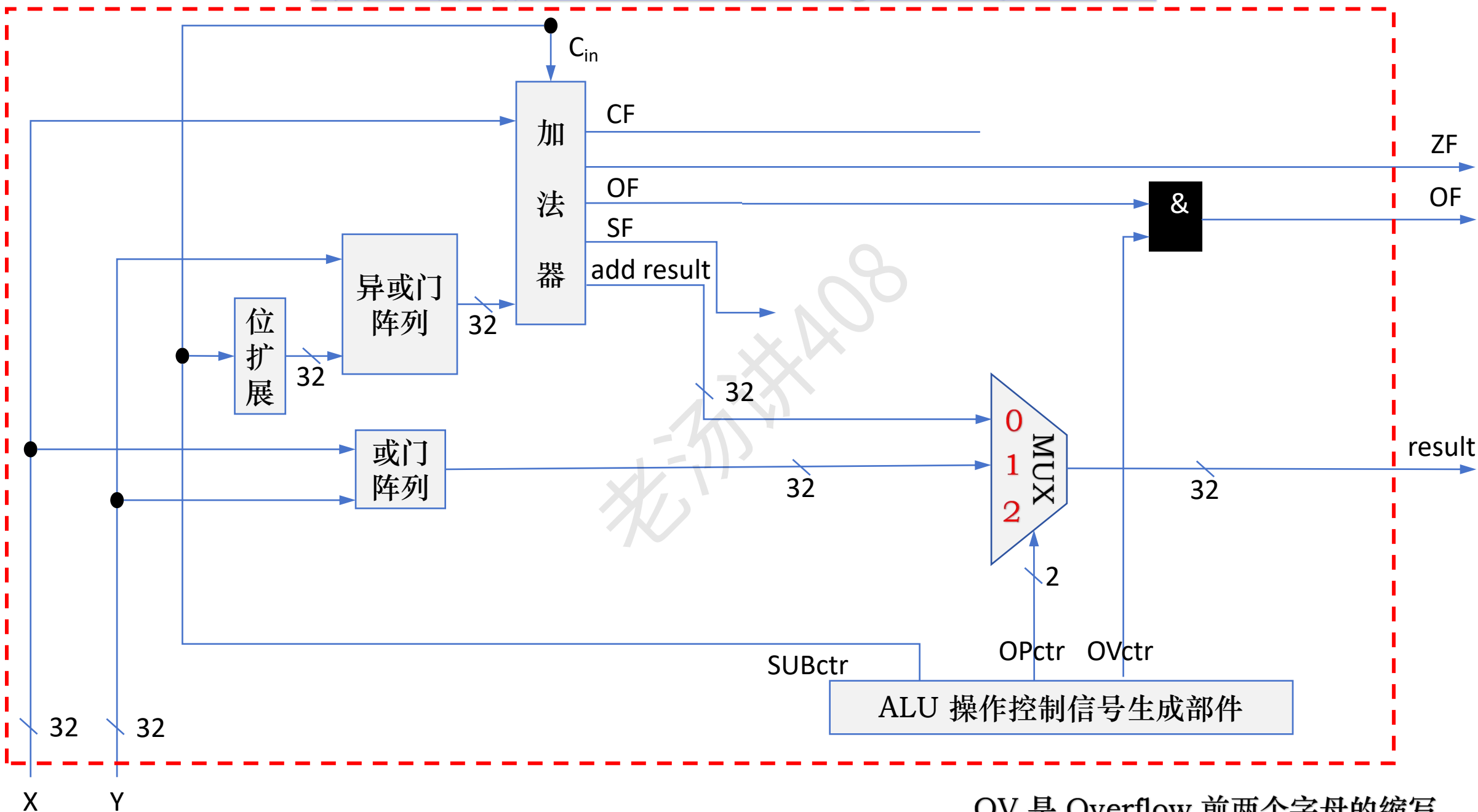
进位/借位标志 (Carry/Borrow Flag, CF)

溢出标志 (Overflow Flag, OF)

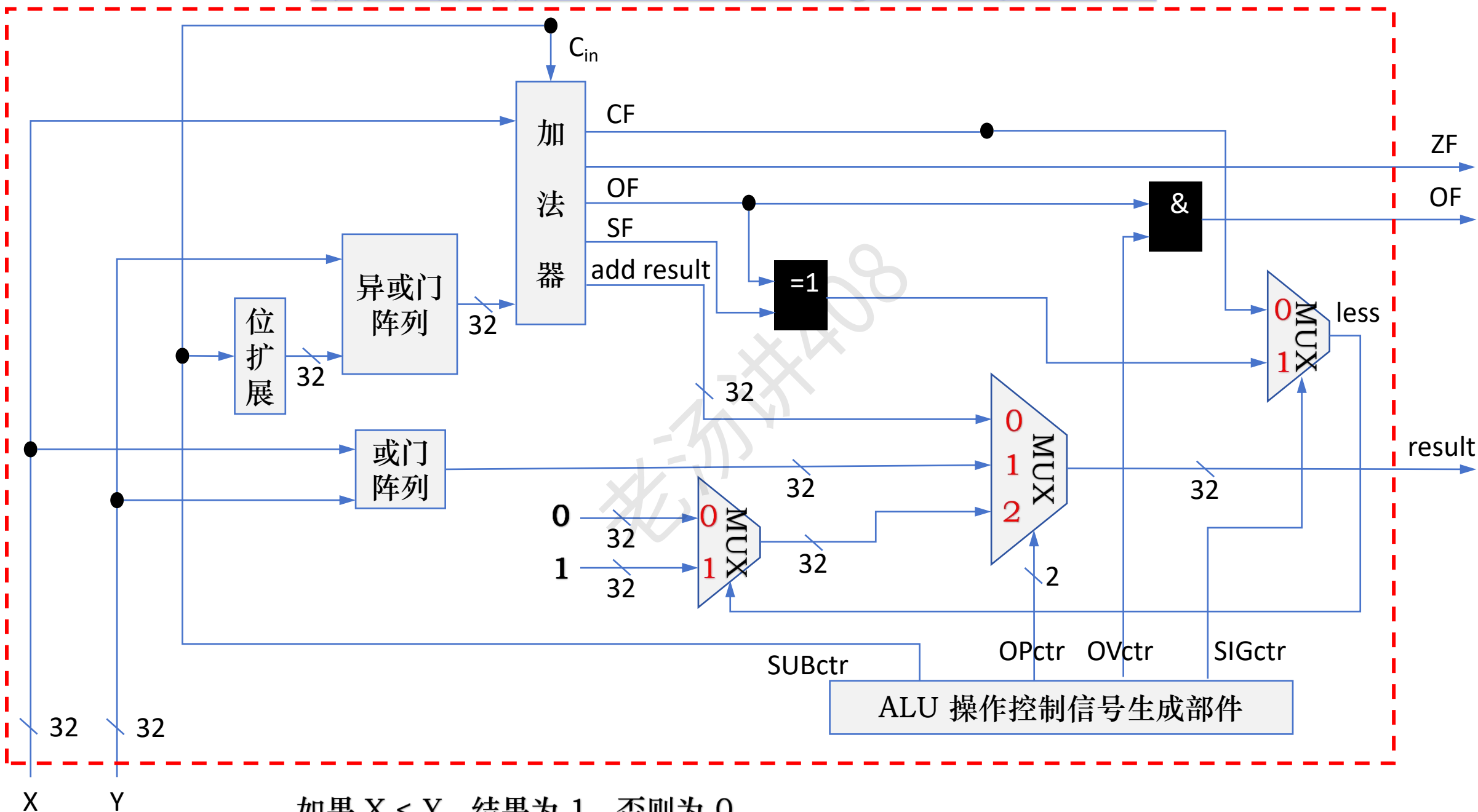
算术逻辑部件 (Arithmetic Logic Unit, ALU)



算术逻辑部件 (Arithmetic Logic Unit, ALU)

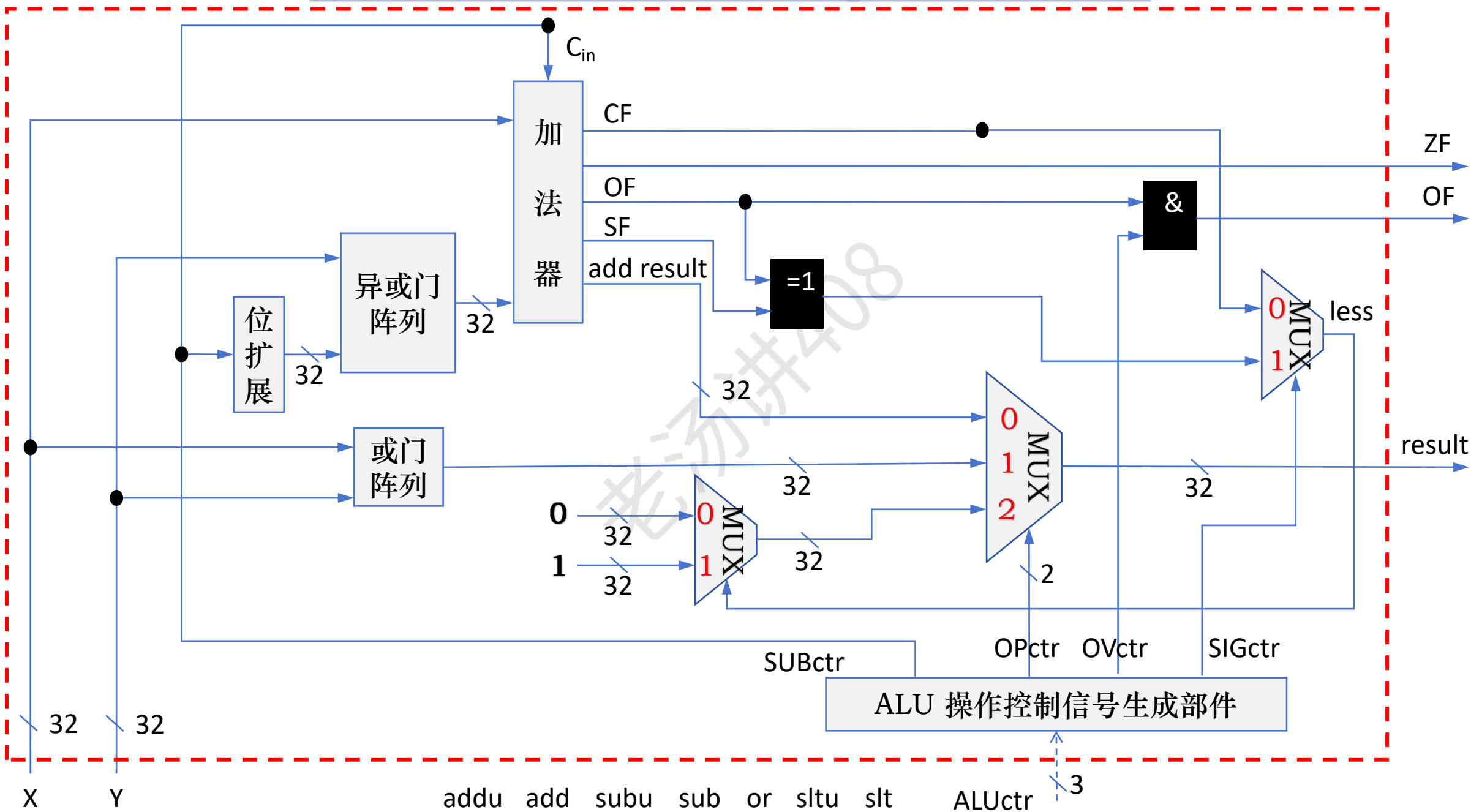


算术逻辑部件 (Arithmetic Logic Unit, ALU)

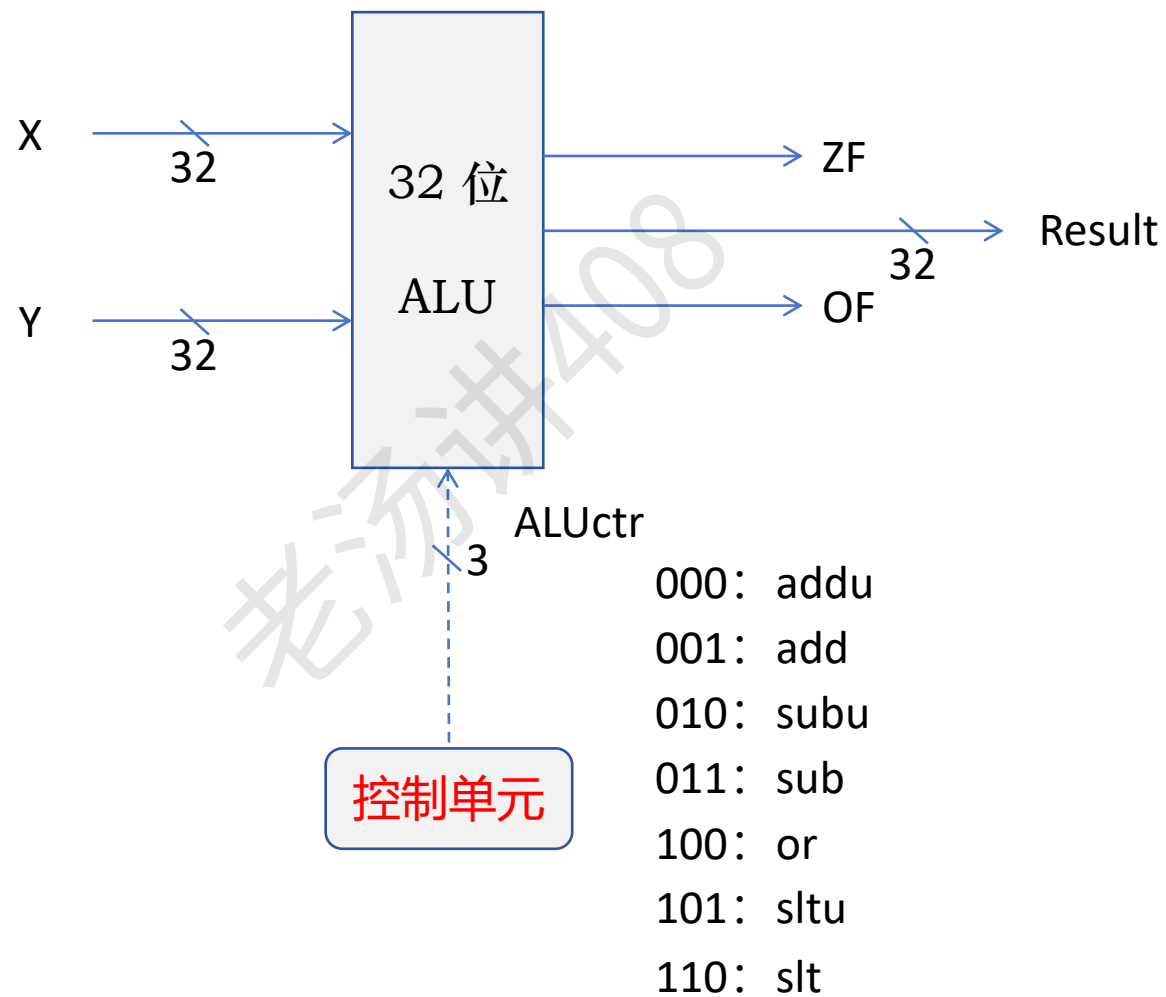


如果 $X < Y$, 结果为 1, 否则为 0

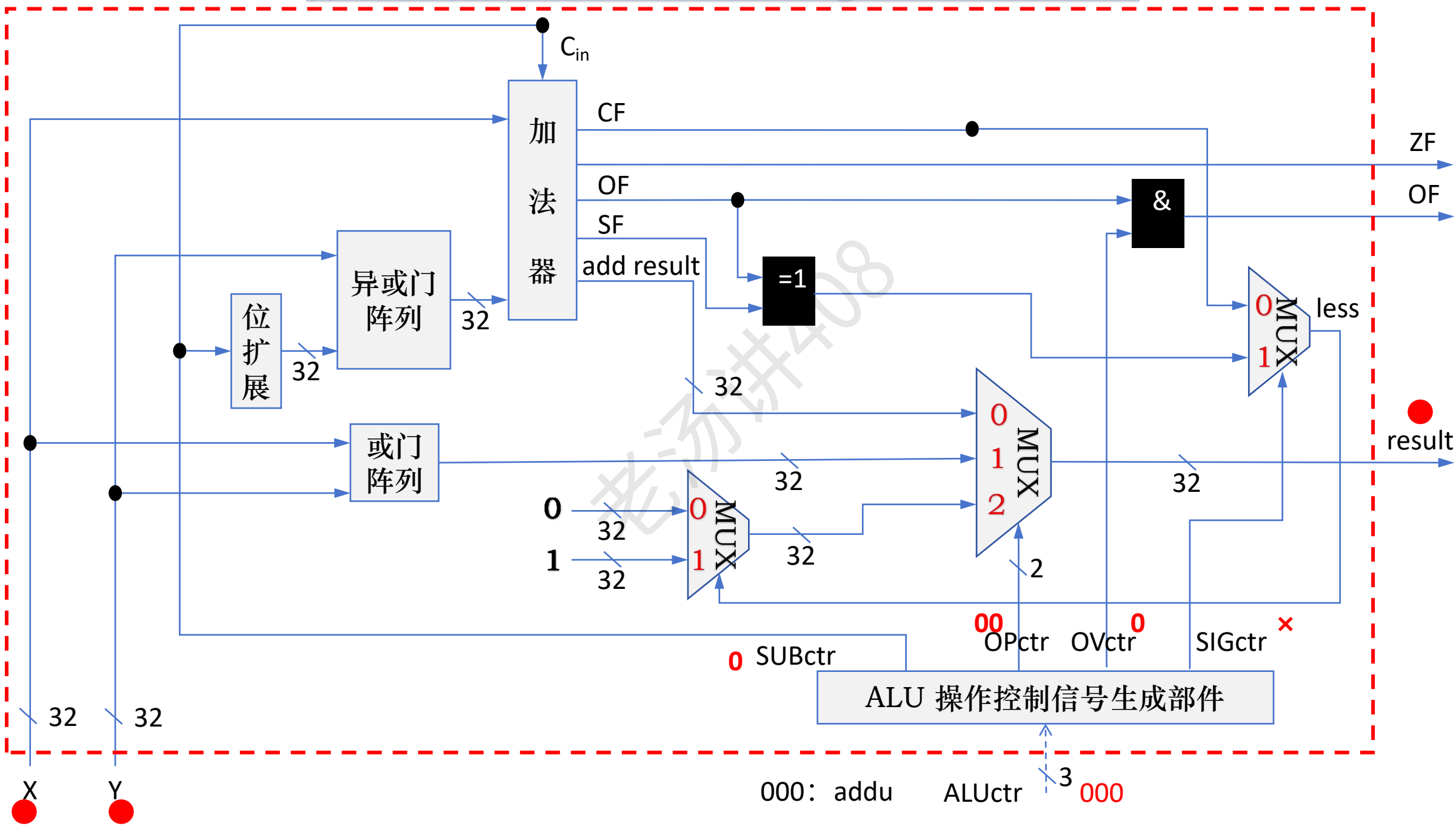
算术逻辑部件 (Arithmetic Logic Unit, ALU)



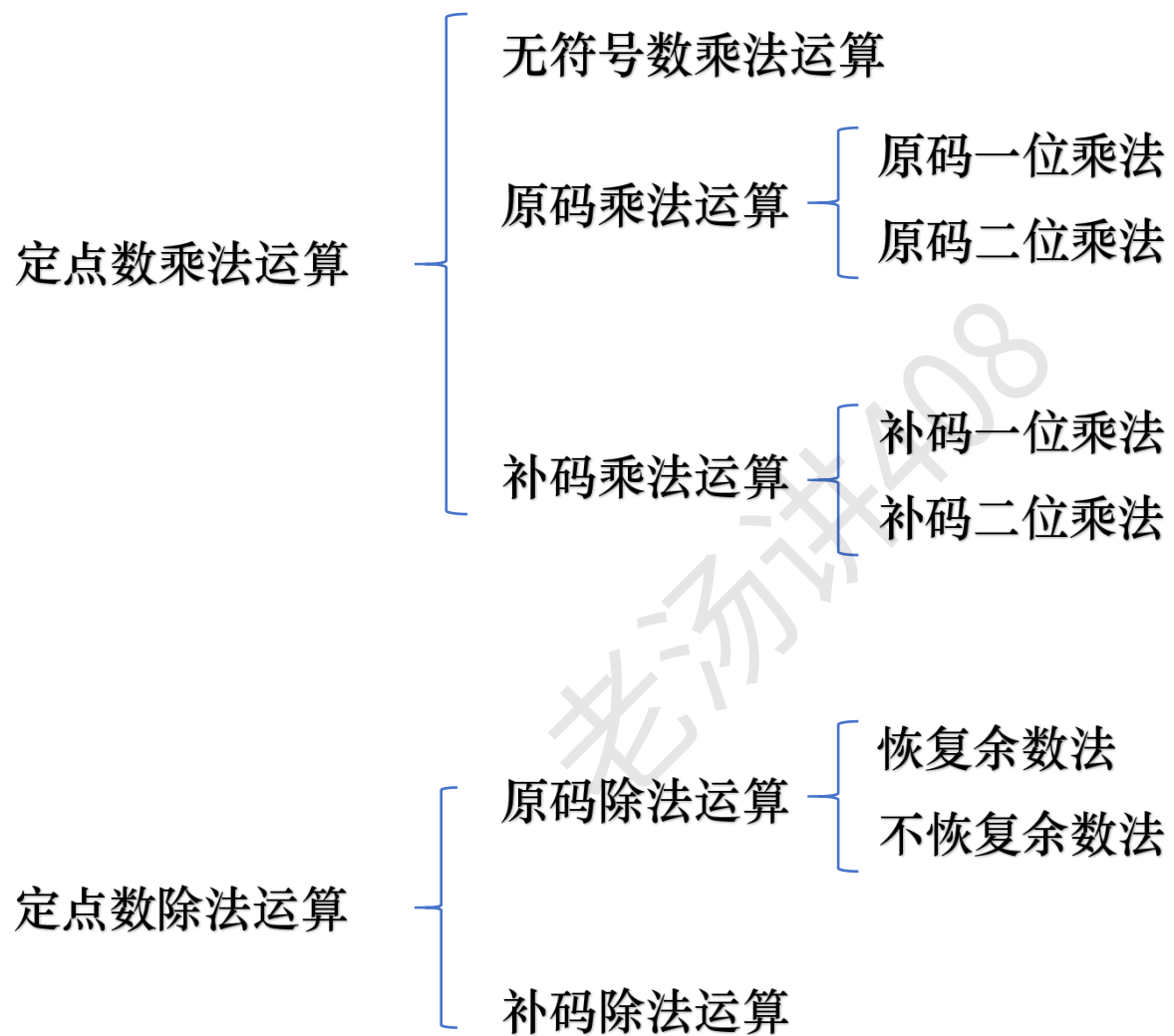
算术逻辑部件 (Arithmetic Logic Unit, ALU)



算术逻辑部件 (Arithmetic Logic Unit, ALU)



定点数乘除运算



$$123 \times 106$$

定点数乘法运算：无符号整数乘法

被乘数：123

1	2	3
---	---	---

乘数：106

×

1	0	6
---	---	---

7	3	8
---	---	---

0	0	0
---	---	---

+

1	2	3
---	---	---

乘积：13038

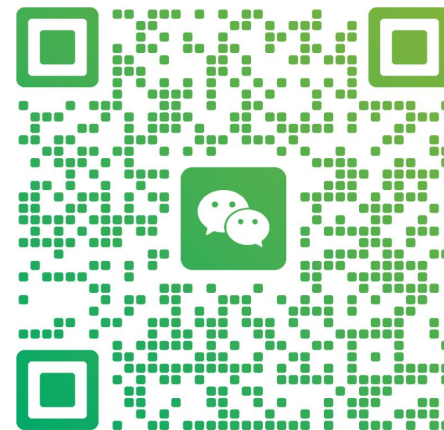
1	3	0	3	8
---	---	---	---	---

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

$$123 \times 106$$

定点数乘法运算：无符号整数乘法

被乘数：123

1	2	3
---	---	---



0	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---

乘数：106

×

1	0	6
---	---	---



×

0	1	1	0	1	0	1	0
---	---	---	---	---	---	---	---

7	3	8
---	---	---

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

0	0	0
---	---	---

0	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---

+

1	2	3
---	---	---

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

乘积：13038

1	3	0	3	8
---	---	---	---	---

0	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

0	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---

0	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---

+

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

乘积：13038

0	1	1	0	0	1	0	1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

定点数乘法运算：无符号整数乘法

123 × 106

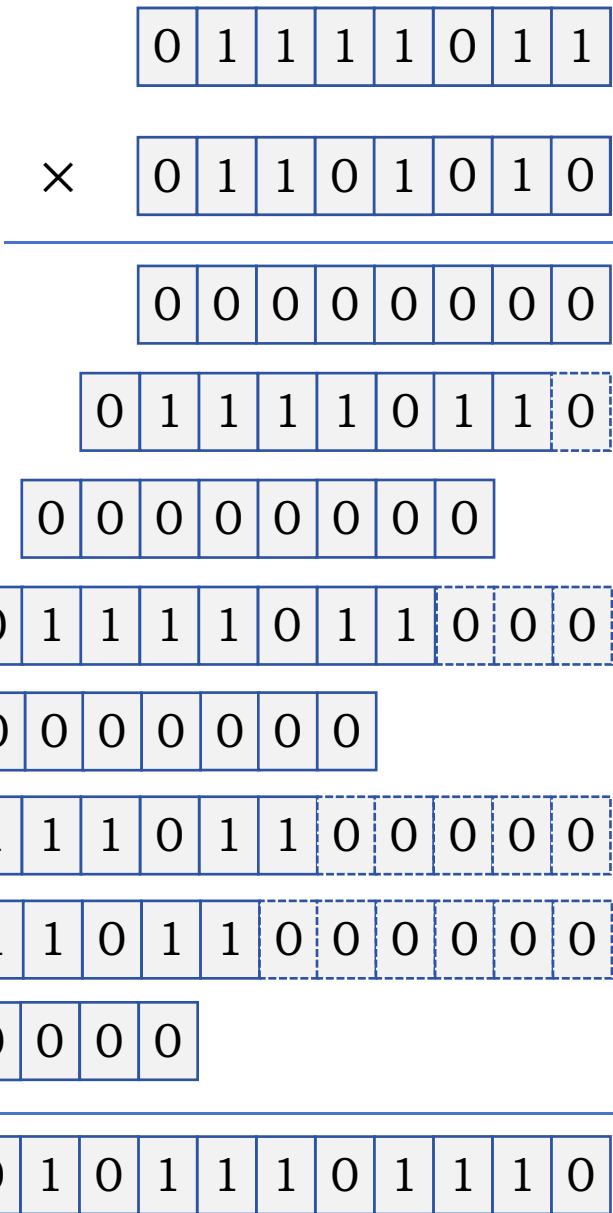
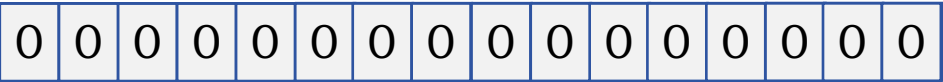
被乘数寄存器 (16 位)



乘数寄存器 (8 位)



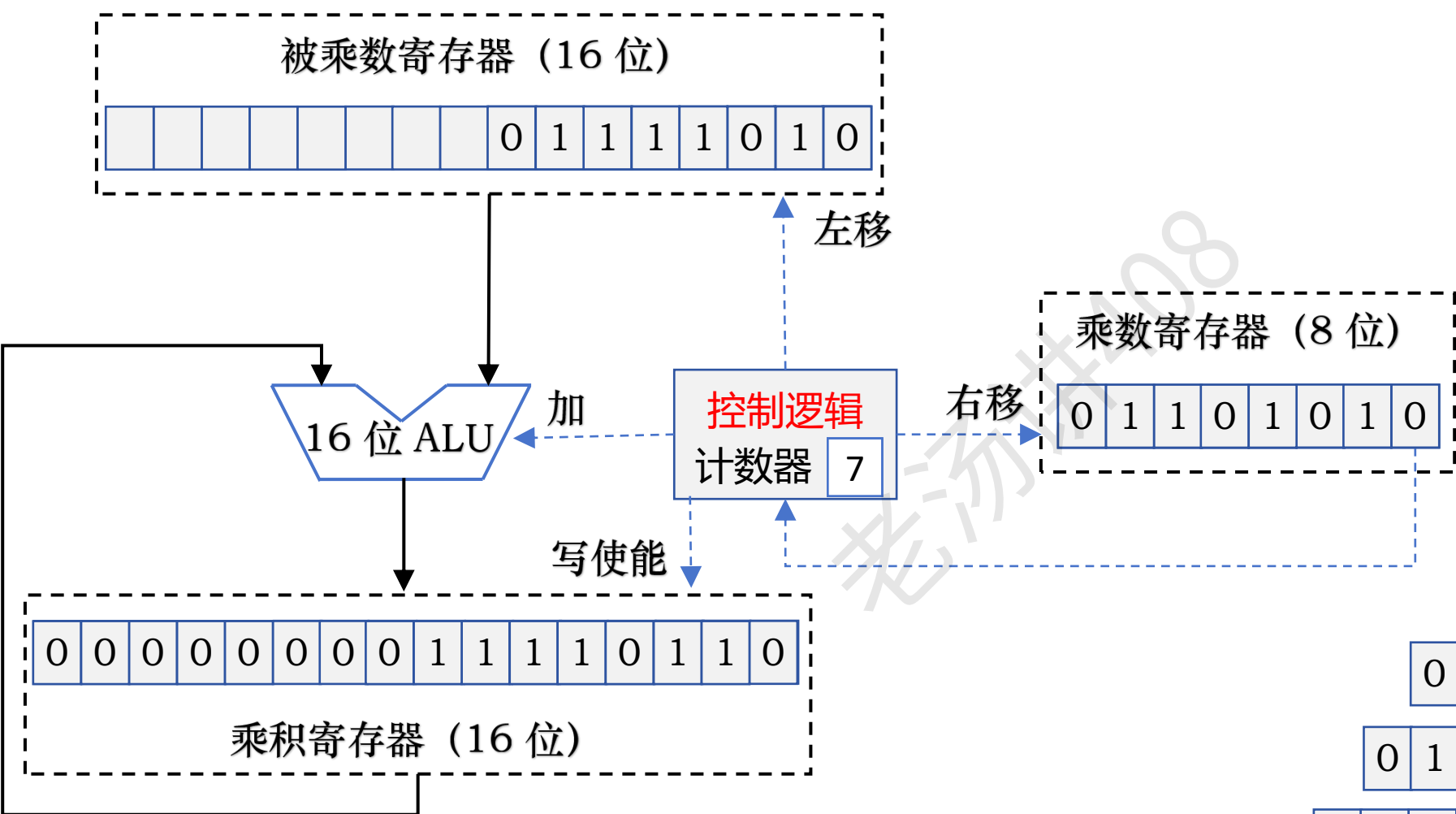
乘积寄存器 (16 位)



乘积：13038

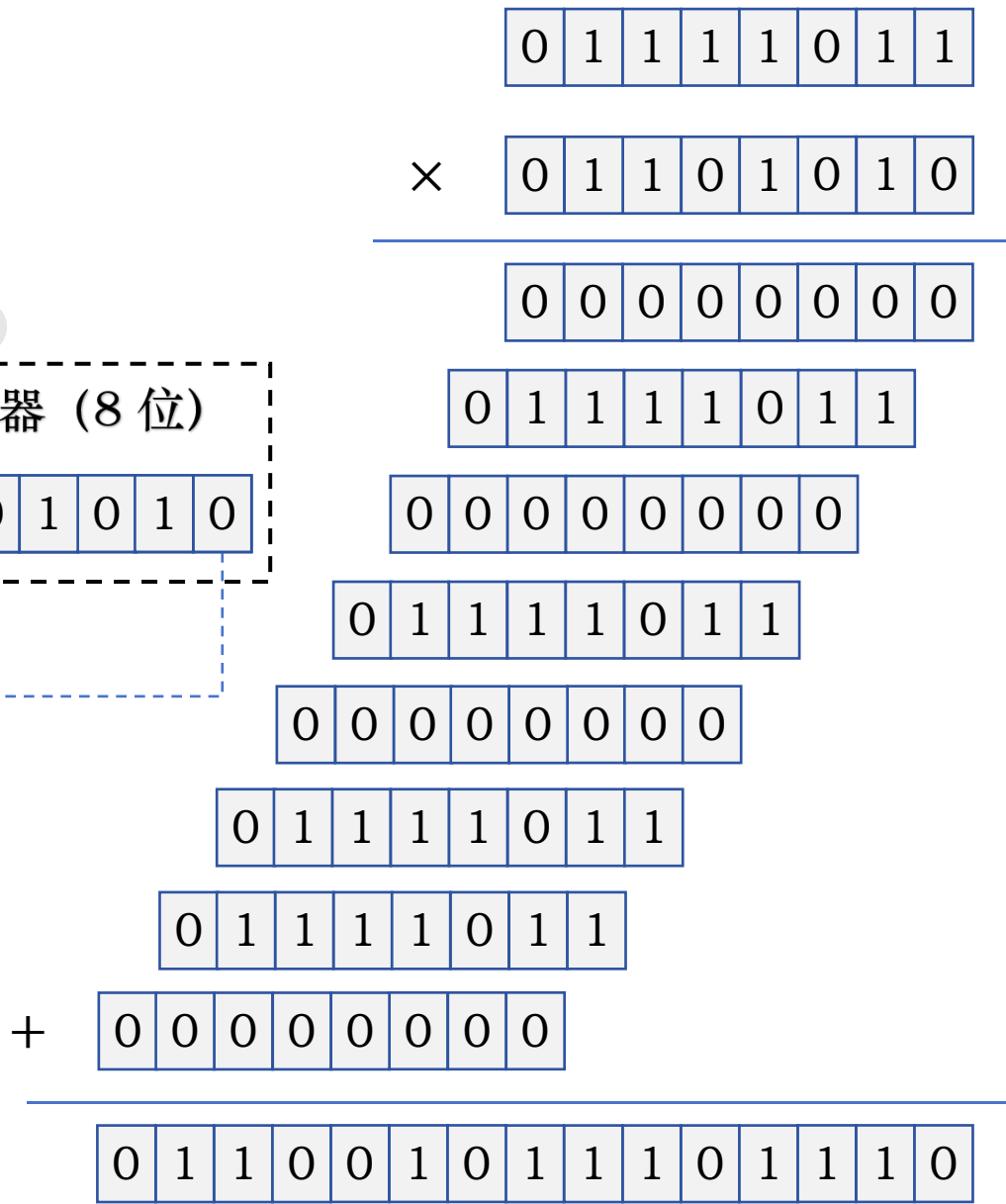
定点数乘法运算：无符号整数乘法电路硬件

123 × 106



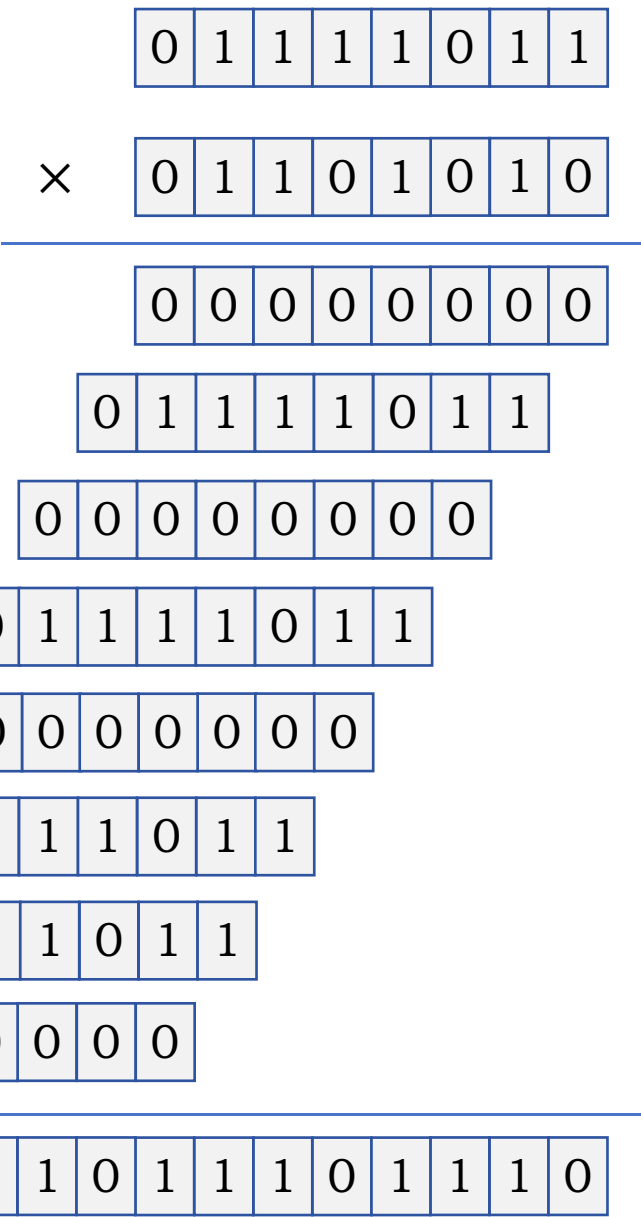
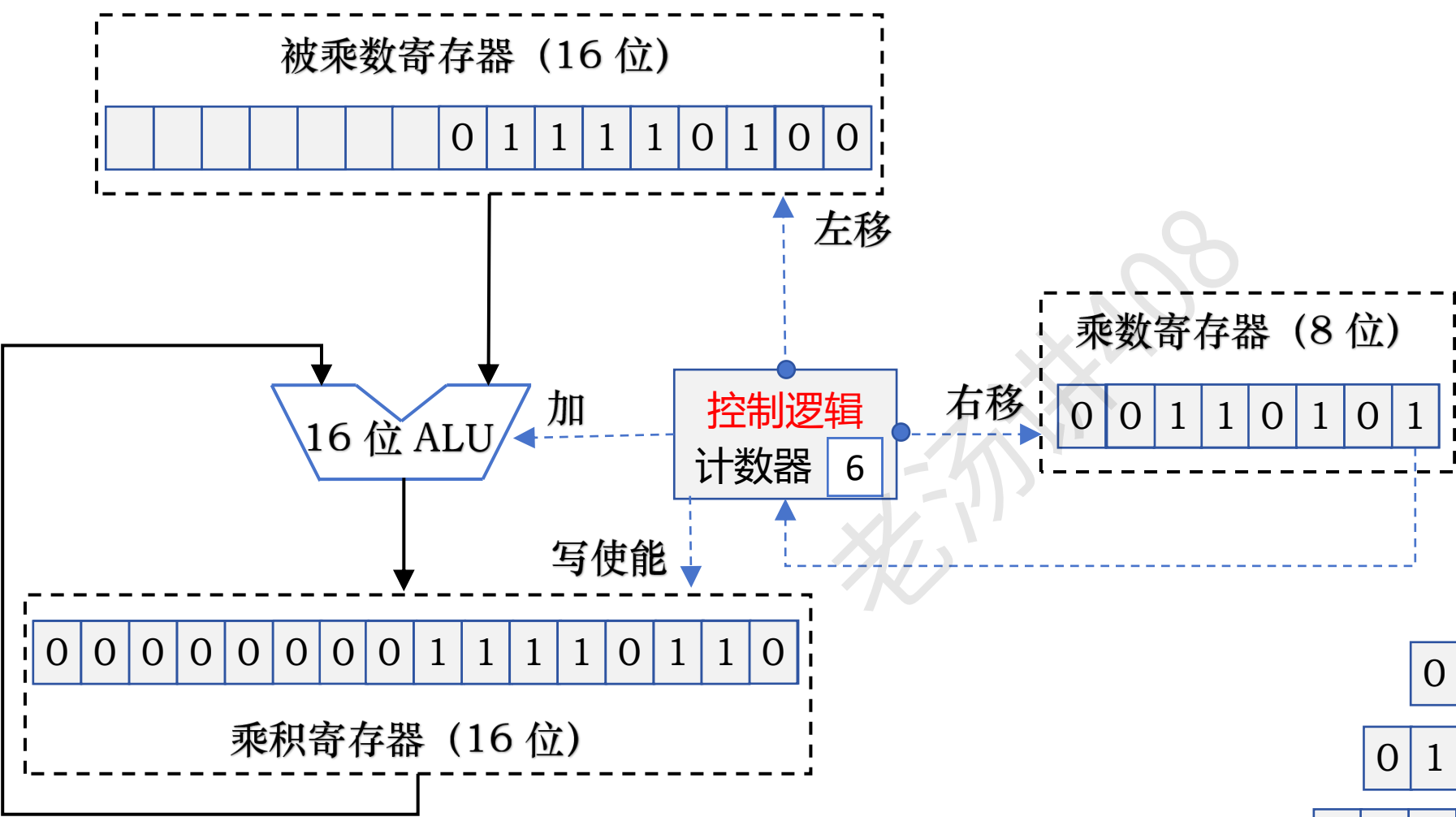
部分积

乘积：13038



定点数乘法运算：无符号整数乘法电路硬件

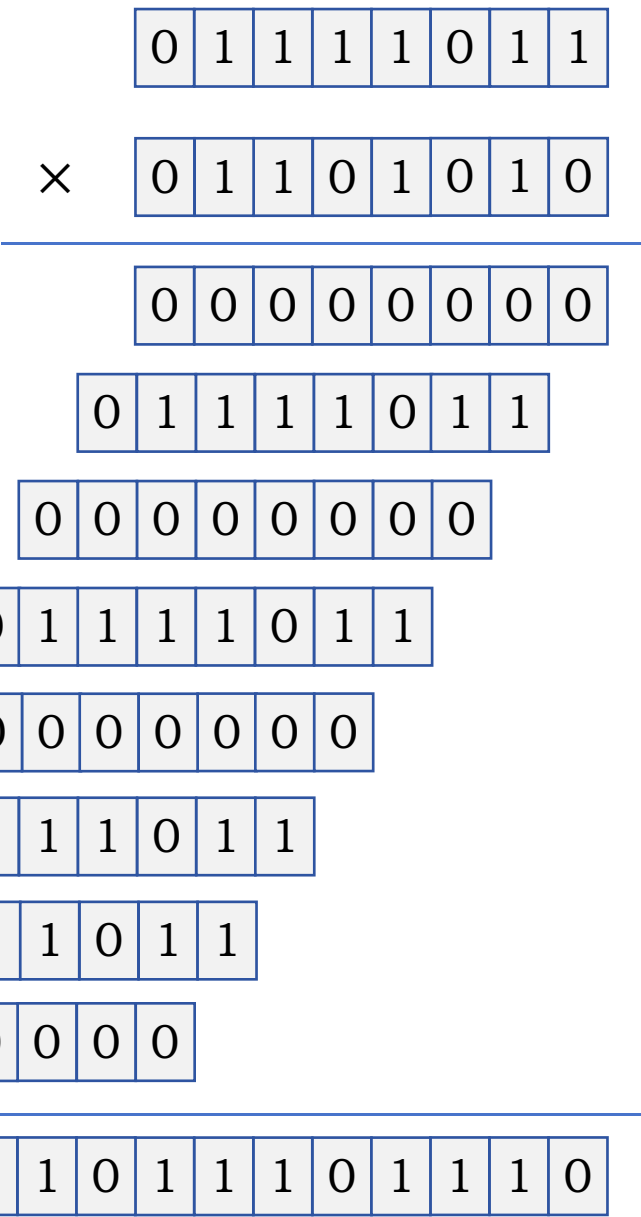
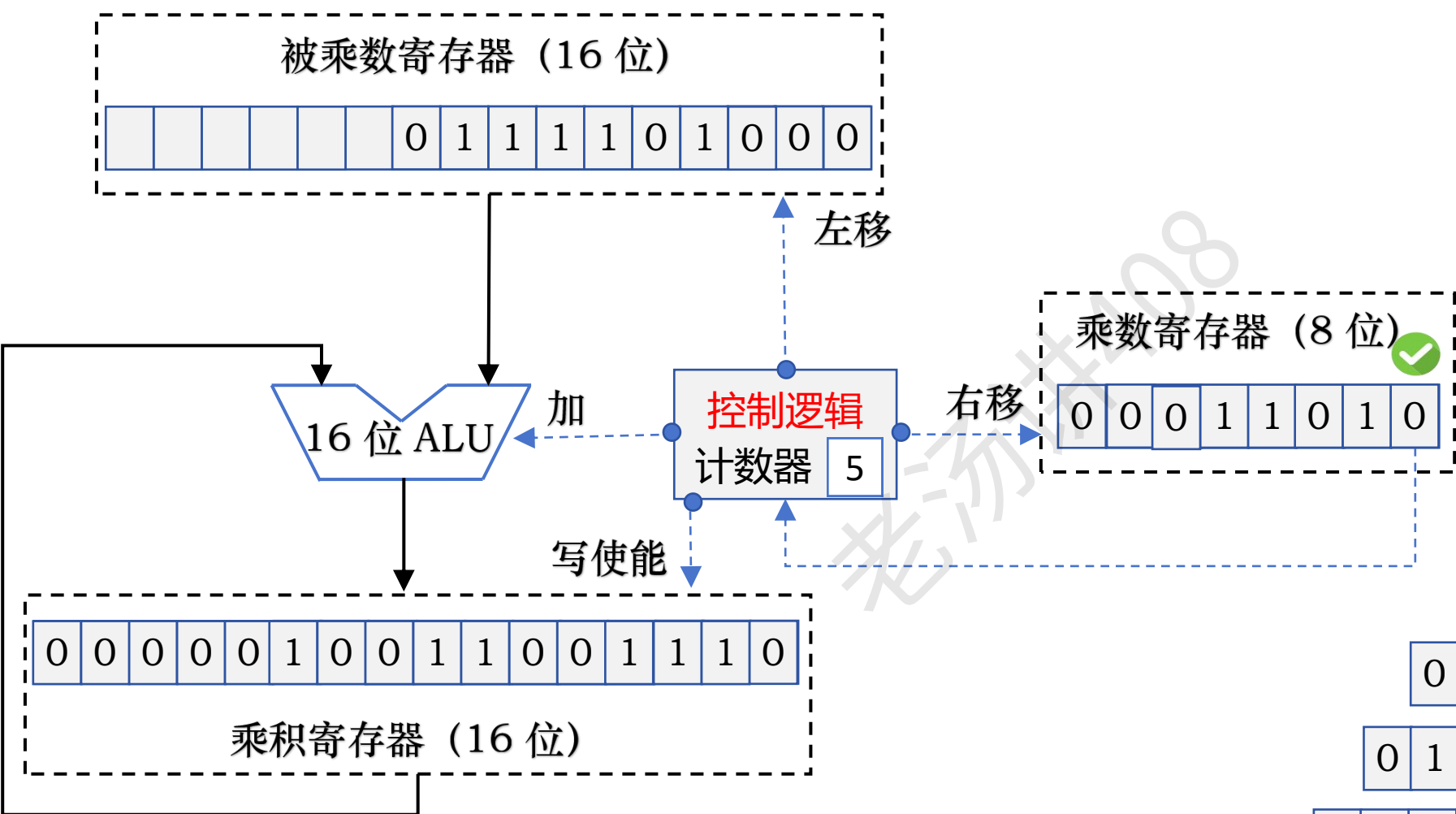
123 × 106



乘积：13038

定点数乘法运算：无符号整数乘法电路硬件

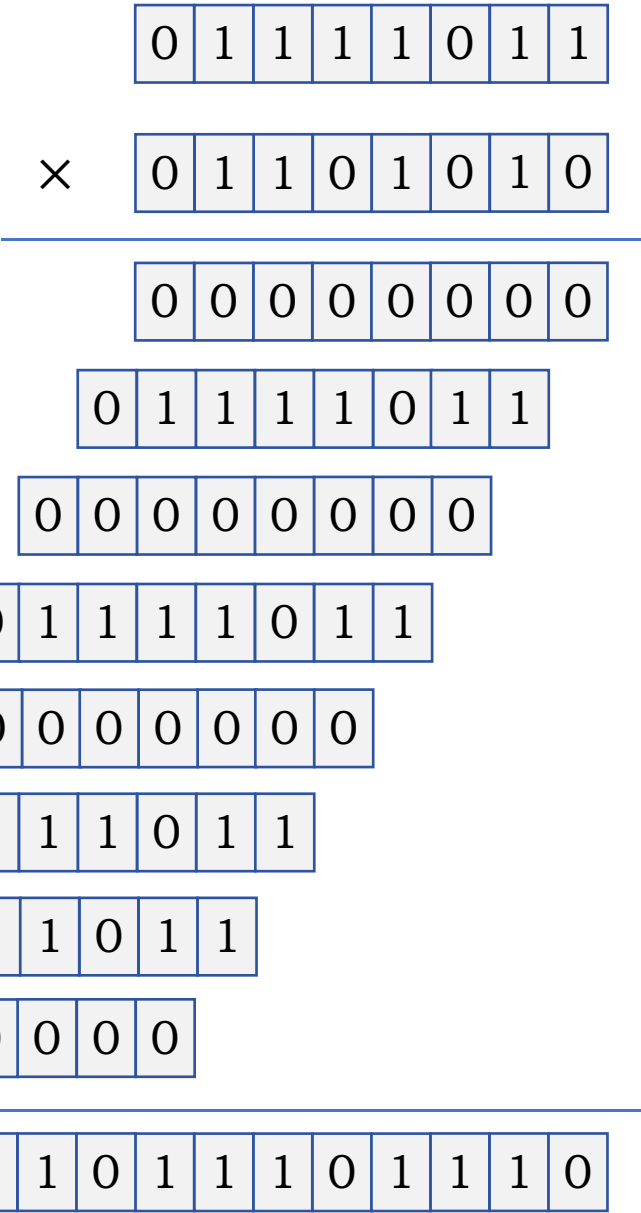
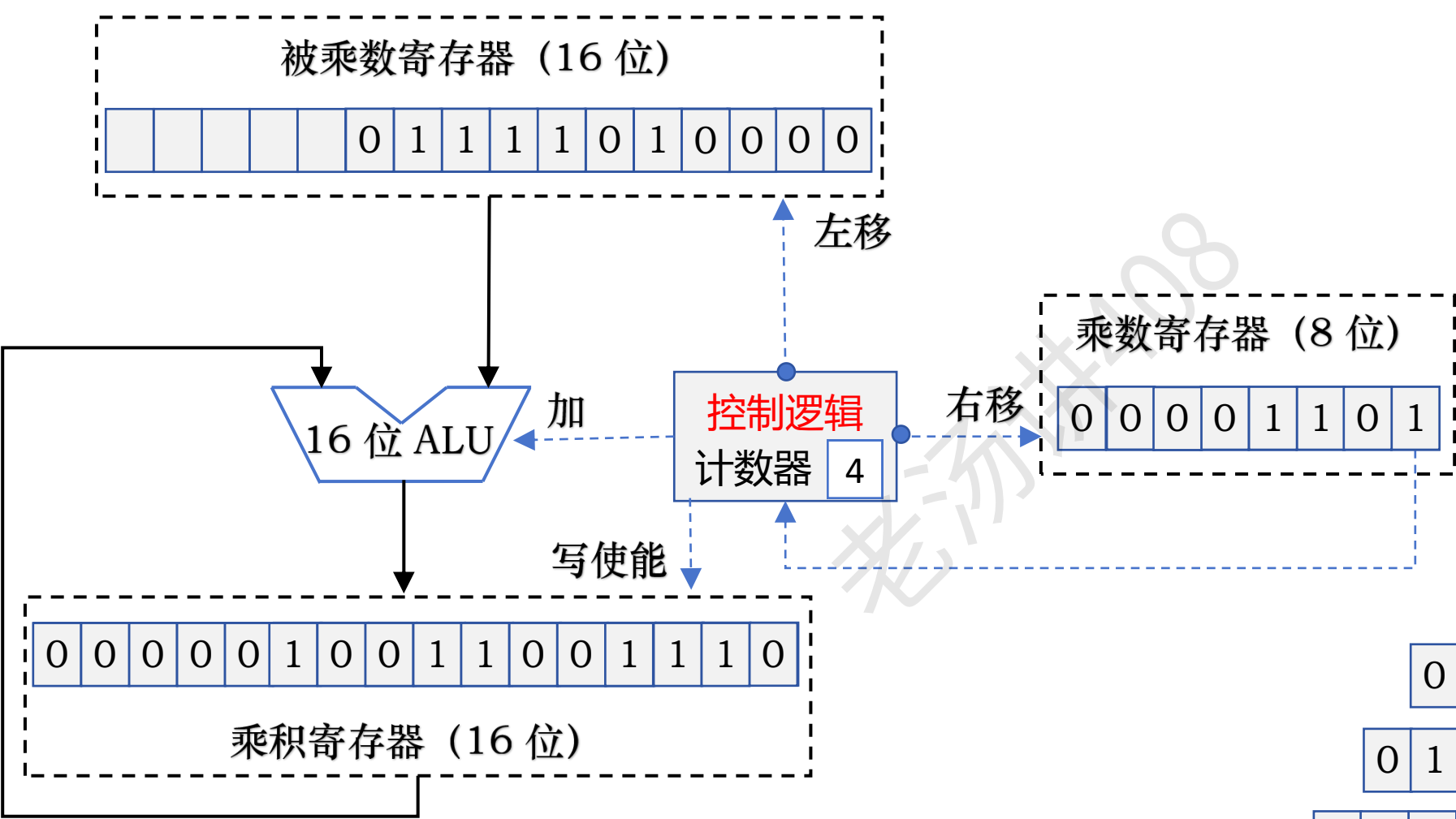
123 × 106



乘积：13038

定点数乘法运算：无符号整数乘法电路硬件

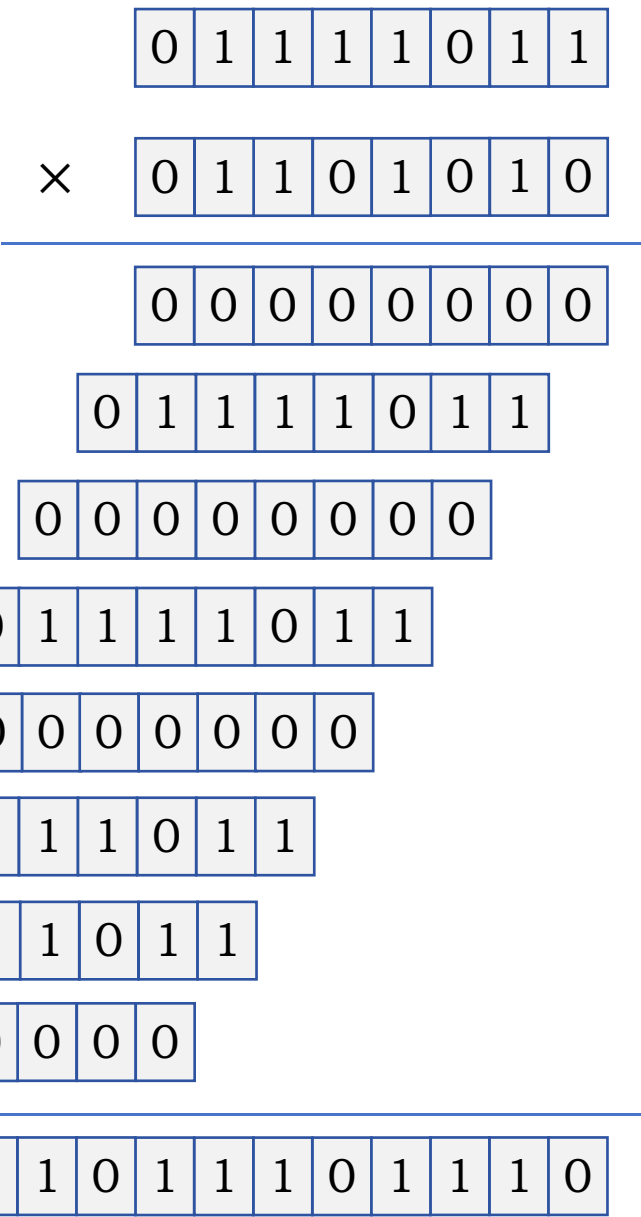
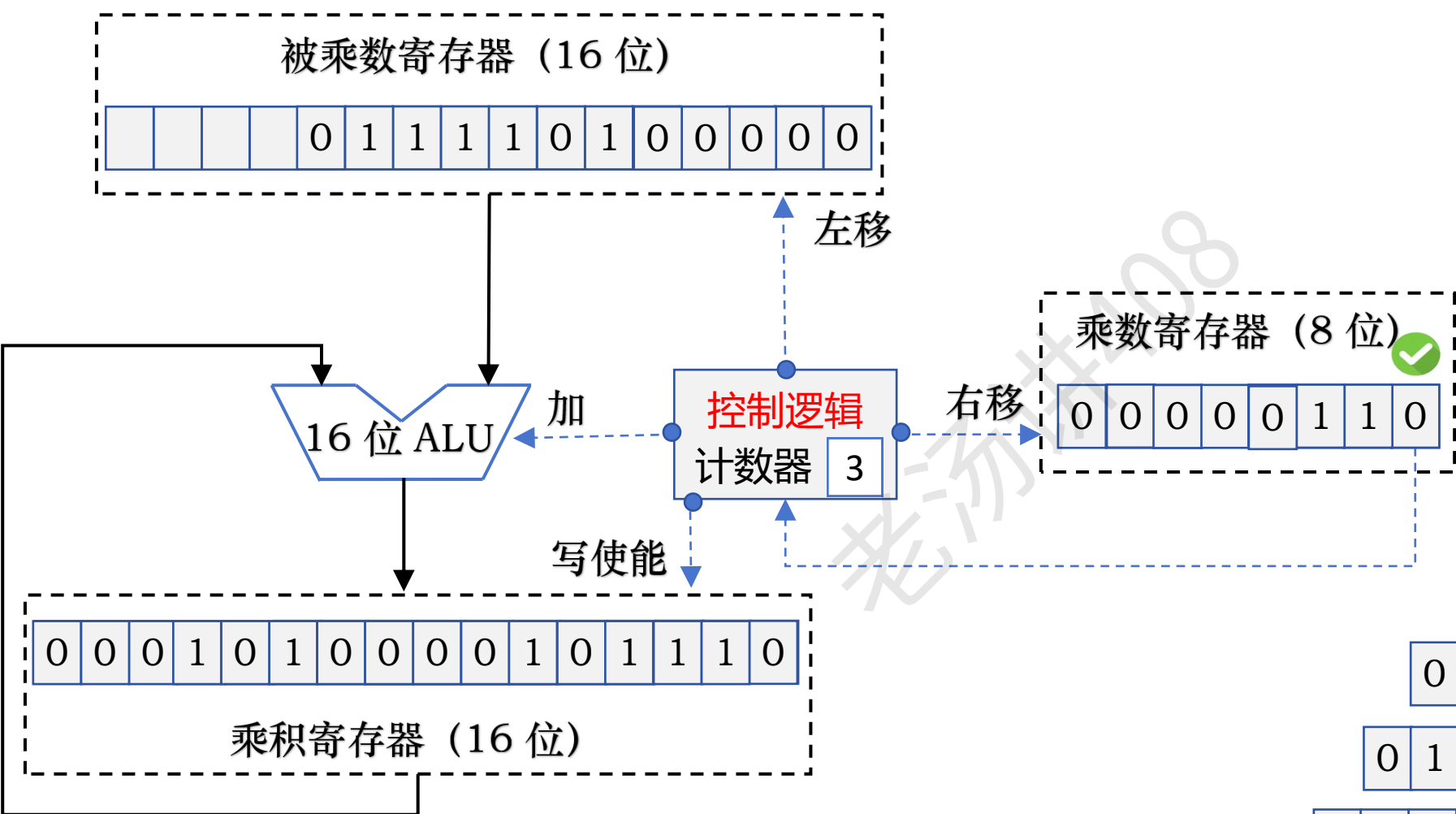
123 × 106



乘积：13038

定点数乘法运算：无符号整数乘法电路硬件

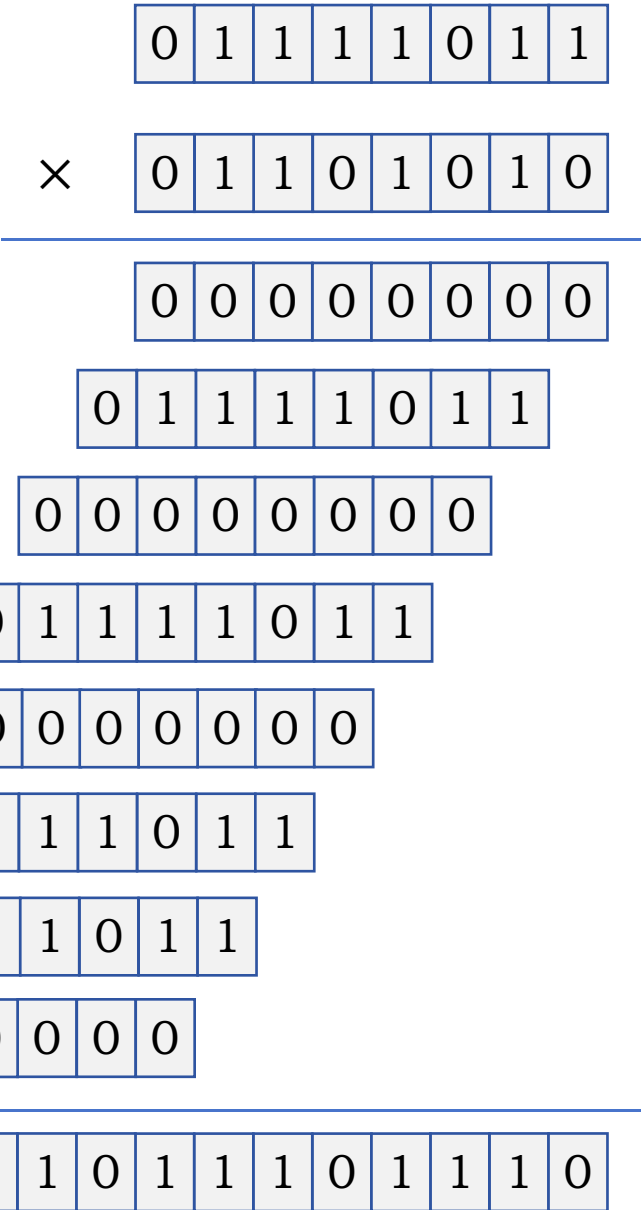
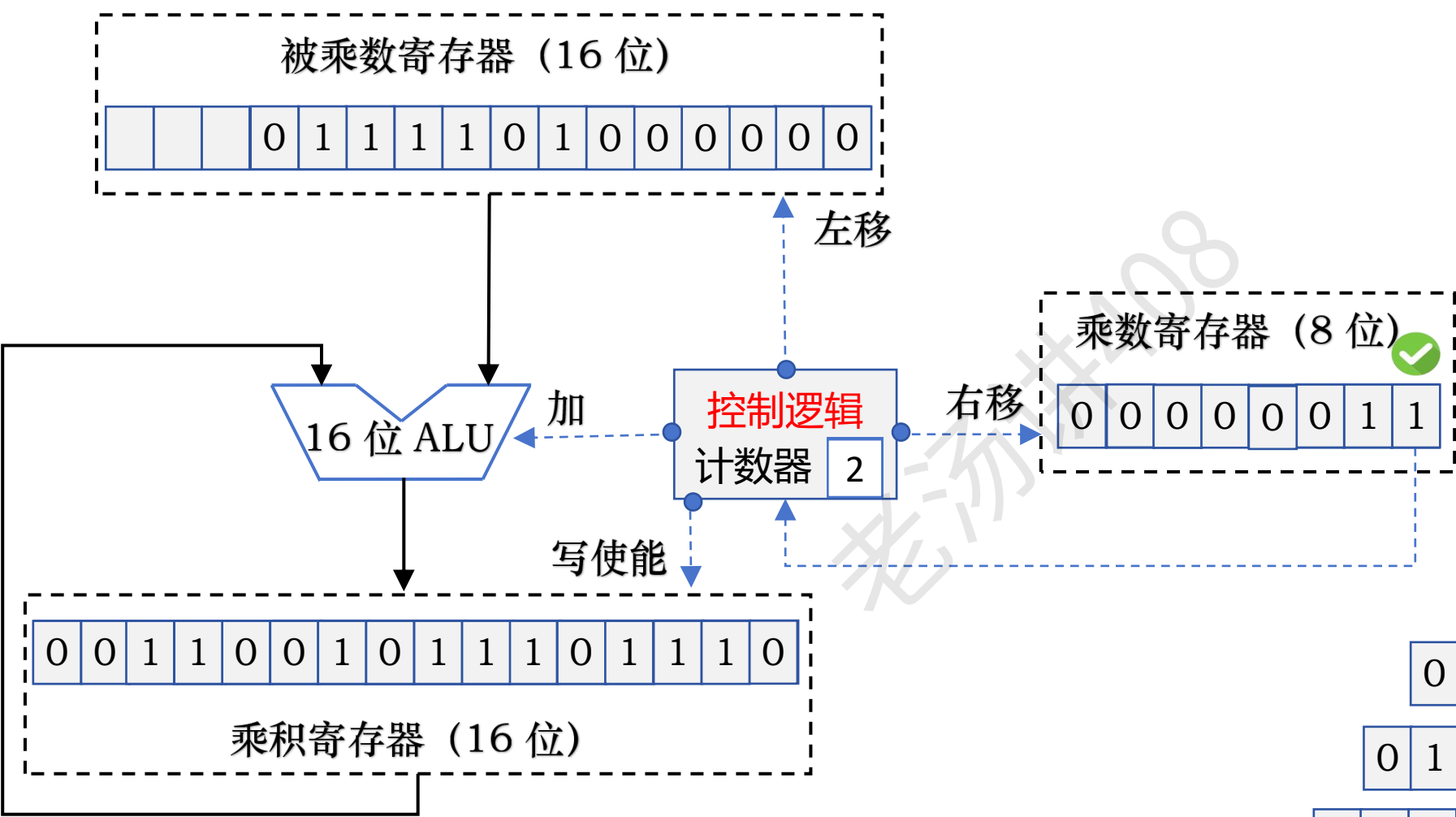
123 × 106



乘积：13038

定点数乘法运算：无符号整数乘法电路硬件

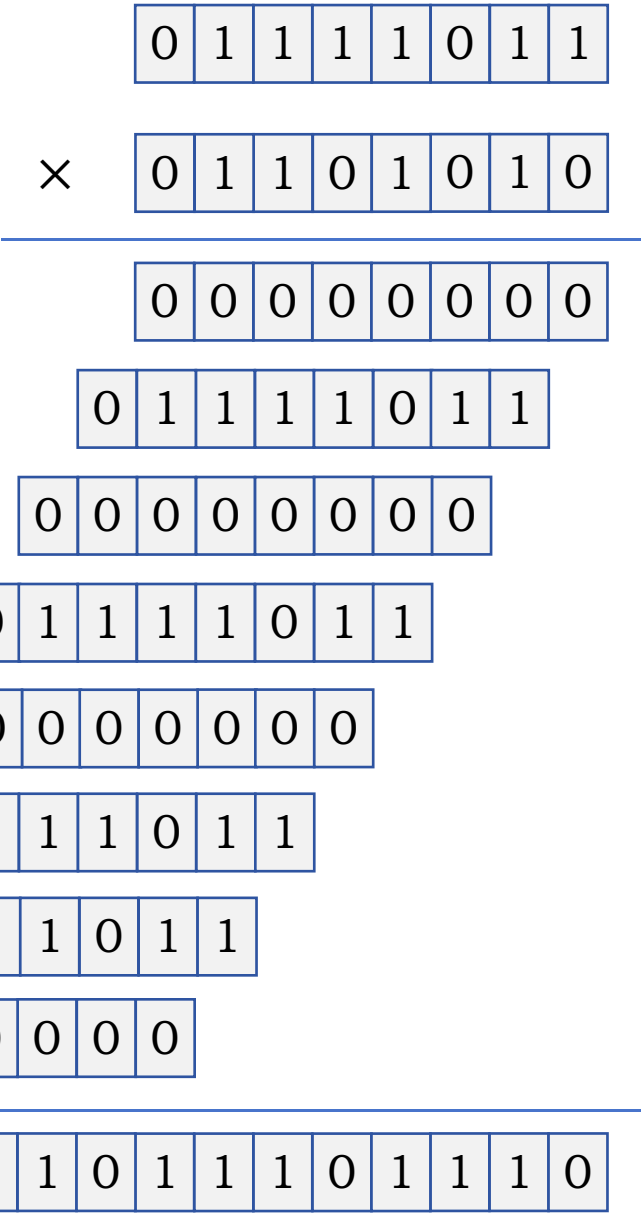
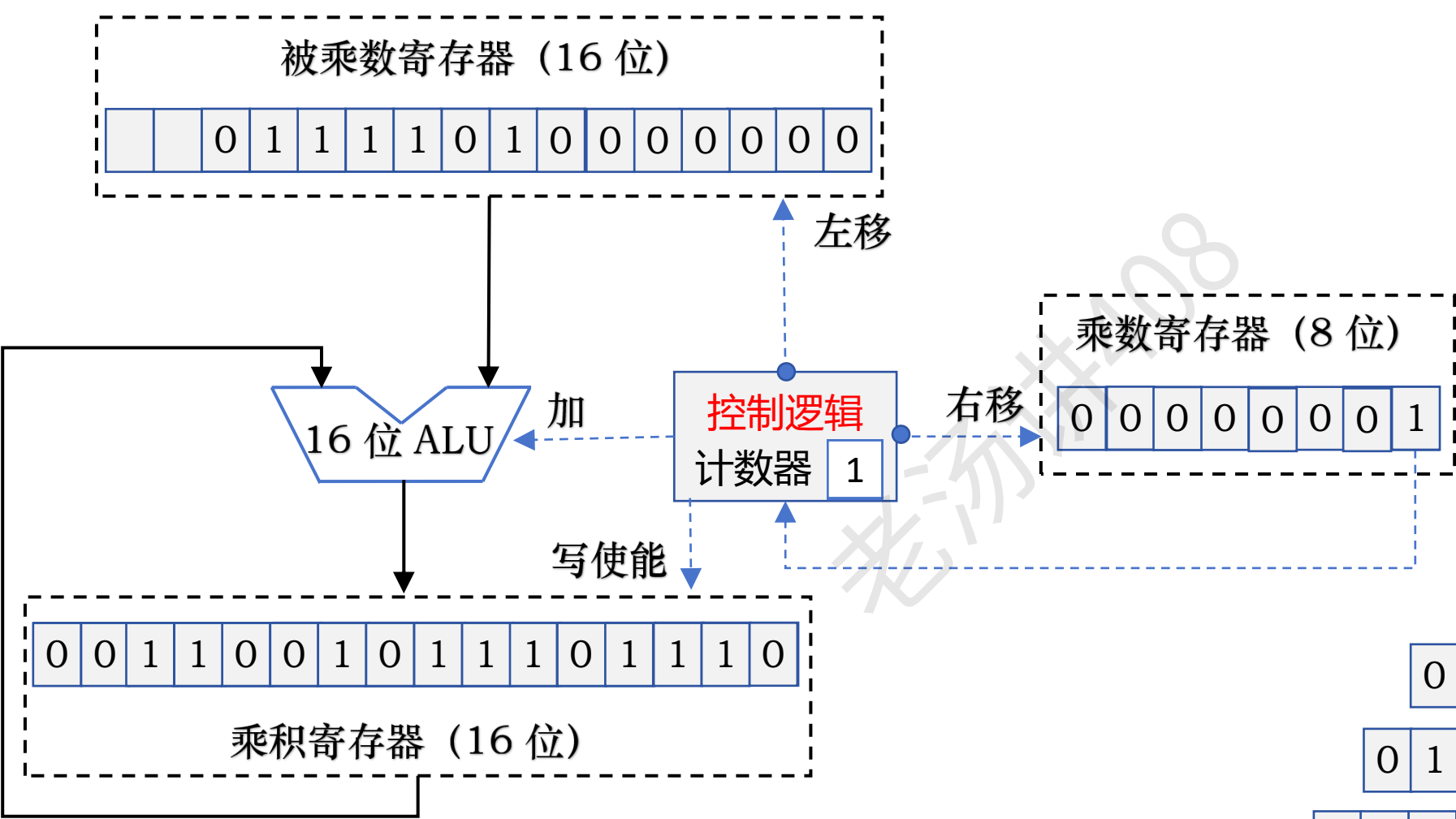
123 × 106



乘积：13038

定点数乘法运算：无符号整数乘法电路硬件

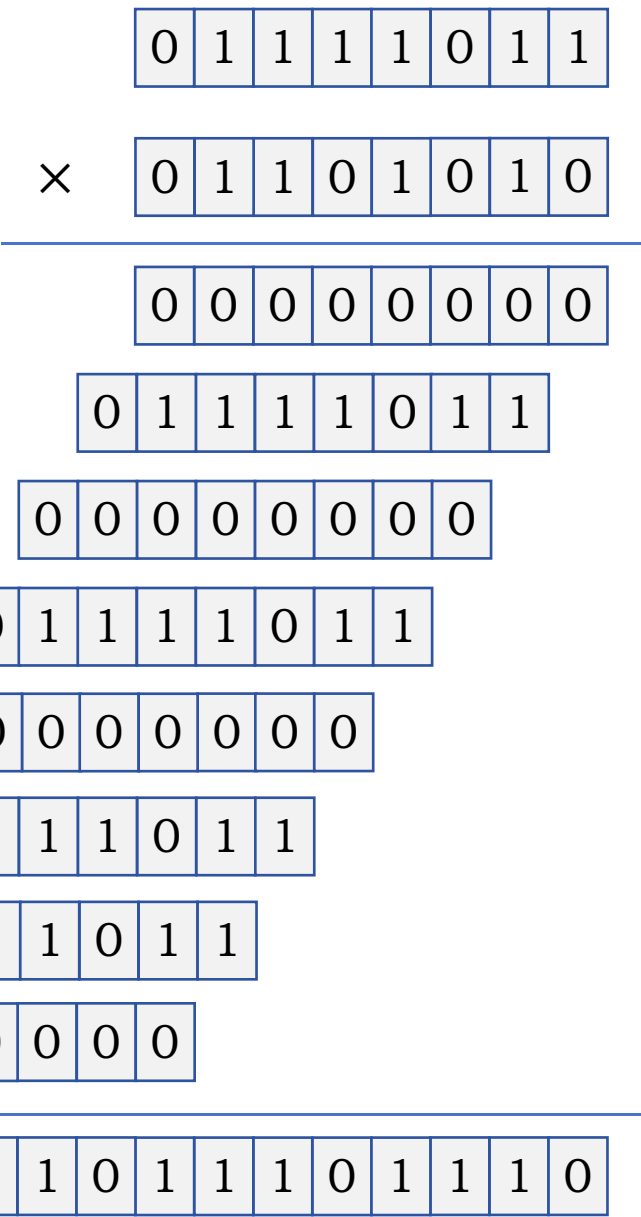
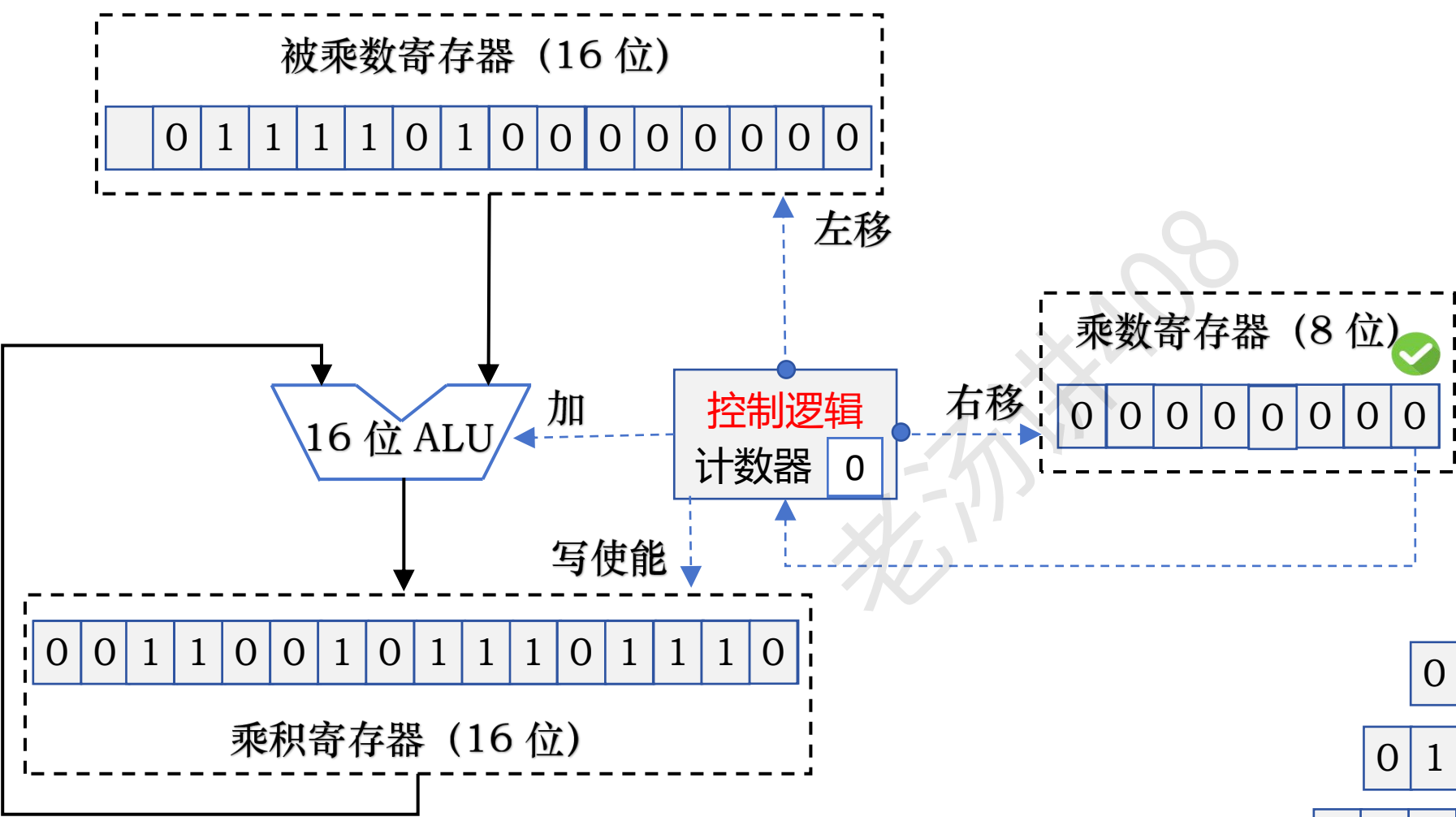
123 × 106



乘积：13038

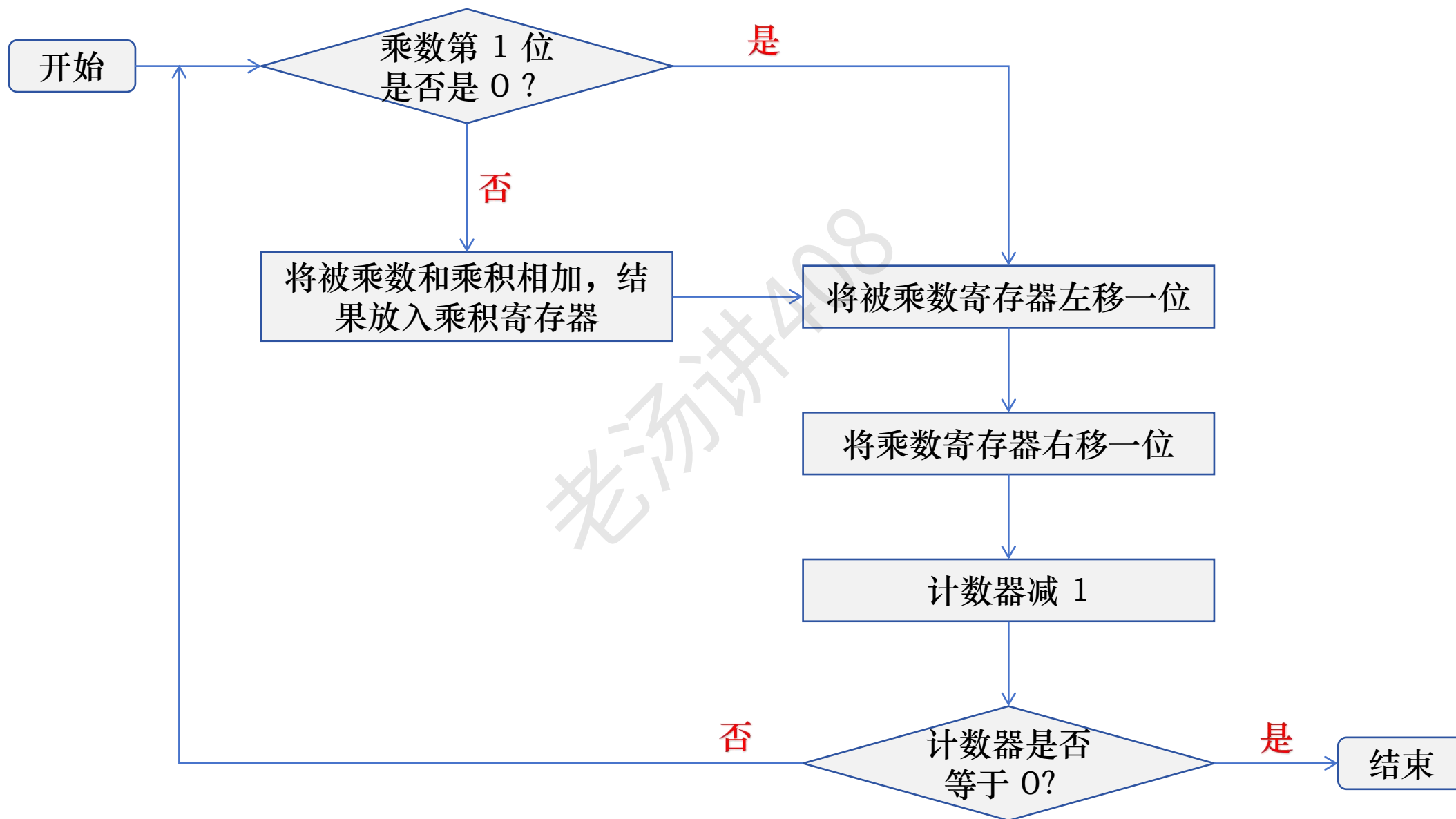
定点数乘法运算：无符号整数乘法电路硬件

123 × 106



乘积：13038

定点数乘法运算：无符号整数乘法电路硬件



定点数乘法运算：无符号整数乘法电路硬件（优化）

$$X = X_7X_6X_5X_4X_3X_2X_1X_0 \quad Y = Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$$

$$X * Y = X * Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$$

$$= X * Y_0 * 2^0 + X * Y_1 * 2^1 + X * Y_2 * 2^2 + X * Y_3 * 2^3 + X * Y_4 * 2^4 + X * Y_5 * 2^5 + X * Y_6 * 2^6 + X * Y_7 * 2^7$$

$$= (X * Y_0 * 2^0 + X * Y_1 * 2^1 + X * Y_2 * 2^2 + X * Y_3 * 2^3 + X * Y_4 * 2^4 + X * Y_5 * 2^5 + X * Y_6 * 2^6 + X * Y_7 * 2^7) * 2^{-8} * 2^8$$

$$= (X * Y_0 * 2^{-8} + X * Y_1 * 2^{-7} + X * Y_2 * 2^{-6} + X * Y_3 * 2^{-5} + X * Y_4 * 2^{-4} + X * Y_5 * 2^{-3} + X * Y_6 * 2^{-2} + X * Y_7 * 2^{-1}) * 2^8$$

$$= ((X * Y_0 * 2^{-7} + X * Y_1 * 2^{-6} + X * Y_2 * 2^{-5} + X * Y_3 * 2^{-4} + X * Y_4 * 2^{-3} + X * Y_5 * 2^{-2} + X * Y_6 * 2^{-1} + X * Y_7) * 2^{-1}) * 2^8$$

$$= (((X * Y_0 * 2^{-6} + X * Y_1 * 2^{-5} + X * Y_2 * 2^{-4} + X * Y_3 * 2^{-3} + X * Y_4 * 2^{-2} + X * Y_5 * 2^{-1} + X * Y_6) * 2^{-1} + X * Y_7) * 2^{-1}) * 2^8$$

$$= (((((X * Y_0 * 2^{-5} + X * Y_1 * 2^{-4} + X * Y_2 * 2^{-3} + X * Y_3 * 2^{-2} + X * Y_4 * 2^{-1} + X * Y_5) * 2^{-1} + X * Y_6) * 2^{-1} + X * Y_7) * 2^{-1}) * 2^8$$

$$= (((((((X * Y_0 * 2^{-4} + X * Y_1 * 2^{-3} + X * Y_2 * 2^{-2} + X * Y_3 * 2^{-1} + X * Y_4) * 2^{-1} + X * Y_5) * 2^{-1} + X * Y_6) * 2^{-1} + X * Y_7) * 2^{-1}) * 2^8$$

$$= (((((((((X * Y_0 * 2^{-3} + X * Y_1 * 2^{-2} + X * Y_2 * 2^{-1} + X * Y_3) * 2^{-1} + X * Y_4) * 2^{-1} + X * Y_5) * 2^{-1} + X * Y_6) * 2^{-1} + X * Y_7) * 2^{-1}) * 2^8$$

$$= (((((((((((X * Y_0 * 2^{-2} + X * Y_1 * 2^{-1} + X * Y_2) * 2^{-1} + X * Y_3) * 2^{-1} + X * Y_4) * 2^{-1} + X * Y_5) * 2^{-1} + X * Y_6) * 2^{-1} + X * Y_7) * 2^{-1}) * 2^8$$

$$= (((((((((((((X * Y_0 * 2^{-1} + X * Y_1) * 2^{-1} + X * Y_2) * 2^{-1} + X * Y_3) * 2^{-1} + X * Y_4) * 2^{-1} + X * Y_5) * 2^{-1} + X * Y_6) * 2^{-1} + X * Y_7) * 2^{-1}) * 2^8$$

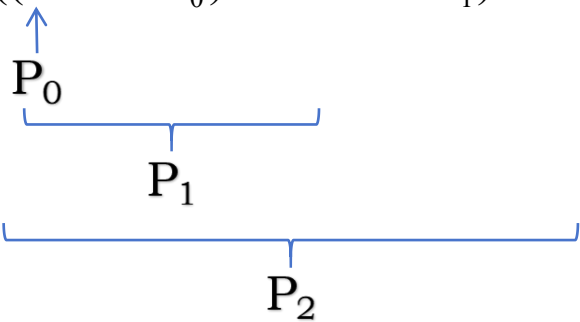
$$= ((((((((((((((0 + X * Y_0) * 2^{-1} + X * Y_1) * 2^{-1} + X * Y_2) * 2^{-1} + X * Y_3) * 2^{-1} + X * Y_4) * 2^{-1} + X * Y_5) * 2^{-1} + X * Y_6) * 2^{-1} + X * Y_7) * 2^{-1}) * 2^8$$

定点数乘法运算：无符号整数乘法电路硬件（优化）

$$X = X_7X_6X_5X_4X_3X_2X_1X_0 \quad Y = Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$$

$$X * Y = X * Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$$

$$= (((((((0 + X * Y_0) * 2^{-1} + X * Y_1) * 2^{-1} + X * Y_2) * 2^{-1} + X * Y_3) * 2^{-1} + X * Y_4) * 2^{-1} + X * Y_5) * 2^{-1} + X * Y_6) * 2^{-1} + X * Y_7) * 2^{-1}) * 2^8$$



初始部分积： $P_0 = 0$

$$\text{部分积： } P_1 = (P_0 + X * Y_0) * 2^{-1}$$

$$P_2 = (P_1 + X * Y_1) * 2^{-1}$$

$\vdots$

$$P_8 = (P_7 + X * Y_7) * 2^{-1}$$

$$P_{i+1} = (P_i + X * Y_i) * 2^{-1}$$

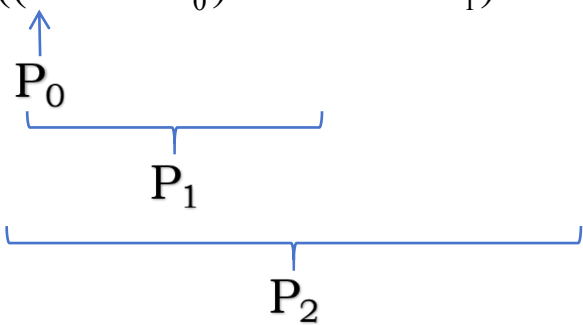
$$X * Y = P_8 * 2^8$$

定点数乘法运算：无符号整数乘法电路硬件（优化）

$$X = X_7X_6X_5X_4X_3X_2X_1X_0 \quad Y = Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$$

$$X * Y = X * Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$$

$$= (((((((0 + X * Y_0) * 2^{-1} + X * Y_1) * 2^{-1} + X * Y_2) * 2^{-1} + X * Y_3) * 2^{-1} + X * Y_4) * 2^{-1} + X * Y_5) * 2^{-1} + X * Y_6) * 2^{-1} + X * Y_7) * 2^{-1}) * 2^8$$



初始部分积： $P_0 = 0$

$$\text{部分积： } P_1 = (P_0 + X * Y_0) * 2^{-1}$$

$$P_2 = (P_1 + X * Y_1) * 2^{-1}$$

$\vdots$

$$P_8 = (P_7 + X * Y_7) * 2^{-1}$$

$$P_{i+1} = (P_i + X * Y_i) * 2^{-1}$$

$$X * Y = P_8 * 2^8$$

① 取乘数的最低位 Y_i 判断

② 若 $Y_i = 1$ ，则将 X 和上一步迭代部分积 P_i 相加，
若 $Y_i = 0$ ，则什么都不做

③ 对部分积逻辑右移一位，产生本次部分积 P_{i+1}

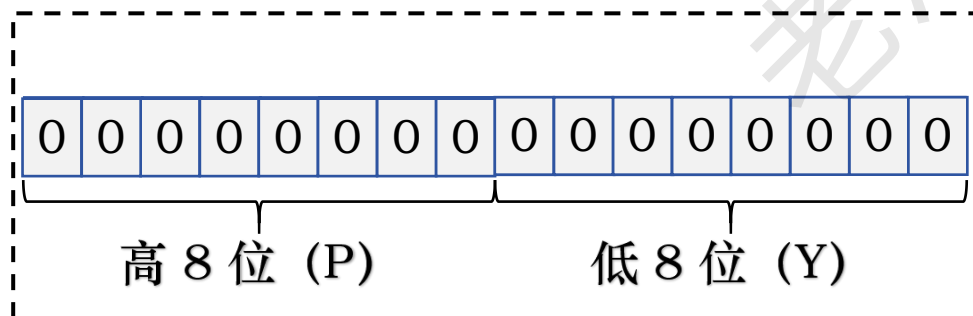
定点数乘法运算：无符号整数乘法电路硬件（优化）

$$123 \times 106$$

被乘数寄存器 X (8 位)

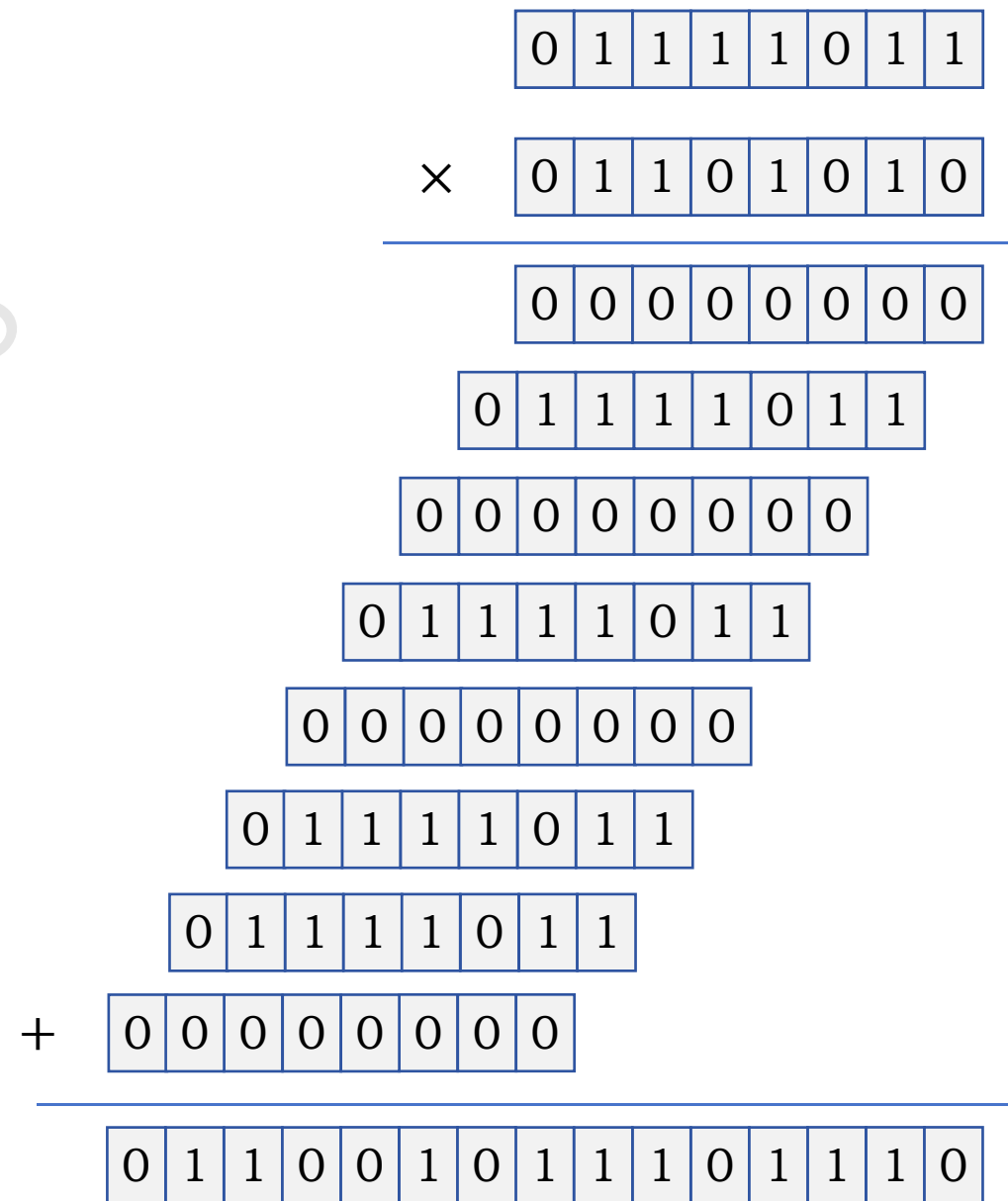


$$X * Y = P_8 * 2^8$$



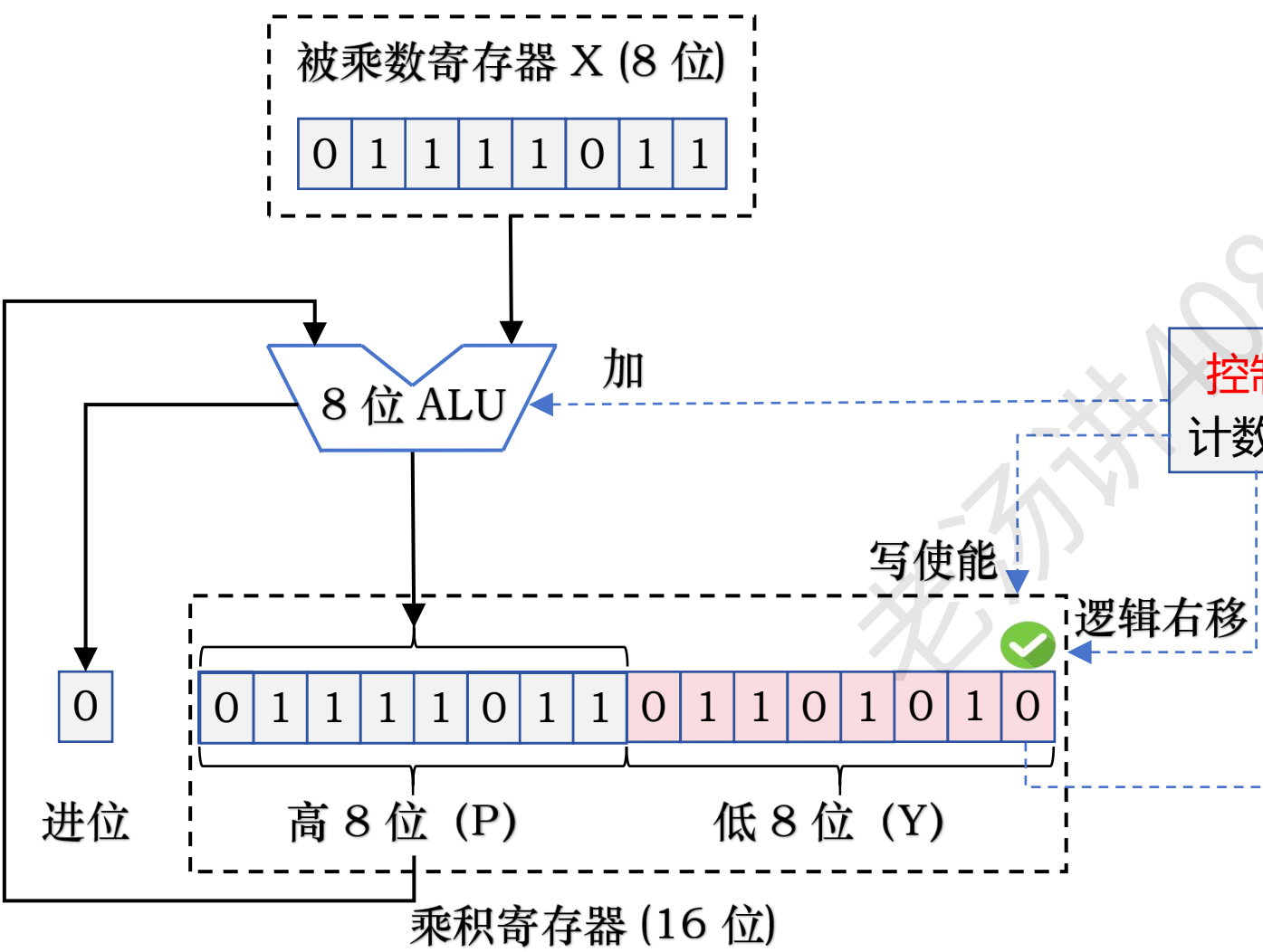
乘积寄存器 (16 位)

乘积: 13038

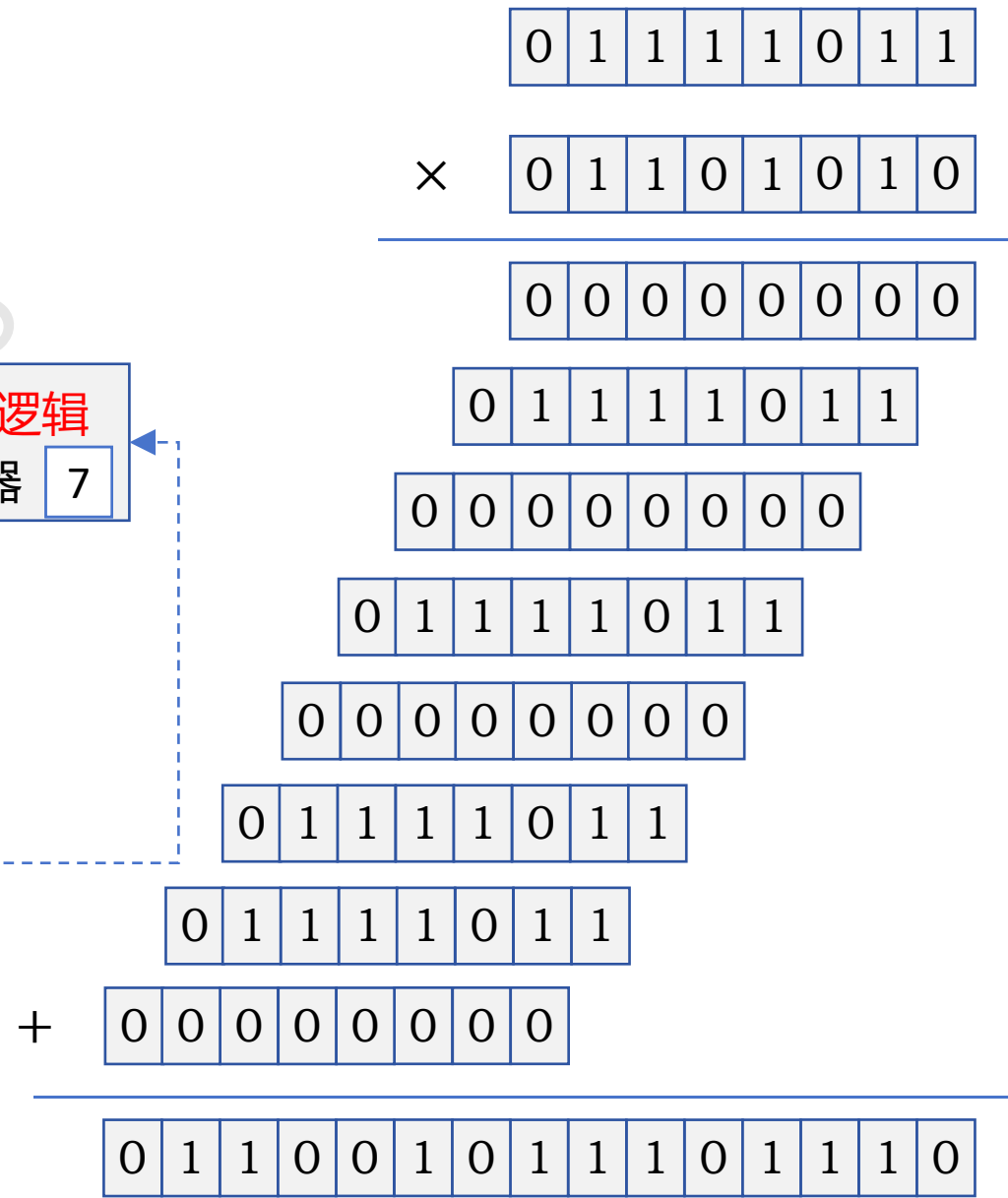


定点数乘法运算：无符号整数乘法电路硬件（优化）

123 × 106

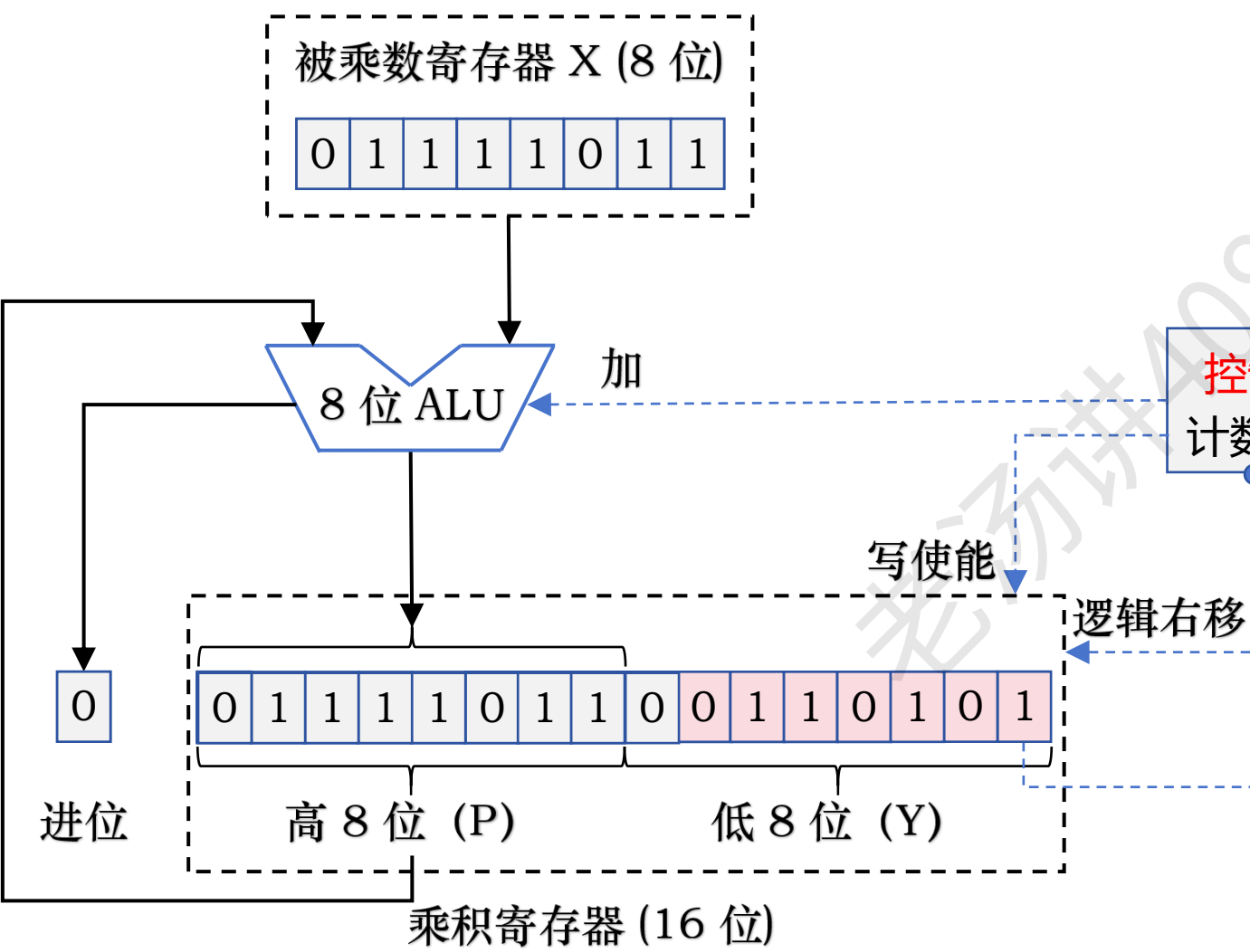


乘积：13038

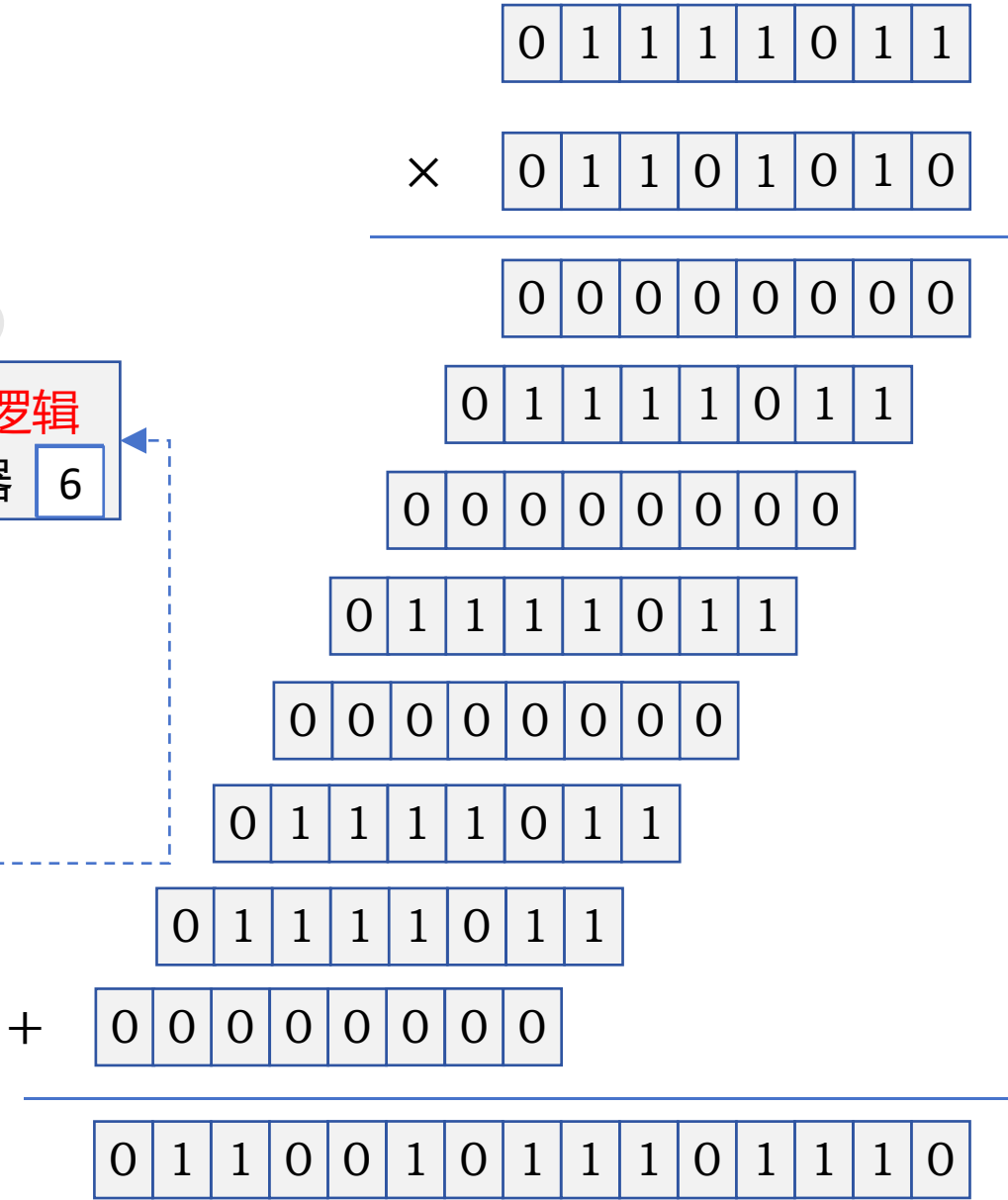


定点数乘法运算：无符号整数乘法电路硬件（优化）

123 × 106

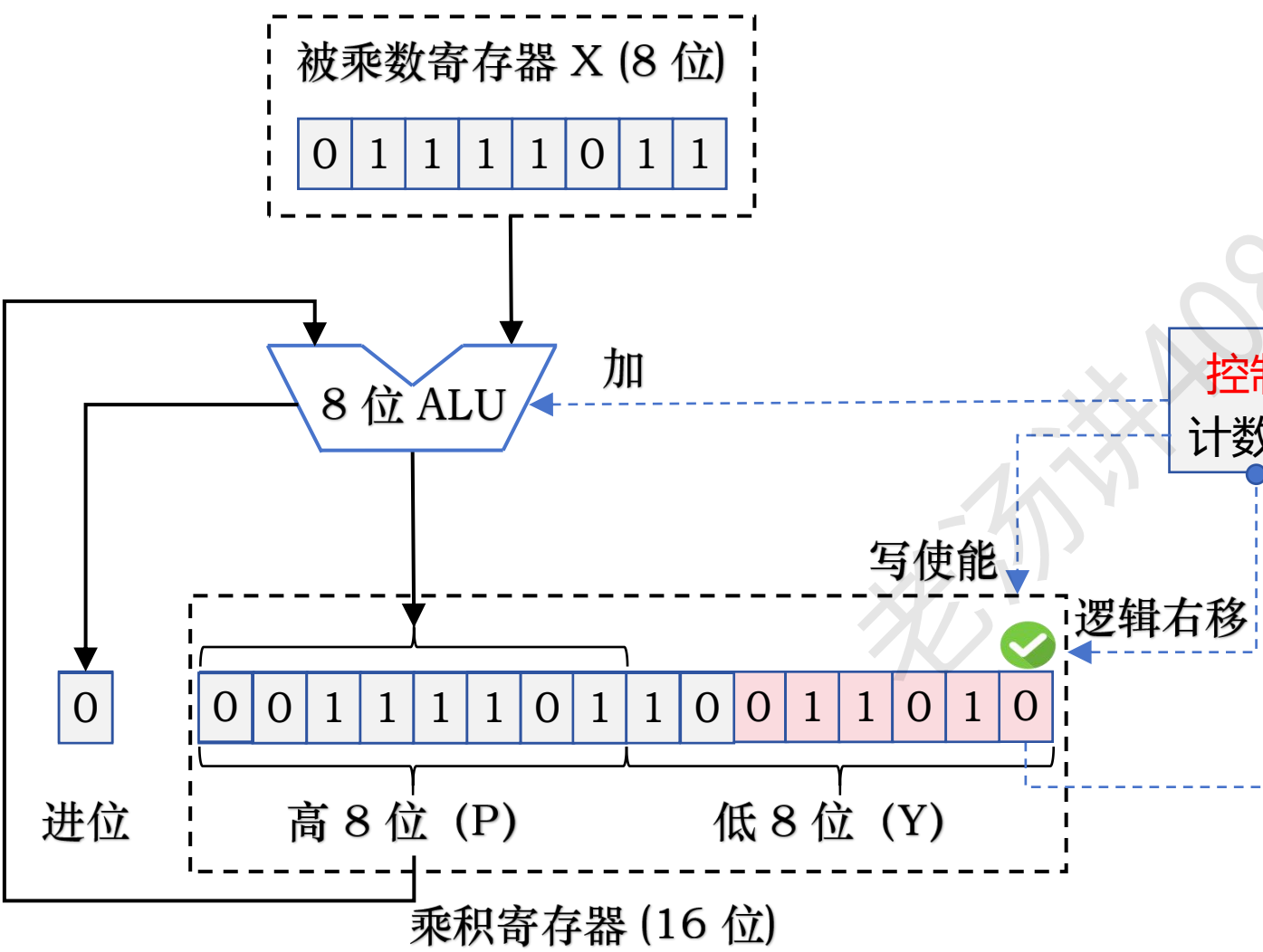


乘积：13038

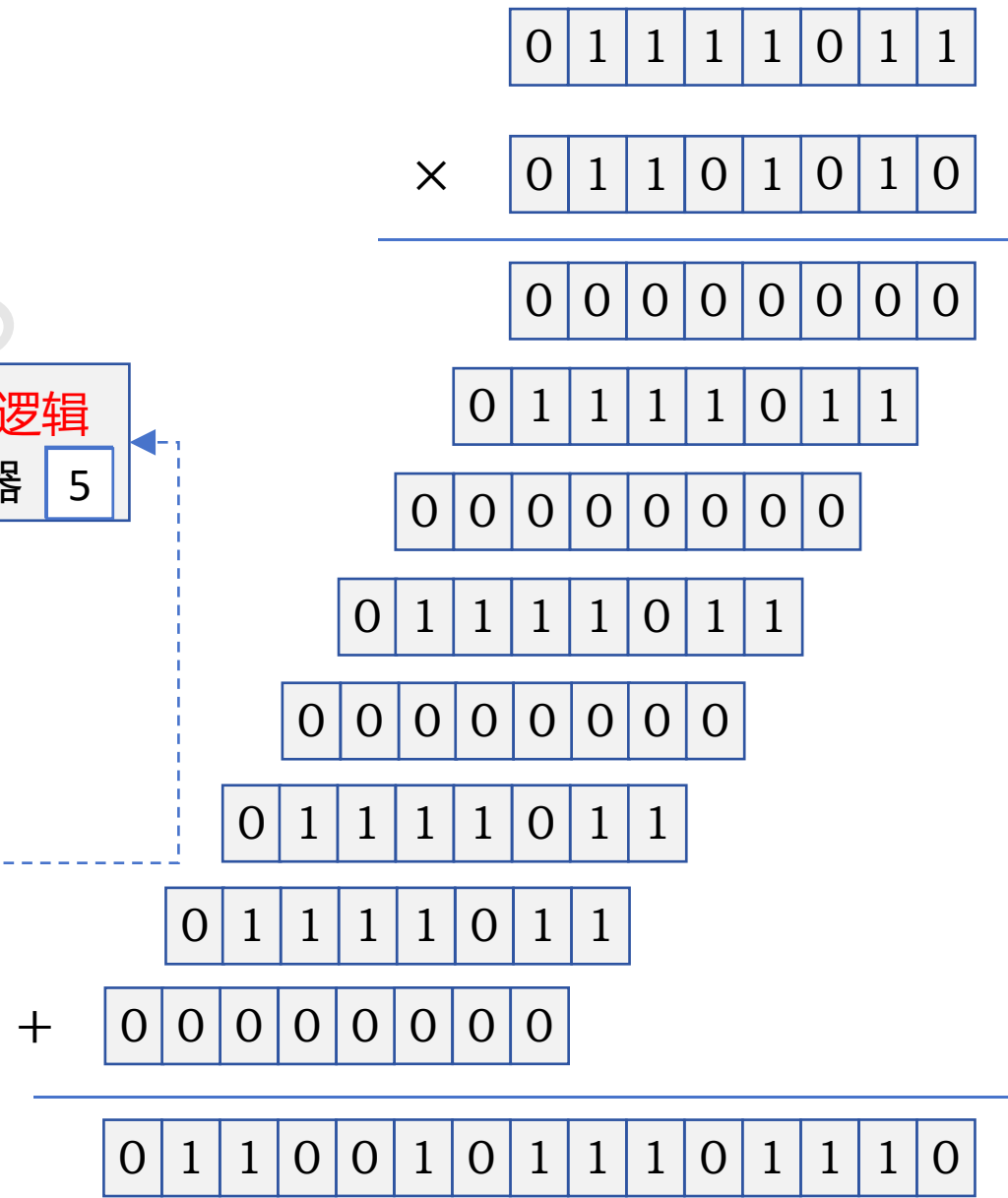


定点数乘法运算：无符号整数乘法电路硬件（优化）

123 × 106

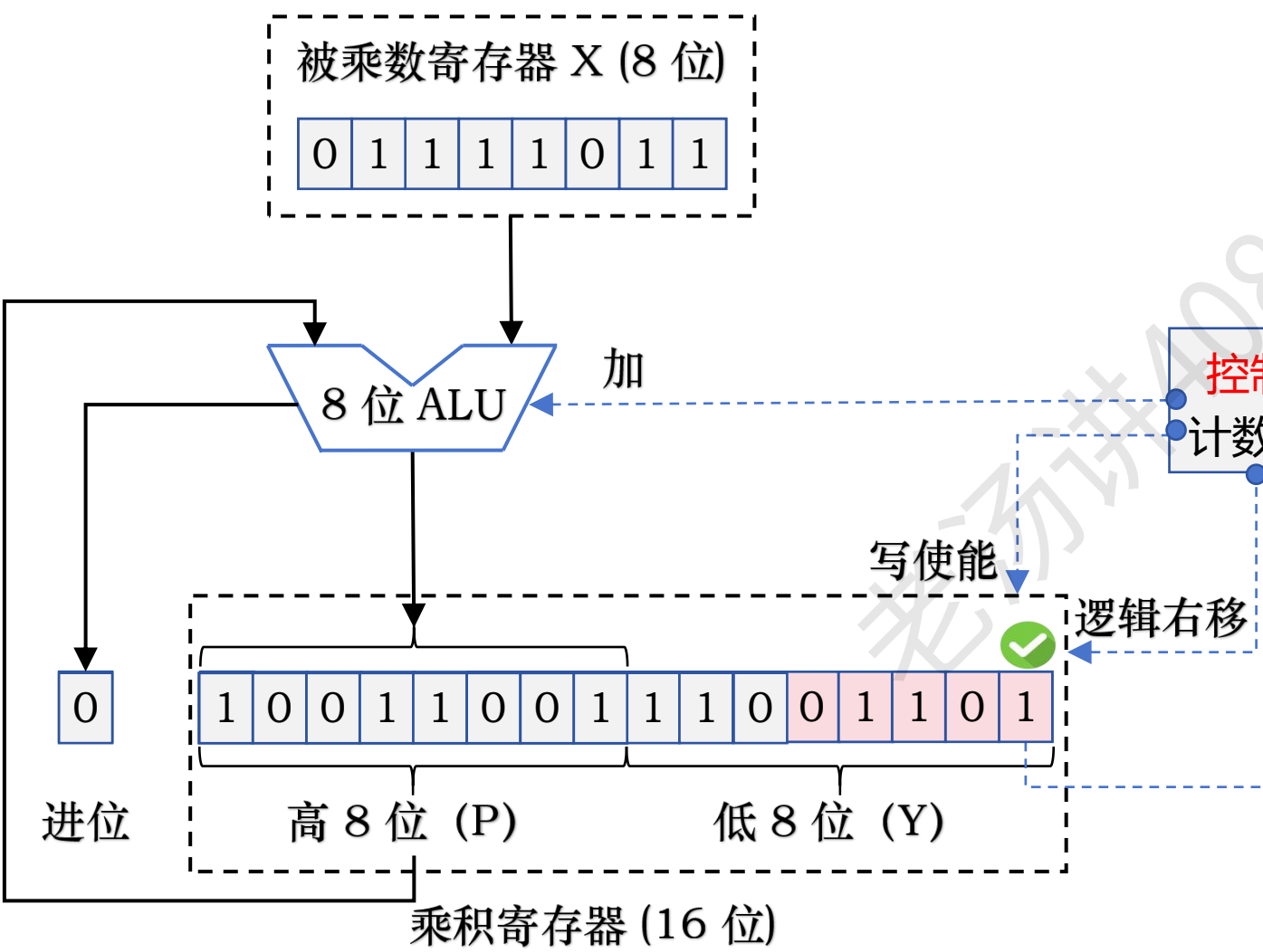


乘积：13038

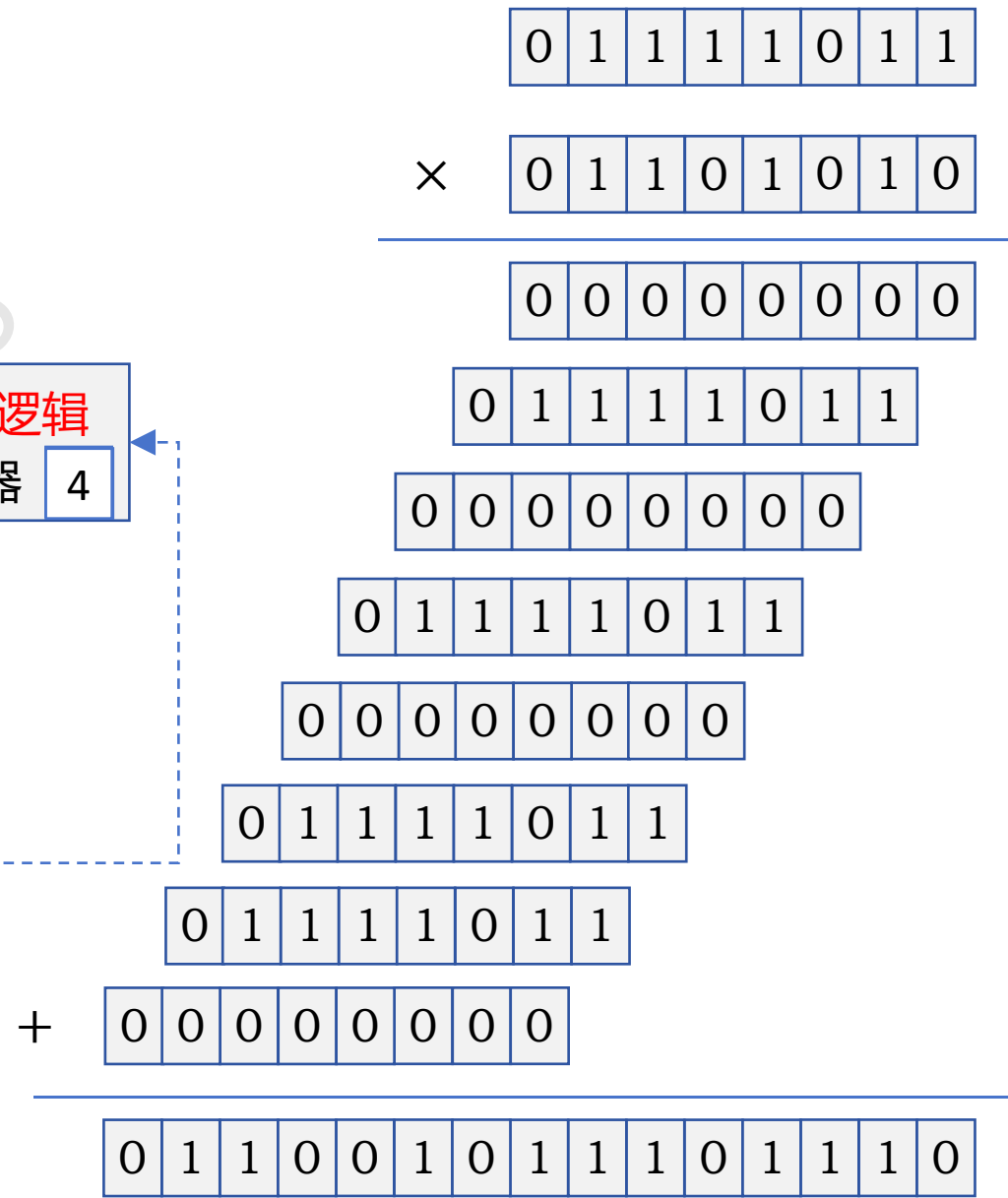


定点数乘法运算：无符号整数乘法电路硬件（优化）

123 × 106

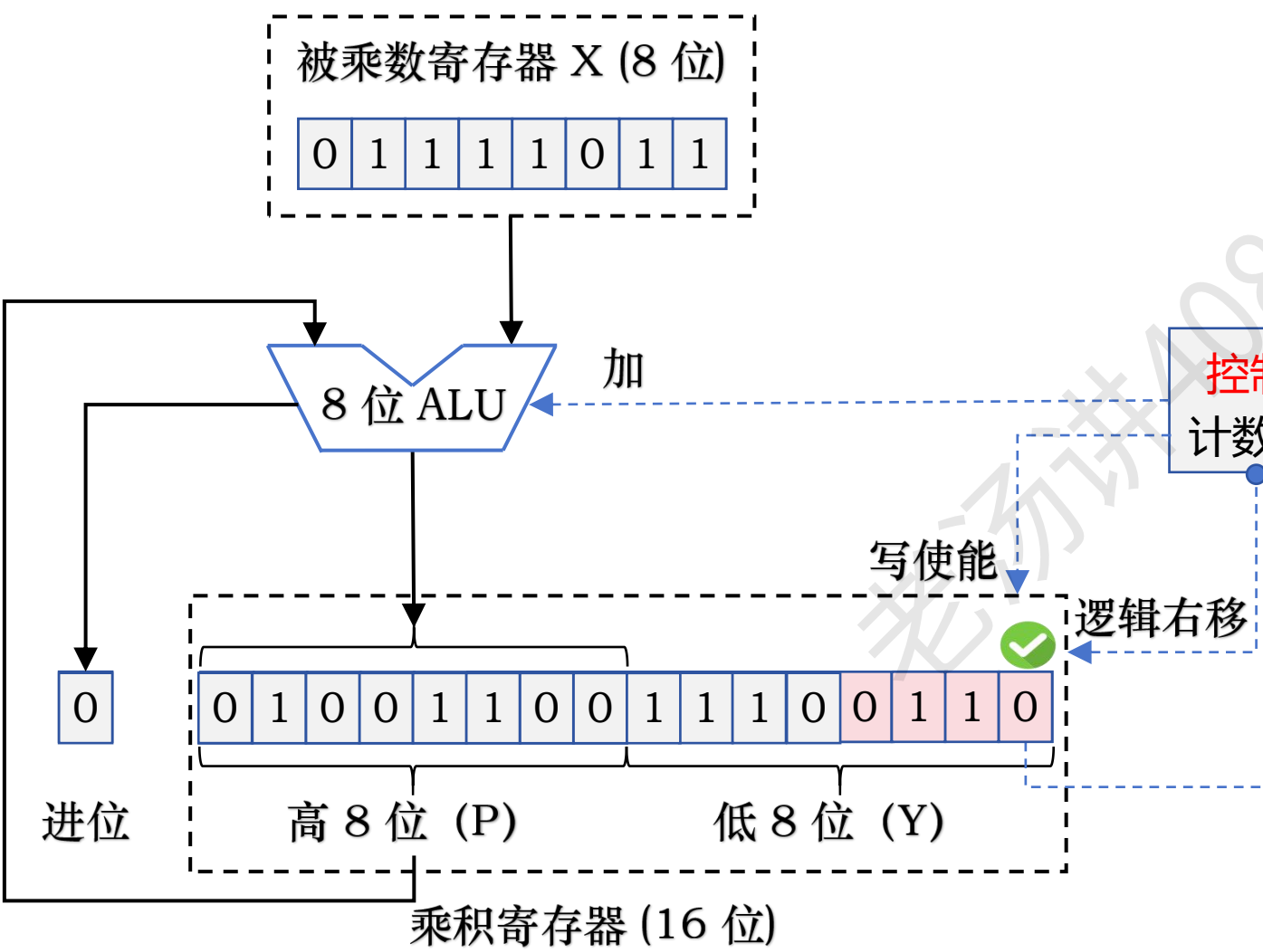


乘积：13038

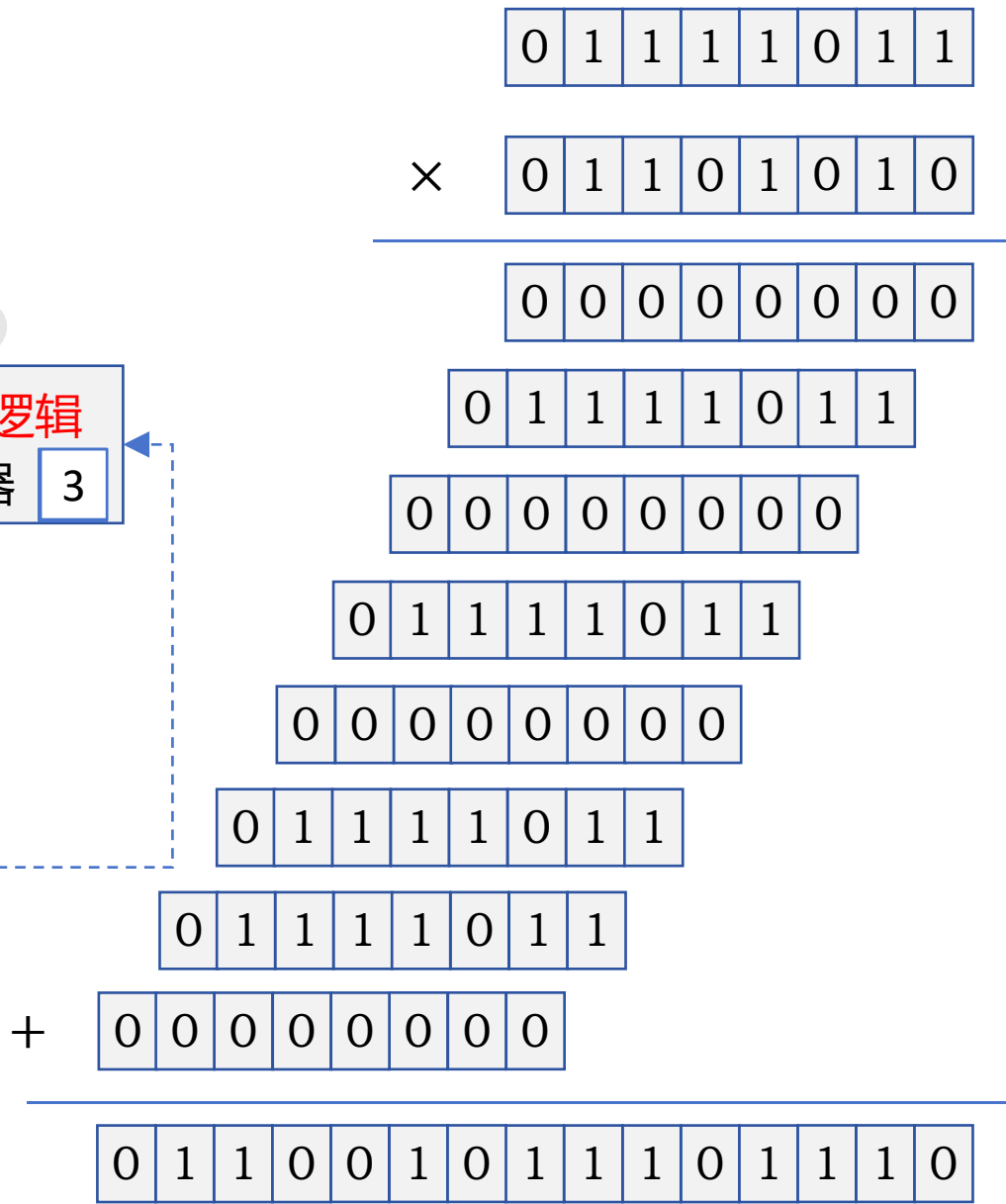


定点数乘法运算：无符号整数乘法电路硬件（优化）

123 × 106

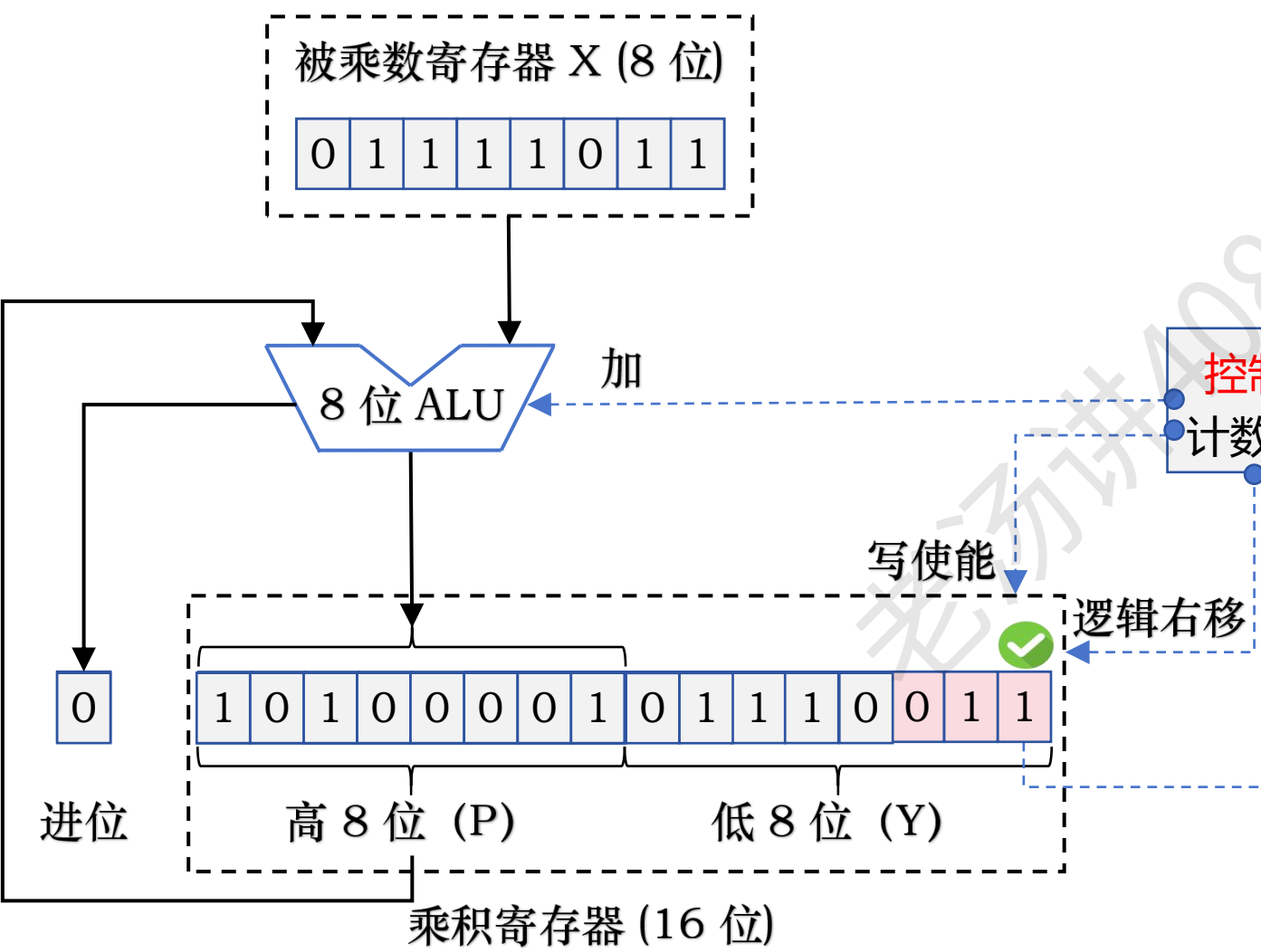


乘积：13038

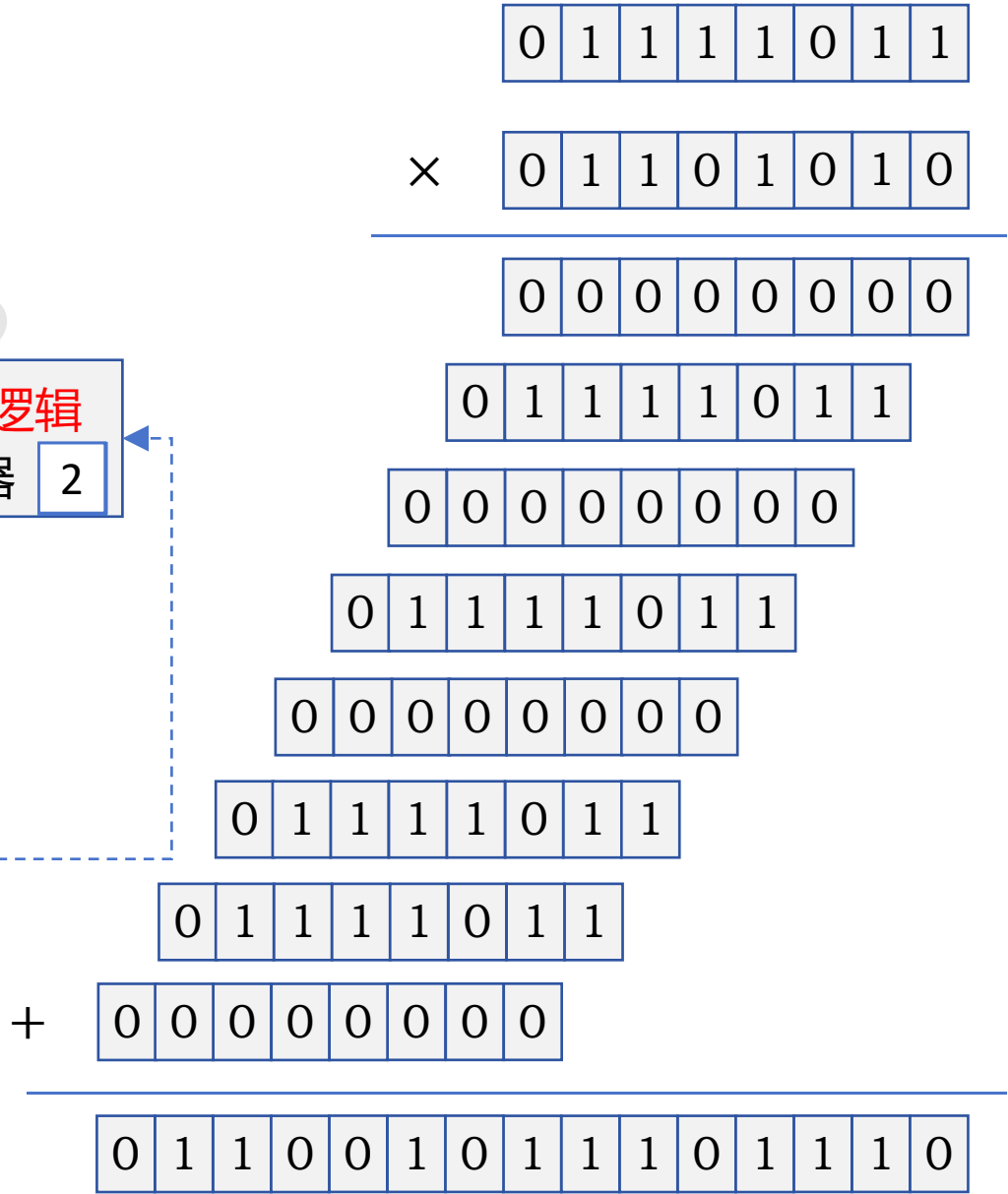


定点数乘法运算：无符号整数乘法电路硬件（优化）

123 × 106

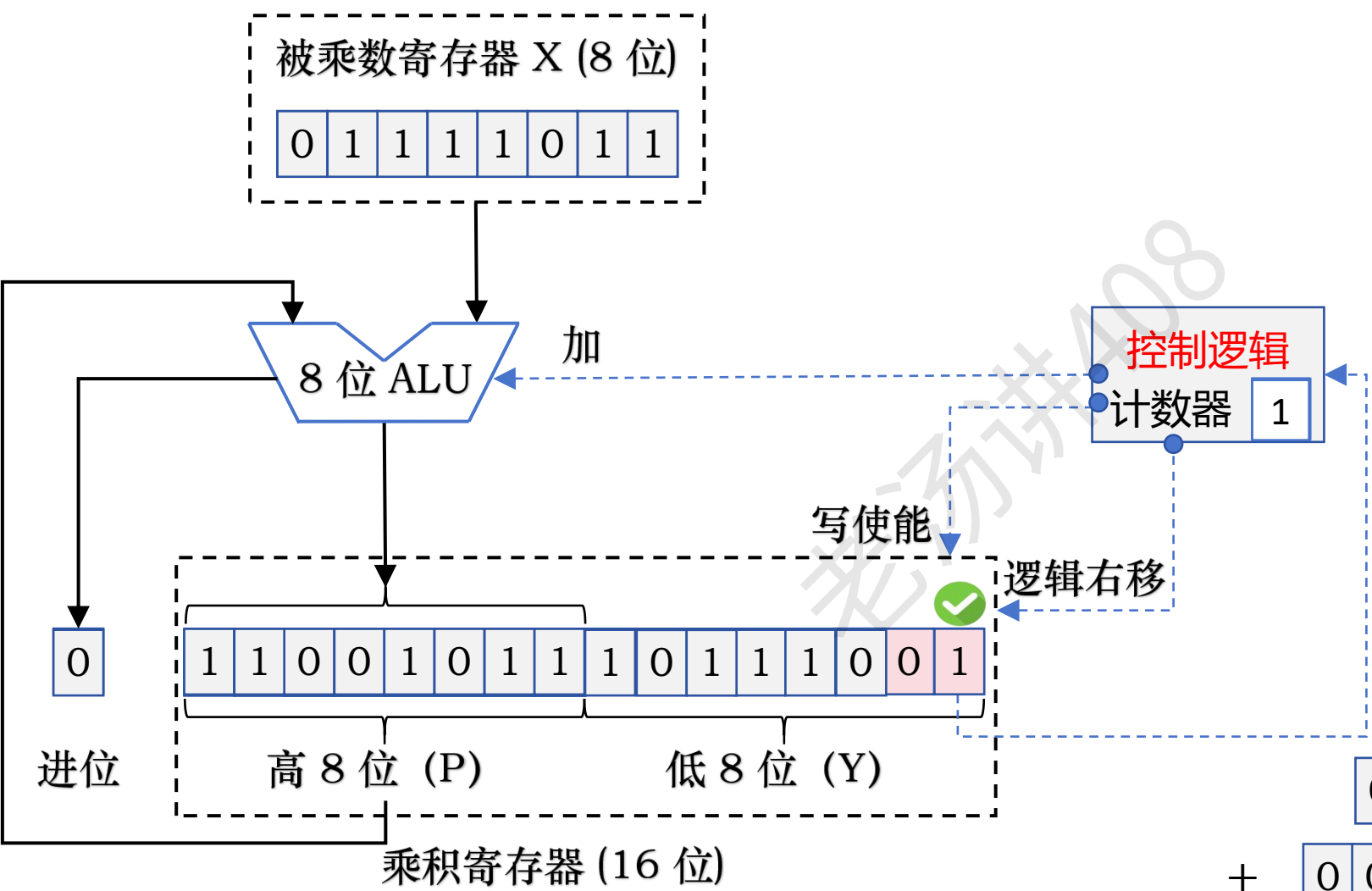


乘积：13038

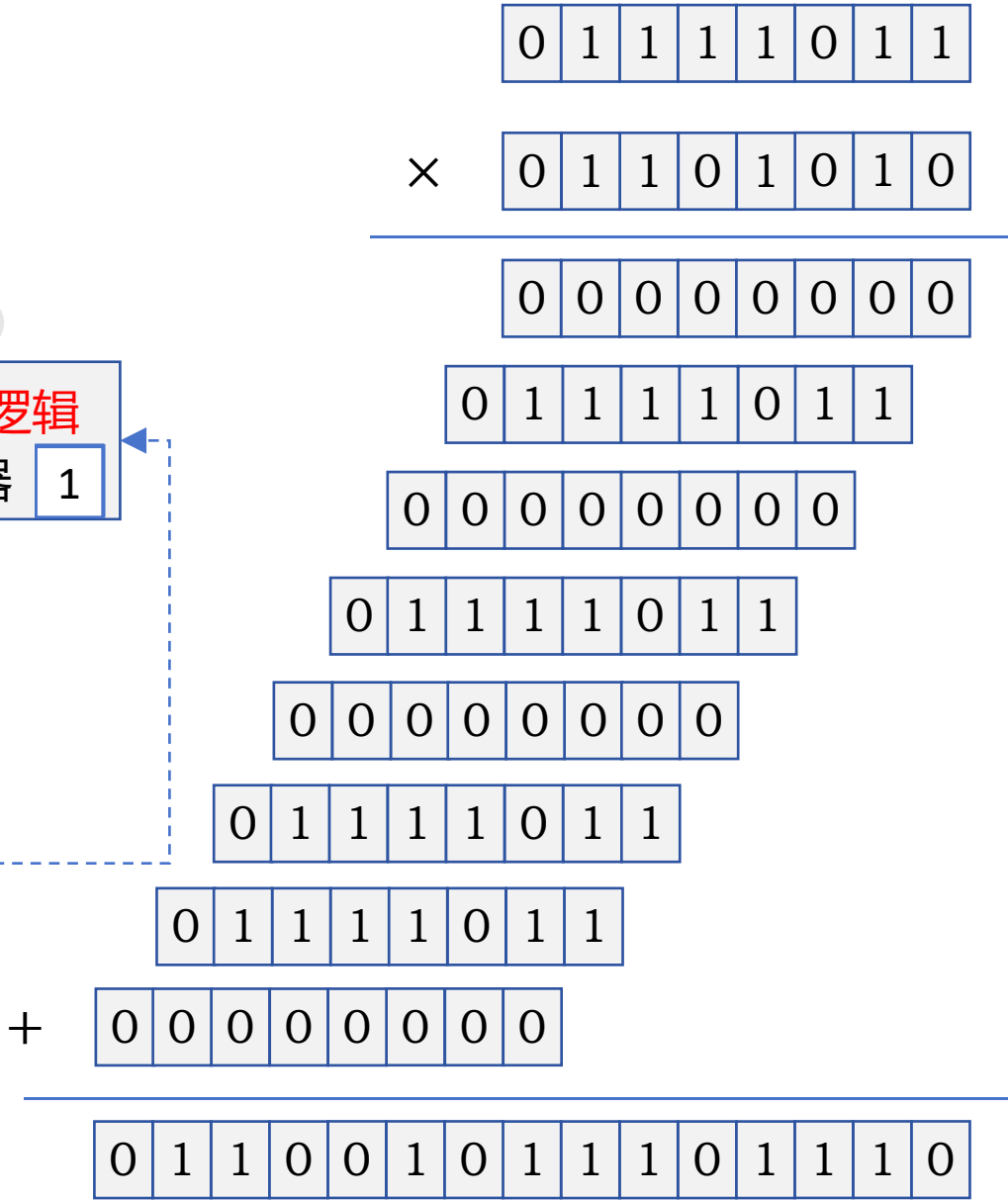


定点数乘法运算：无符号整数乘法电路硬件（优化）

123 × 106

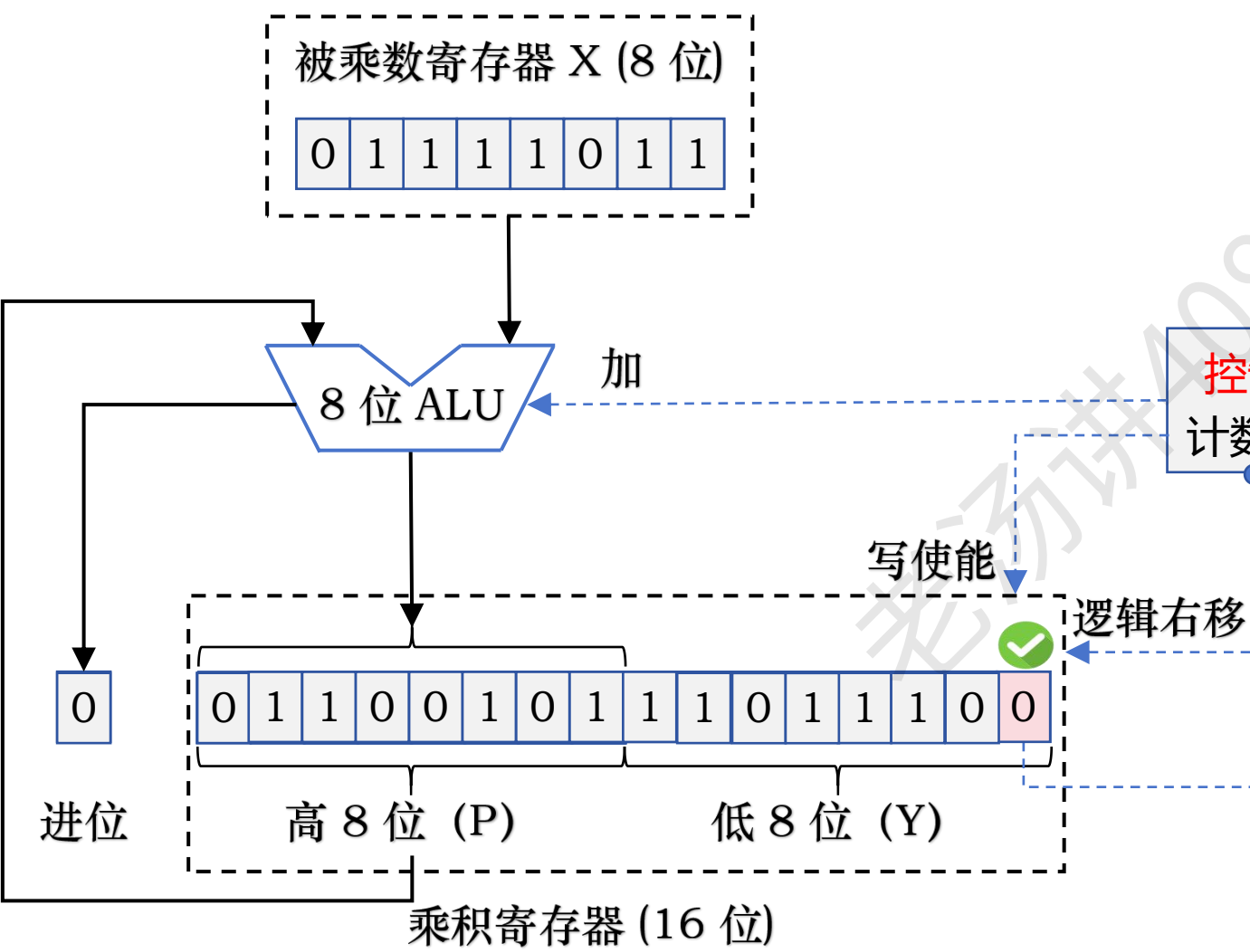


乘积：13038

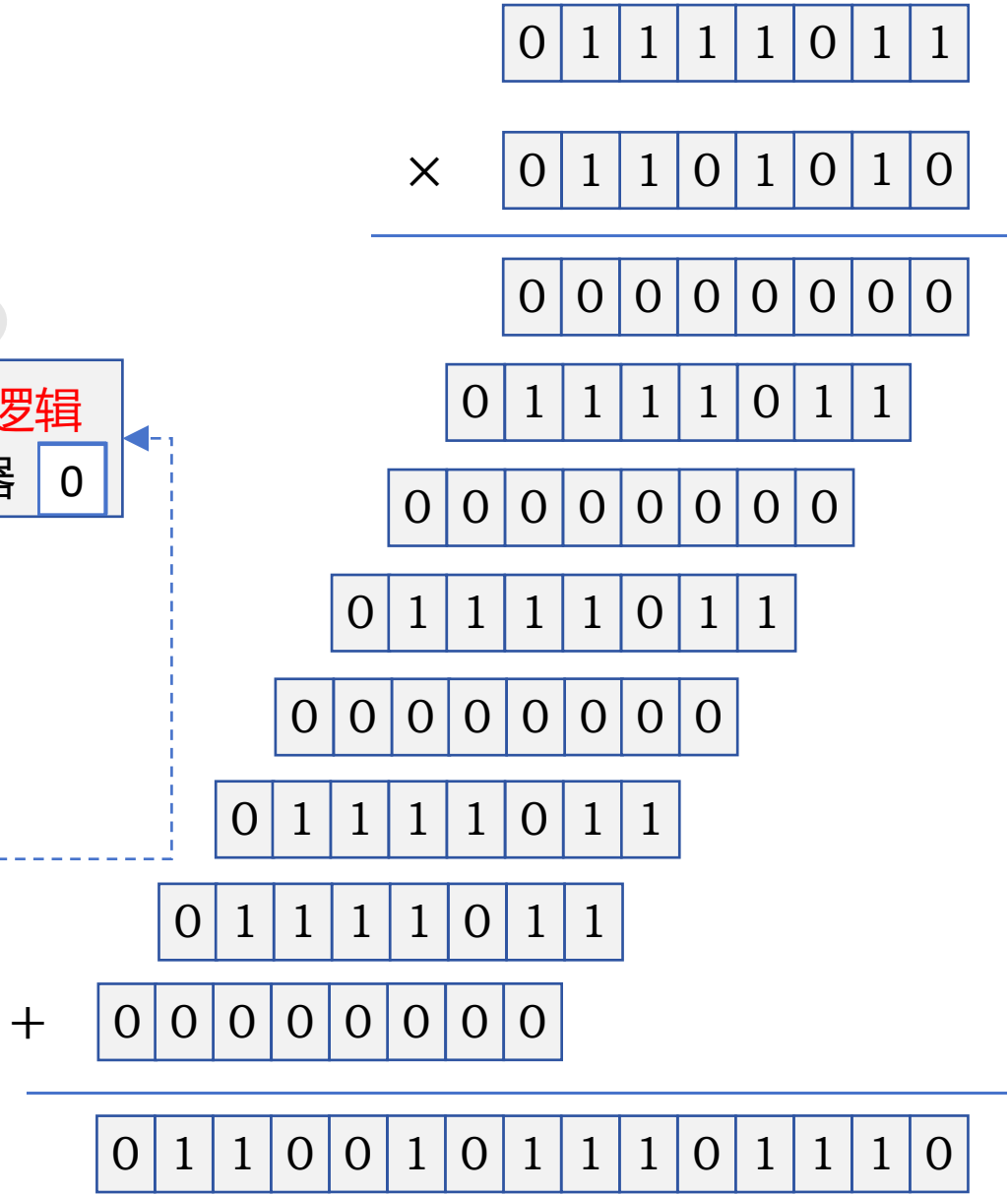


定点数乘法运算：无符号整数乘法电路硬件（优化）

123 × 106

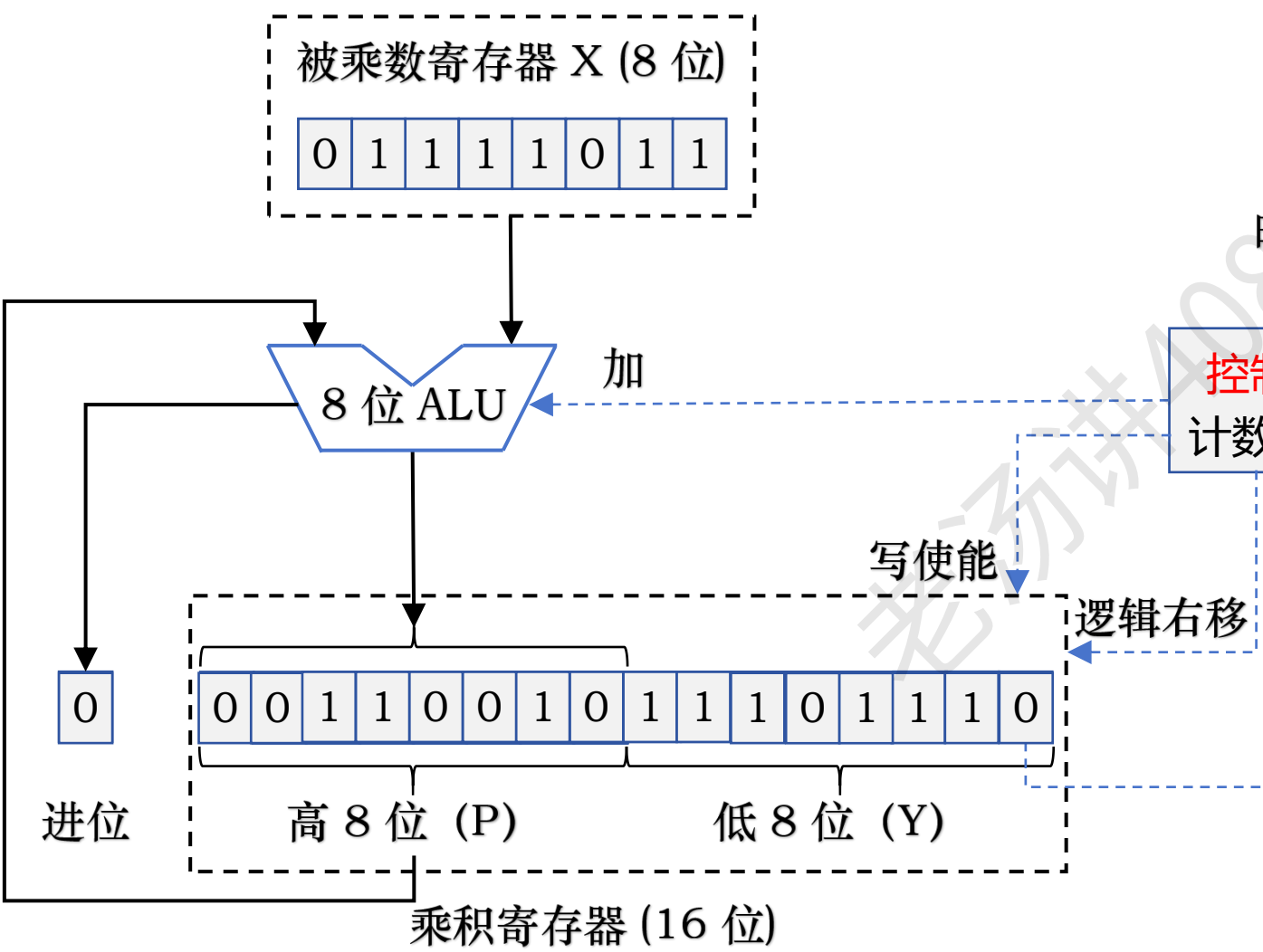


乘积：13038

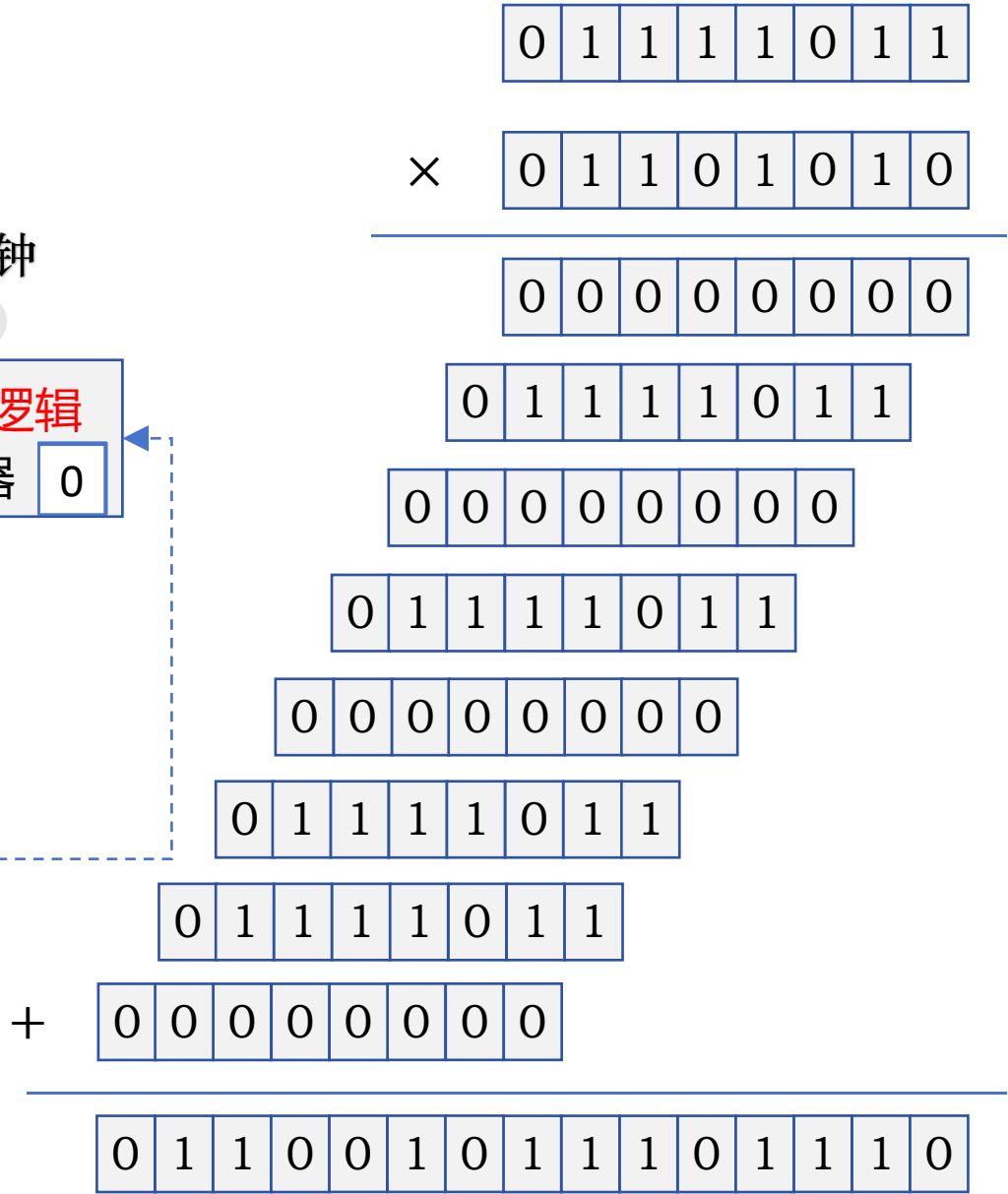


定点数乘法运算：无符号整数乘法电路硬件（优化）

123 × 106

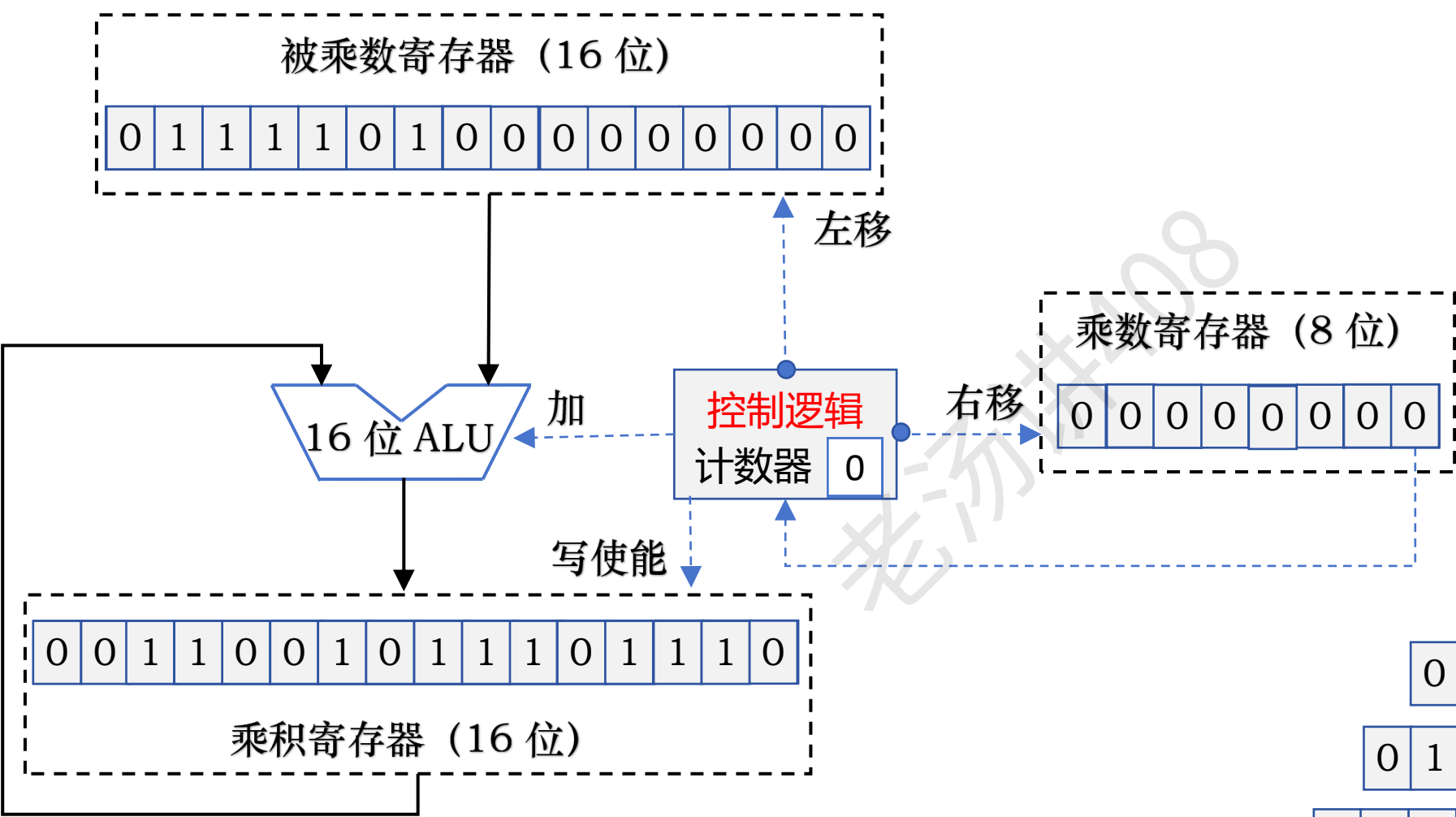


乘积：13038



定点数乘法运算：无符号整数乘法电路硬件

123 × 106



0 1 1 1 1 0 1 1

× 0 1 1 0 1 0 1 0

0 0 0 0 0 0 0 0

0 1 1 1 1 0 1 1

0 0 0 0 0 0 0 0

0 1 1 1 1 0 1 1

0 0 0 0 0 0 0 0

0 1 1 1 1 0 1 1

0 1 1 1 1 0 1 1

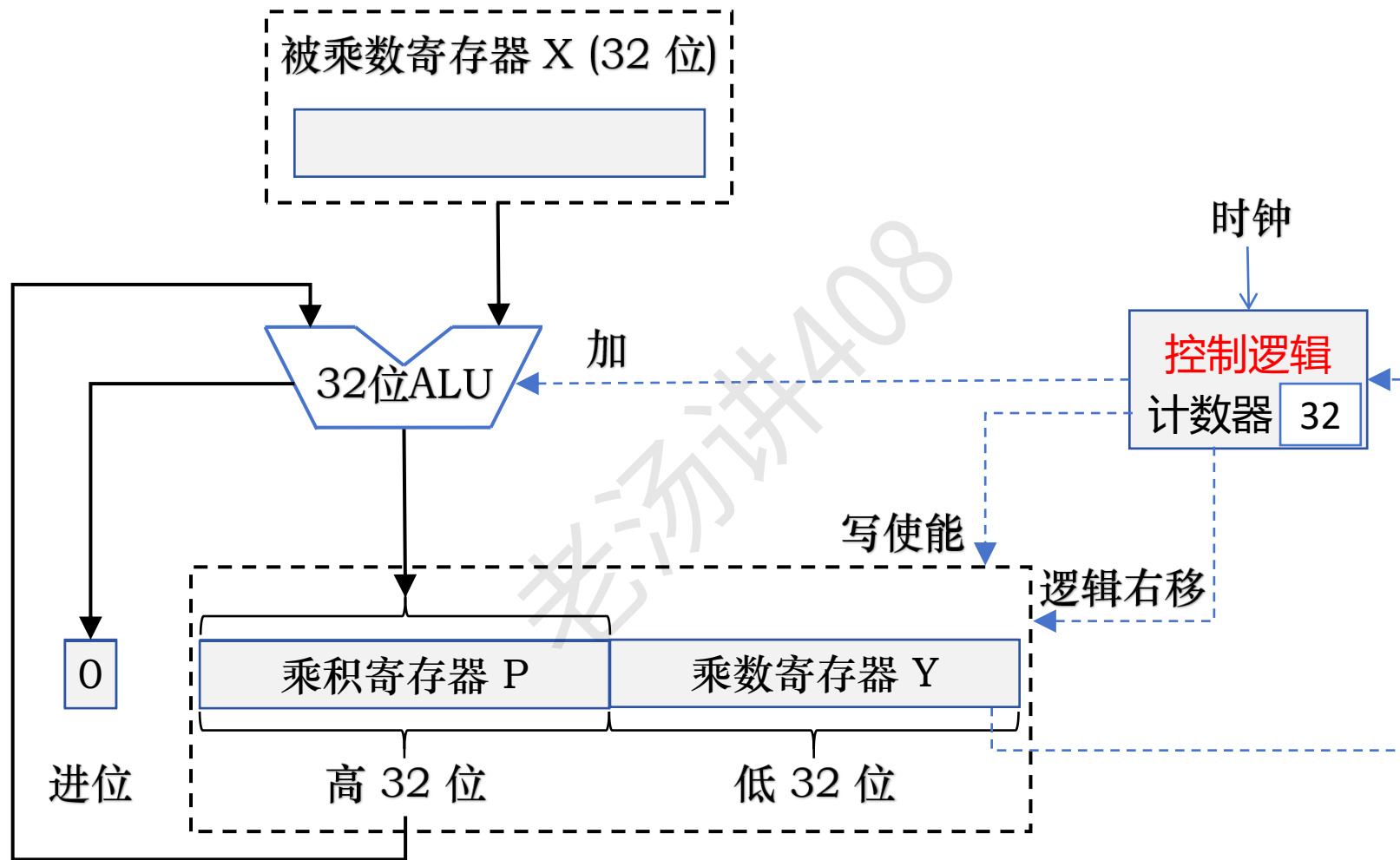
0 1 1 1 1 0 1 1

+ 0 0 0 0 0 0 0 0

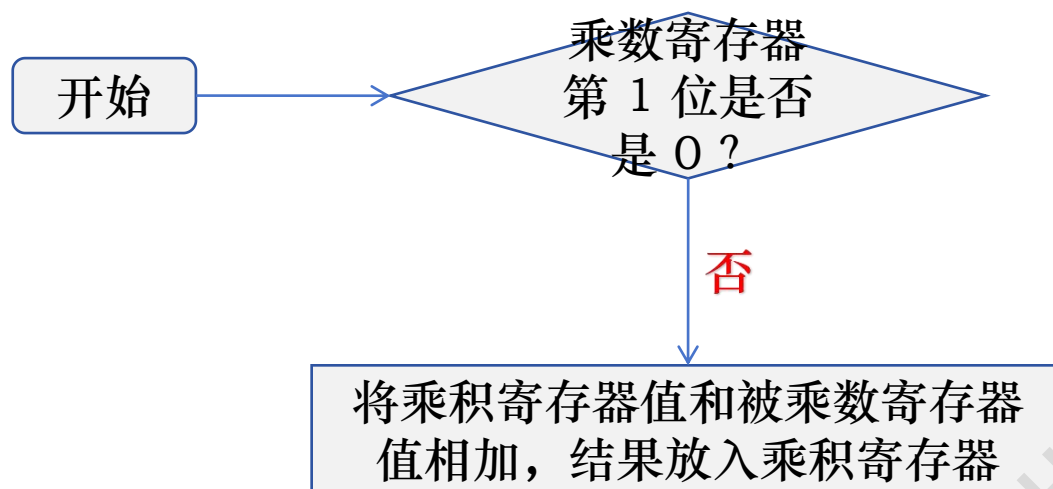
0 1 1 0 0 1 0 1 1 1 0 1 1 1 0

乘积：13038

定点数乘法运算：无符号整数乘法电路硬件（优化）



定点数乘法运算：无符号整数乘法电路硬件（优化）

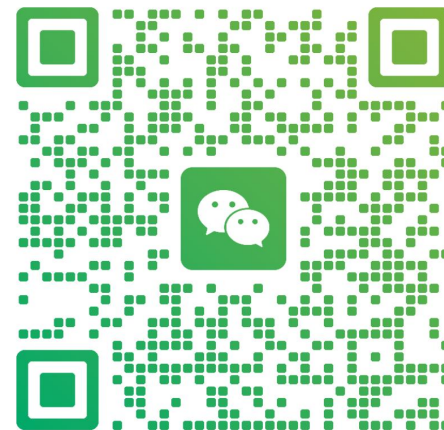


加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



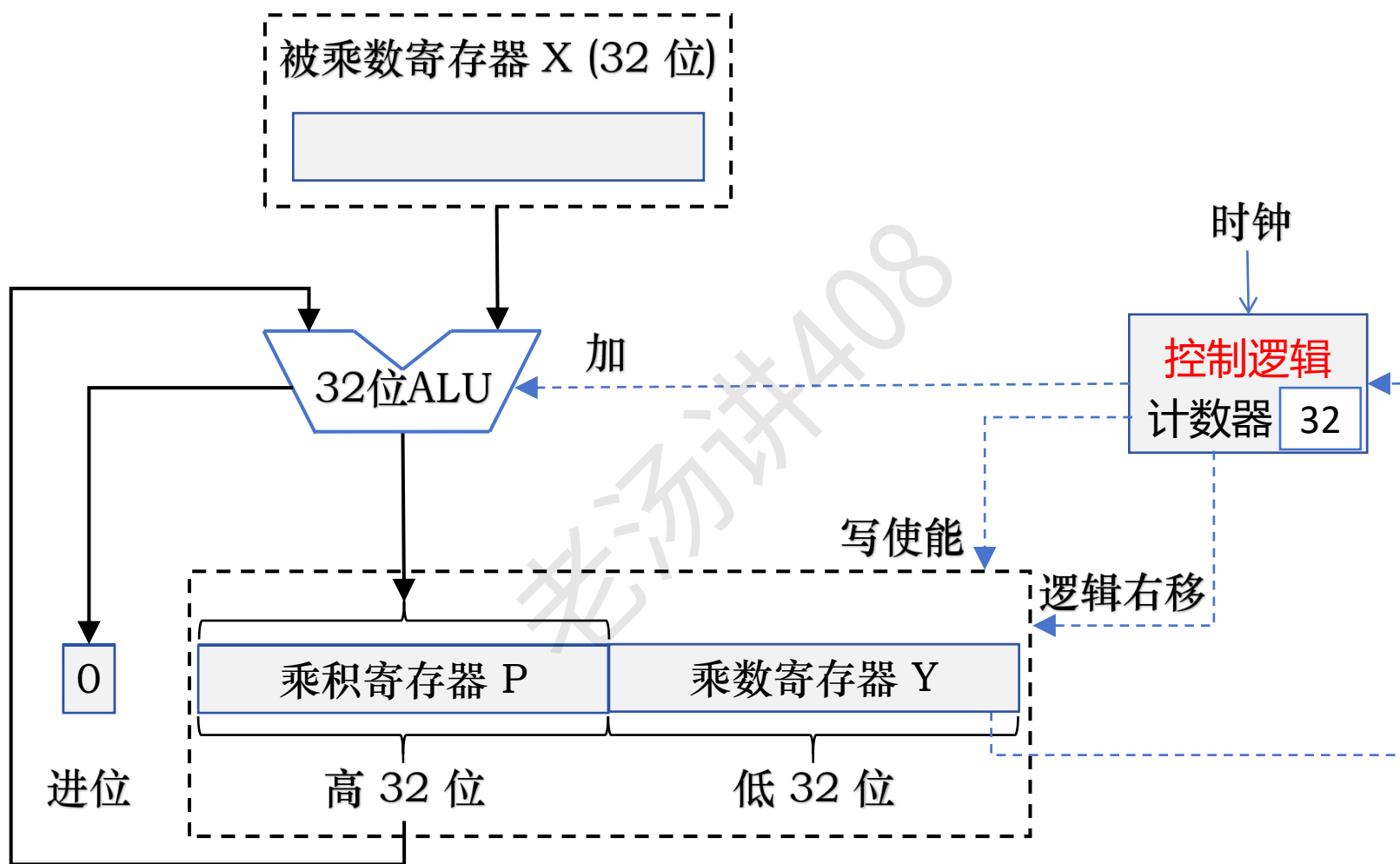
老汤

浙江 杭州

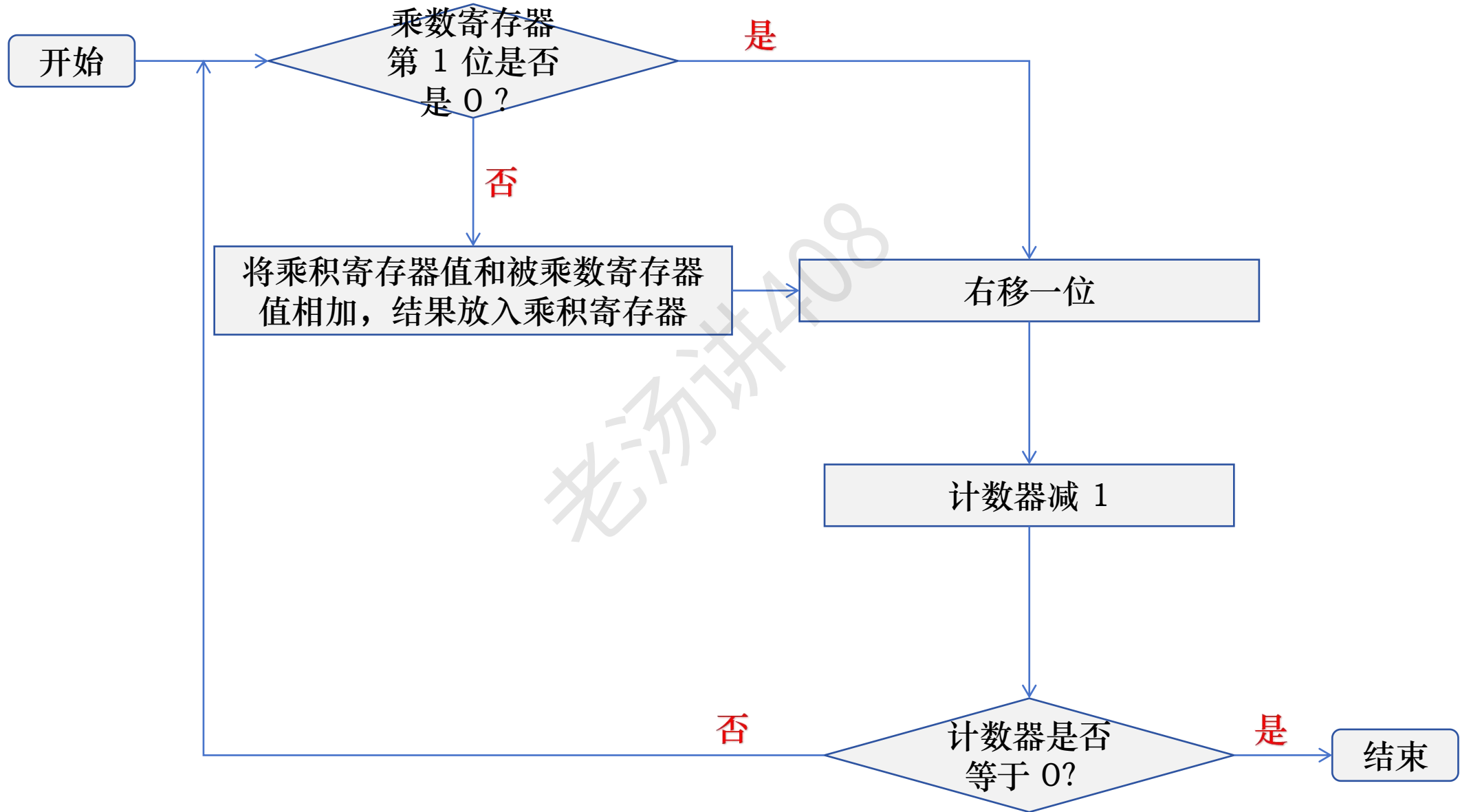


扫一扫上面的二维码图案，加我为朋友。

定点数乘法运算：无符号整数乘法电路硬件（优化 1）



定点数乘法运算：无符号整数乘法电路硬件（优化）



定点数乘法运算：有符号整数乘法（原码一位乘法）

被乘数 $\times$ 乘数 = 乘积

正数 $\times$ 正数 = 正数

负数 $\times$ 负数 = 正数

正数 $\times$ 负数 = 负数

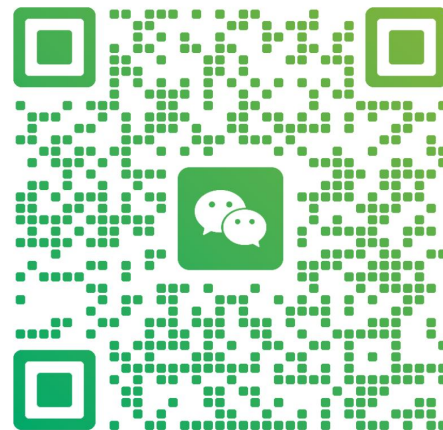
负数 $\times$ 正数 = 负数

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

-123 × 106

定点数乘法运算：有符号整数乘法（原码一位乘法）

被乘数：-123

1	2	3
---	---	---

乘数：106

×	1	0	6
---	---	---	---

	7	3	8
--	---	---	---

	0	0	0
--	---	---	---

+	1	2	3
---	---	---	---

乘积：-13038

1	3	0	3	8
---	---	---	---	---

-123 的原码

1	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---



0	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---

0	1	1	0	1	0	1	0
---	---	---	---	---	---	---	---



×	0	1	1	0	1	0	1	0
---	---	---	---	---	---	---	---	---

106 的原码

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

0	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

0	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

0	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---

+	0	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---

乘积：-13038

1	0	1	1	0	0	1	0	1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

定点数乘法运算：有符号整数乘法（原码一位乘法）

当用原码实现乘法运算时，符号位和数值位分开计算

① 确定乘积的符号位，由两个乘数的符号异或得到

正数 $\times$ 正数 = 正数

$$0 \wedge 0 = 0$$

负数 $\times$ 负数 = 正数

$$1 \wedge 1 = 0$$

正数 $\times$ 负数 = 负数

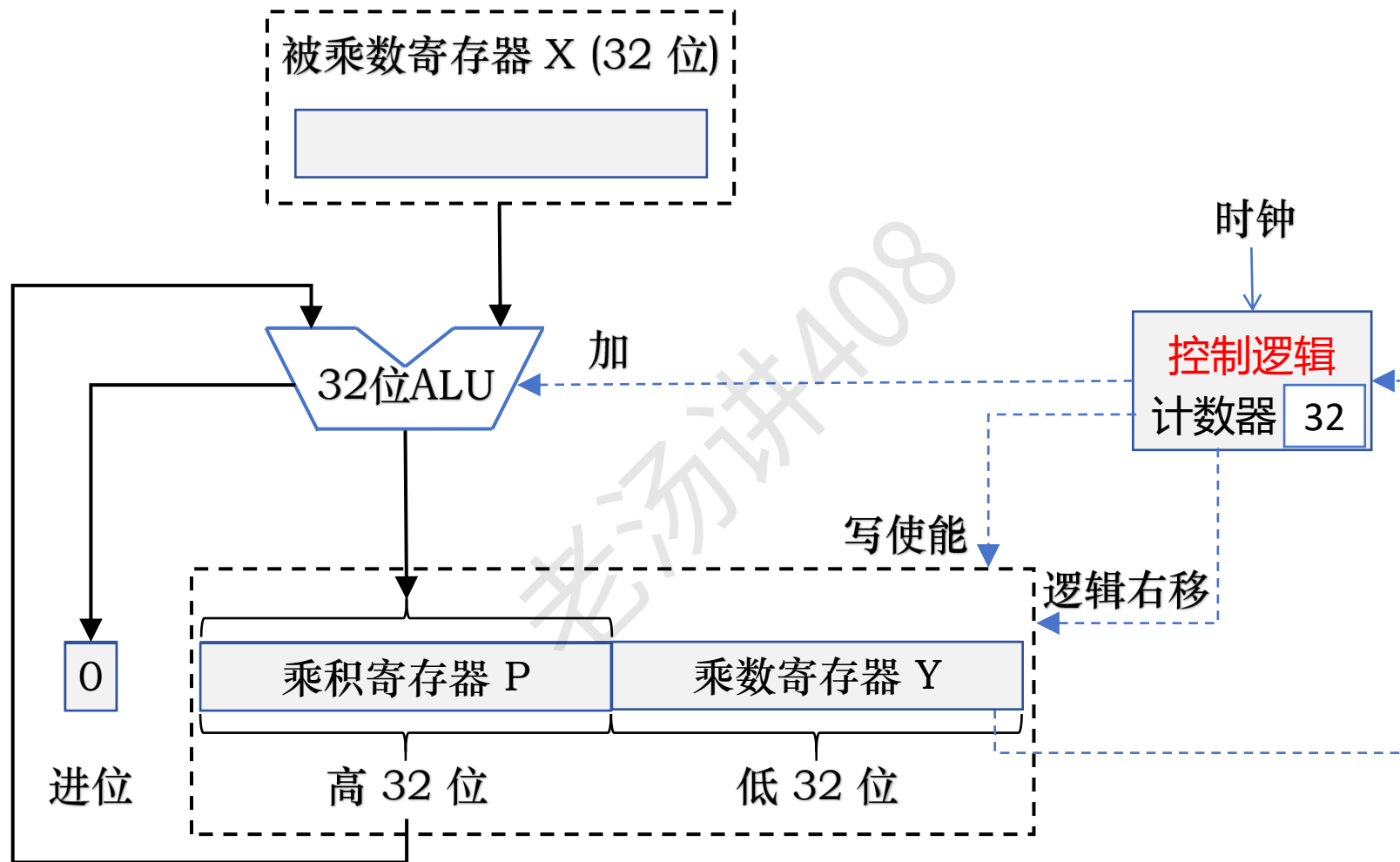
$$0 \wedge 1 = 1$$

负数 $\times$ 正数 = 负数

$$1 \wedge 0 = 1$$

② 计算乘积的数值位，乘积的数值位等于两个乘数的数据部分之积

定点数乘法运算：有符号整数乘法（原码一位乘法）



定点数乘法运算：有符号整数乘法（原码两位乘法）

$$\begin{array}{r} \begin{array}{|c|c|c|c|} \hline 1 & 0 & 1 & 1 \\ \hline \end{array} \\ \times \quad \begin{array}{|c|c|} \hline 1 & 0 \\ \hline \end{array} \\ \hline \begin{array}{|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & \\ \hline \end{array} \\ + \begin{array}{|c|c|c|c|} \hline 1 & 0 & 1 & 1 \\ \hline \end{array} \\ \hline \begin{array}{|c|c|c|c|c|} \hline 1 & 0 & 1 & 1 & 0 \\ \hline \end{array} \end{array}$$

判断

00

01 → 部分积 + 被乘数

10 → 部分积 + 2 * 被乘数

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



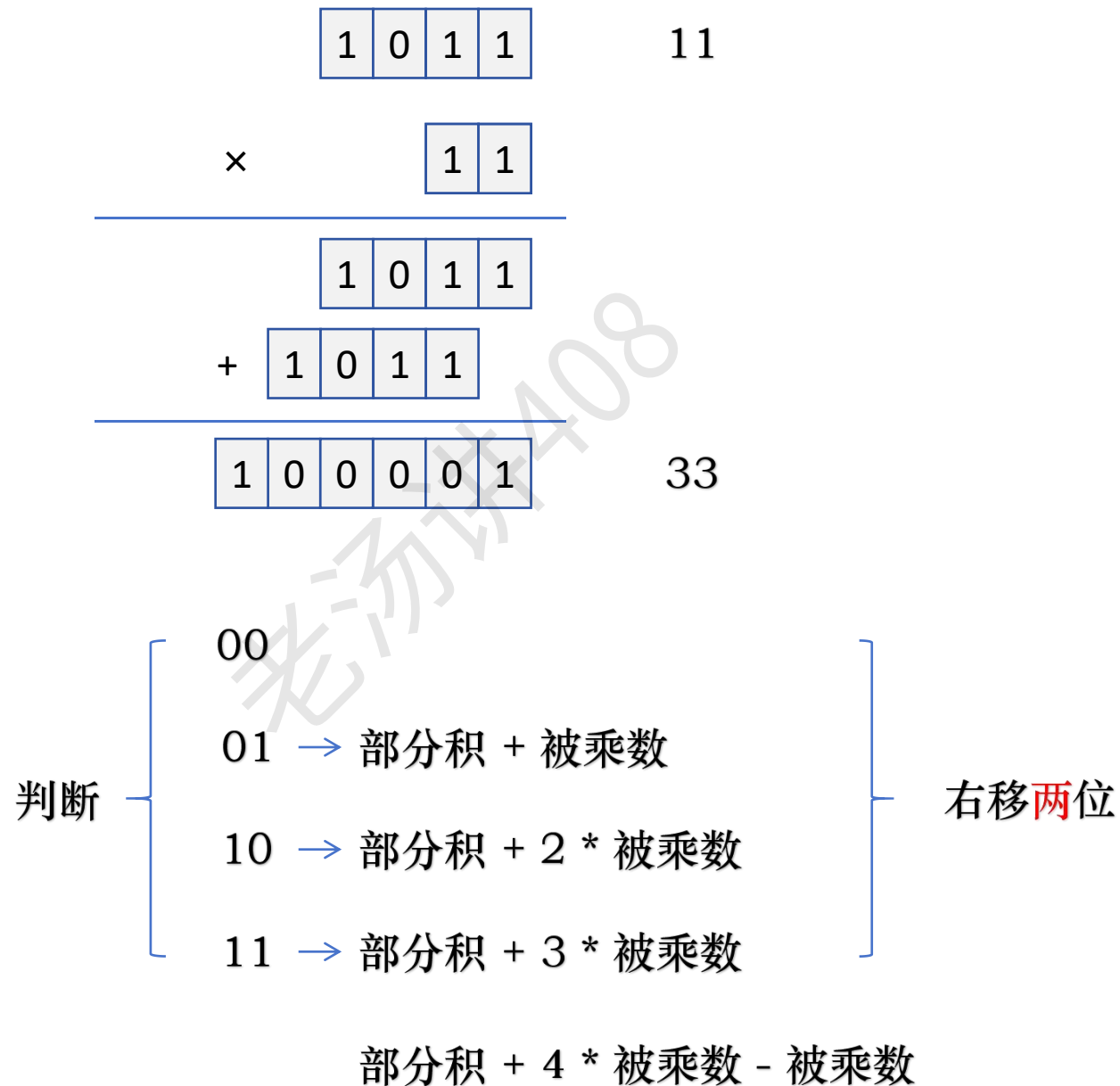
老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

定点数乘法运算：有符号整数乘法（原码两位乘法）



定点数乘法运算：有符号整数乘法（原码两位乘法）

[illegible]

定点数乘法运算：补码一位乘法

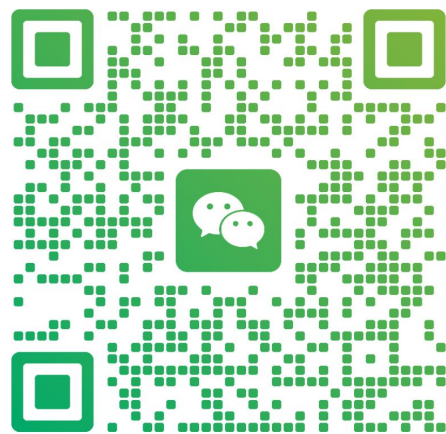
A.D.Booth 提出了一种补码相乘算法，可以将符号位与数值位合在一起参与运算，直接得出用补码表示的乘积，且正数和负数同等对待。这种算法被称为布斯 (Booth) 乘法

加老汤微信，帮你 408 冲 120+



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

定点数乘法运算：补码一位乘法

$$[X]_{\text{补}} = X_7 X_6 X_5 X_4 X_3 X_2 X_1 X_0$$

$$[Y]_{\text{补}} = Y_7 Y_6 Y_5 Y_4 Y_3 Y_2 Y_1 Y_0$$

求： $[X \times Y]_{\text{补}}$

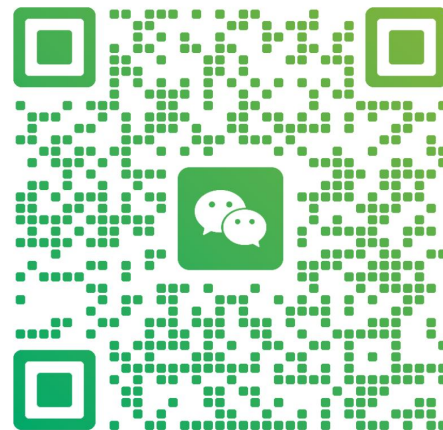
$$Y \text{ 的真值} = -Y_7 2^7 + \sum_{i=0}^6 Y_i 2^i$$

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

有符号整数：什么是补码编码？

$$\text{无符号整数: } 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = 11$$

$$\text{无符号整数: } \underline{x_{n-1} * 2^{n-1}} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = \sum_{i=0}^{n-1} x_i * 2^i$$



无符号编码

n 位二进制: $x_{n-1}x_{n-2}\dots x_2x_1x_0$ 1011



补码编码

最高位的权重变成负数

$$\text{有符号整数: } \underline{-x_{n-1} * 2^{n-1}} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$$

$$\text{有符号整数: } -1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = -5$$

定点数乘法运算：补码一位乘法

$$[X]_{\text{补}} = X_7 X_6 X_5 X_4 X_3 X_2 X_1 X_0$$

$$[Y]_{\text{补}} = Y_7 Y_6 Y_5 Y_4 Y_3 Y_2 Y_1 Y_0$$

$$\text{求: } [X \times Y]_{\text{补}}$$

$$Y \text{ 的真值} = -Y_7 2^7 + \sum_{i=0}^6 Y_i 2^i$$

$$= -Y_7 2^7 + Y_6 2^6 + Y_5 2^5 + Y_4 2^4 + Y_3 2^3 + Y_2 2^2 + Y_1 2^1 + Y_0 2^0$$

$$= -Y_7 2^7 + Y_6 2^7 - Y_6 2^6 + Y_5 2^6 - Y_5 2^5 + Y_4 2^5 - Y_4 2^4 + Y_3 2^4 - Y_3 2^3 + Y_2 2^3 - Y_2 2^2 + Y_1 2^2 - Y_1 2^1 + Y_0 2^1 - Y_0 2^0$$

$$= (Y_6 - Y_7) 2^7 + (Y_5 - Y_6) 2^6 + (Y_4 - Y_5) 2^5 + (Y_3 - Y_4) 2^4 + (Y_2 - Y_3) 2^3 + (Y_1 - Y_2) 2^2 + (Y_0 - Y_1) 2^1 + (Y_{-1} - Y_0) 2^0 \quad Y_{-1} = 0$$

$$= \sum_{i=0}^7 (Y_{i-1} - Y_i) 2^i$$

$$\begin{aligned} [X \times Y]_{\text{补}} &= [X \times \sum_{i=0}^7 (Y_{i-1} - Y_i) 2^i]_{\text{补}} = [X \times \sum_{i=0}^7 (Y_{i-1} - Y_i) 2^i]_{\text{补}} * 2^{-8} * 2^8 \\ &= [X \times \sum_{i=0}^7 (Y_{i-1} - Y_i) 2^{-(8-i)}]_{\text{补}} * 2^8 \end{aligned}$$

定点数乘法运算：补码一位乘法

$$[X]_{\text{补}} = X_7X_6X_5X_4X_3X_2X_1X_0$$

$$[Y]_{\text{补}} = Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$$

$$\text{求}:[X \times Y]_{\text{补}}$$

$$[X \times Y]_{\text{补}} = [X \times \sum_{i=0}^7 (Y_{i-1} - Y_i)2^{-(8-i)}]_{\text{补}} * 2^8$$

$$= [X(Y_{-1} - Y_0)2^{-8} + X(Y_0 - Y_1)2^{-7} + X(Y_1 - Y_2)2^{-6} + X(Y_2 - Y_3)2^{-5} + X(Y_3 - Y_4)2^{-4} + X(Y_4 - Y_5)2^{-3} + X(Y_5 - Y_6)2^{-2} + X(Y_6 - Y_7)2^{-1}]_{\text{补}} * 2^8$$

$$= [(X(Y_{-1} - Y_0)2^{-7} + X(Y_0 - Y_1)2^{-6} + X(Y_1 - Y_2)2^{-5} + X(Y_2 - Y_3)2^{-4} + X(Y_3 - Y_4)2^{-3} + X(Y_4 - Y_5)2^{-2} + X(Y_5 - Y_6)2^{-1} + X(Y_6 - Y_7))2^{-1}]_{\text{补}} * 2^8$$

$$= [(((X(Y_{-1} - Y_0)2^{-6} + X(Y_0 - Y_1)2^{-5} + X(Y_1 - Y_2)2^{-4} + X(Y_2 - Y_3)2^{-3} + X(Y_3 - Y_4)2^{-2} + X(Y_4 - Y_5)2^{-1} + X(Y_5 - Y_6)))2^{-1} + X(Y_6 - Y_7))2^{-1}]_{\text{补}} * 2^8$$

$$= [((((X(Y_{-1} - Y_0)2^{-5} + X(Y_0 - Y_1)2^{-4} + X(Y_1 - Y_2)2^{-3} + X(Y_2 - Y_3)2^{-2} + X(Y_3 - Y_4)2^{-1} + X(Y_4 - Y_5)))2^{-1} + X(Y_5 - Y_6)))2^{-1} + X(Y_6 - Y_7))2^{-1}]_{\text{补}} * 2^8$$

$$= [((((((X(Y_{-1} - Y_0)2^{-4} + X(Y_0 - Y_1)2^{-3} + X(Y_1 - Y_2)2^{-2} + X(Y_2 - Y_3)2^{-1} + X(Y_3 - Y_4)))2^{-1} + X(Y_4 - Y_5)))2^{-1} + X(Y_5 - Y_6)))2^{-1} + X(Y_6 - Y_7))2^{-1}]_{\text{补}} * 2^8$$

$$= [(((((((X(Y_{-1} - Y_0)2^{-3} + X(Y_0 - Y_1)2^{-2} + X(Y_1 - Y_2)2^{-1} + X(Y_2 - Y_3)))2^{-1} + X(Y_3 - Y_4)))2^{-1} + X(Y_4 - Y_5)))2^{-1} + X(Y_5 - Y_6)))2^{-1} + X(Y_6 - Y_7))2^{-1}]_{\text{补}} * 2^8$$

$$= [((((((((X(Y_{-1} - Y_0)2^{-2} + X(Y_0 - Y_1)2^{-1} + X(Y_1 - Y_2)))2^{-1} + X(Y_2 - Y_3)))2^{-1} + X(Y_3 - Y_4)))2^{-1} + X(Y_4 - Y_5)))2^{-1} + X(Y_5 - Y_6)))2^{-1} + X(Y_6 - Y_7))2^{-1}]_{\text{补}} * 2^8$$

$$= [((((((((((X(Y_{-1} - Y_0)2^{-1} + X(Y_0 - Y_1)))2^{-1} + X(Y_1 - Y_2)))2^{-1} + X(Y_2 - Y_3)))2^{-1} + X(Y_3 - Y_4)))2^{-1} + X(Y_4 - Y_5)))2^{-1} + X(Y_5 - Y_6)))2^{-1} + X(Y_6 - Y_7))2^{-1}]_{\text{补}} * 2^8$$

$$= [(((((((((((0 + X(Y_{-1} - Y_0)))2^{-1} + X(Y_0 - Y_1)))2^{-1} + X(Y_1 - Y_2)))2^{-1} + X(Y_2 - Y_3)))2^{-1} + X(Y_3 - Y_4)))2^{-1} + X(Y_4 - Y_5)))2^{-1} + X(Y_5 - Y_6)))2^{-1} + X(Y_6 - Y_7))2^{-1}]_{\text{补}} * 2^8$$

定点数乘法运算：补码一位乘法

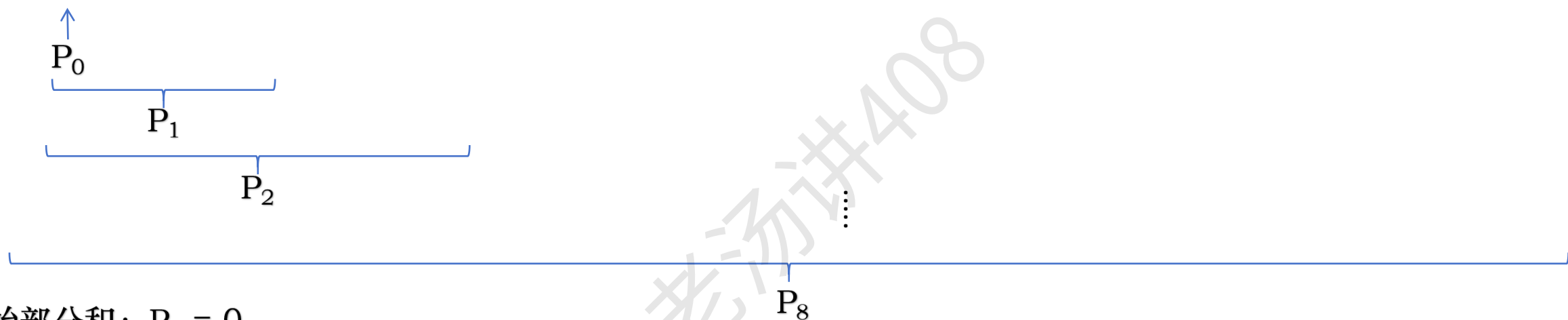
$$[X]_{\text{补}} = X_7 X_6 X_5 X_4 X_3 X_2 X_1 X_0$$

$$[Y]_{\text{补}} = Y_7 Y_6 Y_5 Y_4 Y_3 Y_2 Y_1 Y_0$$

求： $[X \times Y]_{\text{补}}$

$$[X \times Y]_{\text{补}} = [X \times \sum_{i=0}^7 (Y_{i-1} - Y_i) 2^{-(8-i)}]_{\text{补}} * 2^8$$

$$= [((((((((0 + X(Y_{-1} - Y_0))2^{-1} + X(Y_0 - Y_1))2^{-1} + X(Y_1 - Y_2))2^{-1} + X(Y_2 - Y_3))2^{-1} + X(Y_3 - Y_4))2^{-1} + X(Y_4 - Y_5))2^{-1} + X(Y_5 - Y_6))2^{-1} + X(Y_6 - Y_7))2^{-1}]_{\text{补}} * 2^8$$



初始部分积： $P_0 = 0$

$$\left. \begin{array}{l} \text{部分积: } P_1 = (P_0 + X(Y_{-1} - Y_0)) 2^{-1} \\ P_2 = (P_1 + X(Y_0 - Y_1)) 2^{-1} \\ \vdots \\ P_8 = (P_7 + X(Y_6 - Y_7)) 2^{-1} \end{array} \right\} \begin{array}{l} [P_{i+1}]_{\text{补}} = [(P_i + X * (Y_{i-1} - Y_i)) * 2^{-1}]_{\text{补}} \\ [X * Y]_{\text{补}} = [P_8]_{\text{补}} * 2^8 \end{array}$$

定点数乘法运算：补码一位乘法

初始部分积： $P_0 = 0$

$$\left. \begin{array}{l} \text{部分积: } P_1 = (P_0 + X(Y_{-1}-Y_0)) 2^{-1} \\ P_2 = (P_1 + X(Y_0-Y_1)) 2^{-1} \\ \vdots \\ P_8 = (P_7 + X(Y_6-Y_7)) 2^{-1} \end{array} \right\} [P_{i+1}]_{\text{补}} = [(P_i + X * (Y_{i-1} - Y_i)) * 2^{-1}]_{\text{补}}$$

取乘数的最低两位 Y_{i-1} 和 Y_i 判断

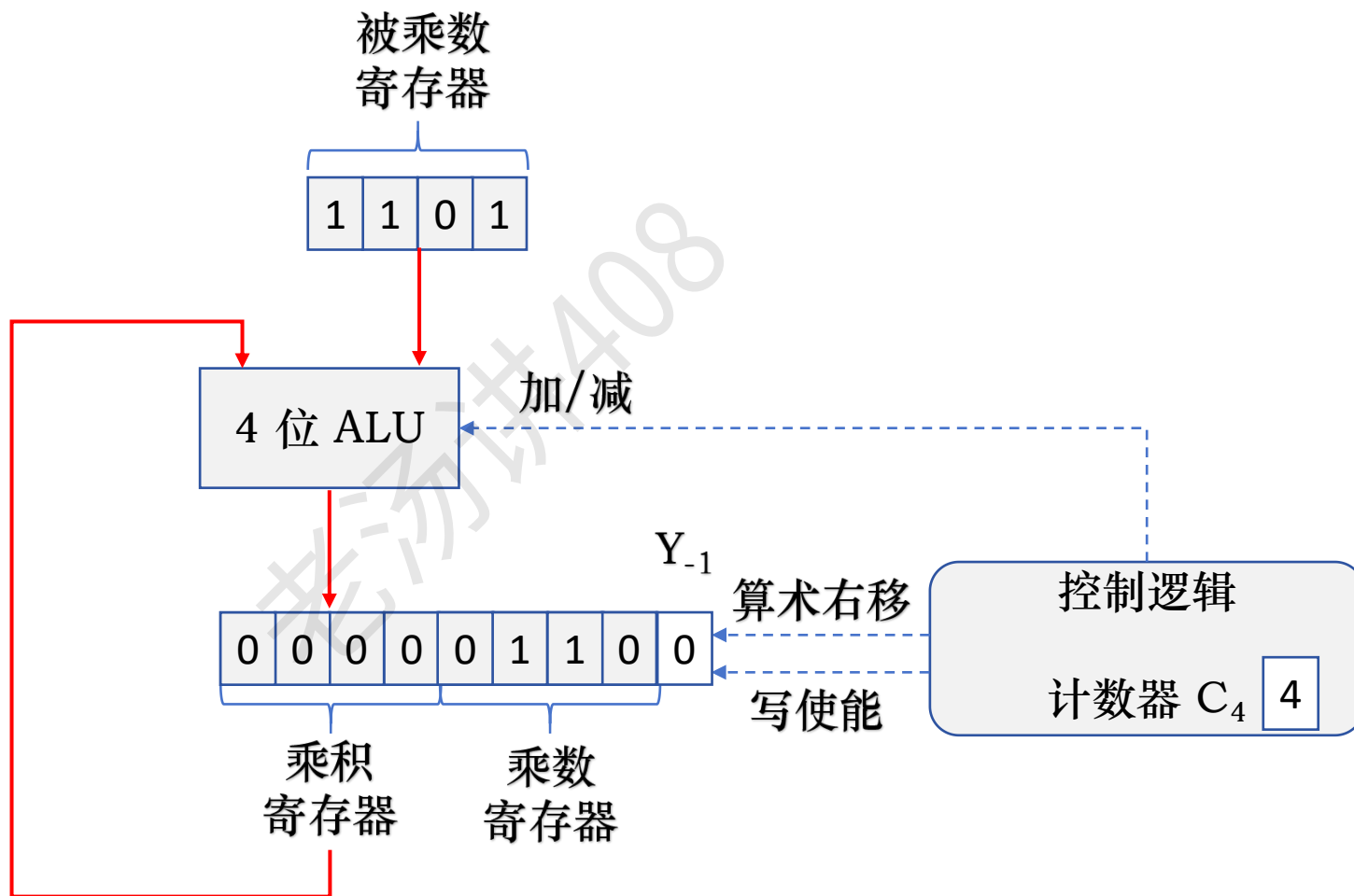
$Y_{i-1}Y_i = 00$ 或者 11 , 则 $[P_{i+1}]_{\text{补}} = [(P_i + 0)2^{-1}]_{\text{补}}$

$Y_{i-1}Y_i = 01$, 则 $[P_{i+1}]_{\text{补}} = [(P_i - X)2^{-1}]_{\text{补}}$

$Y_{i-1}Y_i = 10$, 则 $[P_{i+1}]_{\text{补}} = [(P_i + X)2^{-1}]_{\text{补}}$

定点数乘法运算：补码一位乘法

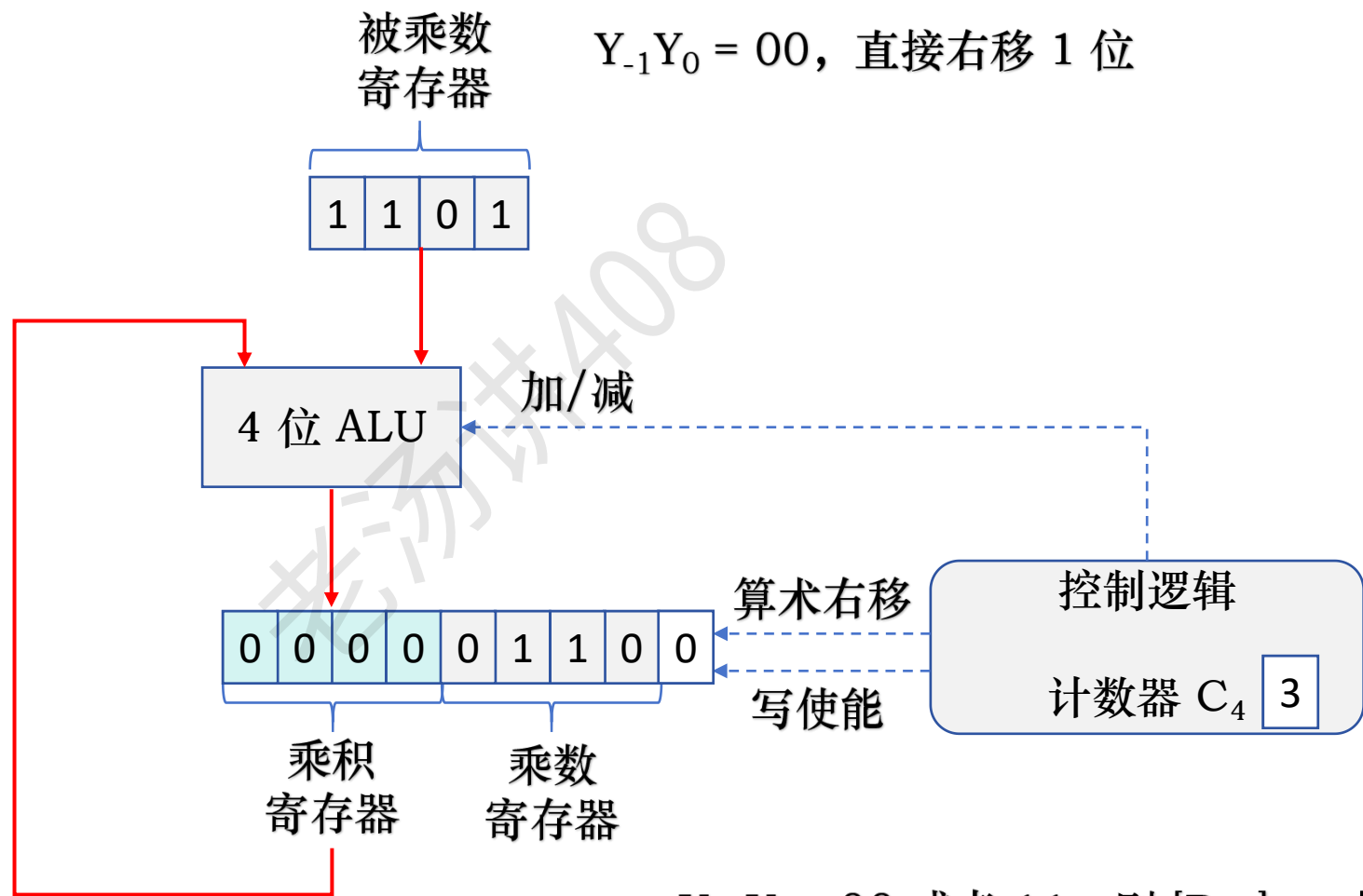
已知 $x_{\text{补}} = 1101$, $y_{\text{补}} = 0110$, 要求用布斯乘法计算 $(x * y)_{\text{补}}$



$$[X * Y]_{\text{补}} = [P_4]_{\text{补}} * 2^4$$

定点数乘法运算：补码一位乘法

已知 $x_{\text{补}} = 1101$, $y_{\text{补}} = 0110$, 要求用布斯乘法计算 $(x * y)_{\text{补}}$



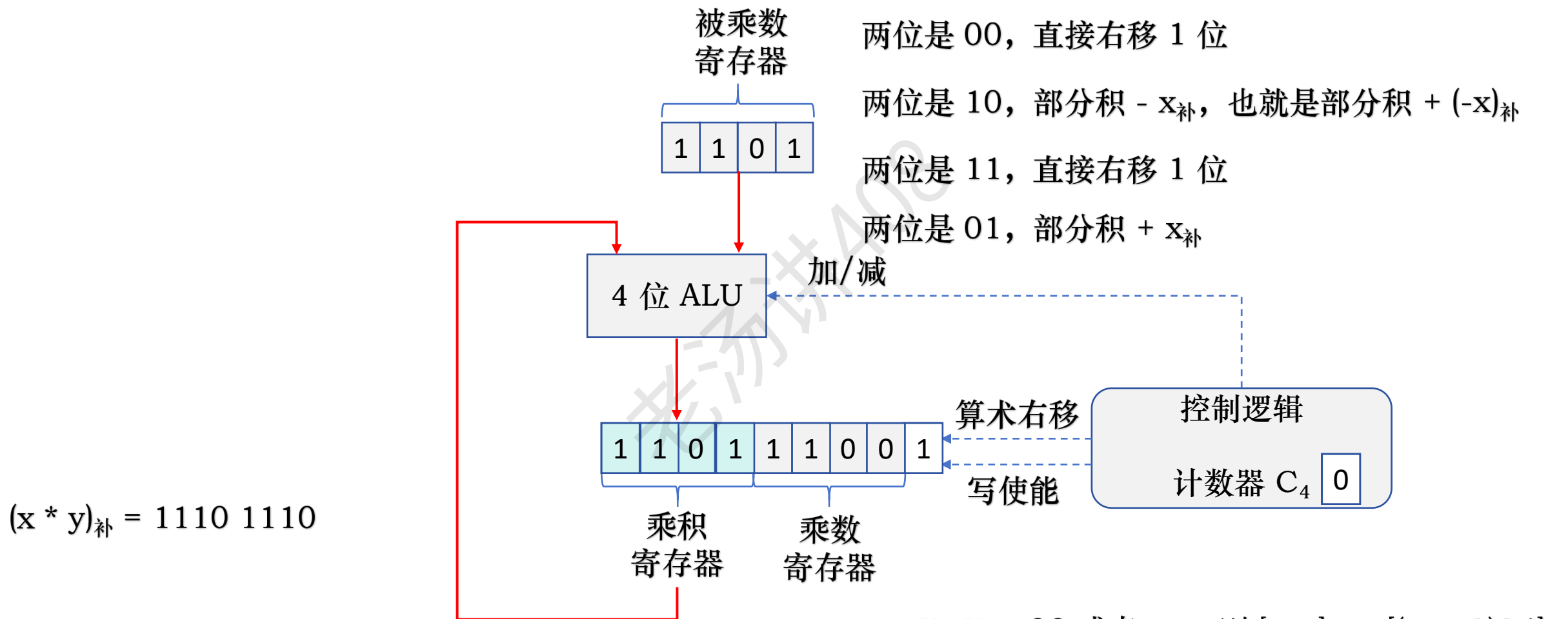
$Y_{i-1}Y_i = 00$ 或者 11 , 则 $[P_{i+1}]_{\text{补}} = [(P_i + 0)2^{-1}]_{\text{补}}$

$Y_{i-1}Y_i = 01$, 则 $[P_{i+1}]_{\text{补}} = [(P_i - X)2^{-1}]_{\text{补}}$

$Y_{i-1}Y_i = 10$, 则 $[P_{i+1}]_{\text{补}} = [(P_i + X)2^{-1}]_{\text{补}}$

定点数乘法运算：补码一位乘法

已知 $x_{\text{补}} = 1101$, $y_{\text{补}} = 0110$, 要求用布斯乘法计算 $(x * y)_{\text{补}}$ $(-x)_{\text{补}} = 0011$



验证: $x = -011B = -3$, $y = +110B = 6$,

$x * y = -001 0010B = -18$, 结果正确

$Y_{i-1}Y_i = 00$ 或者 11 , 则 $[P_{i+1}]_{\text{补}} = [(P_i + 0)2^{-1}]_{\text{补}}$

$Y_{i-1}Y_i = 01$, 则 $[P_{i+1}]_{\text{补}} = [(P_i - X)2^{-1}]_{\text{补}}$

$Y_{i-1}Y_i = 10$, 则 $[P_{i+1}]_{\text{补}} = [(P_i + X)2^{-1}]_{\text{补}}$

定点数乘法运算：补码两位乘法

把乘数分成两位一组，根据**两位二进制**的组合决定加或减被乘数的倍数，形成的部分积**每次右移两位**

定点数乘法运算：阵列乘法器

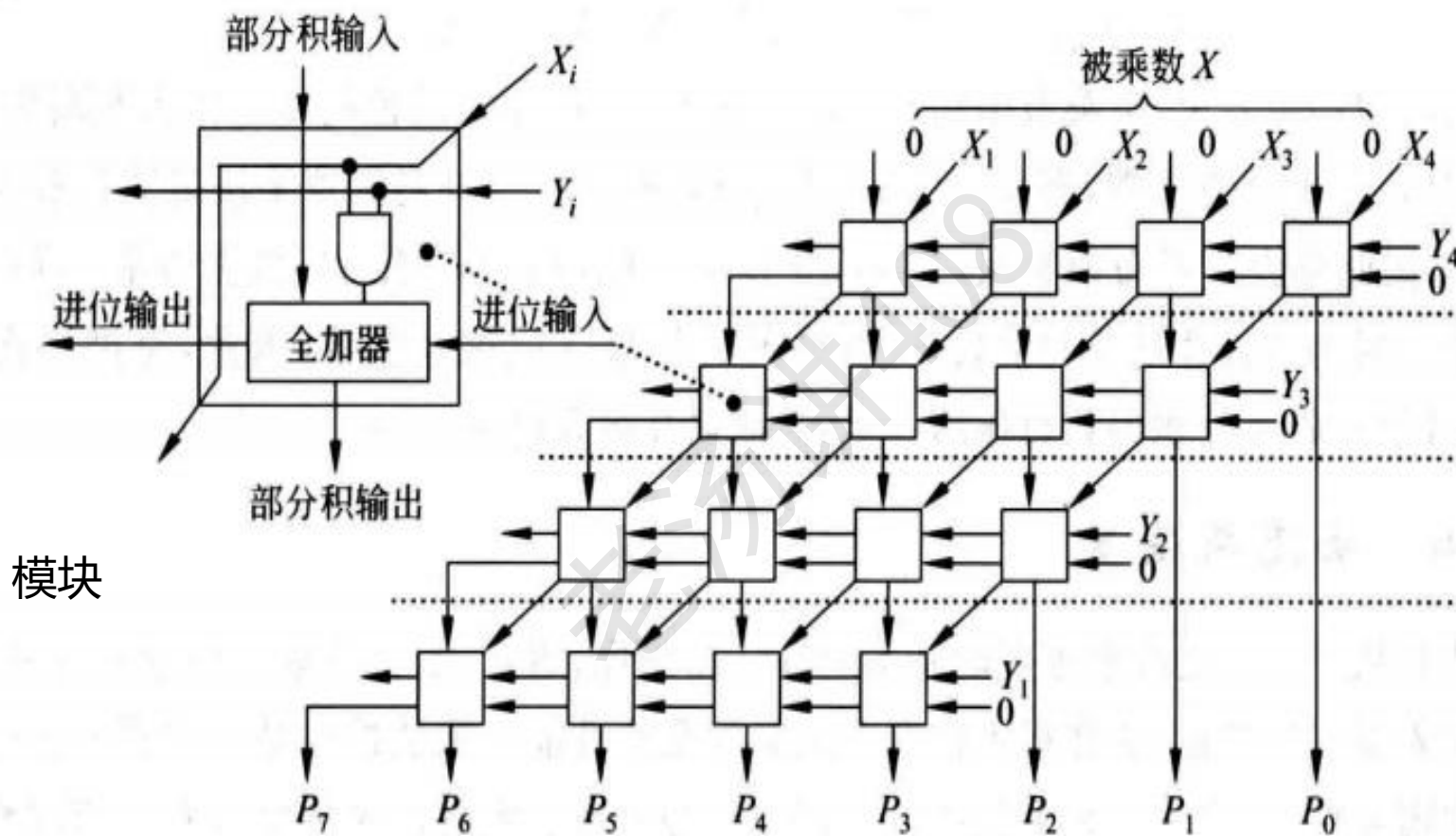


图 3.13 4×4 位基于 CRA 的阵列乘法器

定点数乘法运算：阵列乘法器

被乘数 $X = 1101$ ，乘数 $Y = 0011$

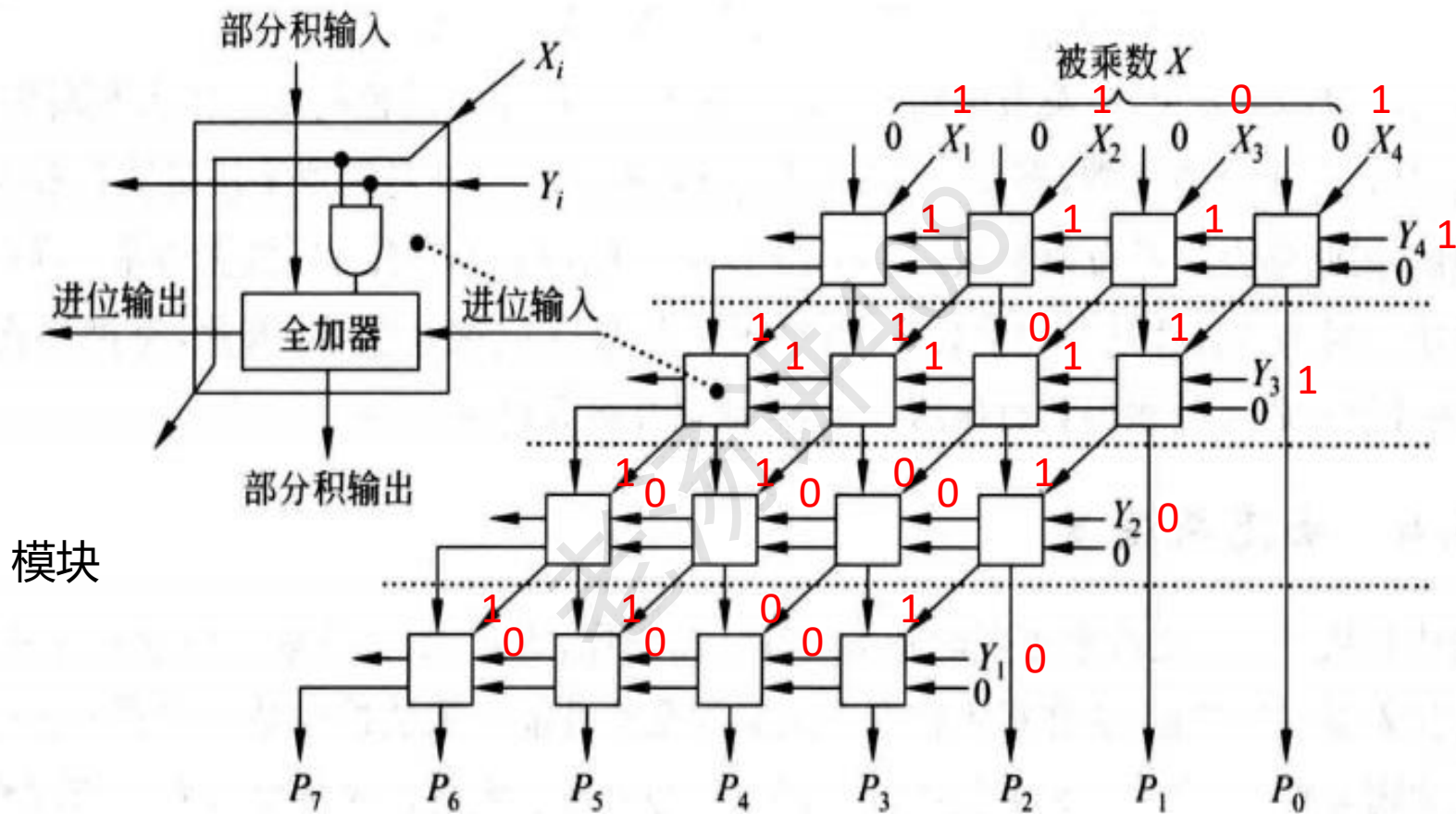
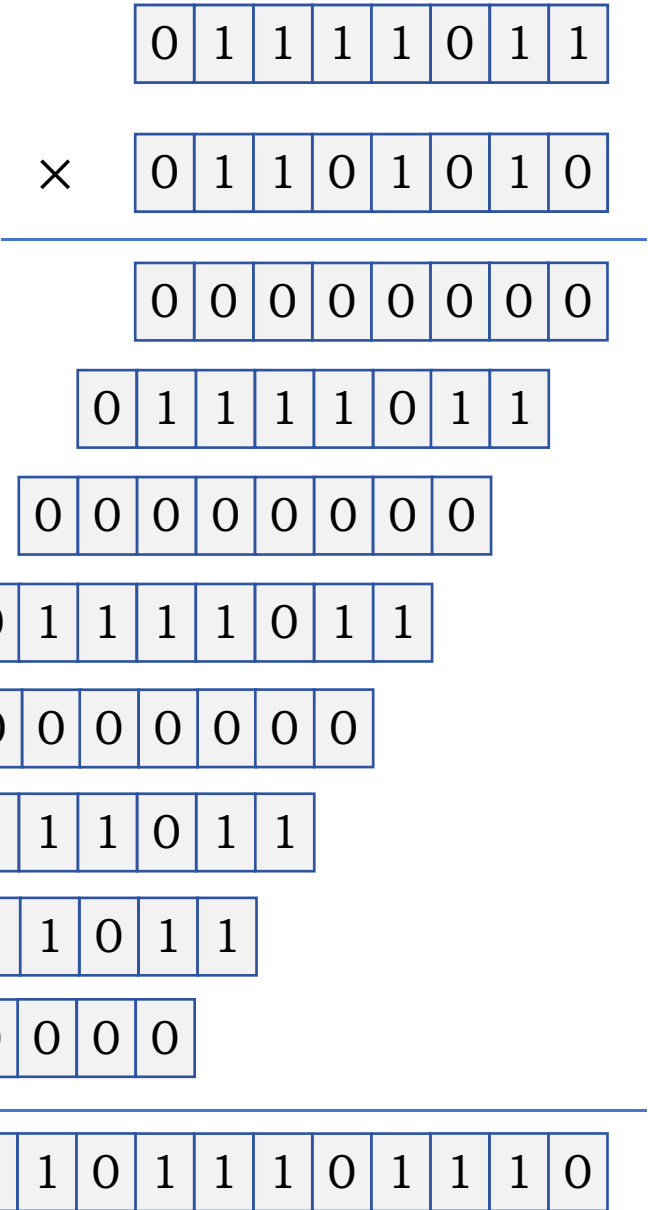


图 3.13 4×4 位基于 CRA 的阵列乘法器

定点数乘法运算：阵列乘法器

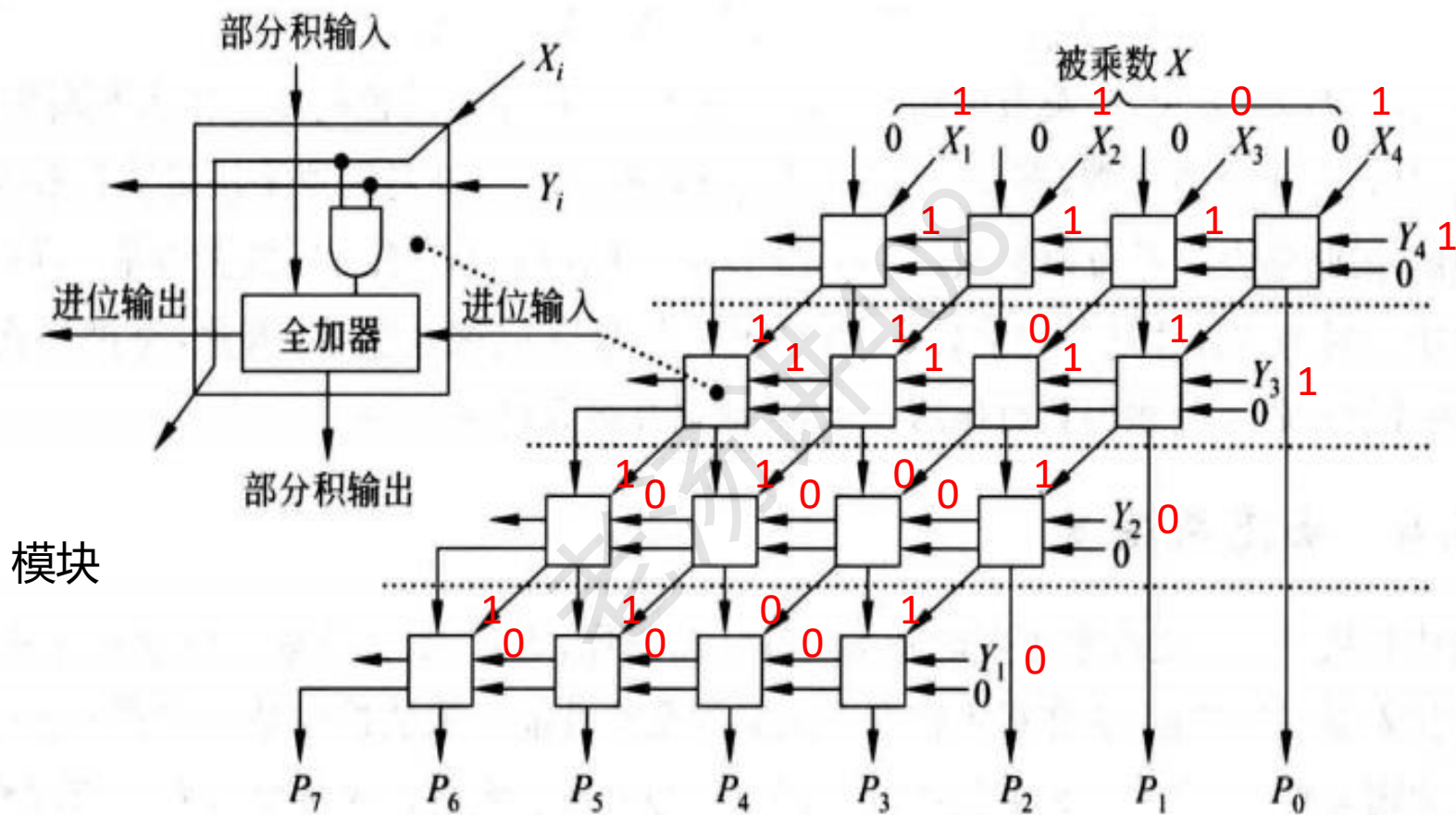
123 × 106



乘积：13038

定点数乘法运算：阵列乘法器

被乘数 $X = 1101$ ，乘数 $Y = 0011$

图 3.13 4×4 位基于 CRA 的阵列乘法器

定点数乘法运算：阵列乘法器

被乘数 $X = 1101$ ，乘数 $Y = 0011$

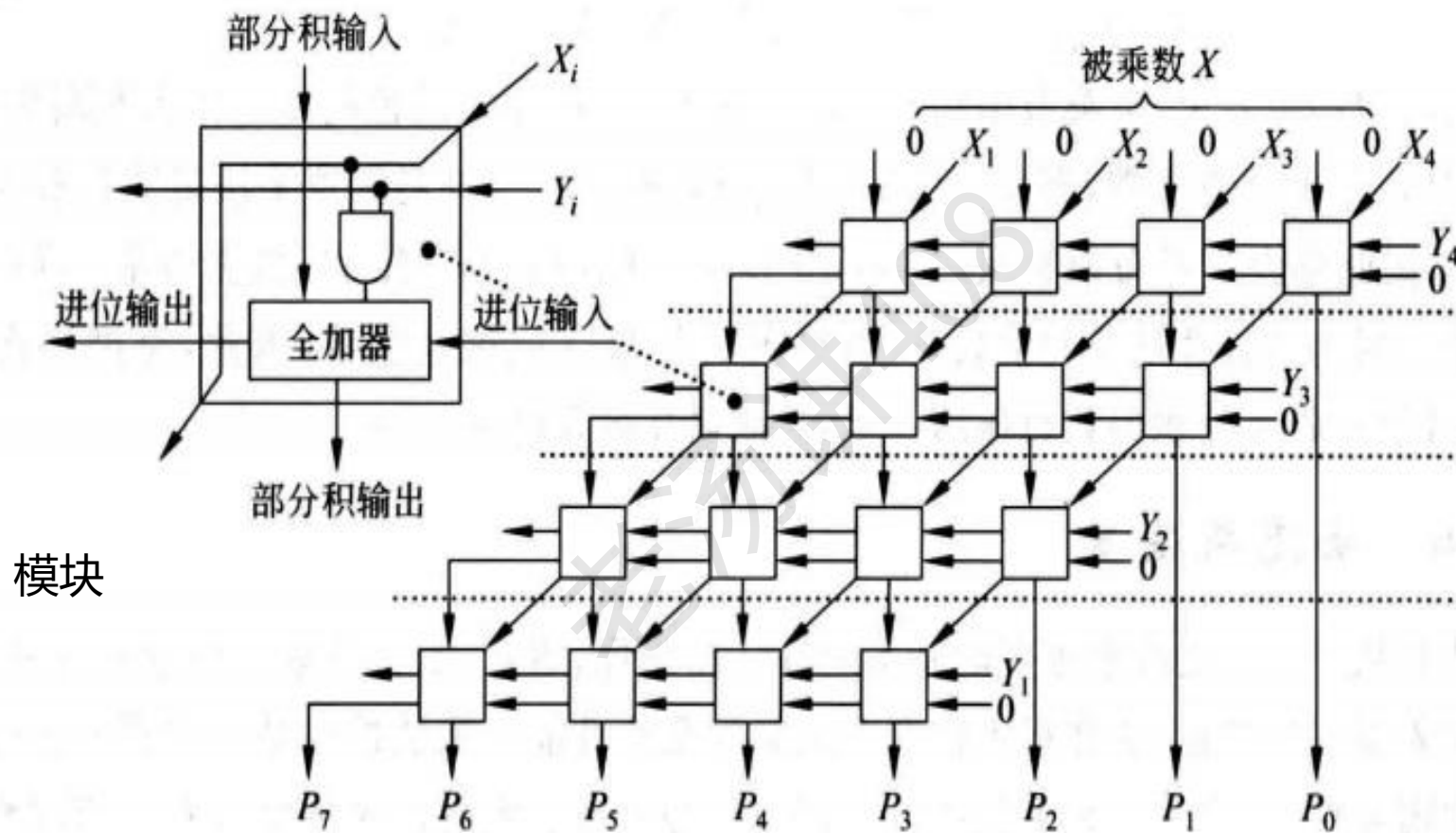


图 3.13 4×4 位基于 CRA 的阵列乘法器

定点数乘法运算：阵列乘法器

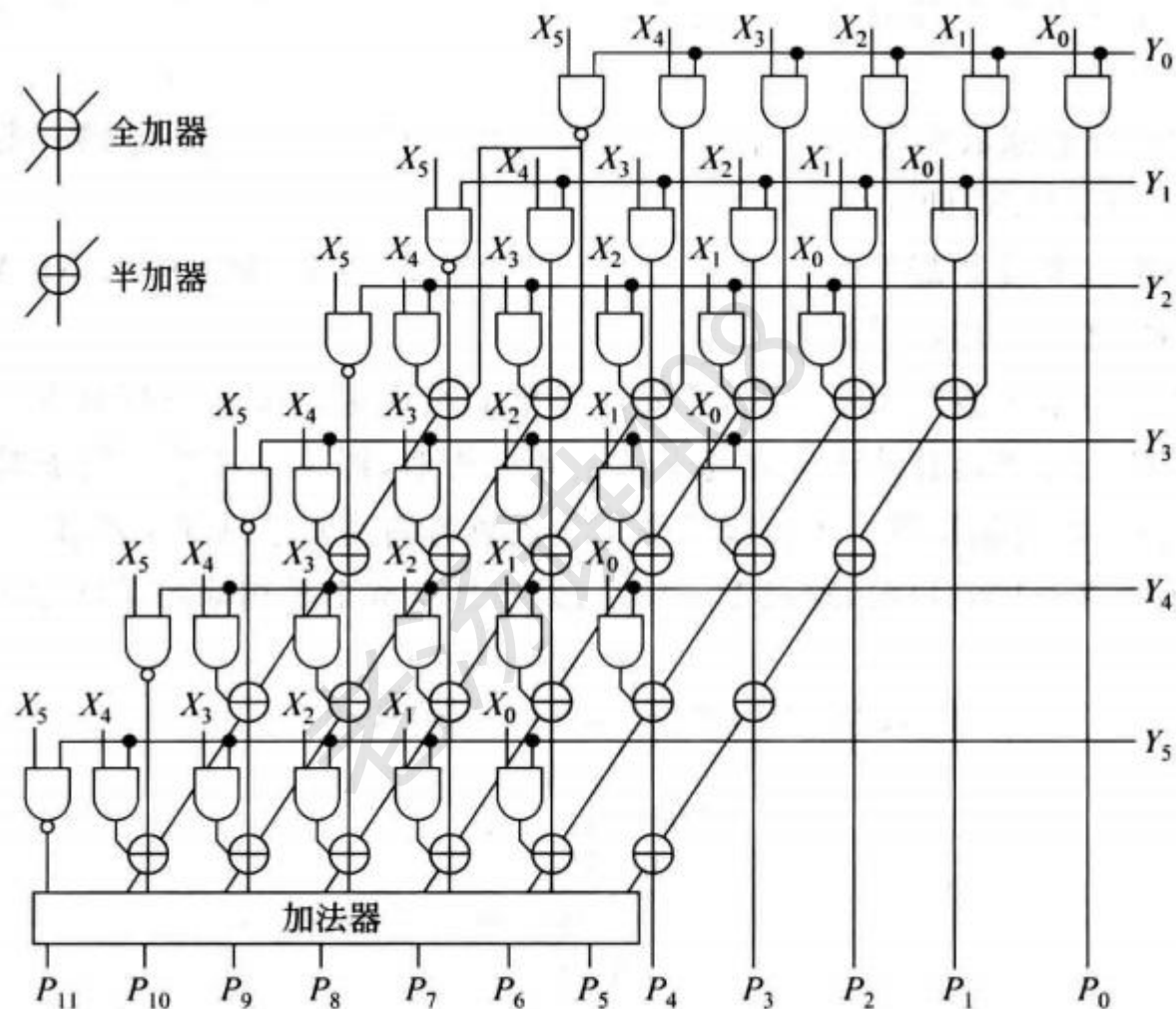
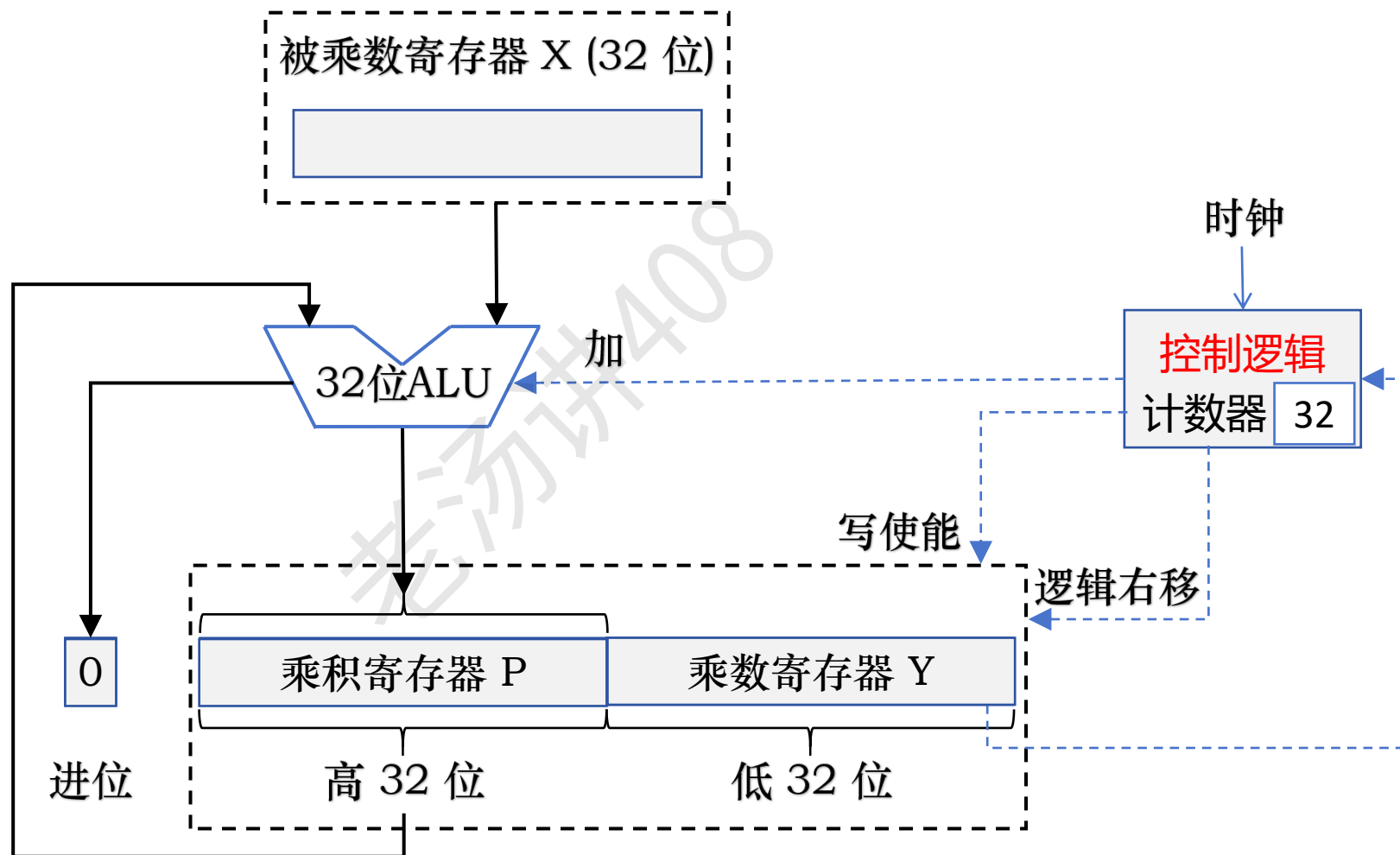


图 3.14 6×6 位基于 CSA 的阵列乘法器

使用阵列乘法器实现的乘法，可以在 1 个时钟周期内完成一次乘法操作

定点数乘法运算：ALU + 移位器



定点数乘法运算：软件实现乘法

变量 $a \times$ 变量 b 机器字长是 8 位 $6 \times 11 = 66$

变量 $a =$

6

0	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---

变量 $b =$

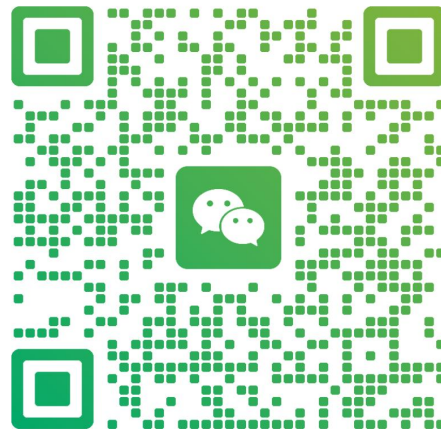
11

0	0	0	0	1	0	1	1
---	---	---	---	---	---	---	---

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

定点数乘法运算：软件实现乘法

变量 a × 变量 b 机器字长是 8 位 6 × 11 = 66

变量 a =

0	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---

变量 b =

0	0	0	0	1	0	1	1
---	---	---	---	---	---	---	---

result =

定点数乘法运算：软件实现乘法

变量 $a \times$ 变量 b 机器字长是 8 位 $6 \times 11 = 66$

变量 $a =$ 12

0	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---

变量 $b =$ 2

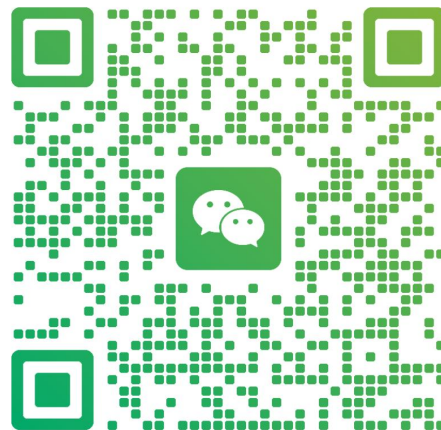
0	0	0	0	0	1	0	1
---	---	---	---	---	---	---	---

result = 18

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

定点数乘法运算：软件实现乘法

变量 $a \times$ 变量 b 机器字长是 8 位 $6 \times 11 = 66$

变量 $a =$ 24

0	0	0	0	1	1	0	0
---	---	---	---	---	---	---	---

变量 $b =$ 1

0	0	0	0	0	0	1	0
---	---	---	---	---	---	---	---

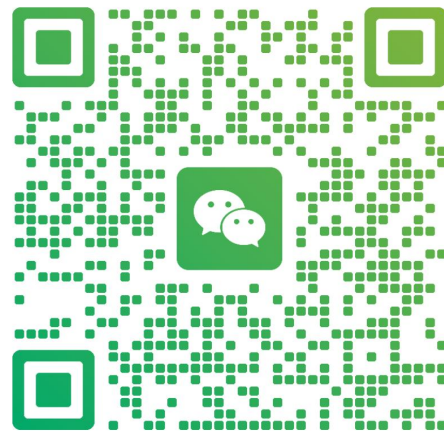
result = 18

加老汤微信，帮你 408 冲 120+



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

定点数乘法运算：软件实现乘法

变量 a × 变量 b 机器字长是 8 位 6 × 11 = 66

变量 a =

0	0	0	1	1	0	0	0
---	---	---	---	---	---	---	---

变量 b =

0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---

result =

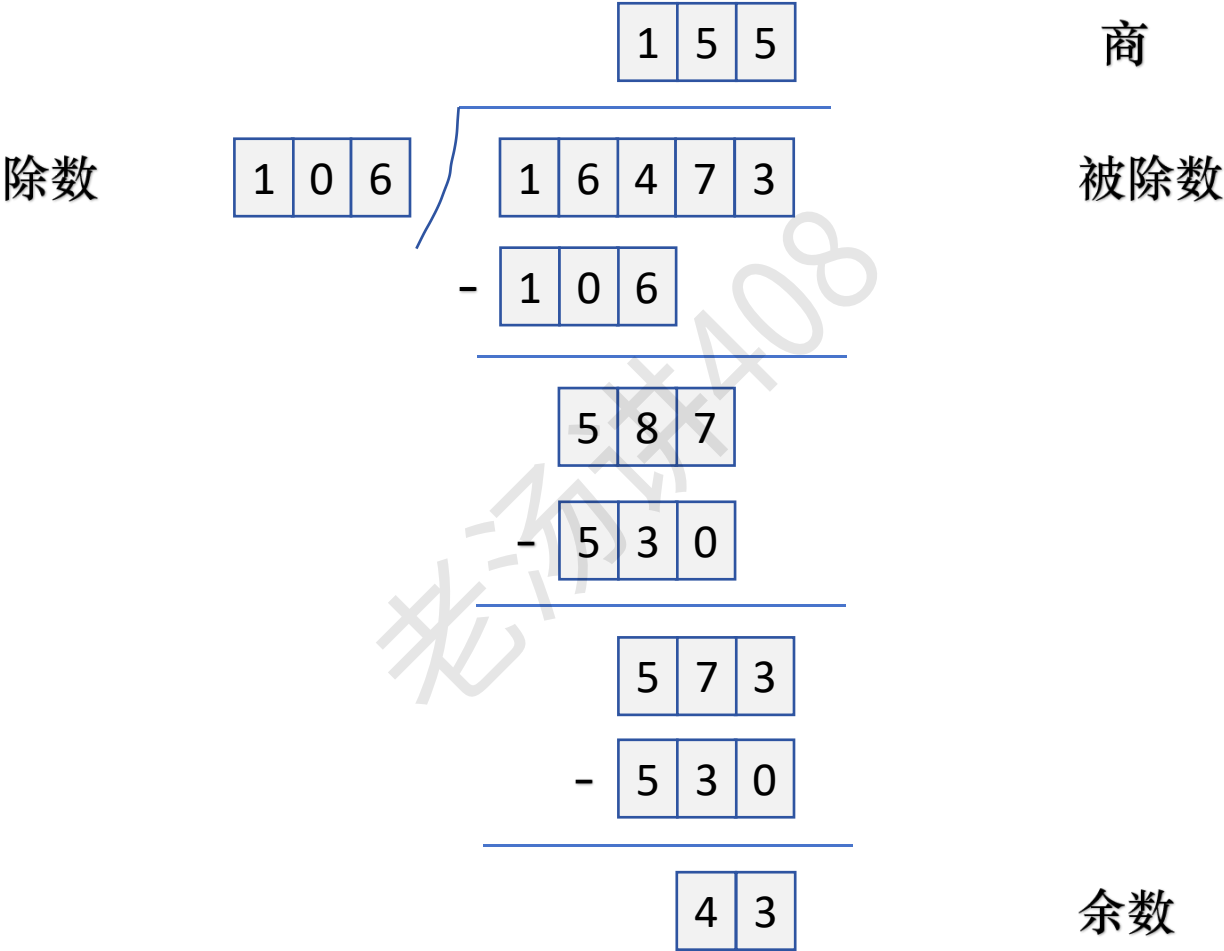
定点数乘法运算：软件实现乘法

// 用移位、加法指令实现乘法

```
int multiply(int a, int b) {  
    int result = 0;  
    int shift = 0;  
    while (b) {  
        if (b & 1)  
            result += a << shift;  
        shift++;  
        b >>= 1;  
    }  
    return result;  
}
```

被除数 ÷ 除数 = 商 余数

定点数除法运算：原码除法运算

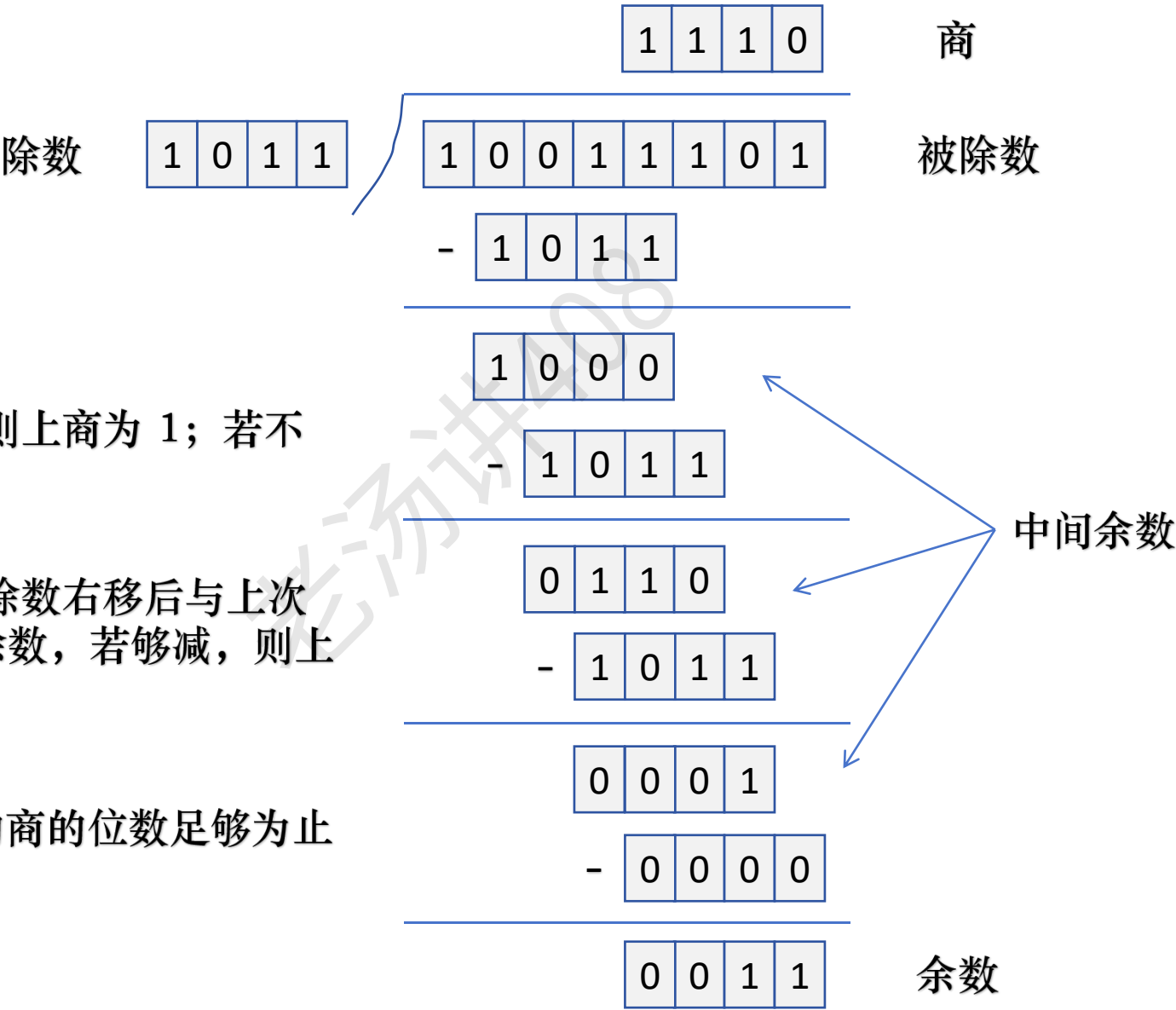


16473 ÷ 106 = 155 43

被除数 ÷ 除数 = 商 余数

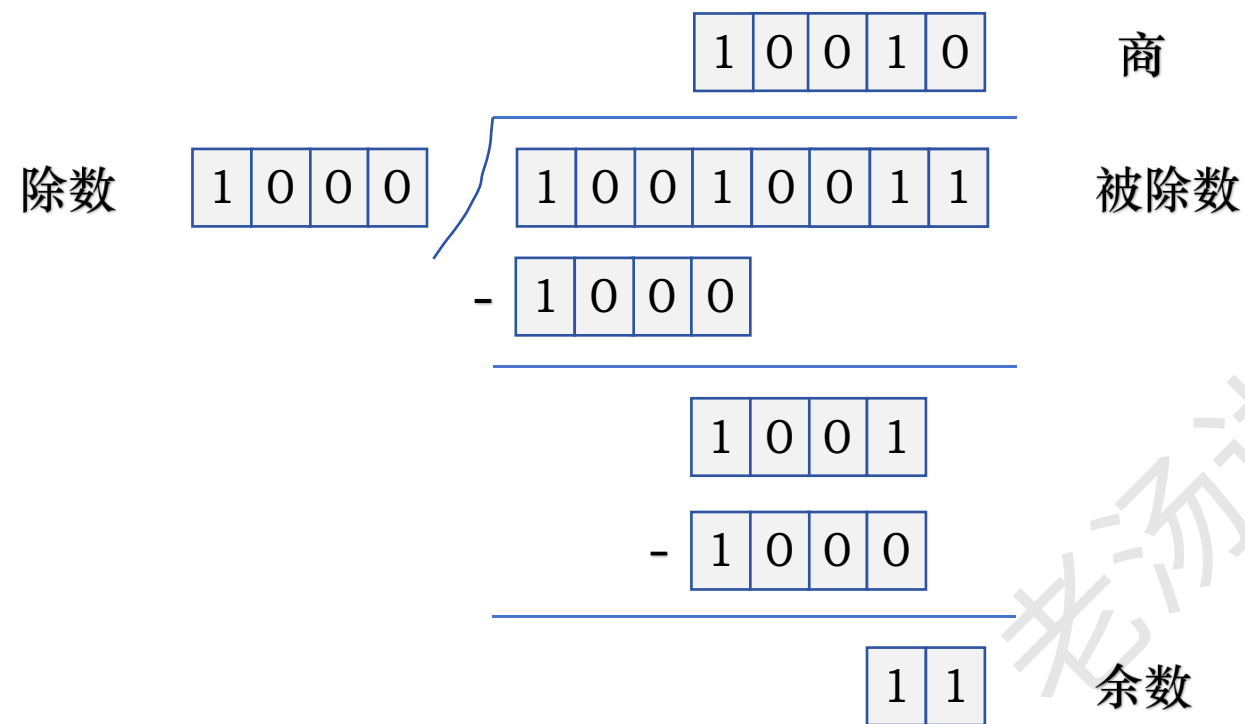
定点数除法运算：原码除法运算

- ① 被除数与除数相减，若够减，则上商为 1；若不够减，则上商为 0
- ② 每次得到的差为中间余数，将除数右移后与上次的中间余数比较。用中间余数减除数，若够减，则上商为 1；若不够减，则上商为 0
- ③ 重复执行第 ② 步，直到求得的商的位数足够为止



被除数 ÷ 除数 = 商 余数

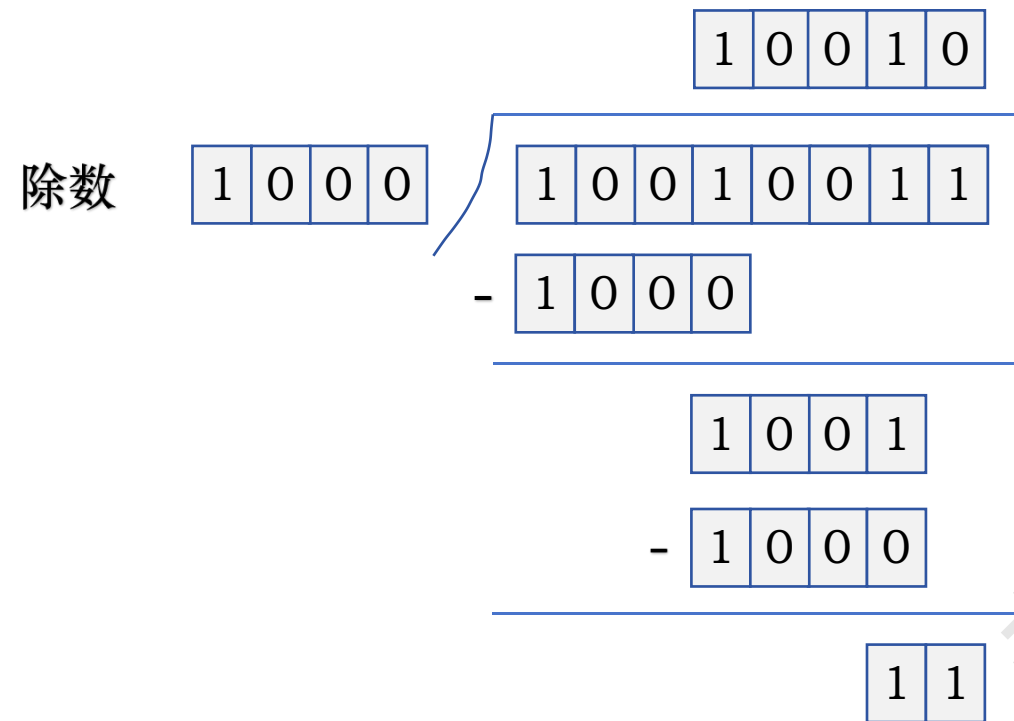
定点数除法运算：原码除法运算



147 ÷ 8 = 18 3

被除数 ÷ 除数 = 商 余数

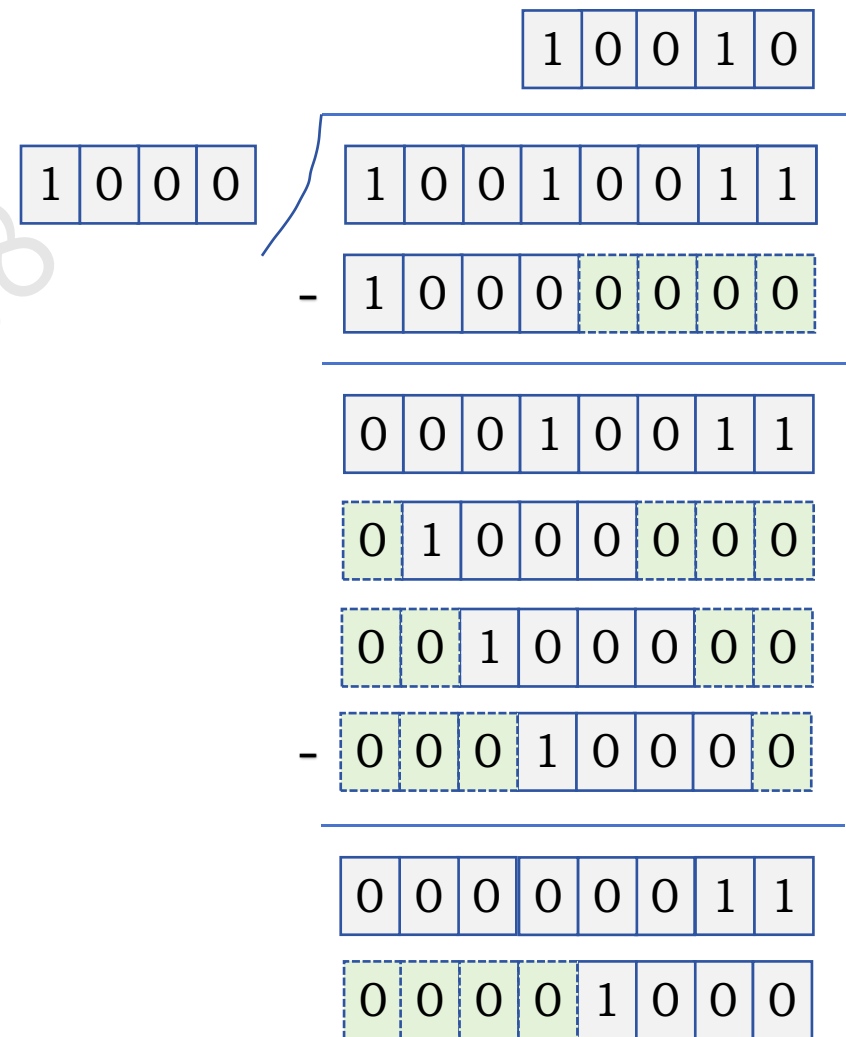
定点数除法运算：原码除法运算



商

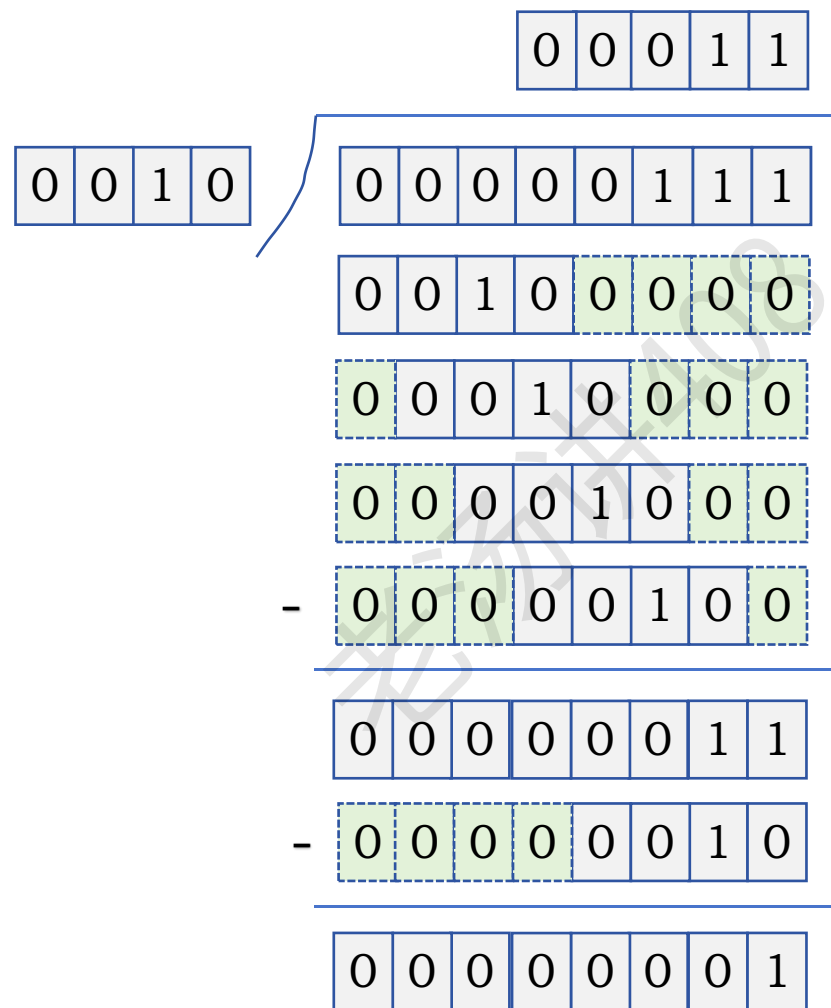
被除数

余数



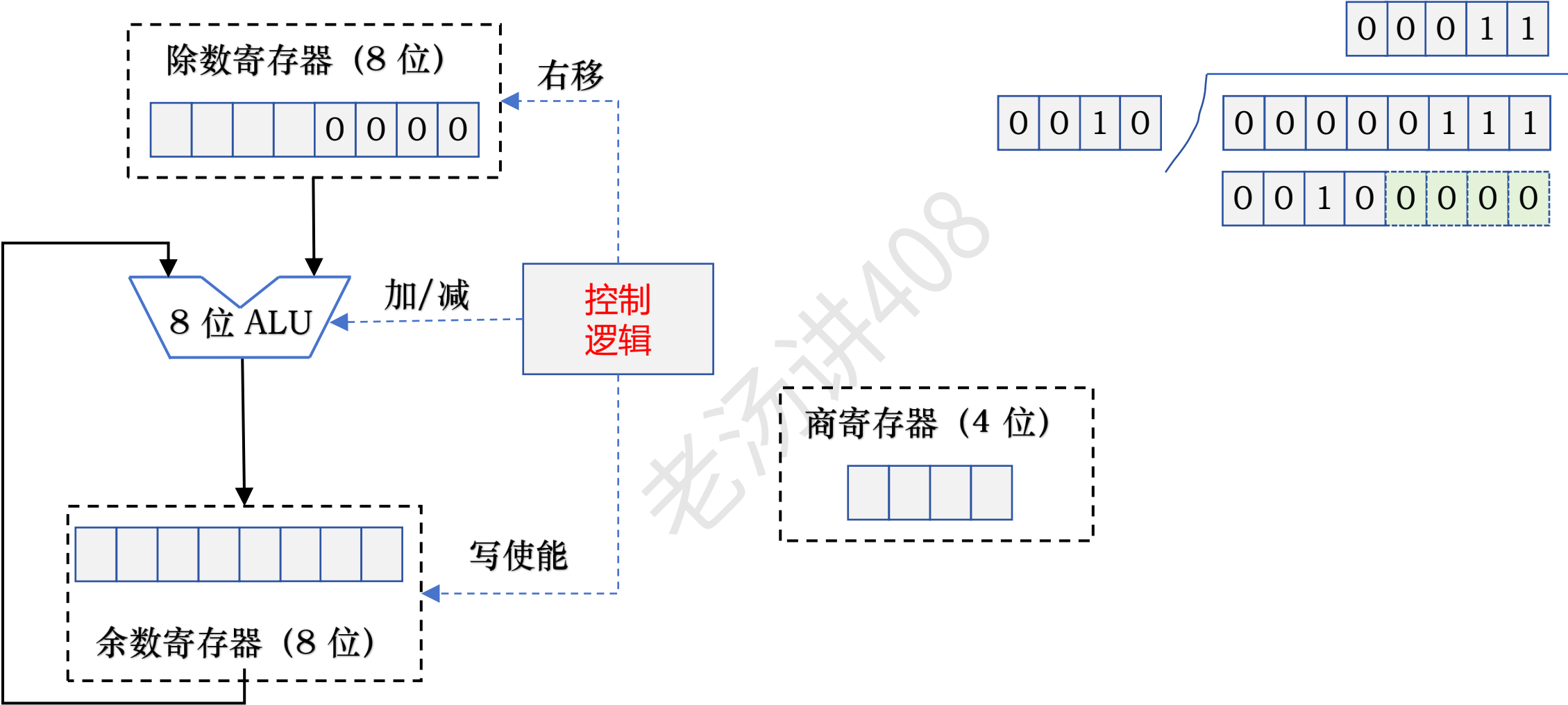
147 ÷ 8 = 18 3

定点数除法运算：原码除法运算

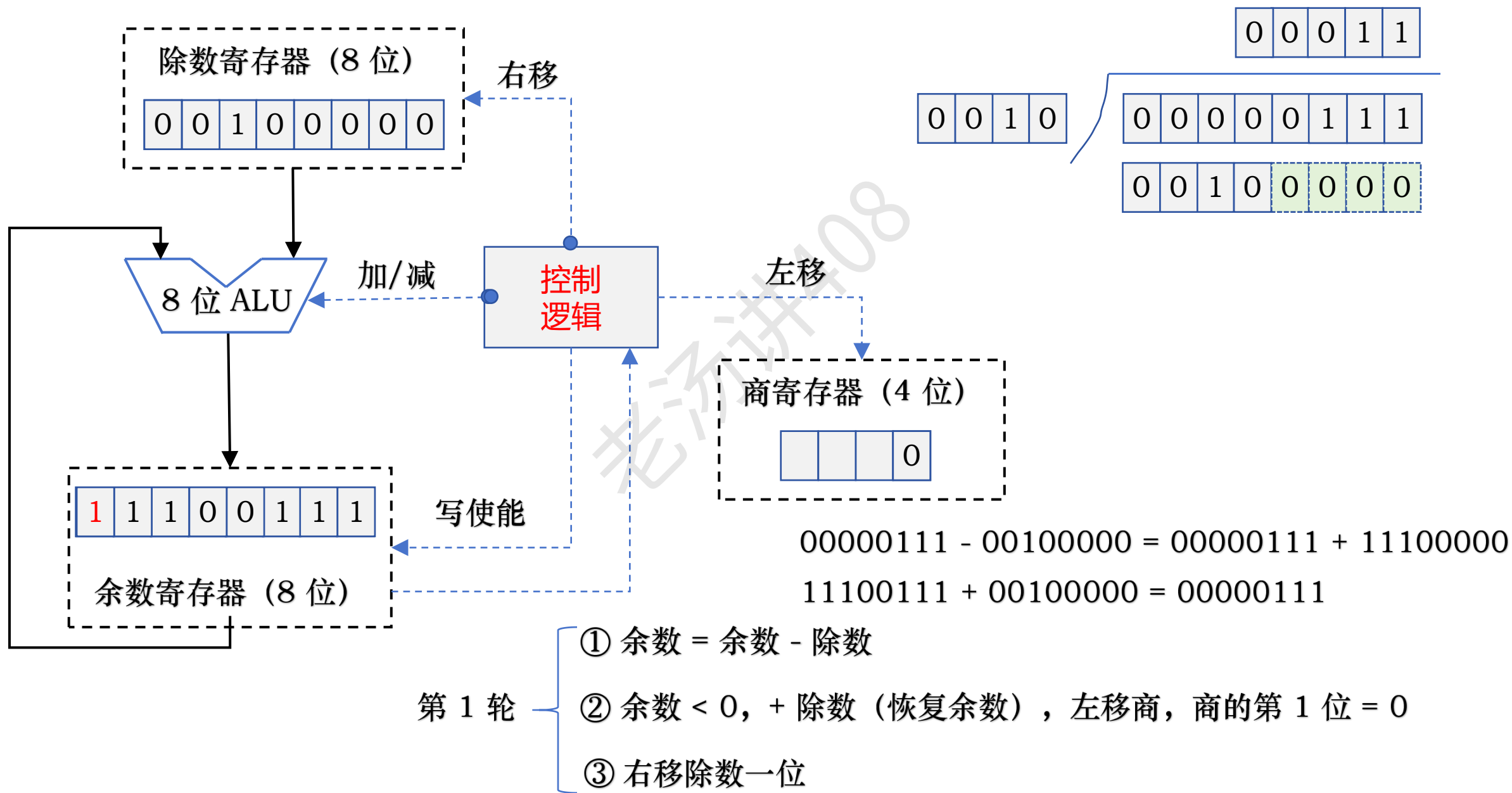


$$7 \div 2 = 3 \dots\dots 1$$

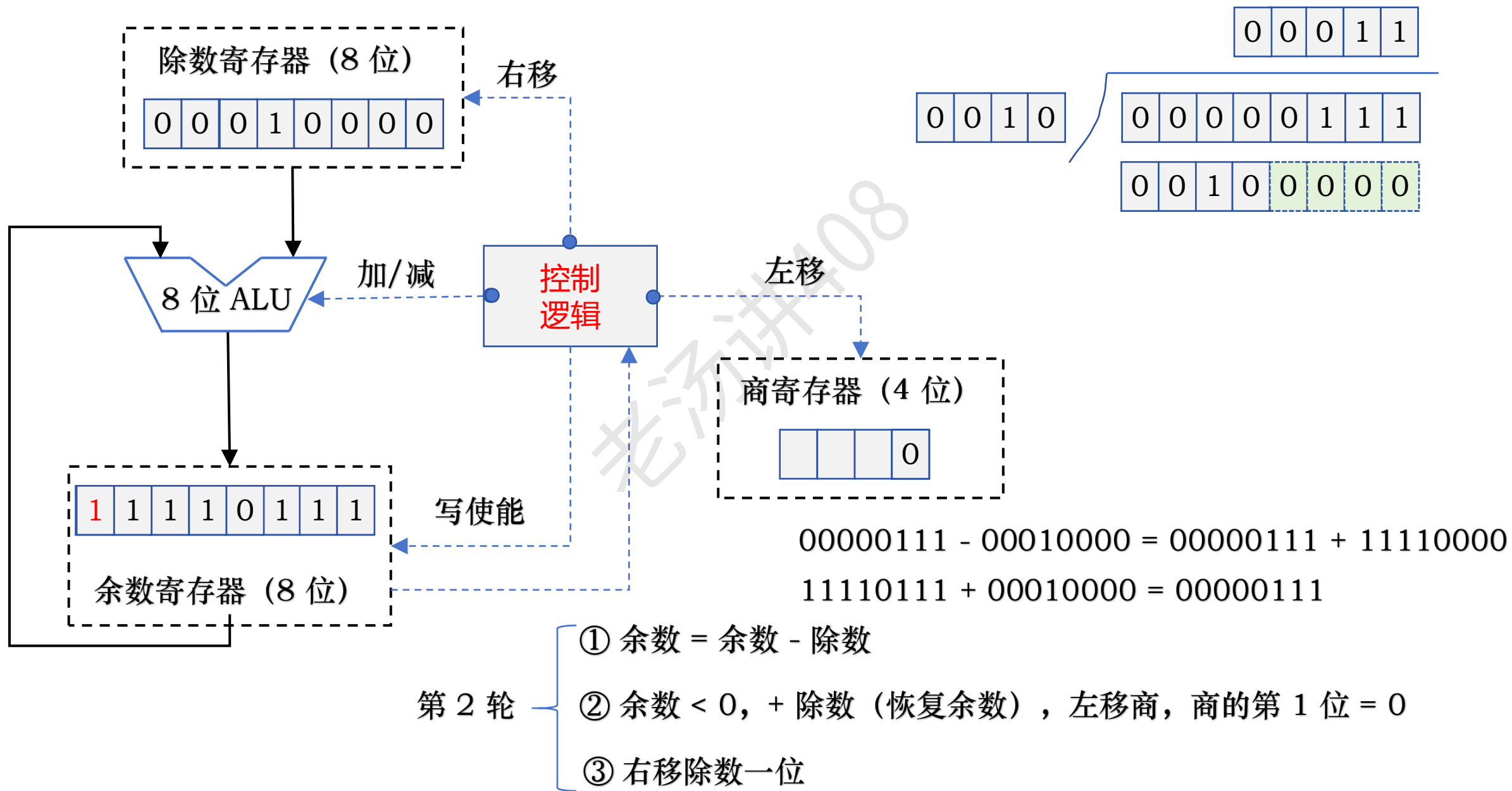
定点数除法运算：原码除法运算



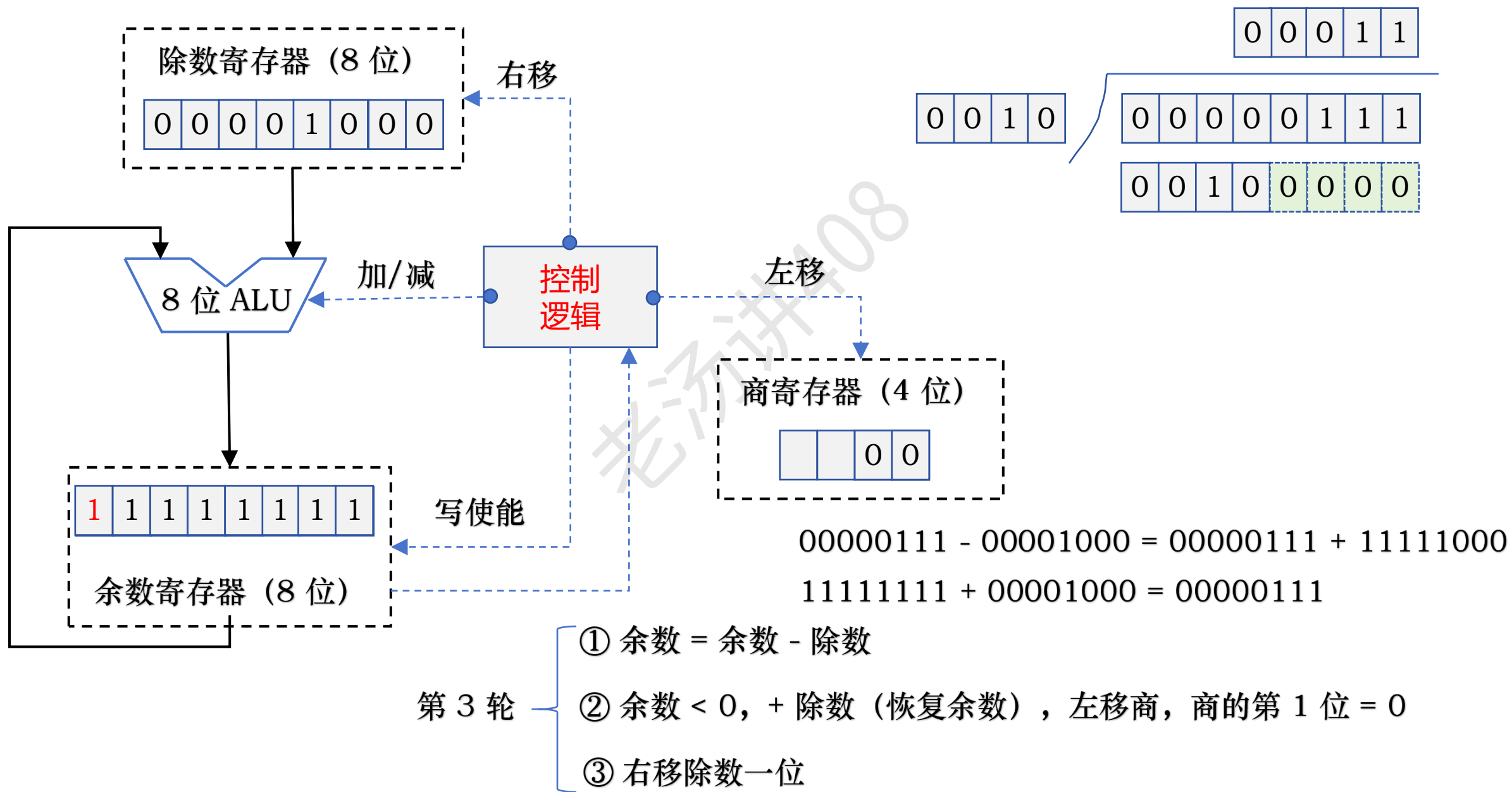
定点数除法运算：原码除法运算



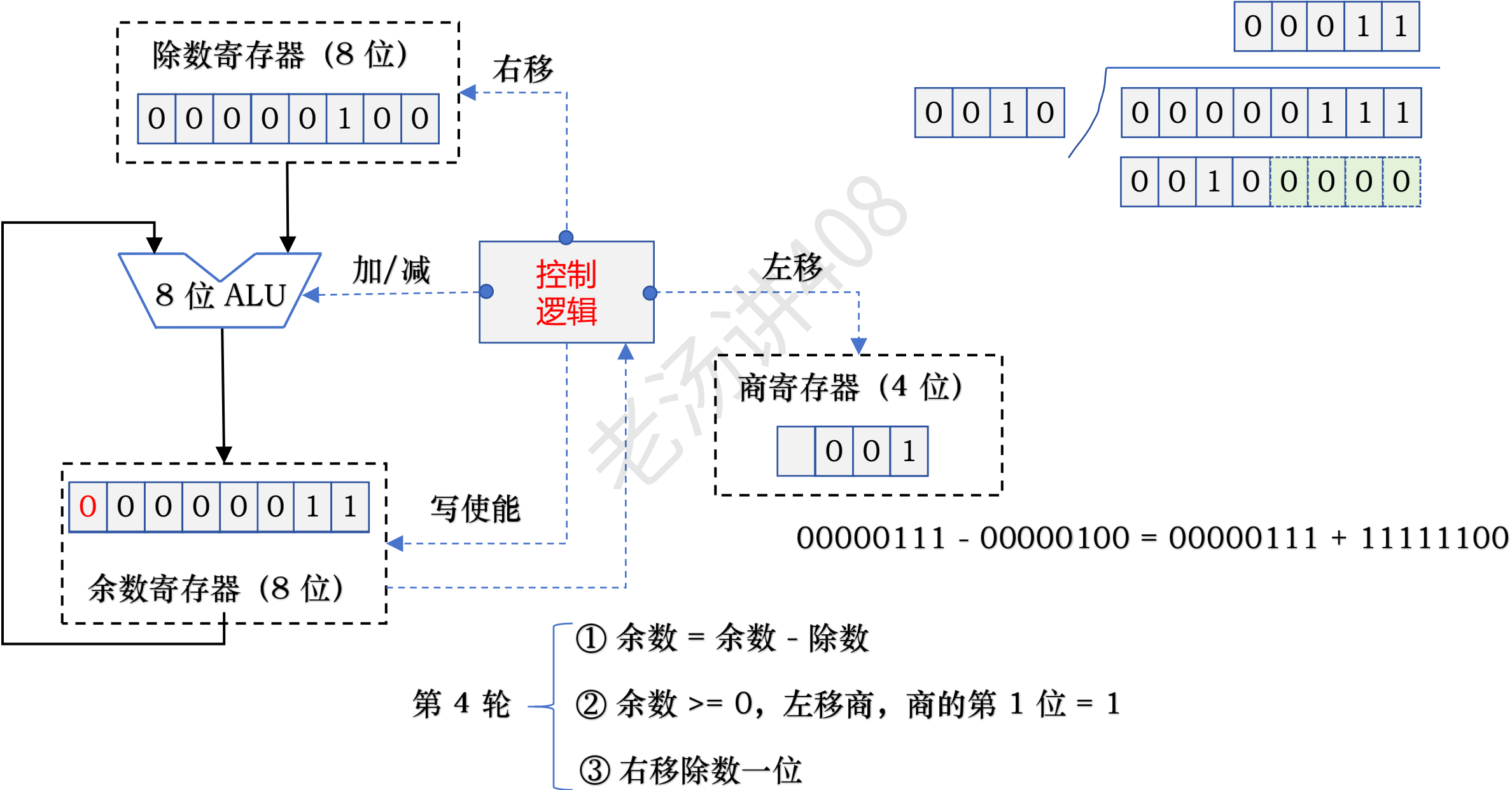
定点数除法运算：原码除法运算



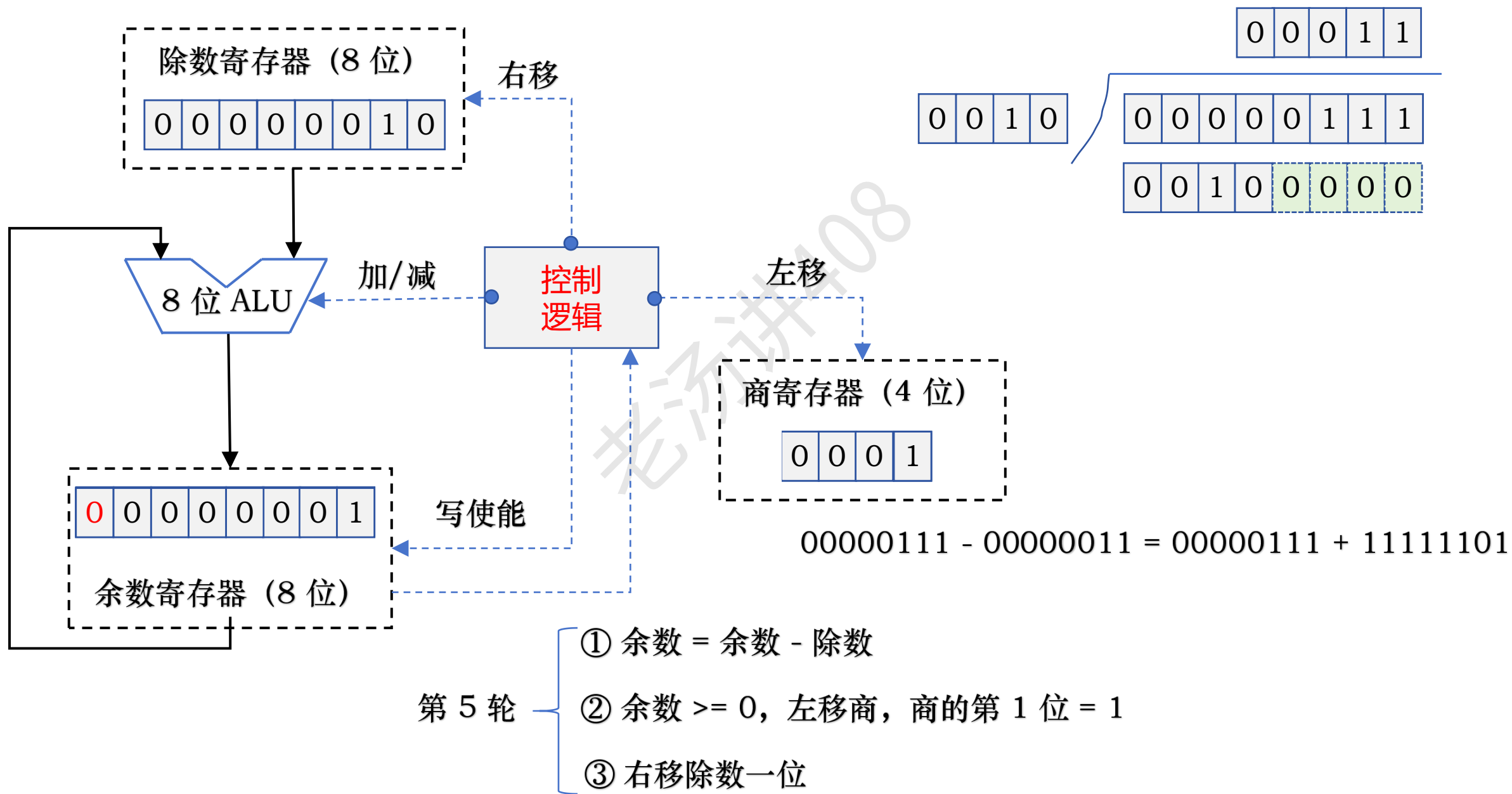
定点数除法运算：原码除法运算



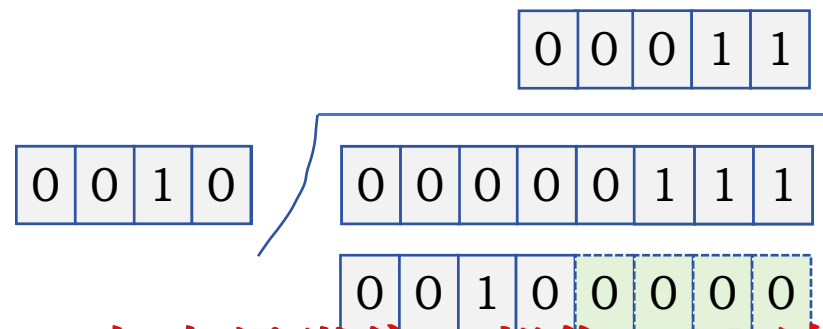
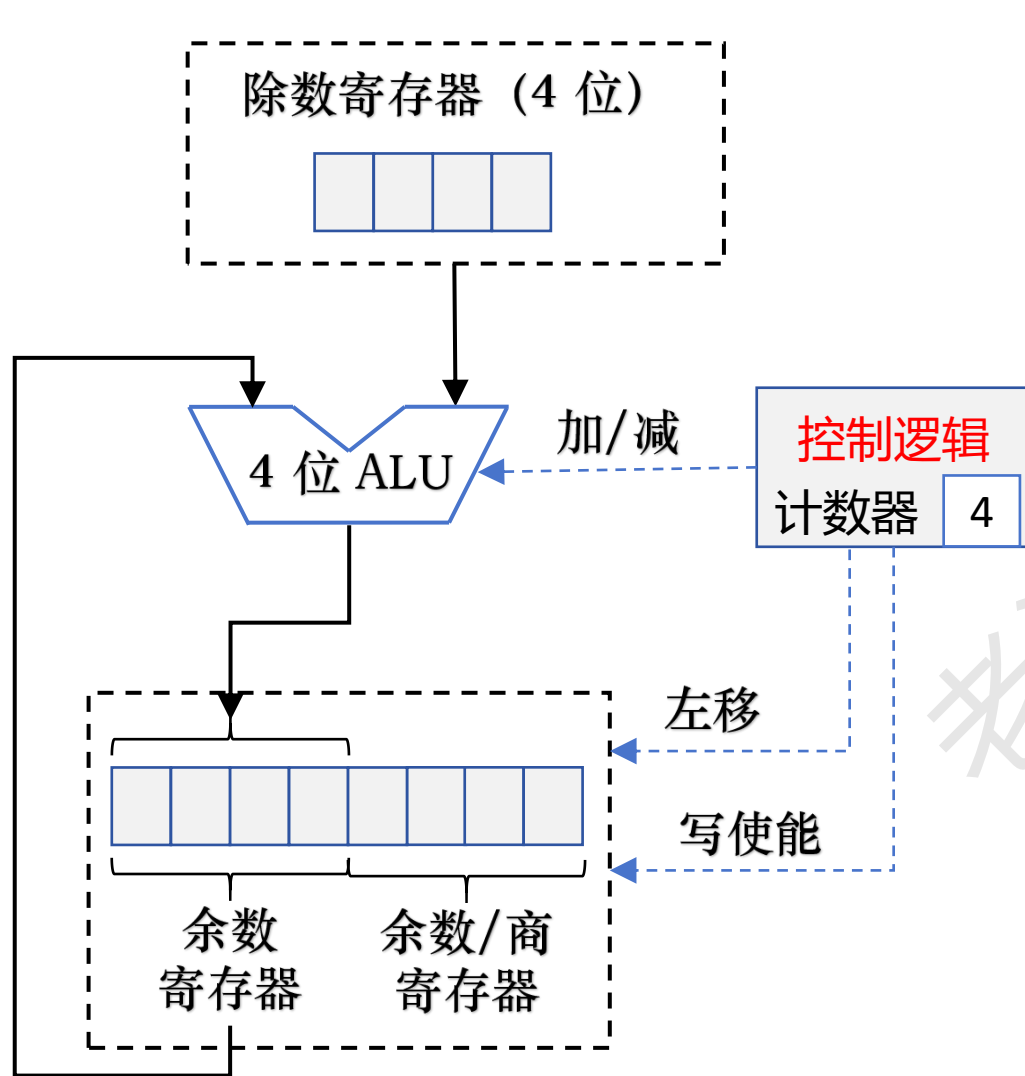
定点数除法运算：原码除法运算



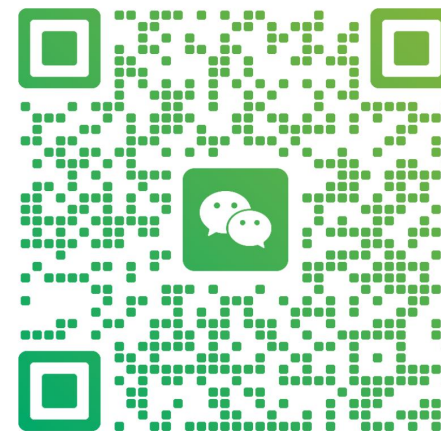
定点数除法运算：原码除法运算



定点数除法运算：原码除法运算（电路优化）

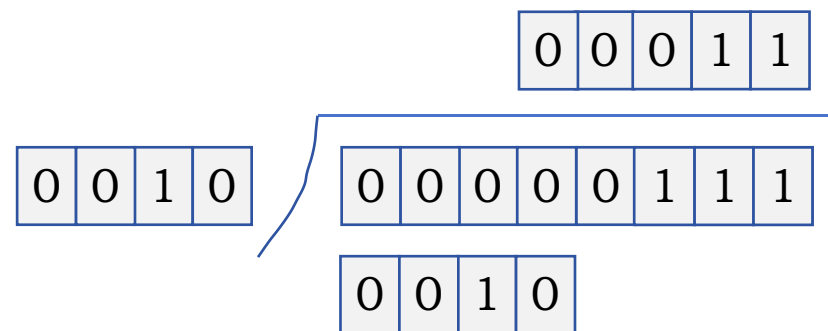
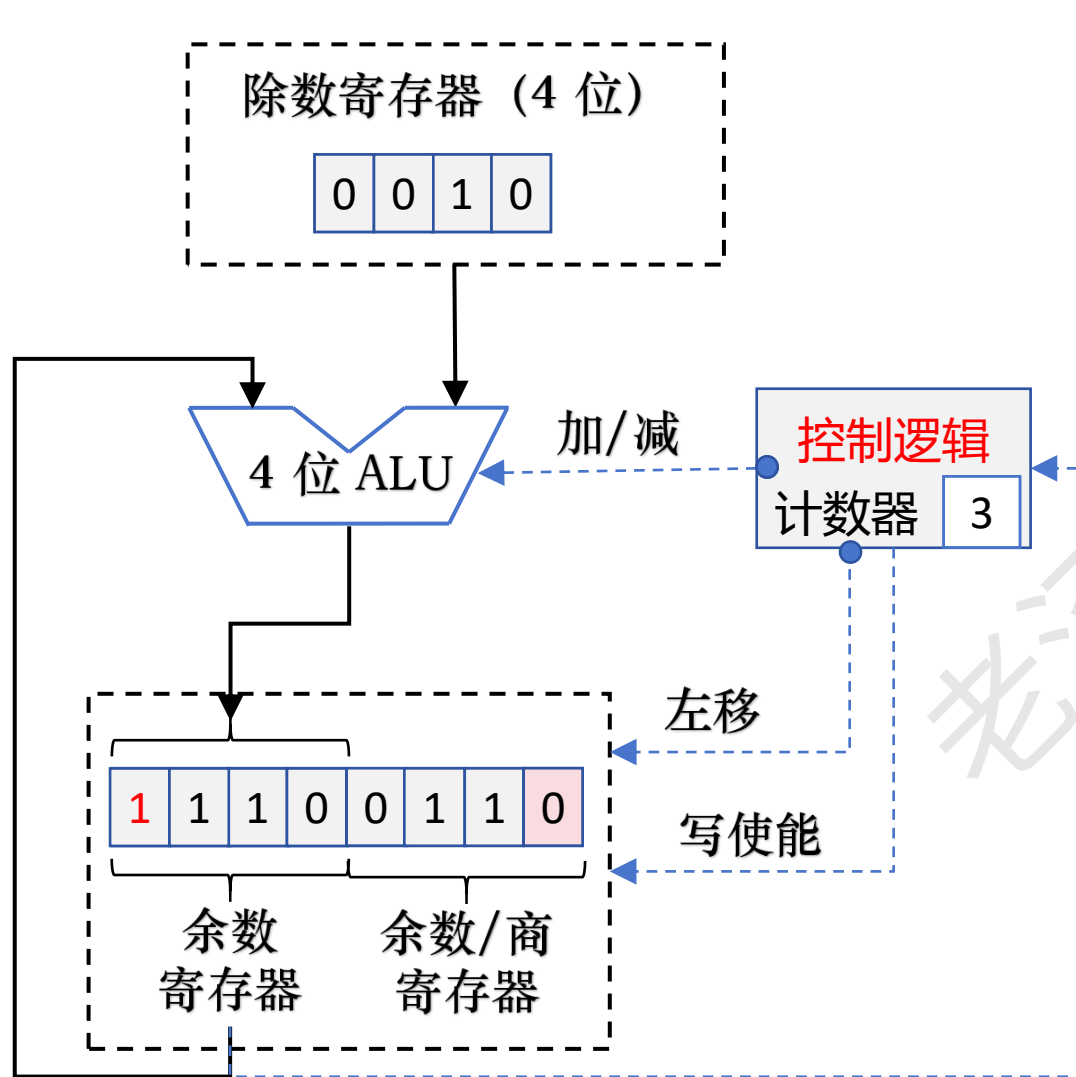


加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



扫一扫上面的二维码图案，加我为朋友。

定点数除法运算：原码除法运算（电路优化）



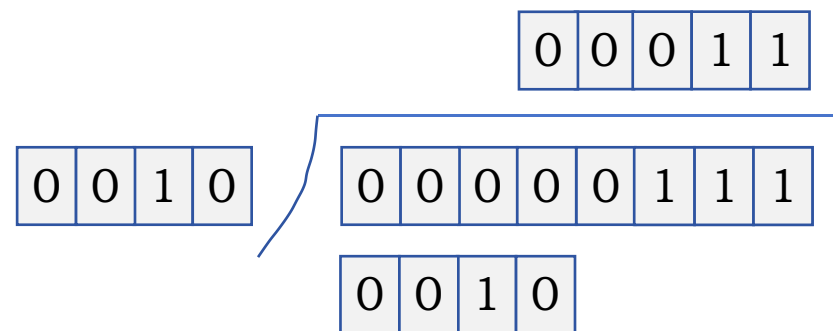
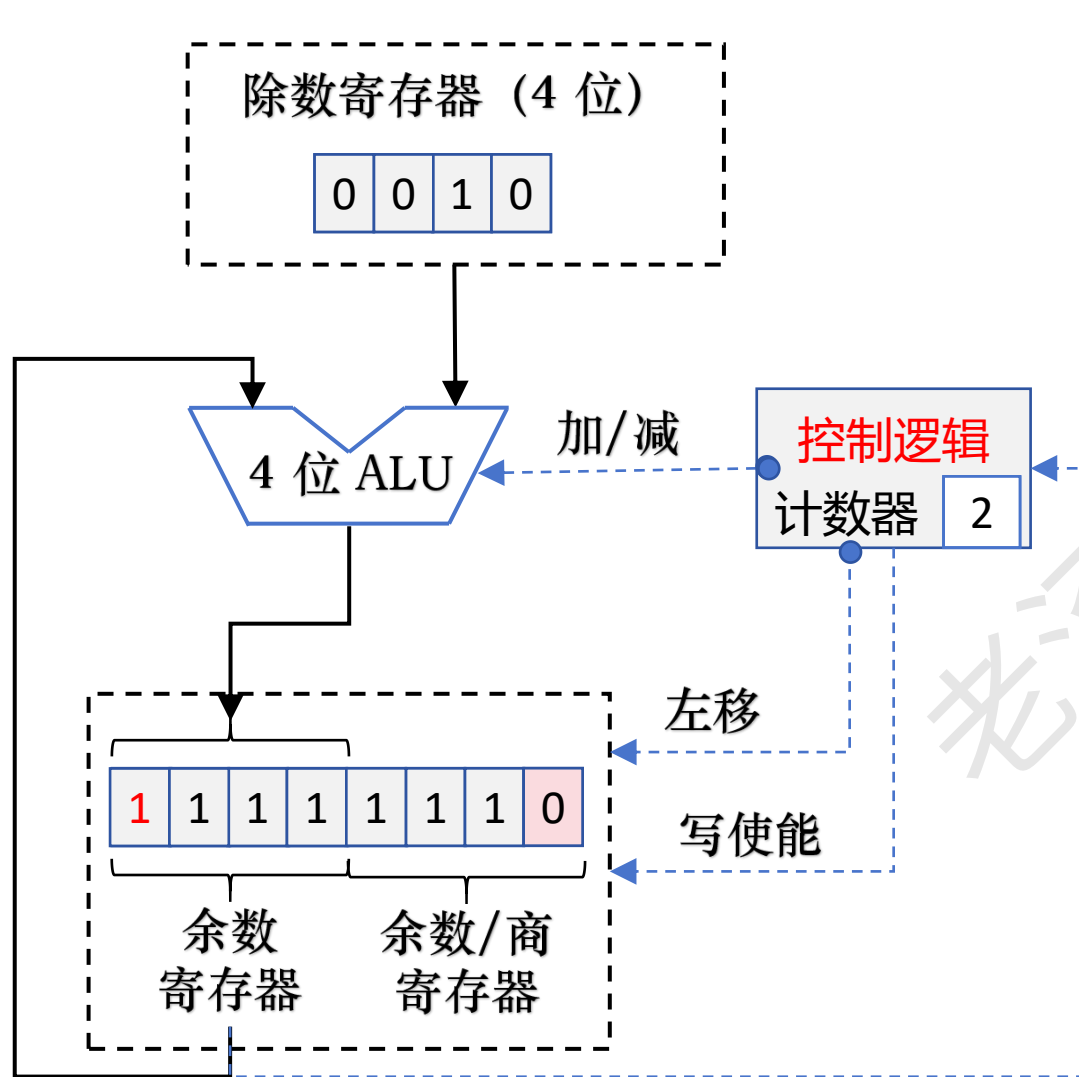
$$0000 - 0010 = 0000 + 1110 = 1110$$

$$1110 + 0010 = 0000$$

第 1 轮

- ① 将余数寄存器以及商寄存器同时左移一位
- ② 余数 = 余数 - 除数
- ③ 余数 < 0, +除数 (恢复余数)
- ④ 上商 0
- ⑤ 计数器减 1

定点数除法运算：原码除法运算（电路优化）



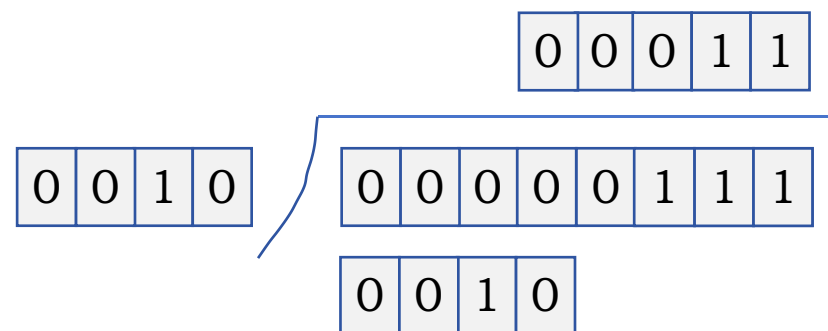
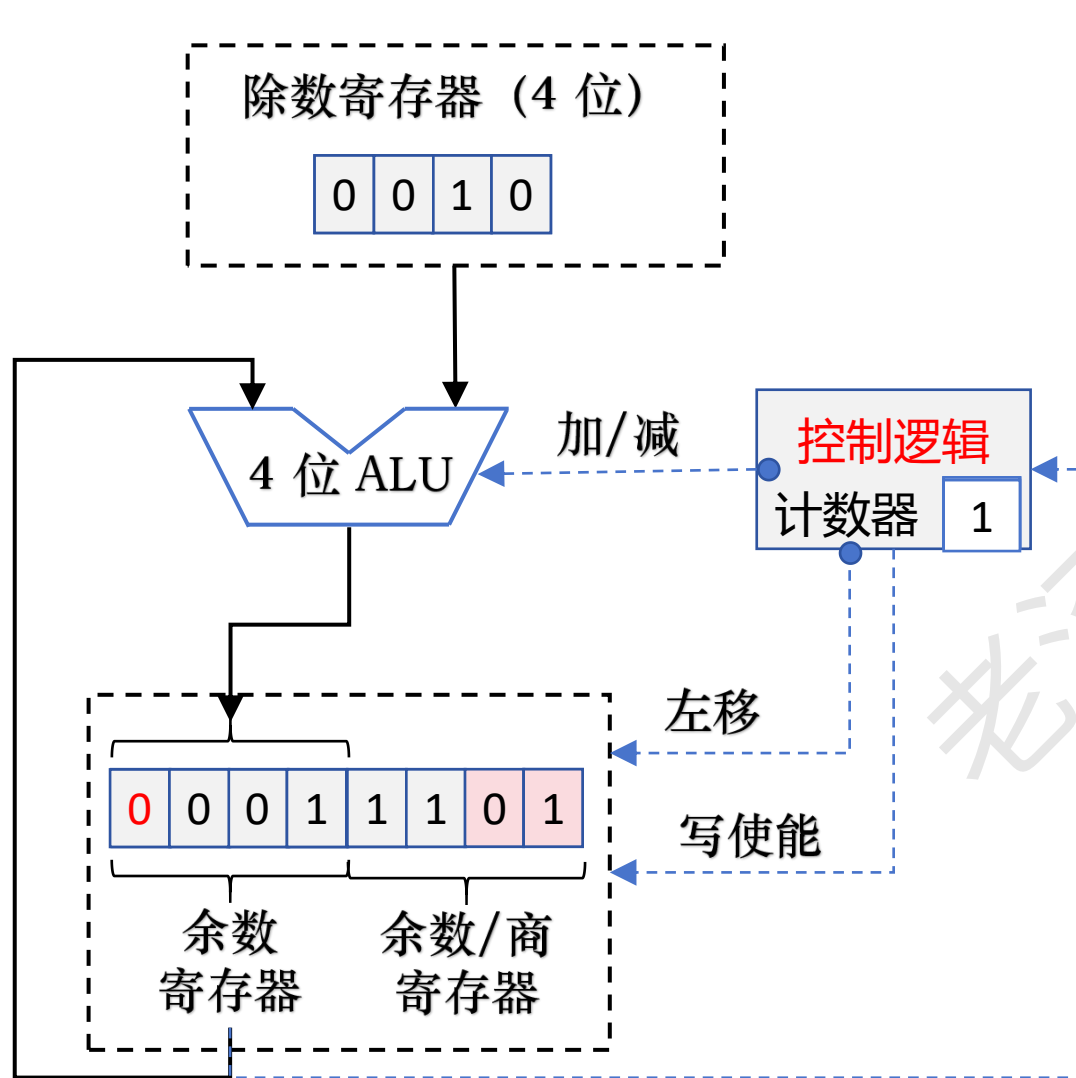
$$0001 - 0010 = 0001 + 1110 = 1111$$

$$1111 + 0010 = 0001$$

第 2 轮

- ① 将余数寄存器以及商寄存器同时左移一位
- ② 余数 = 余数 - 除数
- ③ 余数 < 0, +除数 (恢复余数)
- ④ 上商 0
- ⑤ 计数器减 1

定点数除法运算：原码除法运算（电路优化）

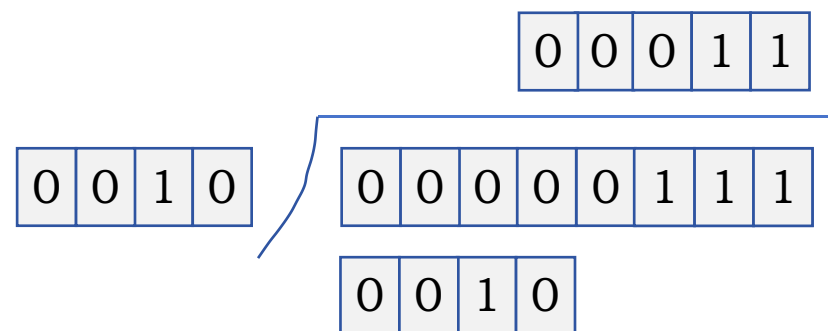
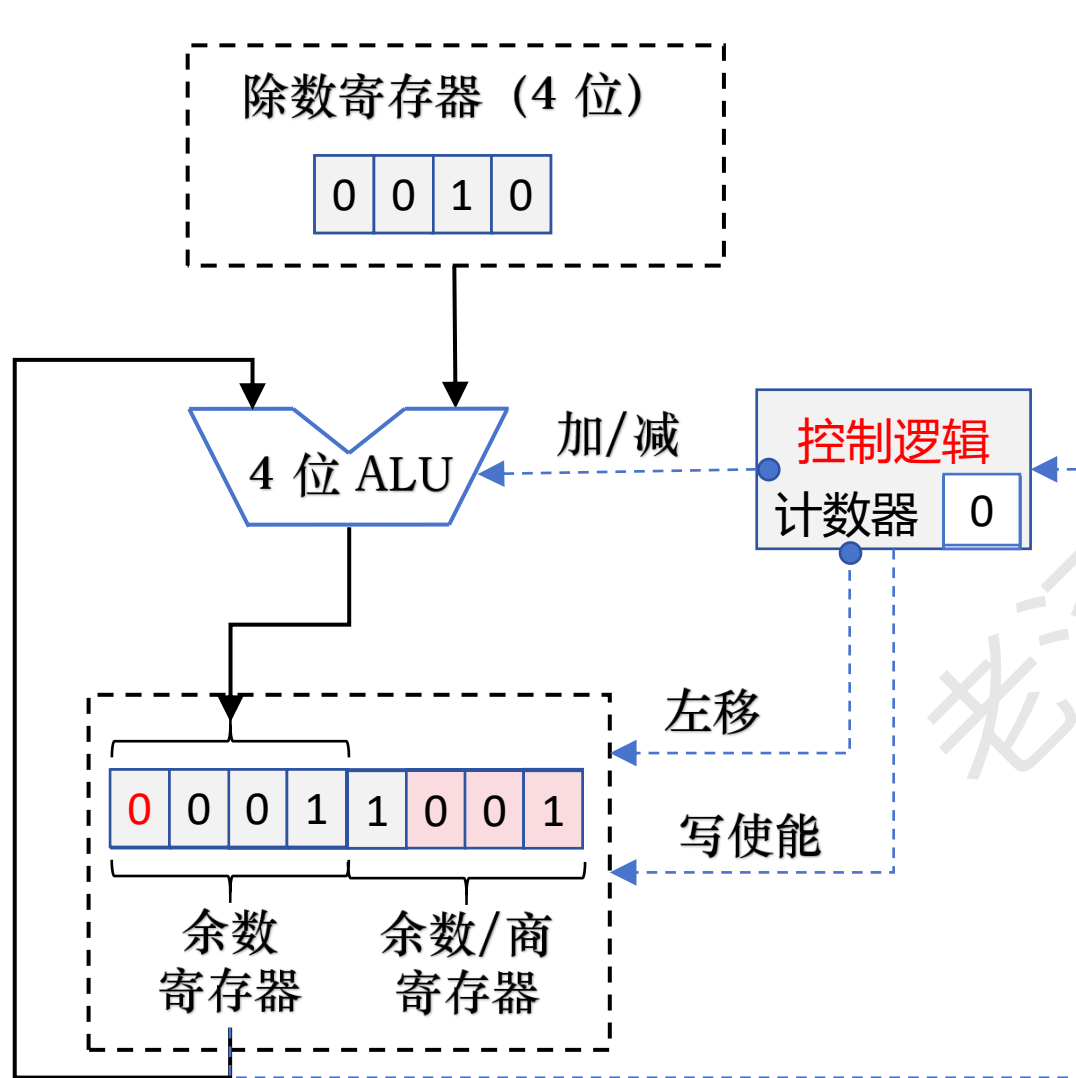


$$0011 - 0010 = 0011 + 1110 = 0001$$

第 3 轮

- ① 将余数寄存器以及商寄存器同时左移一位
- ② 余数 = 余数 - 除数
- ③ 余数 > 0, 不需要恢复余数
- ④ 上商 1
- ⑤ 计数器减 1

定点数除法运算：原码除法运算（电路优化）

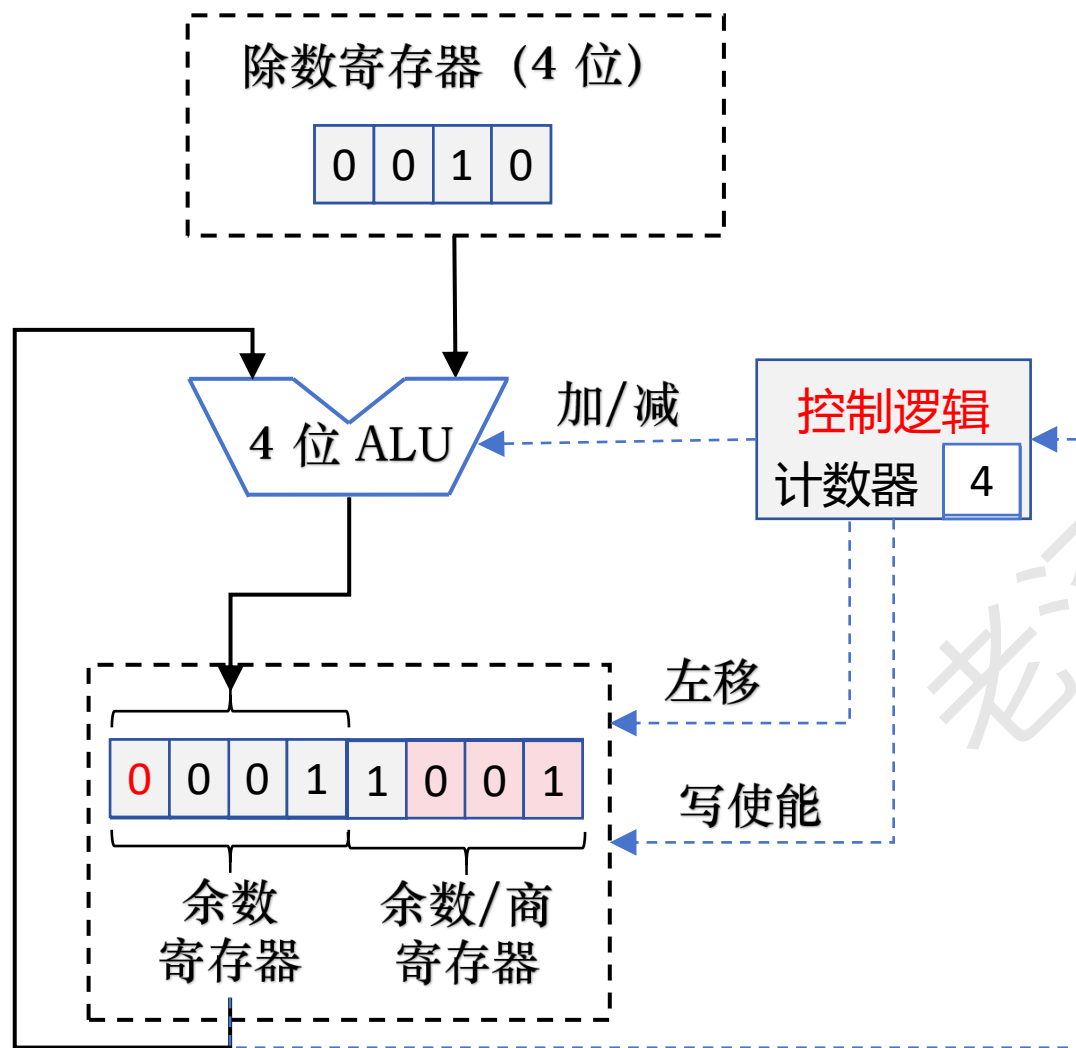


$$0011 - 0010 = 0011 + 1110 = 0001$$

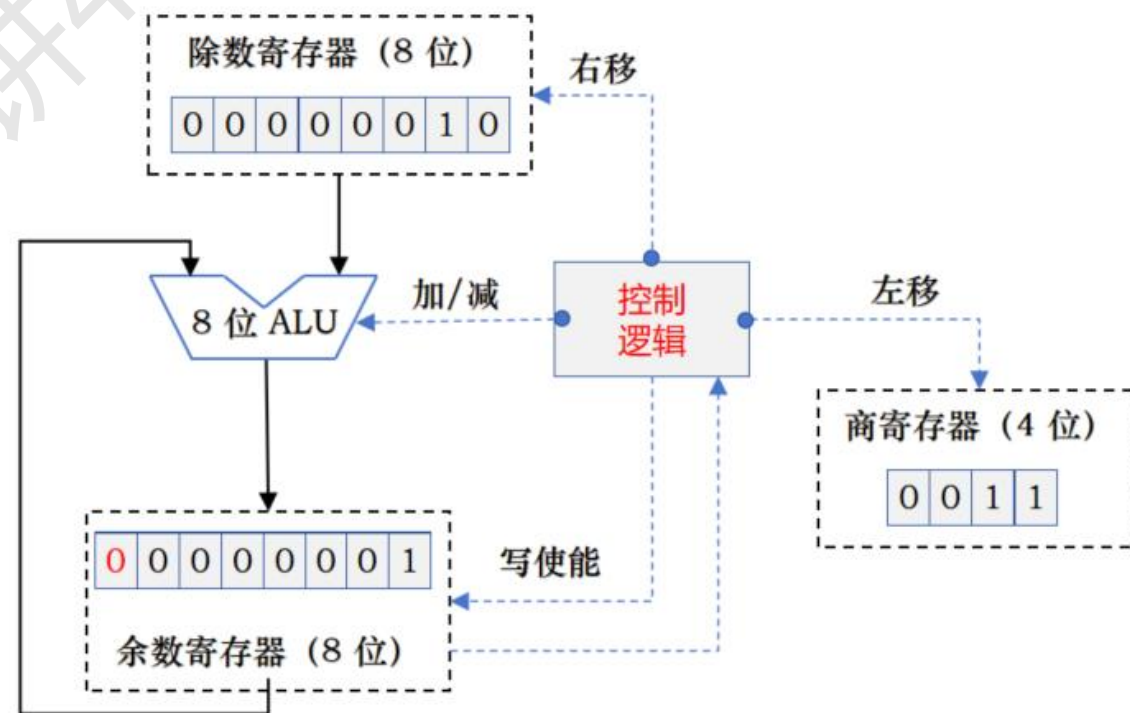
第 4 轮

- ① 将余数寄存器以及商寄存器同时左移一位
- ② 余数 = 余数 - 除数
- ③ 余数 > 0, 不需要恢复余数
- ④ 上商 1
- ⑤ 计数器减 1

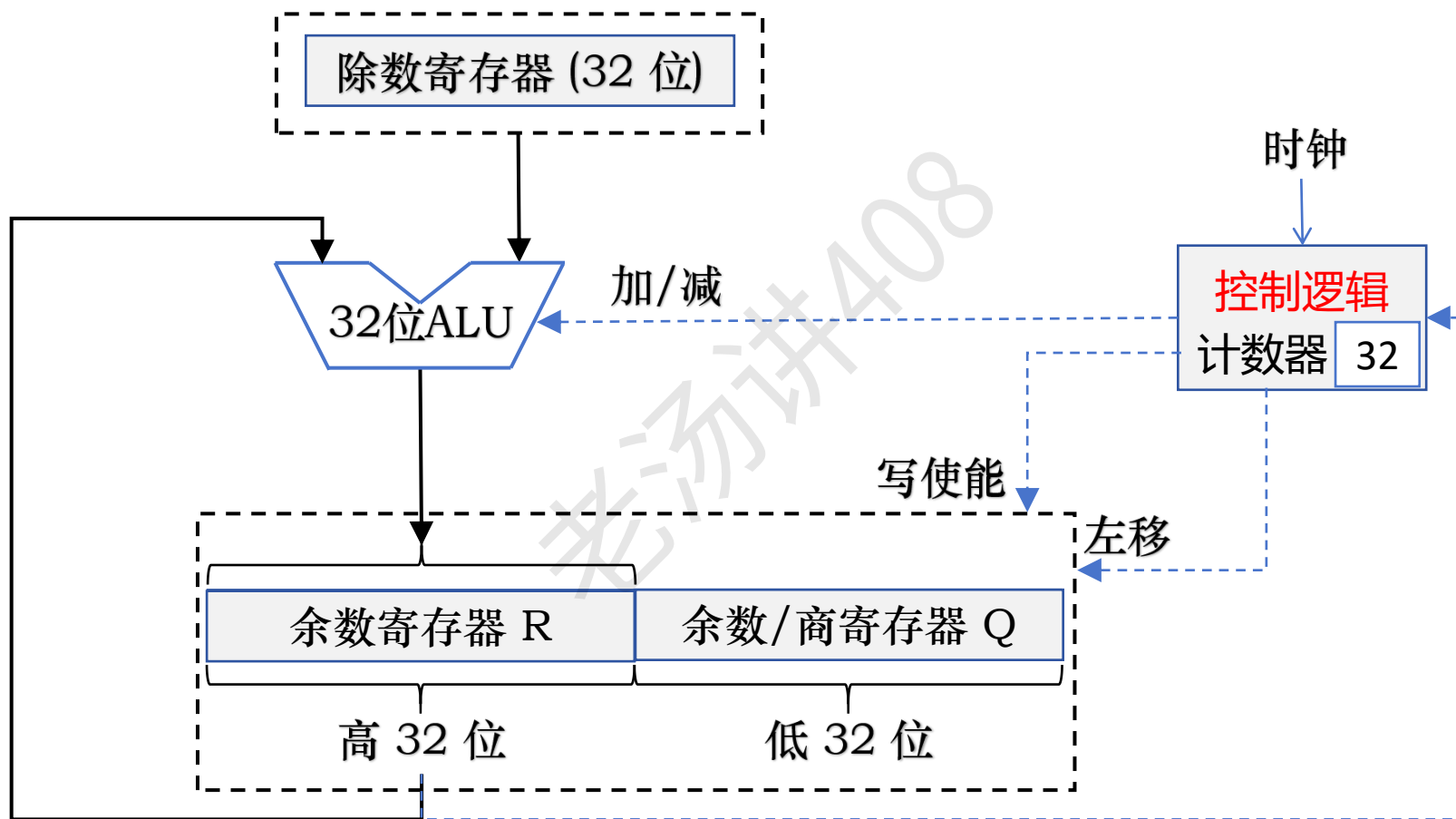
定点数除法运算：原码除法运算（电路优化）



优化前电路



定点数除法运算：原码除法运算（电路优化）



定点数除法运算：原码除法运算（符号处理）

$$7 \div 2 = 3 \dots\dots 1$$

$$-7 \div (-2) = ? \dots\dots ? \qquad -7 \div (-2) = 3 \dots\dots -1$$

$$-7 \div 2 = ? \dots\dots ? \qquad -7 \div 2 = -3 \dots\dots -1$$

$$7 \div (-2) = ? \dots\dots ? \qquad 7 \div (-2) = -3 \dots\dots 1$$

如果两个除数符号相同，那么商的符号就是正数，如果两个除数符号不同，那么商的符号就是负数

余数的符号和被除数的符号保持一致

定点数除法运算：定点小数除法运算

已知 $[x]_{\text{原}} = 0.1011$, $[y]_{\text{原}} = 1.1101$, 计算 $[x/y]_{\text{原}}$

$X = [|x|]_{\text{补}} = 0.1011$

$Y = [|y|]_{\text{补}} = 0.1101$

商的符号位: $0 \wedge 1 = 1$ (负)

余数的符号位和被除数符号位相同, 为 0 (正)

$[-|y|]_{\text{补}} = 1.0011$

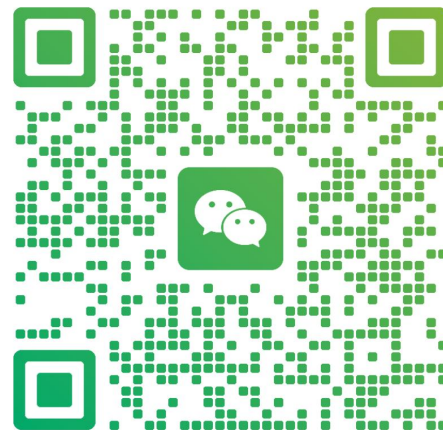
加老汤微信, 帮你 408 冲 120+
咨询【老汤 408 高分特训营】

$$|x| / |y| = 0.1011 / 0.1101$$



老汤

浙江 杭州



扫一扫上面的二维码图案, 加我为朋友。

已知 $[x]_{\text{原}} = 0.1011$, $[y]_{\text{原}} = 1.1101$, 计算 $[x/y]_{\text{原}}$

$$X = [|x|]_{\text{补}} = 0.1011$$

$$Y = [|y|]_{\text{补}} = 0.1101$$

$$[-|y|]_{\text{补}} = 1.0011$$

余数寄存器 R	余数/商寄存器 Q	说明
0 1 0 1 1	0 0 0 0	开始: $R_0 = X$
+ 1 0 0 1 1		$R_1 = X - Y$
1 1 1 1 0	0 0 0 0 0	$R_1 < 0$, 则 $Q_4 = 0$
+ 0 1 1 0 1		恢复余数: $R_1 = R_1 + Y$
0 1 0 1 0		得到 R_1
1 0 1 1 0	0 0 0 0	$2R_1$ (R 和 Q 同时左移, 空出一位商)
+ 1 0 0 1 1		$R_2 = 2R_1 - Y$
0 1 0 0 1	0 0 0 0 1	$R_2 > 0$, 则 $Q_3 = 1$
1 0 0 1 0	0 0 0 1	$2R_2$ (R 和 Q 同时左移, 空出一位商)
+ 1 0 0 1 1		$R_3 = 2R_2 - Y$
0 0 1 0 1	0 0 0 1 1	$R_3 > 0$, 则 $Q_2 = 1$
0 1 0 1 0	0 0 1 1	$2R_3$ (R 和 Q 同时左移, 空出一位商)
+ 1 0 0 1 1		$R_4 = 2R_3 - Y$
1 1 1 0 1	0 0 1 1 0	$R_4 < 0$, 则 $Q_1 = 0$

已知 $[x]_{\text{原}} = 0.1011$, $[y]_{\text{原}} = 1.1101$, 计算 $[x/y]_{\text{原}}$

$$X = [|x|]_{\text{补}} = 0.1011$$

$$Y = [|y|]_{\text{补}} = 0.1101$$

$$[-|y|]_{\text{补}} = 1.0011$$

余数寄存器 R	余数/商寄存器 Q	说明
0 1 0 1 0 + 1 0 0 1 1 1 1 1 0 1 + 0 1 1 0 1 0 1 0 1 0	0 0 1 1 0 0 1 1 0 0 0 1 1 0	$2R_3$ (R 和 Q 同时左移, 空出一位商) $R_4 = 2R_3 - Y$ $R_4 < 0$, 则 $Q_1 = 0$ 恢复余数: $R_4 = R_4 + Y$ 得到 R_4
1 0 1 0 0 + 1 0 0 1 1 0 0 1 1 1	0 1 1 0 0 1 1 0 1	$2R_4$ (R 和 Q 同时左移, 空出一位商) $R_5 = 2R_4 - Y$ $R_5 > 0$, 则 $Q_0 = 1$

$$\begin{array}{r}
 0.1101 \\
 0.1101 \overline{) 0.10110} \\
 \underline{0.01101} \\
 0.010010 \\
 \underline{0.001101} \\
 0.00010100 \\
 \underline{0.00001101} \\
 0.00000111
 \end{array}
 \begin{array}{l}
 \\
 2^{-1} \cdot y \\
 2^{-2} \cdot y \\
 2^{-4} \cdot y
 \end{array}$$

恢复余数除法

商: 1.1101

余数: 0.0111 $\times 2^{-4}$

已知 $[x]_{\text{原}} = 0.1011$, $[y]_{\text{原}} = 1.1101$, 计算 $[x/y]_{\text{原}}$

$$X = [|x|]_{\text{补}} = 0.1011$$

$$Y = [|y|]_{\text{补}} = 0.1101$$

$$[-|y|]_{\text{补}} = 1.0011$$

余数寄存器 R	余数/商寄存器 Q	说明
0 1 0 1 1	0 0 0 0	开始: $R_0 = X$
+ 1 0 0 1 1		$R_1 = X - Y$
1 1 1 1 0	0 0 0 0 0	$R_1 < 0$, 则 $Q_4 = 0$
+ 0 1 1 0 1		恢复余数: $R_1 = R_1 + Y$
0 1 0 1 0		得到 R_1
1 0 1 1 0	0 0 0 0	$2R_1$ (R 和 Q 同时左移, 空出一位商)
+ 1 0 0 1 1		$R_2 = 2R_1 - Y$
0 1 0 0 1	0 0 0 0 1	$R_2 > 0$, 则 $Q_3 = 1$
1 0 0 1 0	0 0 0 1	$2R_2$ (R 和 Q 同时左移, 空出一位商)
+ 1 0 0 1 1		$R_3 = 2R_2 - Y$
0 0 1 0 1	0 0 0 1 1	$R_3 > 0$, 则 $Q_2 = 1$
0 1 0 1 0	0 0 1 1	$2R_3$ (R 和 Q 同时左移, 空出一位商)
+ 1 0 0 1 1		$R_4 = 2R_3 - Y$
1 1 1 0 1	0 0 1 1 0	$R_4 < 0$, 则 $Q_1 = 0$

第 i 次余数: $R_i = 2R_{i-1} - Y$

① 若 $R_i \geq 0$, 则上商 1, 不需要恢复余数, 直接左移一位后再来试商, 得到下次余数:
 $R_{i+1} = 2R_i - Y$

② 若 $R_i < 0$, 则上商 0, 需要恢复余数, 然后再左移一位后再来试商, 得到下次余数:
 $R_{i+1} = 2(R_i + Y) - Y = 2R_i + Y$

当第 i 次中间余数为负时, 可以跳过恢复余数这一步, 直接求第 $i + 1$ 次中间余数, 这种算法称为不恢复余数除法

定点数除法运算：原码除法运算（不恢复余数除法）

$X = [|x|]_{补} = 0.1011$
 $Y = [|y|]_{补} = 0.1101$
 $[-|y|]_{补} = 1.0011$

余数寄存器 R	余数/商寄存器 Q	说明
0 1 0 1 1	0 0 0 0	开始: $R_0 = X$
+ 1 0 0 1 1		$R_1 = X - Y$
1 1 1 1 0	0 0 0 0 0	$R_1 < 0$, 则 $Q_4 = 0$
1 1 1 0 0	0 0 0 0	$2R_1$ (R 和 Q 同时左移, 空出一位商)
+ 0 1 1 0 1		$R_2 = 2R_1 + Y$
0 1 0 0 1	0 0 0 0 1	$R_2 > 0$, 则 $Q_3 = 1$
1 0 0 1 0	0 0 0 1	$2R_2$ (R 和 Q 同时左移, 空出一位商)
+ 1 0 0 1 1		$R_3 = 2R_2 - Y$
0 0 1 0 1	0 0 0 1 1	$R_3 > 0$, 则 $Q_2 = 1$
0 1 0 1 0	0 0 1 1	$2R_3$ (R 和 Q 同时左移, 空出一位商)
+ 1 0 0 1 1		$R_4 = 2R_3 - Y$
0 0 1 0 1	0 0 1 1 0	$R_4 < 0$, 则 $Q_1 = 0$
1 0 1 0 0	0 1 1 0	$2R_4$ (R 和 Q 同时左移, 空出一位商)
+ 0 1 1 0 1		$R_5 = 2R_4 + Y$
0 0 1 1 1	0 1 1 0 1	$R_5 > 0$, 则 $Q_0 = 1$

第 i 次余数: $R_i = 2R_{i-1} - Y$

① 若 $R_i \geq 0$, 则上商 1, 不需要恢复余数, 直接左移一位后再来试商, 得到下次余数:
 $R_{i+1} = 2R_i - Y$

② 若 $R_i < 0$, 则上商 0, 需要恢复余数, 然后再左移一位后再来试商, 得到下次余数:
 $R_{i+1} = 2(R_i + Y) - Y = 2R_i + Y$

当第 i 次中间余数为负时, 可以跳过恢复余数这一步, 直接求第 i + 1 次中间余数, 这种算法称为不恢复余数除法

定点数除法运算：原码除法运算（不恢复余数除法）

$X = [|x|]_{补} = 0.1011$
 $Y = [|y|]_{补} = 0.1101$
 $[-|y|]_{补} = 1.0011$

余数寄存器 R	余数/商寄存器 Q	说明
0 1 0 1 1	0 0 0 0	开始: $R_0 = X$
+ 1 0 0 1 1		$R_1 = X - Y$
1 1 1 1 0	0 0 0 0 0	$R_1 < 0$, 则 $Q_4 = 0$
1 1 1 0 0	0 0 0 0	$2R_1$ (R 和 Q 同时左移, 空出一位商)
+ 0 1 1 0 1		$R_2 = 2R_1 + Y$
0 1 0 0 1	0 0 0 0 1	$R_2 > 0$, 则 $Q_3 = 1$
1 0 0 1 0	0 0 0 1	$2R_2$ (R 和 Q 同时左移, 空出一位商)
+ 1 0 0 1 1		$R_3 = 2R_2 - Y$
0 0 1 0 1	0 0 0 1 1	$R_3 > 0$, 则 $Q_2 = 1$
0 1 0 1 0	0 0 1 1	$2R_3$ (R 和 Q 同时左移, 空出一位商)
+ 1 0 0 1 1		$R_4 = 2R_3 - Y$
0 0 1 0 1	0 0 1 1 0	$R_4 < 0$, 则 $Q_1 = 0$
1 0 1 0 0	0 1 1 0	$2R_4$ (R 和 Q 同时左移, 空出一位商)
+ 0 1 1 0 1		$R_5 = 2R_4 + Y$
0 0 1 1 1	0 1 1 0 1	$R_5 > 0$, 则 $Q_0 = 1$

不恢复余数除法算法要点：正、1、减、负、0、加

定点数除法运算：补码除法运算

当中间余数与除数同号时，做减法；异号时，做加法

若加减运算后，得到的新余数与原余数符号一致（即余数符号未变），则够减；否则不够减

$$\begin{array}{r} -7 \quad 1001 \\ -6 \quad - \quad 1010 \\ \hline -1 \quad 1111 \end{array}$$

$$\begin{array}{r} 7 \quad 0111 \\ 6 \quad - \quad 0110 \\ \hline 1 \quad 0001 \end{array}$$

$$\begin{array}{r} 3 \quad 0011 \\ -2 \quad + \quad 1110 \\ \hline 1 \quad 0001 \end{array}$$

$$\begin{array}{r} -2 \quad 1110 \\ 5 \quad + \quad 0110 \\ \hline 1 \quad 0100 \end{array}$$

定点数除法运算：补码除法运算（恢复余数法）

例子：用 4 位补码恢复余数法计算 $-7/3$ 的值

-7 的 8 位补码 $X = [x]_{\text{补}} = 1111\ 1001$ （进行符号扩展）

3 的 4 位补码 $Y = [y]_{\text{补}} = 0011$

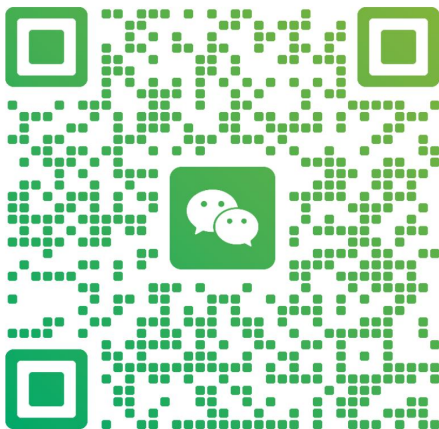
$-Y = [-y]_{\text{补}} = 1101$

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

定点数除法运算：补码除法运算（恢复余数法）

例子：用 4 位补码恢复余数法计算 -7/3 的值

余数寄存器 R	余数/商寄存器 Q	说明
1111	1001	开始：R ₀ = X
1111 0011	001	2R ₀ (R 和 Q 同时左移，空出一位商) R ₁ 和 Y 异号，做加法
0010 1101	001 0	R 符号已变，说明不够减，上商 0 恢复余数（减），得 R ₁
1111 0011	010	2R ₁ (R 和 Q 同时左移，空出一位商) R ₂ 和 Y 异号，做加法
0001 1101	010 0	R 符号已变，说明不够减，上商 0 恢复余数（减），得 R ₂
1110 0011	100	2R ₂ (R 和 Q 同时左移，空出一位商) R ₃ 和 Y 异号，做加法
1111	1001	R 符号未变，说明够减，上商 1

-7 的 8 位补码 X = [x]_补 = 1111 1001

3 的 4 位补码 Y = [y]_补 = 0011

-Y = [-y]_补 = 1101

定点数除法运算：补码除法运算（恢复余数法）

例子：用 4 位补码恢复余数法计算 -7/3 的值

余数寄存器 R	余数/商寄存器 Q	说明
1 1 1 1	1 0 0 1	接上面
1 1 1 1 + 0 0 1 1	0 0 1	2R ₃ (R 和 Q 同时左移，空出一位商)
0 0 1 0 + 1 1 0 1	0 0 1 0	R ₄ 和 Y 异号，做加法
1 1 1 1		R 符号已变，说明不够减，上商 0
		恢复余数（减），得 R ₄

-7 的 8 位补码 $X = [x]_{补} = 1111\ 1001$

3 的 4 位补码 $Y = [y]_{补} = 0011$

$-Y = [-y]_{补} = 1101$

如果被除数和除数同号，则 Q 中是真正的商；否则，将 Q 中的数值求补后作为商

求补为（取反加 1）：1110，其真值是 -2

余数是补码 1111，真值是 -1

$-7/3 = -2.....-1$

定点数除法运算：补码除法运算（不恢复余数法）

例子：已知 $x = -9$, $y = 2$ ，用补码除法计算 $[x/y]_{\text{补}}$

$$X = [x]_{\text{补}} = 1\ 0111, Y = [y]_{\text{补}} = 0\ 0010$$

对被除数符号扩展 $X = 11111\ 10111$

$$-Y = [-y]_{\text{补}} = 1\ 1110$$

补码除法运算的不恢复余数法 6 字口诀：同、1、减、异、0、加

如果这一轮的中间余数的符号和除数的符号相同，那么上商 1，下一轮做减法；

如果这一轮的中间余数的符号和除数的符号不同，那么上商 0，下一轮做加法

定点数除法运算：补码除法运算（不恢复余数法）

余数寄存器 R	余数/商寄存器 Q	说明	例子：已知 $x = -9, y = 2$ ，用补码除法计算 $[x/y]_{\text{补}}$
1 1 1 1 1	1 0 1 1 1	开始： $R_0 = X$	$X = [x]_{\text{补}} = 1\ 0111, Y = [y]_{\text{补}} = 0\ 0010$
+ 0 0 0 1 0		$R_1 = X + Y$	对被除数符号扩展 $X = 11111\ 10111$
0 0 0 0 1	1 0 1 1 1	R_1 与 Y 同号，故 $Q_5 = 1$	
0 0 0 1 1	0 1 1 1 1	$2R_1$ (R 和 Q 同时左移，空出一位上商 1)	$-Y = [-y]_{\text{补}} = 1\ 1110$
+ 1 1 1 1 0		$R_2 = 2R_1 - Y$	
0 0 0 0 1	0 1 1 1 1	R_2 与 Y 同号，故 $Q_4 = 1$	
0 0 0 1 0	1 1 1 1 1	$2R_2$ (R 和 Q 同时左移，空出一位上商 1)	
+ 1 1 1 1 0		$R_3 = 2R_2 - Y$	
0 0 0 0 0	1 1 1 1 1	R_3 与 Y 同号，故 $Q_3 = 1$	
0 0 0 0 1	1 1 1 1 1	$2R_3$ (R 和 Q 同时左移，空出一位上商 1)	
+ 1 1 1 1 0		$R_4 = 2R_3 - Y$	
1 1 1 1 1		R_4 与 Y 异号，故 $Q_2 = 0$	
1 1 1 1 1	1 1 1 1 0	$2R_4$ (R 和 Q 同时左移，空出一位上商 0)	
+ 0 0 0 1 0		$R_5 = 2R_4 + Y$	
0 0 0 0 1	1 1 1 1 0	R_5 与 Y 同号，故 $Q_1 = 1$	

6 字口诀：
同、1、减、异、0、加

定点数除法运算：补码除法运算（不恢复余数法）

余数寄存器 R	余数/商寄存器 Q	说明	例子：已知 $x = -9, y=2$ ，用补码除法计算 $[x/y]_{\text{补}}$
0 0 0 0 1	1 1 1 1 0	开始： $R_0 = X$	$X = [x]_{\text{补}} = 1\ 0111, Y = [y]_{\text{补}} = 0\ 0010$
0 0 0 1 1 + 1 1 1 1 0	1 1 1 0 1	$2R_5$ (R 和 Q 同时左移，空出一位上商 1) $R_6 = 2R_5 - Y$	对被除数符号扩展 $X = 11111\ 10111$
0 0 0 0 1 + 1 1 1 1 0	1 1 0 1 1 + 1	R_6 与 Y 同号，Q 左移，上商 $Q_0 = 1$	$-Y = [-y]_{\text{补}} = 1\ 1110$
1 1 1 1 1	1 1 1 0 0	商为负数，末位加 1 减除数以修正余数	

商的修正：若被除数与除数同号，则 Q 中就是真正的商，否则 Q 中的商末位加 1

6 字口诀：
同、1、减、异、0、加

余数的修正：若余数符号和被除数符号相同，则不需要修正，R 寄存器中就是余数；
否则按照这个规则进行修正：

1. 当被除数和除数符号相同，最后余数加除数
2. 否则，最后余数减除数

$[x/y]_{\text{补}} = 111100$ ，余数是 11111 $x/y = -0100B = -4$ ，余数是 $-0001B = -1$ $-9/2 = -4\ldots -1$

整数乘除运算

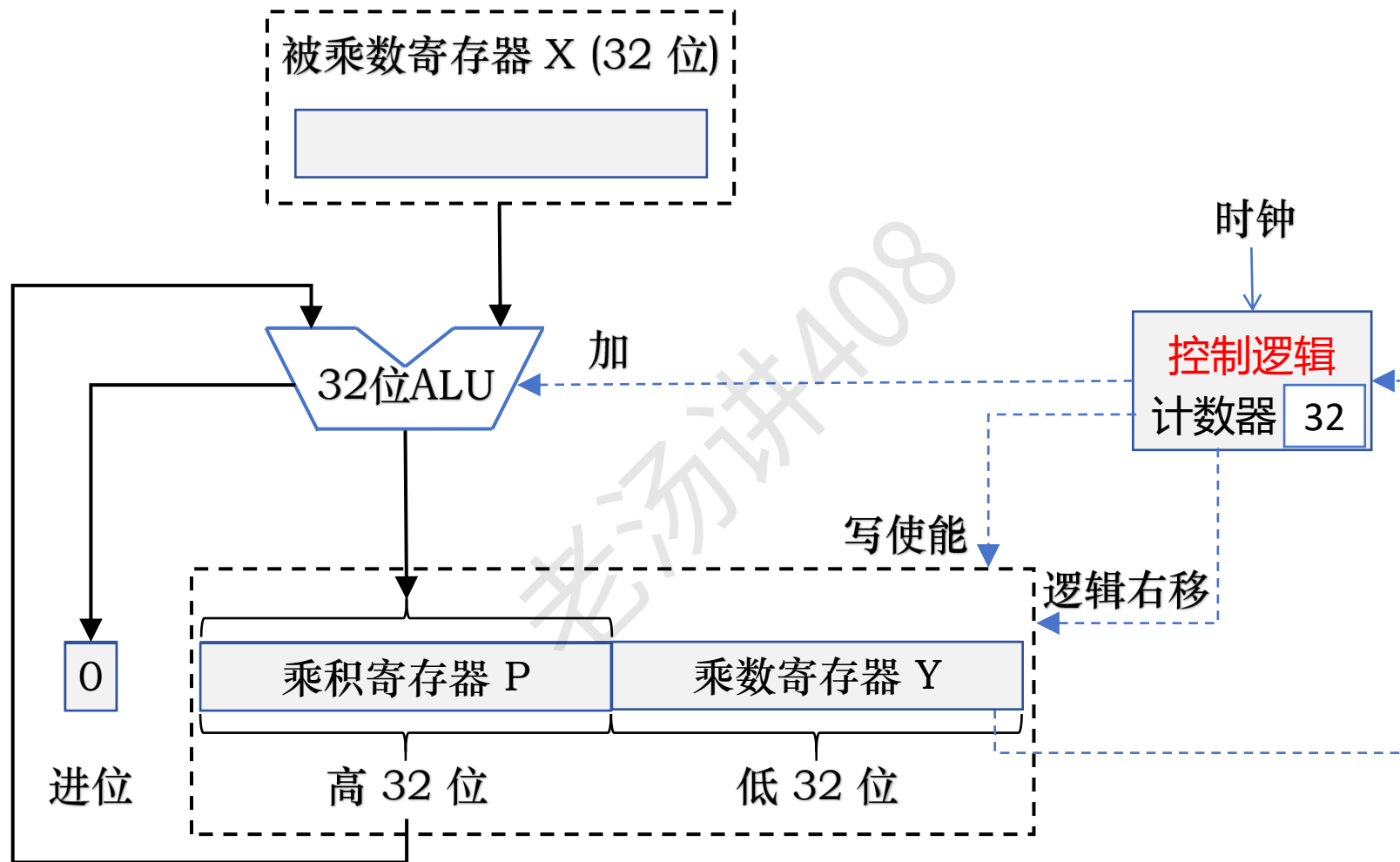
```
short a = 23;  
short b = 34;  
short res = a * b;
```

```
int a = 23;  
short b = 34;  
int res = a * b;
```

在高级语言中，两个 n 位整数相乘得到的结果通常也是一个 n 位整数

两个 n 位整数相乘，得到的是一个 $2n$ 位的整数

定点数乘法运算：有符号整数乘法（原码一位乘法）



整数乘除运算：无符号整数相乘的溢出判断

当丢弃的高 n 位乘积为非 0，则发生溢出

$$0010 \times 0110 = \textcolor{red}{0000} 1100 \quad \longrightarrow \quad 0010 \times 0110 = 1100$$

$$2 \times 6 = 12$$

未溢出

$$0110 \times 1010 = \textcolor{red}{0011} 1100 \quad \longrightarrow \quad 0110 \times 1010 = 1100$$

$$6 \times 10 = 12$$

溢出

整数乘除运算：带符号整数相乘的溢出判断

若高 n 位中每一位都与低 n 位的最高位相同，则不溢出；否则溢出

$$0110 \times 1010 = \textcolor{red}{1101} 1100 \quad \longrightarrow \quad 0010 \times 1010 = 1100$$

$$6 \times (-6) = -4$$

溢出

$$1101 \times 1110 = \textcolor{red}{0000} 0110 \quad \longrightarrow \quad 1101 \times 1110 = 0110$$

$$(-3) \times (-2) = 6$$

未溢出

$$0010 \times 1100 = \textcolor{red}{1111} 1000 \quad \longrightarrow \quad 0010 \times 1100 = 1000$$

$$2 \times (-4) = -6$$

未溢出

$$1000 \times 0010 = \textcolor{red}{1111} 0000 \quad \longrightarrow \quad 1000 \times 0010 = 0000$$

$$-8 \times 2 = 0$$

溢出

被除数 $\div$ 除数 = 商 余数 整数乘除运算：整数除法的溢出判断

商的绝对值不可能比被除数的绝对值更大，因而除法一般不会发生溢出

只有当 int 的最小值 $\div (-1)$ 时会发生溢出

$$-2147483648 / (-1) = 2147483648$$

超过了 int 的最大值 $2147483647 (2^{31} - 1)$ ，所以溢出

浮点数加减运算

$$0.123 \times 10^5 + 0.456 \times 10^2$$

$$= 0.123 \times 10^5 + 0.000456 \times 10^5$$

对阶（小阶向大阶看齐）

$$= 0.123456 \times 10^5$$

尾数相加

$$= 0.123 \times 10^5$$

尾数舍入处理

就近舍入到偶数（类似 4 舍 5 入）

浮点数加减运算：IEEE 754 提供的四种尾数舍入策略

	2.052 ₄₀	2.052 ₅₀	2.052 ₆₀	-2.053 ₄₀	-2.053 ₅₀	-2.053 ₆₀
就近舍入到偶数	2.052	2.052	2.053	-2.053	-2.054	-2.054
朝 $+\infty$ 方向舍入						
朝 $-\infty$ 方向舍入						
朝 0 方向舍入						

就近舍入到偶数：相当于十进制的 4 舍 5 入（不完全是），二进制的 0 舍 1 入

当结果正好在两个可表示数中间时，根据就近舍入的原则无法操作，此时结果强迫为偶数

浮点数加减运算：IEEE 754 提供的四种尾数舍入策略

	2.052 ₄₀	2.052 ₅₀	2.052 ₆₀	-2.053 ₄₀	-2.053 ₅₀	-2.053 ₆₀
就近舍入到偶数	2.052	2.052	2.053	-2.053	-2.054	-2.054
朝 $+\infty$ 方向舍入	2.053	2.053	2.053	-2.053	-2.053	-2.053
朝 $-\infty$ 方向舍入	2.052	2.052	2.052	-2.054	-2.054	-2.054
朝 0 方向舍入	2.052	2.052	2.052	-2.053	-2.053	-2.053

就近舍入到偶数：相当于十进制的 4 舍 5 入（不完全是），二进制的 0 舍 1 入

当结果正好在两个可表示数中间时，根据就近舍入的原则无法操作，此时结果强迫为偶数

朝 $+\infty$ 方向舍入：总是取数轴上右边的最近可表示数，也称为正向舍入或朝上舍入

朝 $-\infty$ 方向舍入：总是取数轴上左边的最近可表示数，也称为负向舍入或朝下舍入

朝 0 方向舍入：直接截取所需位数，丢弃后面所有位，也称为截取、截断或恒舍法

对于正数和负数来说，都是取数轴上更靠近原点 (0 点) 的那个可表示数，是一种趋向原点的舍入，又称为趋向 0 舍入

IEEE 754 标准浮点数加减运算

例题 1: 若 x 和 y 为 float 型, $x = 1.5$, $y = 2.75$, 请给出 $x + y$ 的计算过程

① 十进制转换成二进制规格化浮点数

$$x = 1.5 = 1.1B$$

$$\text{IEEE 754 规格化浮点数: } 1.1 \times 2^0$$

$$y = 2.75 = 10.11B$$

$$\text{IEEE 754 规格化浮点数: } 1.011 \times 2^1$$

② 按照 IEEE 754 标准表示为单精度浮点数

对于 $x = 1.5$

符号位 $S = 0$ (正数)

阶码 $E = 0 + 127$, 二进制表示为 01111111

尾数 M 去掉整数部分的 1, 取小数部分并补 0 到 23 位

0 01111111 100000000000000000000000

对于 $y = 2.75$

符号位 $S = 0$ (正数)

阶码 $E = 1 + 127$, 二进制表示为 10000000

尾数 M 去掉整数部分的 1, 取小数部分并补 0 到 23 位

0 10000000 011000000000000000000000

IEEE 754 标准浮点数加减运算

例题 1: 若 x 和 y 为 float 型, $x = 1.5$, $y = 2.75$, 请给出 $x + y$ 的计算过程

$x = 1.5 = 1.1B \rightarrow$ IEEE 754 规格化浮点数: $1.1 \times 2^0 \rightarrow 0\ 01111111\ 100000000000000000000000$

$y = 2.75 = 10.11B \rightarrow$ IEEE 754 规格化浮点数: $1.011 \times 2^1 \rightarrow 0\ 10000000\ 011000000000000000000000$

	$x + y = 1.1 \times 2^0 + 1.011 \times 2^1$	0.11
第一步: 对阶 (小阶看向大阶)	$= 0.11 \times 2^1 + 1.011 \times 2^1$	$+ \ 1.011$
第二步: 尾数相加	$= 10.001 \times 2^1$	10.001
第三步: 尾数规格化 (右规)	$= 1.0001 \times 2^2 \rightarrow 0\ 10000001\ 000100000000000000000000$	
第四步: 尾数舍入处理	尾数没有超过 23 位, 不需要舍入处理	

结果验证: $x + y = 1.5 + 2.75 = 1.0001 \times 2^2 = 100.01B = 4.25$

IEEE 754 标准浮点数加减运算

例题 2: 若 x 和 y 为 float 型, $x = 2.11$, $y = 43.25$, 请给出 $x + y$ 的计算过程

① 十进制转换成二进制规格化浮点数

$x = 2.11 = 10.00011100001001100111010111..._B$ 规格化: $1.000011100001001100111010111... \times 2^1$

$y = 43.25 = 101011.01_B$

规格化: 1.0101101×2^5

② 按照 IEEE 754 标准表示为单精度浮点数

对于 $x = 2.11$

符号位 $S = 0$ (正数)

阶码 $E = 1 + 127$, 二进制表示为 10000000

尾数 M 去掉整数部分的 1, 取小数部分的 23 位

0 10000000 00001110000100110011101

```
float b = 2.11;
```

```
printf("%.9f\n", b); // 2.109999895
```

对于 $y = 43.25$

符号位 $S = 0$ (正数)

阶码 $E = 5 + 127$, 二进制表示为 10000100

尾数 M 去掉整数部分的 1, 取小数部分并补 0 到 23 位

0 10000100 010110100000000000000000

IEEE 754 标准浮点数加减运算

例题 2: 若 x 和 y 为 float 型, $x = 2.11$, $y = 43.25$, 请给出 $x + y$ 的计算过程

$$x = 1.00001110000100110011101 \times 2^1 \quad \rightarrow \quad 0\ 10000000\ 00001110000100110011101$$

$$y = 1.0101101 \times 2^5 \quad \rightarrow \quad 0\ 10000100\ 010110100000000000000000$$

$$x + y = 1.00001110000100110011101 \times 2^1 + 1.0101101 \times 2^5$$

第一步: 对阶 (小阶看向大阶)

$$= 0.000100001110000100110011101 \times 2^5 + 1.0101101 \times 2^5$$

$$= 0.000100001110000100110011101 \times 2^5 + 1.0101101 \times 2^5$$

在 IEEE 754 标准规定, 浮点数运算的中间结果右边必须至少额外保留两位附加位

保护位, 或者警戒位: 用以保护尾数右移的位

舍入位: 左规时可以根据其值进行舍入

粘位: 只要舍入位的右边有任何非 0 数字, 粘位被置 1, 否则, 粘位置 0

IEEE 754 标准浮点数加减运算

在就近舍入到偶数的舍入策略中，使用粘位可以减少运算结果正好在两个可表示数中间的情况

$$1.24 \times 10^4 + 5.03 \times 10^1 \quad \text{要求结果保留 2 位小数}$$

$$= 1.24 \times 10^4 + 0.00503 \times 10^4$$

不使用粘位:

$$= 1.24\textcolor{red}{00} \times 10^4 + 0.00\textcolor{red}{50} \times 10^4$$

$$= 1.24\textcolor{red}{50} \times 10^4 \quad \text{就近舍入到偶数} = 1.24 \times 10^4$$

$$= 1.24 \times 10^4 + 0.00503 \times 10^4$$

使用粘位:

$$= 1.24\textcolor{red}{000} \times 10^4 + 0.00\textcolor{red}{503} \times 10^4$$

$$= 1.24\textcolor{red}{503} \times 10^4 \quad \text{就近舍入到偶数} = 1.25 \times 10^4$$

IEEE 754 标准浮点数加减运算

例题 2: 若 x 和 y 为 float 型, $x = 2.11$, $y = 43.25$, 请给出 $x + y$ 的计算过程

$$x = 1.00001110000100110011101 \times 2^1 \quad \rightarrow \quad 0\ 10000000\ 00001110000100110011101$$

$$y = 1.0101101 \times 2^5 \quad \rightarrow \quad 0\ 10000100\ 010110100000000000000000$$

$$x + y = 1.00001110000100110011101 \times 2^1 + 1.0101101 \times 2^5$$

第一步: 对阶 (小阶看向大阶)

$$= 0.000100001110000100110011101 \times 2^5 + 1.0101101 \times 2^5$$

$$= 0.000100001110000100110011101 \times 2^5 + 1.0101101 \times 2^5$$

第二步: 尾数相加

$$= 1.011010101110000100110011101 \times 2^5$$

第三步: 尾数规格化

第四步: 尾数舍入处理

$$= 1.01101010111000010011010 \times 2^5 \quad (\text{就近舍入到偶数: 0 舍 1 入})$$

结果验证: $x + y = 2.11 + 43.25 \approx 1.01101010111000010011010 \times 2^5 = 45.35995$

IEEE 754 标准浮点数加减运算：溢出判断

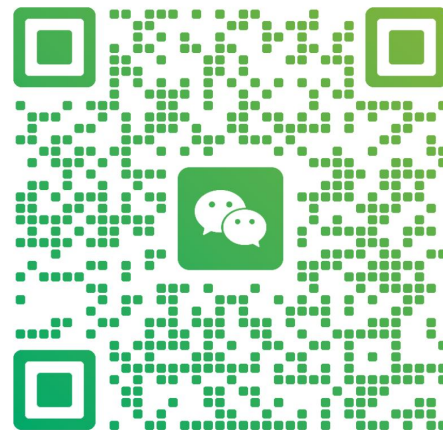
在进行尾数规格化和尾数舍入时，可能会对结果的阶码执行加、减运算，因此必须考虑结果的指数溢出问题

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

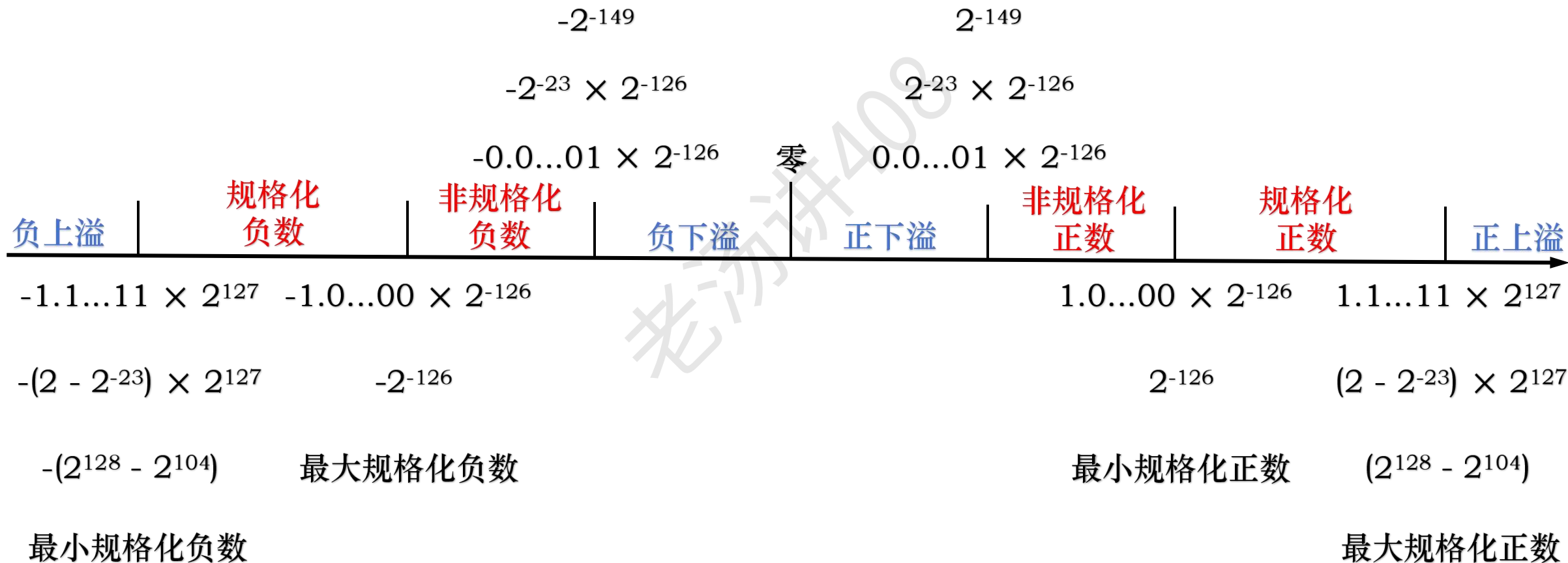
IEEE 754 浮点数标准：32 位单精度表示范围

单精度指数最大允许值：127

单精度指数最小允许值：-149

最大非规格化负数

最小非规格化正数



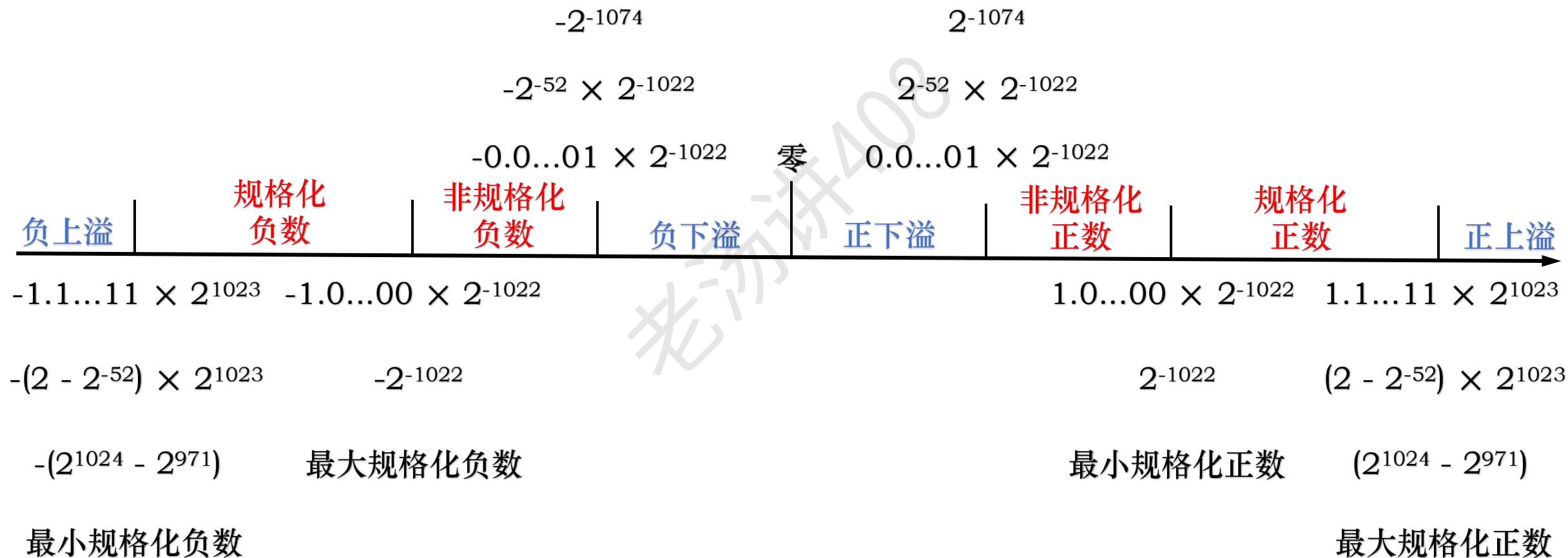
IEEE 754 浮点数标准：64 位双精度表示范围

双精度指数最大允许值：1023

双精度指数最大允许值：-1074

最大非规格化负数

最小非规格化正数



IEEE 754 标准浮点数加减运算：溢出判断

在进行尾数规格化和尾数舍入时，可能会对结果的阶码执行加、减运算，因此必须考虑结果的指数溢出问题

单精度指数范围：-149 ~ 127

双精度指数范围：-1074 ~ 1023

如果一个正指数超过了最大允许值（127 或 1023），则发生指数上溢，机器产生异常，也有的机器把结果设置为 $+\infty$ （数符为 0 时）或 $-\infty$ （数符为 1 时）后，继续执行下去

如果一个负指数超过了最小允许值（-149 或 -1074），则发生指数下溢，此时，一般把结果设置为 $+0$ （数符为 0 时）或 -0 （数符为 1 时），也有的机器引起异常

IEEE 754 标准浮点数加减运算：溢出判断

$$\begin{array}{r} 1.111\ 1111\ 1111\ 1111\ 1111\ 0000 \times 2^{124} \\ +\ 1.000\ 0000\ 0000\ 0000\ 0000\ 0111 \times 2^{124} \\ \hline \end{array}$$

尾数溢出：

$$10.111\ 1111\ 1111\ 1111\ 1111\ 0111 \times 2^{124}$$



右规

$$1.011\ 1111\ 1111\ 1111\ 1111\ 1011\ \textcolor{red}{1} \times 2^{125}$$



尾数舍入

$$1.011\ 1111\ 1111\ 1111\ 1111\ 1100 \times 2^{125} \quad (\text{就近舍入到偶数：0 舍 1 入})$$

尾数溢出并不是真正的浮点数溢出

IEEE 754 标准浮点数加减运算：溢出判断

$$\begin{array}{r} 1.111\ 1111\ 1111\ 1111\ 1111\ 0000 \times 2^{127} \\ +\ 0.000\ 0000\ 0000\ 0000\ 0000\ 0111 \times 2^{127} \\ \hline \end{array}$$

尾数溢出：

$$10.111\ 1111\ 1111\ 1111\ 1111\ 0111 \times 2^{127}$$



右规

$$1.011\ 1111\ 1111\ 1111\ 1111\ 1011\ \textcolor{red}{1} \times 2^{128}$$

单精度指数范围：-149 ~ 127

尾数溢出并不是真正的浮点数溢出

IEEE 754 标准浮点数加减运算：溢出判断

0.000 1111 1111 1111 1111 0111 $\times 2^{-148}$



左规

0.111 1111 1111 1111 0111 0000 $\times 2^{-151}$

单精度指数范围：-149 ~ 127

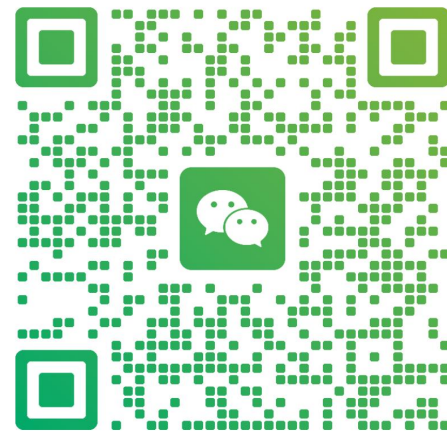
IEEE 754 标准浮点数加减运算：溢出判断

- 右规一次，指数加 1，可能导致指数上溢
- 左规一次，指数减 1，可能导致指数下溢
- 尾数溢出并不是真正的浮点数溢出

加老汤微信，帮你 408 冲 120+
浮点数溢出



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。